



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

An Asynchronous Tree Network-on-Chip for Exact Hierarchical Multicast

Master Thesis

Marco Ferroni

September 30, 2026

Advisors: Prof. Dr. Giacomo Indiveri, Prof. Dr. Kwabena Boahen

Department of Information Technology and Electrical Engineering, ETH Zürich

Abstract

Multicast communication is a fundamental requirement in many-core systems characterized by sparse and irregular one-to-many data movement, including neuromorphic and other event-driven architectures. Existing multicast schemes trade routing precision against addressing overhead: exact representations can encode arbitrary destination sets but often require large headers, whereas compact approximate encodings may deliver packets to non-target endpoints, introducing unnecessary network activity and requiring destination-side filtering. This thesis presents a fully asynchronous quaternary-tree Network-on-Chip (NoC) with native support for exact multicast. The proposed hierarchical routing scheme represents the active multicast subtree and progressively repartitions the route as the packet traverses the network. Each router consumes its local routing information, forwards only the subroutes required by its active children, and replicates the common payload exclusively along branches leading to intended destinations. The routing overhead therefore scales with the number and spatial structure of the active destinations rather than with the total network size, while avoiding false-positive traffic. A batching mechanism can further allow multiple source addresses sharing the same destination set to reuse a single multicast route, amortizing its generation, transmission, and parsing overhead. The router is developed using a quasi-delay-insensitive design methodology and characterized at transistor level using predictive 65-nm CMOS models. The extracted timing and energy parameters are used to parameterize a discrete-event simulator that evaluates the architecture from controlled unicast and multicast traffic up to application-driven Spiking Neural Network workloads. The results expose the main trade-offs of the hierarchical topology: long-distance unicast benefits from the reduced number of router traversals, whereas multicast performance is strongly affected by route-parsing overhead and is therefore most favorable for spatially localized and dense destination sets. Application-level results show that batching reduces the latency of the proposed architecture by more than one order of magnitude between $B_{\max} = 1$ and $B_{\max} = 64$. At the largest evaluated batch size, the proposed exact multicast scheme also reduces dynamic NoC energy by up to 18% relative to the state-of-the-art Hierarchical Bit String (HBS) approach, despite the current router being characterized in predictive 65-nm technology rather than the post-layout 22-nm implementation used for the reference. These results demonstrate that exact hierarchical routing combined with batching can substantially reduce redundant network activity, even in the presence of significant circuit-level implementation overhead.

Contents

Contents	ii
1 Acknowledgements	1
2 Introduction	2
3 Background	6
3.1 Asynchronous Design Flow	6
3.2 Terminology	7
4 Network Architecture	11
4.1 Topology	11
4.1.1 Structural Scaling	13
4.1.2 Deadlock-Free Up-Down Routing	14
4.1.3 Physical Interconnect Trade-Off	15
4.2 Packet-Level Communication	15
4.2.1 Flit Format and Control Bits	15
4.2.2 Unicast Transmission	16
4.2.3 Multicast Transmission	18
5 Multicast Routing Scheme	20
5.1 Route Encoding	20
5.1.1 The Broadcasting Case	21
5.1.2 The Multicasting Case	22
5.2 Route Parsing	23
5.3 Comparison with Existing Routing Schemes	25
5.3.1 Exact Packet-Carried Encodings	25
5.3.2 Approximate Destination Encodings	27
6 Router Microarchitecture	28
6.1 Node Description	28

6.2	Node Decomposition	29
6.2.1	ChildRouter	30
6.2.2	ParentRouter	33
6.2.3	MulticastUnit	35
7	Implementation Flow	43
7.1	Communicating Hardware Processes	43
7.1.1	Top-Level Specification	44
7.1.2	Process Decomposition	44
7.1.3	Bit-Sliced Datapath Decomposition: Omitted Step . . .	44
7.1.4	Functional Verification	45
7.2	Handshaking Expansion	45
7.2.1	HSE Reshuffling	46
7.2.2	State-Variable Insertion	47
7.3	Production Rule Sets	48
7.3.1	PRS Optimization	49
7.3.2	Isochronic-Fork Analysis	49
7.3.3	Bubble Reshuffling	50
7.4	Physical Implementation	51
7.4.1	SPICE Netlist Generation	51
7.4.2	Transistor-Level Verification and Characterization . . .	52
7.4.3	Custom Cell Library: Omitted Step	52
7.4.4	Placement and Routing: Omitted Step	52
7.4.5	Post-Layout Verification: Omitted Step	53
8	Results and Discussion	54
8.1	Methodology	54
8.1.1	Router Characterization	54
8.1.2	Network Characterization	54
8.1.3	Network Benchmarks	57
8.2	Router Characterization	60
8.2.1	Timing Characteristics	60
8.2.2	Dynamic Energy	62
8.2.3	Area	65
8.3	Network-Level Characterization	65
8.3.1	Unicast Results	66
8.3.2	Multicast Results	68
8.4	Network Benchmarks	72
8.4.1	Naturally Occurring Multicast Batches	72
8.4.2	Inference Latency	75
8.4.3	Inference Energy	77
8.5	Discussion	79
9	Conclusion	82

9.1 Limitations and Future Work	83
Bibliography	84
A Common Building Blocks	86
A.1 Reg4	87
A.1.1 PRS	87
A.1.2 Bubble Reshuffling	87
A.2 Reg8	87
A.2.1 PRS	87
A.2.2 Bubble Reshuffling	88
A.3 FullAdder	89
A.3.1 PRS	89
A.3.2 Bubble Reshuffling	89
A.4 HalfAdder	90
A.4.1 PRS	90
A.4.2 Bubble Reshuffling	90
A.5 FullAdder'	91
A.5.1 PRS	91
A.5.2 Bubble Reshuffling	91
A.6 HalfAdder'	92
A.6.1 PRS	92
A.6.2 Bubble Reshuffling	92
A.7 Sequencer7	93
A.7.1 CHP	93
A.7.2 HSE	93
A.7.3 PRS	94
A.7.4 Bubble Reshuffling	95
A.8 Merge4Bits	96
A.8.1 CHP	97
A.8.2 HSE	97
A.8.3 PRS	97
A.8.4 Bubble Reshuffling	98
A.9 Merge8Bits	98
A.9.1 PRS	98
A.9.2 Bubble Reshuffling	99
A.10 Merge32Bits	99
A.10.1 PRS	99
A.10.2 Bubble Reshuffling	99
A.11 Split4Bits	100
A.11.1 CHP	100
A.11.2 HSE	100
A.11.3 PRS	101
A.11.4 Bubble Reshuffling	101

A.12 Arb	101
A.12.1 CHP	101
A.12.2 HSE	103
A.12.3 PRS	104
A.12.4 Bubble Reshuffling	108
B Sub-Router Building Blocks	111
B.1 StopcodeSel	111
B.1.1 CHP	111
B.1.2 HSE	112
B.1.3 PRS	113
B.1.4 Bubble Reshuffling	117
B.2 UpdownSel	120
B.2.1 CHP	121
B.2.2 HSE	121
B.2.3 PRS	124
B.2.4 Bubble Reshuffling	128
B.3 ChildSel	130
B.3.1 CHP	131
B.4 MltcSel	132
B.4.1 CHP	132
B.4.2 HSE	132
B.4.3 PRS	134
B.4.4 Bubble Reshuffling	136
C Multicast Unit Building Blocks	138
C.1 InputRouter	138
C.1.1 CHP	138
C.1.2 HSE	138
C.1.3 PRS	140
C.1.4 Bubble Reshuffling	141
C.2 Serializer	143
C.2.1 CHP	144
C.2.2 HSE	144
C.2.3 PRS	145
C.2.4 Bubble Reshuffling	147
C.3 QuadRouter	149
C.3.1 CHP	149
C.3.2 HSE	153
C.3.3 PRS	157
C.3.4 Bubble Reshuffling	165
C.4 Counter	172
C.4.1 CHP	172
C.4.2 HSE	172

C.4.3	PRS	173
C.4.4	Bubble Reshuffling	177
C.5	Deserializer	181
C.5.1	CHP	181
C.5.2	HSE	183
C.5.3	PRS	186
C.5.4	Bubble Reshuffling	191
C.6	PayloadDispatch	194
C.6.1	CHP	195
C.6.2	HSE	195
C.6.3	PRS	196
C.6.4	Bubble Reshuffling	198
D	Router Characterization Lookup Tables	201
D.1	Unicast Forwarding	201
D.1.1	Forward Latency	202
D.1.2	Initiation Interval	203
D.1.3	Dynamic Energy	204
D.2	Multicast Payload Distribution	204
D.2.1	Forward Latency	205
D.2.2	Initiation Interval	206
D.2.3	Dynamic Energy	207
D.3	Multicast Setup	207
D.4	Multicast Route Parsing	208
D.4.1	Initial Route Word	208
D.4.2	Continuation Route Words	209

Chapter 1

Acknowledgements

First of all, I would like to thank Prof. Kwabena Boahen and Prof. Giacomo Indiveri for their guidance and support throughout this thesis. I am extremely grateful to have had the opportunity to carry out this work at Stanford University.

This thesis will likely mark the end of my academic journey, and I would therefore like to dedicate a few words to the people who have supported me along the way.

My deepest thanks go to my family. In particular, I would like to thank my dad, Daniele, for giving me the financial freedom to pursue my interests and ambitions without having to worry about money; my dear grandma Germana, whose company and food I miss more every day, for her immense love and support; and my dear grandpa Fernando, whose memory continues to guide and inspire me every day, for being the very first supporter of my engineering journey. I would also like to thank my sister Luce, my mom Silvia, and my grandma Leonella for always being there for me.

A heartfelt thank you goes to my friend Togno, for being a constant presence throughout all these years.

I would also like to thank my friends from Villanova for their immense support, my friends in Zurich, and everyone at Vinzenzheim. A special thank you goes to Marco for giving me the opportunity to become part of this big family.

Finally, I would like to thank the members of the Brains in Silicon Lab—Leo, Kyrus, Hery, Caleb, and Dev—for their help and for creating such a stimulating research environment.

Chapter 2

Introduction

As the number of processing elements integrated on a chip continues to increase, communication becomes a first-order architectural constraint. Shared buses and dedicated point-to-point links scale poorly in both wiring complexity and contention, motivating the adoption of packet-switched Networks-on-Chip (NoCs) as scalable communication substrates [1, 2]. However, there is no universally optimal NoC topology: the most appropriate architecture strongly depends on the spatial and temporal characteristics of the target communication workload. This thesis focuses on neuromorphic systems, and particularly on workloads in which communication is *globally sparse but locally dense*. Neural activity is inherently event-driven, so only a subset of the system communicates at any given time, while the connectivity of the neural model can generate dense communication among relatively localized groups of processing elements. These properties make asynchronous hierarchical networks particularly attractive. Local handshakes allow inactive portions of the network to remain dynamically quiescent, while a tree topology provides short hierarchical paths between distant regions and naturally confines localized communication within small subtrees [3, 4]. Previous neuromorphic systems have already demonstrated the suitability of tree-based communication for this class of workloads, and the results presented in Chapter 8 further show that the reduced number of router traversals can provide substantial latency advantages for sparse long-distance communication.

Multicast communication arises naturally in this context. Spiking Neural Networks (SNNs) are commonly partitioned across multiple processing tiles, while the fan-out of a single presynaptic neuron may include postsynaptic neurons located on several different tiles. A single spike must therefore frequently be delivered from one source to multiple destinations. Sending an independent unicast packet to every destination would repeatedly transport the same information through common portions of the network, unnecessarily increasing both traffic and energy. Native one-to-many communication

instead allows a packet to remain shared along common paths and to be replicated only where its destination routes diverge [5, 4, 6]. Moreover, an appropriate partitioning of the neural network can spatially cluster connected populations, making these multicast destination sets locally dense and therefore particularly compatible with a hierarchical tree topology.

A central problem is how the destination set of such a multicast transmission should be represented. One important class of solutions uses compact, approximate destination representations. The multicast tree router introduced by Merolla *et al.* for neuromorphic systems demonstrated how hierarchical routing information can be used to efficiently distribute spikes over a tree [4]. More recently, the Hierarchical Bit String (HBS) scheme proposed by Su *et al.* further improved the scalability of this approach by hierarchically encoding multicast regions [7]. The key advantage of these schemes is the small amount of routing information required: rather than identifying every destination independently, a compact representation activates regions of the network containing the desired endpoints. This compression, however, is obtained by sacrificing destination accuracy. Some activated branches may contain endpoints for which the packet was not intended, producing *false-positive* deliveries that must be discarded through destination-side filtering. Although such filtering is functionally acceptable in many neuromorphic architectures, where the receiving tile may already store sufficient connectivity information to reject the packet, the unnecessary transmission has already consumed network resources. False-positive traffic therefore causes additional switching activity in routers and links, requires bandwidth and arbitration resources, and can increase network congestion even though the corresponding payload is ultimately discarded.

Exact multicast schemes take the opposite approach: they guarantee that the payload reaches only the intended destinations, at the cost of carrying a more expressive routing representation. Existing exact encodings expose different scalability limitations. A Flat Bit String (FBS) associates one bit with every possible endpoint, requiring $O(N)$ routing information independently of the actual multicast fan-out [7, 6]. An explicit destination list instead carries a complete $\lceil \log_2 N \rceil$ -bit address for each of the K destinations, resulting in $O(K \log N)$ routing information [5]. Exact routing therefore avoids false-positive traffic but can introduce a significant setup overhead because the multicast route itself must be transmitted and processed. Crucially, however, this routing overhead is paid once per multicast transaction and can consequently be amortized over a sufficiently long common payload. This observation is particularly relevant for neuromorphic communication. A conventional spike packet may carry only the identifier of a single presynaptic neuron and therefore contain a very short payload, providing little opportunity to amortize an expensive exact route. In many workloads, however, multiple spike occurrences generated within the same timestep originate

from the same source tile and share exactly the same destination set. Their source identifiers can then be *batched* into a single multicast transaction, transforming several short messages into one longer common payload without changing the semantics of the neural computation. The application traces evaluated in this work show that this opportunity is frequent: 92.4%, 96.9%, and 89.1% of the spike addresses generated by MAP-SNN/N-MNIST, MAP-SNN/SHD, and LSM/N-MNIST, respectively, belong to naturally occurring compatible groups containing at least 16 addresses.

This thesis investigates a new exact multicast-route encoding designed specifically for a quaternary tree. Rather than allocating routing information to every endpoint or explicitly listing complete destination addresses, the proposed representation encodes the active multicast subtree itself. Each participating internal node is represented by a local four-bit child-selection mask, and these masks are arranged hierarchically according to the structure of the tree. As the message descends through the network, every router consumes the information corresponding to its own level and dynamically repartitions the remaining encoded route among the active child branches. The size of the representation therefore adapts naturally to the structure and density of the destination set: localized or dense multicast groups can be represented without paying the fixed full-network cost of an FBS, while exact destination information prevents the false-positive traffic generated by approximate representations.

This flexibility comes with an important cost. Because the route is hierarchical, it cannot simply be forwarded unchanged through the network: each router must parse the incoming sequence of child masks, determine which masks belong to each active child, and construct the corresponding subroutes dynamically. As the LCA of a multicast group moves upward, the represented subtree grows rapidly; in a quaternary tree, each additional level increases the number of possible leaf destinations by a factor of four. Consequently, larger multicast regions require increasingly long routes and more parsing operations before the common payload can be distributed. At the network level, the same hierarchy also concentrates increasingly global traffic onto progressively fewer upper-level resources, reducing the sustainable throughput per endpoint as the multicast region expands. This limitation is amplified by the present circuit implementation: the 32-bit dual-rail datapath was not decomposed through bit slicing because of project-time constraints, and the final production-rule sets were optimized primarily for correctness, CMOS implementability, and bubble reshuffling rather than minimum cycle time. As a result, multicast-route parsing is currently the main latency bottleneck, with continuation route words requiring approximately 23.1–51.3 ns and initial route words reaching up to 74.9 ns.

Despite the substantial route-parsing overhead of the current non-optimized

implementation, batching reduces the latency of the proposed architecture by more than one order of magnitude between $B_{\max} = 1$ and $B_{\max} = 64$. In the application-driven evaluation, the proposed NoC also reduces dynamic communication energy by up to 18% compared with the state-of-the-art HBS scheme, despite being characterized in predictive 65-nm technology rather than the 22-nm post-layout technology used for the reference. These results indicate that the reduction in redundant network activity enabled by exact hierarchical routing and batching can compensate for significant circuit-level implementation overhead.

The main contributions of this work are:

- the design and transistor-level implementation of a fully asynchronous quaternary-tree NoC router with native support for unicast and multicast communication. The architecture is developed from Communicating Hardware Processes (CHP) down to production-rule implementations following established quasi-delay-insensitive design methodologies [8, 3];
- a novel exact hierarchical multicast-routing scheme that can represent arbitrary destination sets without false-positive deliveries, while scaling its routing overhead with the number and structure of the active destinations rather than with the total network size;
- a multi-level evaluation combining transistor-level characterization, synthetic network experiments, and application-driven SNN benchmarks. Router timing and dynamic energy are extracted from predictive 65-nm CMOS simulations and used to parameterize a discrete-event model including physical link delay, multicast replication, buffering, arbitration, contention, and backpressure. This evaluation identifies the operating regimes and limitations of the proposed topology and demonstrates the advantages of batching in the evaluated workloads.

Chapter 3

Background

This chapter introduces the concepts and notation required to understand the architecture and implementation developed in this thesis. Section 3.1 presents the asynchronous synthesis methodology used throughout the work, introducing the main abstraction levels from Communicating Hardware Processes (CHP) to Handshaking Expansions (HSE) and Production Rule Sets (PRS). Section 3.2 then defines the principal networking and circuit terms used consistently throughout the following chapters.

3.1 Asynchronous Design Flow

The QDI synthesis methodology used in this work progressively refines a behavioral description into a circuit implementation through three principal representations: *Communicating Hardware Processes* (CHP), *Handshaking Expansions* (HSE), and *Production Rule Sets* (PRS) [8, 3]. Only their basic roles are introduced here; the complete implementation procedure followed for the router is described in Chapter 7.

CHP is the highest-level representation and describes a system as a collection of concurrent processes that interact through communication channels. It captures computation, concurrency, and synchronization while abstracting away the signals and gates used to implement them. HSE lowers these abstract communication actions into explicit Boolean signal transitions implementing handshake protocols. Finally, PRS associates Boolean guards with individual node transitions and forms the representation from which CMOS pull-up and pull-down networks can be synthesized. The ACT toolchain used in this work supports these abstraction levels within a common design environment [9].

Table 3.1 summarizes the notation used throughout this thesis. The notation follows the conventional CHP, HSE, and PRS representations.

Communication	
C	Communicate on dataless port C
$X?x$	Read a value from input port X into x
$Y!y$	Write the value y to output port Y
$Y!X?$	Receive a value from X and directly forward it to Y
$x.s$	Extract field s from x
Probe	
$\overline{C}$	True if a communication is pending on C
$\overline{X?}$	Value pending on X , or null otherwise
Program Composition	
$S_1; S_2$	Execute S_1 , then S_2
$S_1 \parallel S_2$ or S_1, S_2	Execute S_1 and S_2 concurrently
$[e \rightarrow S]$	Execute S when e becomes true
$*[e \rightarrow S]$	Repeatedly execute S while e is true
$[e_1 \rightarrow S_1 \mid e_2 \rightarrow S_2]$	Non-deterministic selection
$[e_1 \rightarrow S_1 \parallel e_2 \rightarrow S_2]$	Deterministic selection; guards are mutually exclusive
$\langle ; v : j..0 : S_v \rangle$	Sequentially execute S_v for $v = j, \dots, 0$
$\langle , v : j..0 : S_v \rangle$	Execute in parallel S_v for $v = j, \dots, 0$
Handshaking Expansion	
$s \uparrow, s \downarrow$	Set Boolean signal s high or low, respectively
$c_r \uparrow; [c_a]; c_r \downarrow; [\neg c_a]$	Four-phase active handshake on channel C
$[d_r]; d_a \uparrow; [\neg d_r]; d_a \downarrow$	Four-phase passive handshake on channel D
Production Rules	
$a \rightarrow b \uparrow$	If guard a is true, eventually set b high
$a \wedge b \Rightarrow c$	Combinational operator: $a \wedge b \rightarrow c \uparrow, \neg a \vee \neg b \rightarrow c \downarrow$
$a \wedge b \Rightarrow c$	State-holding C-element: $a \wedge b \rightarrow c \uparrow, \neg a \wedge \neg b \rightarrow c \downarrow$

Table 3.1: Notation used for Communicating Hardware Processes (CHP), Handshaking Expansions (HSE), and Production Rule Sets (PRS).

3.2 Terminology

The following terminology is used consistently throughout this thesis.

Broadcast. A special case of multicast in which every eligible endpoint in the network is selected as a destination.

C-element. A state-holding asynchronous logic element whose output changes only when all of its inputs agree: it is driven high when all inputs are high, driven low when all inputs are low, and otherwise retains its previous value.

CHP (Communicating Hardware Processes). A high-level formalism for specifying concurrent hardware processes and their communication. In this work, CHP is used as the starting behavioral description before decomposition into lower-level asynchronous circuit representations.

Cycle time / Initiation Interval (II). The minimum time between consecutive operations of an asynchronous pipeline stage under sustained activity. It determines the maximum local steady-state throughput of that stage.

Endpoint. A component connected to the leaves of the network that can inject or receive traffic. In the proposed architecture, an endpoint can act as both a source and a destination.

Fan-out. In a networking context, the number of destinations reached by a multicast transmission. When discussing circuit implementation, *electrical fan-out* instead refers to the number of gate inputs driven by a circuit node. The intended meaning is stated whenever ambiguity is possible.

Flit. The flow-control unit transferred by the router in a single channel communication. A packet is divided into one or more flits. In this work, each flit contains 32 bits.

Forwarding latency (L_f). The local propagation time through a router, measured from the arrival of valid data at an input to the appearance of the corresponding valid data at the selected output. It characterizes the forward data path only and does not include the subsequent return-to-neutral phase; therefore, it is distinct from the cycle time or initiation interval.

Handshake. The sequence of request/data and acknowledge transitions through which two asynchronous components synchronize and transfer information without a global clock.

HBS (Hierarchical Bit String). A hierarchical multicast destination encoding in which routing masks are associated with successive levels of the network hierarchy. In the baseline considered in this work, the representation may introduce false-positive destinations that must subsequently be filtered.

Header. The initial routing information of a packet. Depending on the transmission type, the header identifies either a unicast path or the beginning of a multicast route description.

Hop. The traversal of one network link between adjacent nodes. Hop count is therefore a logical measure of path length and should not be confused with physical propagation delay.

Latency (L). The elapsed time between the initiation of an operation or communication and its completion. At network level, packet latency is measured from packet release at the source until the final flit reaches the destination, or the last destination in the case of multicast.

- Link.** A communication channel connecting two neighboring network nodes. In the physical implementation, a link includes the wires and the asynchronous protocol used to transfer information between them.
- Lowest Common Ancestor (LCA).** In a rooted tree, the deepest router that is an ancestor of all nodes in a given set. For multicast routing, the LCA of the destinations is the highest point that must be reached before all selected destinations can be reached exclusively by descending the tree.
- Liquid State Machine (LSM).** A recurrent spiking neural-network model composed of a fixed recurrent reservoir, or *liquid*, whose activity is interpreted by a trainable readout layer.
- Lookup Table (LUT).** A table of pre-characterized values indexed by one or more operating conditions. In the network simulator, LUTs provide router timing and energy values extracted from transistor-level simulations.
- MAP-SNN.** The published spiking neural-network architecture used in this work to generate application-level communication traces for the N-MNIST and SHD workloads.
- Mask.** A bit vector used to represent a set of selected entities. Unless otherwise specified, an asserted bit indicates that the corresponding child, endpoint, or other indexed object belongs to the selected set.
- MSCODE.** A control code used by the proposed multicast protocol to identify the end of the meaningful route information within a multicast-route word.
- Multicast.** A one-to-many transmission in which the same payload is delivered from one source to a selected subset of network endpoints. Packet replication may occur inside the network so that shared portions of the route are traversed only once.
- N-MNIST (Neuromorphic MNIST).** An event-based version of the MNIST handwritten-digit dataset generated using a neuromorphic vision sensor and used in this work for the SNN application benchmarks.
- Network-on-Chip (NoC).** An on-chip communication infrastructure composed of routers, links, and endpoints that transports data between processing or memory elements.
- Packet.** The complete routed communication unit transmitted through the NoC, consisting of routing information followed by the associated payload. The term *message* is used informally with the same meaning when the distinction is not relevant.

- Payload.** The information carried by a packet after its routing information. For multicast transmissions, the same payload is replicated toward all selected destinations.
- PCFB (Pre-Charge Full Buffer).** An asynchronous pipeline stage based on precharge logic and full buffering, used to decouple neighboring stages and pipeline long combinational or interconnect paths.
- PRS (Production Rule Set).** A transistor-level asynchronous logic representation in which guarded production rules specify the conditions under which circuit nodes are driven high or low.
- QDI (Quasi-Delay-Insensitive).** An asynchronous circuit model that remains functionally correct under arbitrary gate and wire delays except for explicitly assumed isochronic forks.
- Route.** The sequence of routing decisions required to move a packet from its source to its destination or destinations.
- Router.** An internal network node that receives flits from one or more input ports and forwards them toward one or more output ports according to the routing information contained in the packet.
- SHD (Spiking Heidelberg Digits).** An event-based spoken-digit dataset represented as spike trains and used in this work as one of the SNN application workloads.
- Source / Destination.** The *source* is the endpoint that originates a transmission, whereas a *destination* is an endpoint selected to receive it. A multicast transmission contains more than one destination.
- Spiking Neural Network (SNN).** A neural-network model in which information is communicated through discrete spike events evolving over time. In the application benchmarks, neuron spikes generate the communication traffic supplied to the NoC simulator.
- Tail flit.** The final flit of a packet. Its identification indicates that the current transmission has completed and that network resources reserved for the packet can be released.
- Unicast.** A point-to-point transmission from one source to exactly one destination.
- Wormhole routing.** A flow-control scheme in which a packet is divided into flits and forwarded progressively through the network. Once the header establishes a path, subsequent flits follow it without requiring an intermediate router to store the complete packet.

Network Architecture

This chapter presents the network-level architecture developed in this work. Section 4.1 describes the logical quad-tree organization of the network and its physical H-tree embedding, together with its structural scaling, routing properties, and physical interconnect trade-offs. Section 4.2 then defines the packet-level communication protocol, including the supported unicast and multicast transmission modes, flit organization, and routing-header formats.

4.1 Topology

The proposed Network-on-Chip is organized logically as a *quad-tree*. The network endpoints occupy the leaves of the tree and can act as both sources and destinations, while each internal router connects one parent branch to four child subtrees, as illustrated in Fig. 4.1. A communication between endpoints contained within the same small subtree remains confined to the lower levels of the hierarchy, whereas communication spanning progressively more distant endpoint groups naturally progresses toward routers at higher levels.

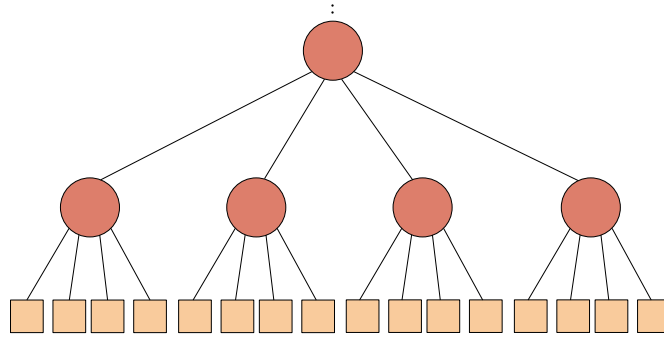


Figure 4.1: Logical organization of the proposed quad-tree network. Internal nodes correspond to routers, while the leaves represent the network source and destination endpoints.

This hierarchical organization is well suited to workloads with dense local connectivity and sparse long-range communication, a property already exploited in hierarchical and small-world neuromorphic interconnects [10, 11]. Recent comparisons further show that tree-based routing becomes increasingly attractive as communication distance, fan-out, and system size grow [12]. The proposed quad-tree follows this principle by explicitly embedding locality into the network structure.

Although the logical network is a tree, the target system consists physically of processing tiles distributed over a two-dimensional square region. The quad-tree is therefore embedded recursively as an *H-tree*, as illustrated in Fig. 4.2. Each router is positioned near the center of the spatial region represented by its subtree, and its four child branches connect progressively smaller quadrants of that region.

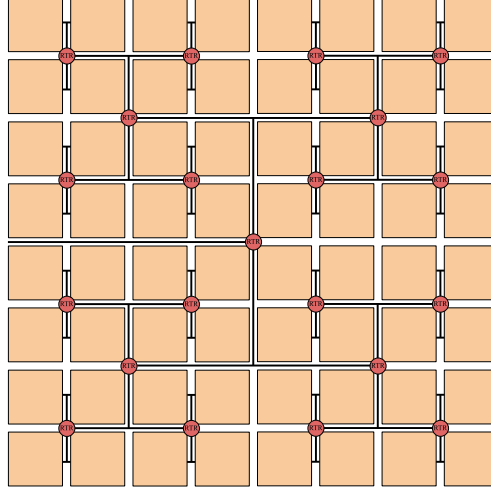


Figure 4.2: Physical H-tree embedding of the logical quad-tree over a two-dimensional array of endpoints.

This embedding preserves the correspondence between network hierarchy and physical locality: traffic between nearby endpoints is resolved through short lower-level branches, while the longer links close to the root are used only when communication spans larger chip regions. The topology therefore concentrates global communication resources only where they are required rather than imposing a uniform global interconnect on every endpoint.

4.1.1 Structural Scaling

A first consequence of the quad-tree organization is that its logical depth grows logarithmically with the number of endpoints. Let N denote the number of leaves of a complete tree and h the number of routing levels between the root and the endpoints. Since every internal router has four children,

$$N = 4^h, \quad (4.1)$$

and therefore

$$h = \log_4 N. \quad (4.2)$$

The maximum number of router stages traversed between two leaves is consequently

$$H_{\text{tree,max}} = 2 \log_4 N, \quad (4.3)$$

corresponding to two endpoints whose lowest common ancestor is the root. For comparison, a conventional one-router-per-tile $\sqrt{N} \times \sqrt{N}$ two-dimensional mesh has Manhattan diameter

$$H_{\text{mesh,max}} = 2(\sqrt{N} - 1). \quad (4.4)$$

The tree therefore provides an $O(\log N)$ logical diameter rather than the $O(\sqrt{N})$ diameter of the mesh. This comparison refers only to router stages and does not imply the same scaling for physical propagation delay, since the H-tree contains non-uniform link lengths; this distinction is discussed in Section 4.1.3.

The hierarchy also changes the number of routing nodes required by the network. A conventional mesh with one router associated with each endpoint requires

$$R_{\text{mesh}}(N) = N. \quad (4.5)$$

For a complete quad-tree, endpoints occupy only the leaves and the internal router count is instead

$$R_{\text{tree}}(N) = \frac{N - 1}{3}. \quad (4.6)$$

Hence,

$$\Delta R(N) = R_{\text{mesh}}(N) - R_{\text{tree}}(N) = \frac{2N + 1}{3}, \quad (4.7)$$

and

$$\lim_{N \rightarrow \infty} \frac{R_{\text{mesh}}(N)}{R_{\text{tree}}(N)} = 3. \quad (4.8)$$

These results highlight an important structural advantage of the quad-tree: as the network grows, it requires only about one third as many routers as a conventional one-router-per-endpoint mesh. The reduction becomes increasingly significant for large systems, providing a more compact hierarchical communication fabric while retaining connectivity to all endpoints.

4.1.2 Deadlock-Free Up-Down Routing

The absence of topological cycles in a tree enables a particularly simple deterministic routing discipline. A packet first travels *upward* from the source to the lowest common ancestor (LCA) of the source and destination, and then reverses direction and travels only *downward* toward the destination. This corresponds to the up-down organization used in previous tree-based multicast networks [4].

Under this discipline, channel dependencies follow a strict ordering: an upward channel may be followed by another upward channel or by a downward channel, whereas a downward channel is never followed by an upward channel. A cyclic dependency would therefore require a packet to return from the downward phase to the upward phase, which the routing rule explicitly prohibits. The same ordering can be preserved for multicast by allowing replication only after the packet has entered the downward phase. Individual copies may then descend through different child subtrees, but none of them is allowed to request a parent channel [4].

This property is especially useful for the wormhole-style communication protocol adopted in this work, since packets may retain network resources while subsequent flits continue to traverse the selected route. Maintaining a globally acyclic channel ordering avoids requiring additional virtual channels solely to break routing-induced dependency cycles.

4.1.3 Physical Interconnect Trade-Off

The principal cost of the hierarchical topology is that its physical links are not uniform. In the H-tree embedding, each successive level toward the root spans a larger spatial region and therefore requires longer wires. For an approximately square endpoint array, the characteristic length of an inter-router branch increases by approximately a factor of two for each level toward the root.

Consequently, the logarithmic logical diameter of the tree does not translate directly into logarithmic physical latency. Upper-level links have larger resistance and capacitance and may contribute a significant fraction of the propagation delay and communication energy. In contrast, a conventional mesh uses a larger number of shorter and more uniform local links. The topology therefore exchanges routing-stage count and hierarchical reach for increasingly expensive global wires, a trade-off also observed in comparisons of hierarchical and spatially local communication fabrics [12].

4.2 Packet-Level Communication

The previous section described the spatial organization of the network. This section defines how messages are represented and transmitted over that organization. The proposed Network-on-Chip supports two transmission modes: *unicast*, for point-to-point communication between a single source and a single destination, and *multicast*.

Both modes use wormhole-style packet transmission. A message is divided into fixed-width flits that propagate through the network sequentially, without requiring a router to buffer the complete message. Once an output has been assigned to a transmission, the corresponding arbitration state is retained until the final flit is observed, preventing flits belonging to different messages from being interleaved on the same output [4].

4.2.1 Flit Format and Control Bits

The datapath width used throughout this work is $W = 32$ bits and, consequently, every flit transmitted through the network consists of one 32-bit word. The four least-significant bits, $b[3 : 0]$, are reserved for control informa-

tion. Only two are currently used, while the remaining two are reserved for possible future extensions.

- **Bit 3 – Reserved:** currently unused and available for future extensions of the communication protocol.
- **Bit 2 – Reserved:** currently unused and available for future extensions of the communication protocol.
- **Bit 1 – *mltc*:** identifies a flit belonging to the multicast-route portion of a transmission. When asserted, the routing information contained in the flit is interpreted by the multicast-routing datapath.
- **Bit 0 – *tail*:** identifies the final flit of the transmission. Detection of this flag terminates packet-level arbitration and releases the resources reserved by the message. A multicast message may contain several route flits followed by an arbitrary number of payload flits, but *tail* is asserted only on the final payload flit.

4.2.2 Unicast Transmission

Unicast implements point-to-point communication between one source endpoint and one destination endpoint. As illustrated in Fig. 4.3, a unicast message consists of one header flit followed by one or more payload flits. The current implementation assumes that at least one payload flit follows the header; therefore, the *tail* bit is deasserted in the header and asserted only on the final payload flit.

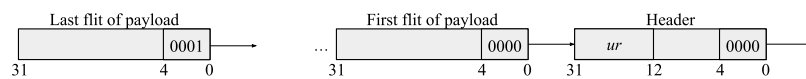


Figure 4.3: Structure of a unicast transmission. A single header flit establishes the route and is followed by one or more payload flits. The *tail* flag is asserted only on the final flit of the transmission.

The 19 most-significant bits of the unicast header, $b[31 : 13]$, form the *unicast route* field, denoted by ur . The route is encoded relative to the source position and is progressively consumed while the header traverses the tree.

Routing consists of an upward phase, a direction-reversal operation, and a downward phase. During the upward phase, the most-significant bit of ur determines whether the packet must continue toward the root. A value of 1 forwards the packet to the parent and shifts the route by one position.

The first 0 encountered at the most-significant position terminates the upward phase. Since the network is a quad-tree, two additional bits identify the child selected after direction reversal. The turnaround router therefore consumes three bits: one direction bit and one two-bit child identifier.

After entering the downward phase, the direction no longer needs to be explicitly represented. Every subsequent router consumes two bits from ur , directly interpreting them as one of its four child identifiers.

For the maximum-depth network considered in this work, 19 routing bits are sufficient for the longest supported path:

$$|ur| = 5 + 1 + 6 \cdot 2 + 1 = 19. \quad (4.9)$$

The final useful route bit acts as a termination marker. Once the preceding routing decisions have been consumed, this marker reaches the most-significant position while all remaining route bits are zero, producing the unicast stop code. This code is used to identify the router at which a multicast transmission begins, as described in Section 4.2.3.

$$USCODE = 1 \underbrace{00 \dots 0}_{18 \text{ bits}}. \quad (4.10)$$

The additional terminator bit is also required for unicast routing to distinguish the end of the route from a valid sequence of child selections followed by zero padding. For example, the pattern $1 \underbrace{00 \dots 0}_{18 \text{ bits}}$ could otherwise be interpreted either as the unicast stop code or as a valid route that first selects child 2 and then repeatedly selects child 0 at all subsequent levels. Appending an explicit termination bit removes this ambiguity and allows the router to determine unambiguously when the encoded route has been fully consumed.

Figure 4.4 summarizes the route encoding and its progressive consumption.

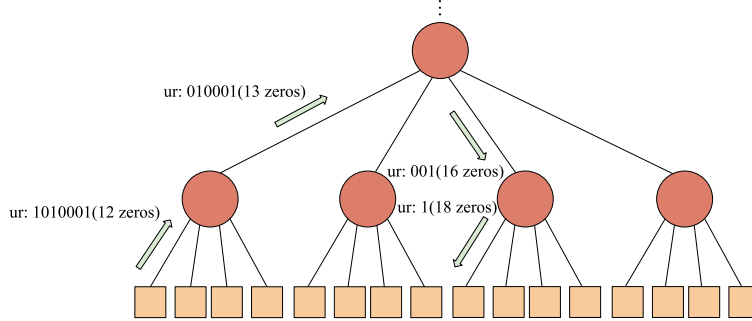


Figure 4.4: Unicast route encoding and progressive consumption of the *ur* field while traversing the quad-tree.

4.2.3 Multicast Transmission

A multicast transmission delivers a common payload to the exact destination set defined by the source, according to the semantics introduced in Section 5.

The transmission does not necessarily begin multicast replication at the source. Instead, the packet is first routed using the conventional unicast mechanism toward the router chosen as the root of the multicast tree. For the destination set considered by the transmission, this router corresponds to the lowest common ancestor from which all selected destination subtrees can be reached.

As shown in Fig. 4.5, the message therefore starts with a conventional unicast header encoded as described in Section 4.2.2. When *USCODE* is detected at the multicast root, the unicast header is consumed locally and the router switches to multicast processing.

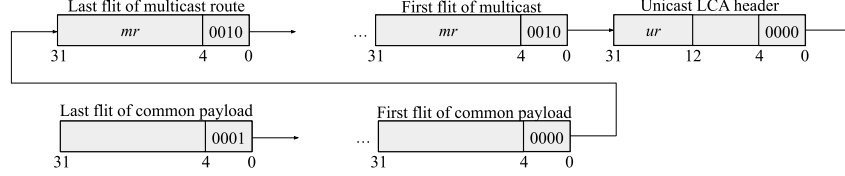


Figure 4.5: Structure of a multicast transmission. The message starts with a unicast route toward the multicast root, followed by one or more multicast-route flits and finally by the common payload.

The multicast phase consists of a *route-setup stage* followed by a *payload-dispatch stage*. During route setup, one or more multicast-route flits describe the destination subtree. Their routing field, denoted by *mr*, occupies the most-significant $W - 4$ bits of each flit.

Unlike a fixed destination mask that is forwarded unchanged throughout the network, the proposed route is progressively consumed and decomposed. Each router determines which of its children belong to the multicast tree and constructs the corresponding subroute for each active child. Consequently, different branches of the tree receive only the routing information required to describe their own descendants.

Route setup, and hence its parsing, terminates when the multicast stop code *MSCODE* is encountered. In the proposed encoding,

$$MSCODE = 0000, \quad (4.11)$$

where the code is interpreted at the quadruplet level. Once the termination code is detected, subsequent flits are treated as the common payload and are replicated toward the children selected during route setup.

The payload-dispatch stage continues until the `tail` bit is observed on the final payload flit. At that point the multicast transmission terminates and the routing resources reserved by the message are released.

This section defines only the network-level semantics and packet structure of multicast communication. The multicast route representation itself and its recursive parsing, are developed in Chapter 5.

Multicast Routing Scheme

This chapter presents the multicast routing scheme adopted in the proposed tree-based Network-on-Chip. The scheme relies on a recursive encoding of the multicast route, designed to represent the destination set compactly while allowing each router to locally extract the information required to forward the packet.

To simplify the explanation, we make the following assumptions:

1. The multicast route is represented as a single bit stream, read from left to right, although in the actual implementation it is transmitted as a sequence of 32-bit words.
2. We consider the route from the perspective of a router located high enough in the tree that the packet only needs to travel downward within the considered subtree.
3. Each router is assumed to have two children rather than four. This simplifies the illustrations without changing the underlying principle of the encoding.
4. Each router node is associated with a two-bit child-selection mask. Each bit corresponds to one of its children: a value of '1' indicates that the packet must be forwarded to that child, whereas a value of '0' indicates that the corresponding branch does not belong to the multicast subtree.

The two-bit value is therefore a mask rather than a one-hot code, since both bits may be asserted when the packet must be forwarded to both children.

5.1 Route Encoding

The multicast route encoding plays an important role in limiting the overhead introduced by the multicast setup phase. A compact representation reduces

the number of routing flits that must be transmitted and processed before the payload can be distributed, thereby reducing both communication latency and dynamic energy consumption.

5.1.1 The Broadcasting Case

We begin with the simplest case, in which the packet must reach every destination in the considered subtree. The multicast operation therefore degenerates into a broadcast.

The route can be constructed level by level, conceptually traversing each level of the tree from left to right. Each internal node contributes one two-bit child-selection mask indicating which of its children must be traversed. The first bit pair is consumed by the current router, while the subsequent pairs correspond to routers located progressively deeper in the subtree, as illustrated in Fig. 5.1.

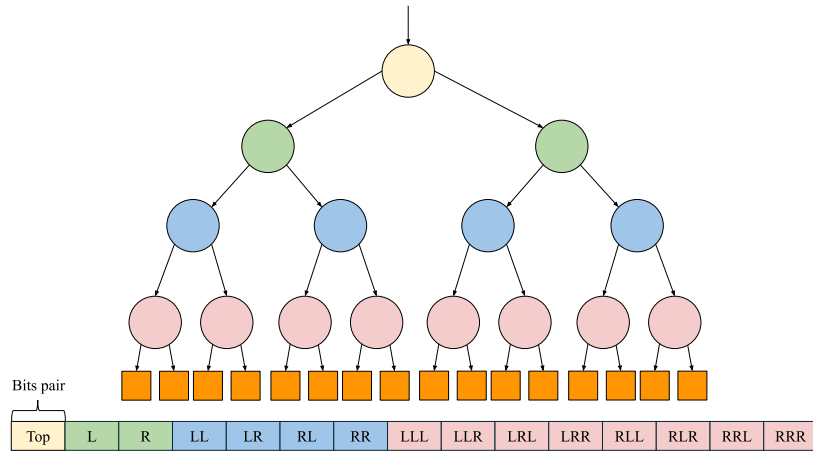


Figure 5.1: Route encoding for a broadcast over a binary subtree. The bit pairs are ordered level by level from left to right. The first pair is consumed by the current router, while the remaining pairs are associated with routers at deeper levels of the tree.

In this case, every internal node is traversed and therefore every node must contribute its own mask. Consequently, the proposed encoding does not provide any compression in the broadcasting case.

Broadcast communication itself can be implemented more simply. For instance, a router may recognize a dedicated broadcast command and replicate the packet toward all of its children. Other common solutions include flooding the packet over all eligible output links or using a predefined distribution

tree. In a tree-based NoC, broadcasting is particularly straightforward because the topology already provides a unique loop-free structure over which the packet can be replicated.

The usefulness of the proposed encoding therefore becomes more evident when only a subset of the destinations must be reached.

5.1.2 The Multicasting Case

Consider now a transmission targeting only a subset of the destinations in the subtree. In this case, some branches of the tree are never traversed, and consequently the routers belonging exclusively to those branches never process the packet.

The key idea of the proposed encoding is that the masks associated with such routers do not need to appear in the route at all. The bit stream therefore represents only the nodes that belong to the *multicast subtree*, as illustrated in Fig. 5.2.

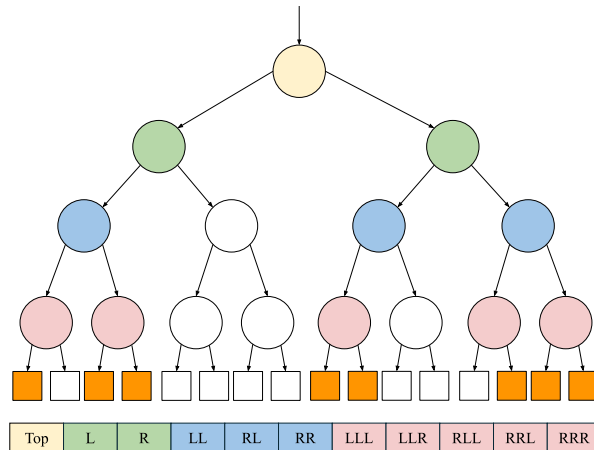


Figure 5.2: Route encoding for a multicast transmission. Masks are included only for the routers that belong to the multicast subtree. Nodes belonging exclusively to unvisited branches are omitted from the encoded route.

This provides a natural form of route compression. Instead of describing the complete physical subtree, the encoded route describes only the portion of the network required to connect the current router to the selected destinations. The number of routing bits therefore depends on the structure of the multicast subtree rather than directly on the total number of nodes in the physical tree.

5.2 Route Parsing

Having described how the multicast route is encoded, we now consider how it is parsed by each router along the multicast subtree. The parsing procedure exploits the hierarchical structure of the encoding and progressively partitions the incoming route into one sub-route for each active child.

The first child-selection mask in the incoming route is associated with the current router. Each bit of this mask determines whether the corresponding child participates in the multicast transmission. The remaining masks describe the routing decisions that must be taken by routers located deeper in the tree and must therefore be distributed among the sub-routes forwarded to the active children.

The key observation is that the number of masks belonging to each child at a specific tree level can be inferred directly from the masks already assigned to that child. In the binary example considered here, every asserted bit in a mask corresponds to one router that will have to be described at the next level. Therefore, while a mask is appended to the sub-route of child i , the number of asserted bits in that mask is accumulated in a counter. Once the current level has been completely processed, this counter indicates how many masks must be assigned to the same child at the next level.

The procedure is therefore recursive. The first mask determines how many masks must be consumed for each child at the first level; those masks, in turn, determine how many masks must be consumed at the following level, and so on. In this way, the original route can be partitioned without explicitly storing the position of each router in the tree.

Route parsing terminates when the multicast stop code, denoted throughout this work as `MSCODE`, is encountered. In the binary representation used in this section, `MSCODE = 00`. This value cannot occur as a valid child-selection mask inside the multicast subtree: a 00 mask would indicate that neither child is traversed, and the corresponding router would therefore not belong to the encoded subtree in the first place. The zero-valued mask can consequently be reserved as an unambiguous route-termination symbol.

By construction, `MSCODE` can only occur at the boundary between two tree levels. The termination condition therefore needs to be evaluated only after all masks belonging to the current level have been processed.

Figure 5.3 illustrates the parsing procedure on a representative multicast route. The input stream is consumed level by level, while the masks are progressively assigned to the two child sub-routes. The counters associated with each child determine how many masks must be extracted at every iteration.

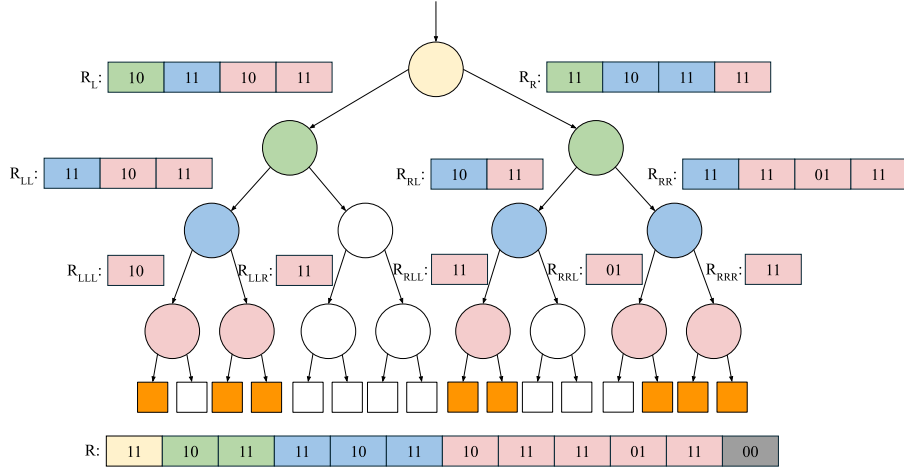


Figure 5.3: Example of multicast route parsing. The first child-selection mask is consumed by the current router. The remaining masks are progressively partitioned into the sub-routes R_0 and R_1 , while the number of asserted bits determines how many masks must be processed for each child at the following tree level.

Algorithm 1 formalizes this procedure. The following quantities are used:

- R : encoded multicast route received by the current router;
- R_i : sub-route generated for child i . Here $i = 0$ means left, while $i = 1$ means right;
- b : child-selection mask currently being processed;
- $q_{i,n}$: number of masks at the current level that still need to be appended to R_i ;
- $q_{i,n+1}$: number of masks that will need to be appended to R_i at the following level.

The operation $\text{popcount}(b)$ returns the number of asserted bits in b . Since each asserted bit identifies an active child of the router represented by that mask, its value corresponds to the number of routing masks that will have to be processed at the following level. Here, n is used only as a notational label for the current tree level and does not represent an explicit algorithmic variable.

Algorithm 1 Multicast route parsing

Input: Encoded multicast route R
Output: Child sub-routes R_0, R_1

```

1: Read  $b$  from  $R$ 
2:  $q_{0,n} \leftarrow b[0], q_{1,n} \leftarrow b[1]$ 
3:  $q_{0,n+1} \leftarrow 0, q_{1,n+1} \leftarrow 0$ 
4: while  $b \neq \text{MSCODE}$  do
5:   for  $i \in \{0, 1\}$  do
6:     while  $q_{i,n} \neq 0$  do
7:       Read  $b$  from  $R$ 
8:       Append  $b$  to  $R_i$ 
9:        $q_{i,n} \leftarrow q_{i,n} - 1$ 
10:       $q_{i,n+1} \leftarrow q_{i,n+1} + \text{popcount}(b)$ 
11:    end while
12:  end for
13:   $q_{0,n} \leftarrow q_{0,n+1}, q_{1,n} \leftarrow q_{1,n+1}$ 
14:   $q_{0,n+1} \leftarrow 0, q_{1,n+1} \leftarrow 0$ 
15: end while
16: Forward  $R_0$  and  $R_1$  to the corresponding children

```

5.3 Comparison with Existing Routing Schemes

As outlined in Chapter 2, exact and approximate multicast destination-set representations involve different trade-offs in routing-information overhead and delivery accuracy. Building on that discussion, this section examines the problem in greater detail by formalizing the main differences among representative schemes from both categories and directly comparing their scaling properties and destination-set capabilities with those of the proposed encoding.

5.3.1 Exact Packet-Carried Encodings

The most direct exact representation is the *Flat Bit String* (FBS), in which one bit is associated with every possible destination [7, 6]. FBS can represent any subset of N endpoints and requires no multicast-group setup, but its routing information scales as $O(N)$. Practical implementations may mask or partition the bit string according to the destinations reachable through each output, as in [6], but the representation remains defined over the complete endpoint space.

An alternative is the *Explicit Destination List* (EDL), in which each endpoint is assigned a unique identifier and the identifiers of the K selected destinations are carried directly in the packet [5]. Since representing one out of N

5.3. Comparison with Existing Routing Schemes

possible destinations requires $\lceil \log_2 N \rceil$ bits, the routing-information cost is approximately $K \lceil \log_2 N \rceil$ bits. This makes EDL attractive for low fan-out, but increasingly expensive as the multicast set grows.

A comparison between these schemes and the proposed one is shown in Fig. 5.4. The hierarchical representation is particularly advantageous for sparse destination sets, where it requires substantially fewer routing bits than a full-network bitmap. As the destination density increases, its cost grows with the size of the active multicast subtree and eventually approaches and exceeds the fixed cost of FBS when the destination density exceeds 25% of the total, while remaining significantly more compact than EDL for moderate and large multicast sets.

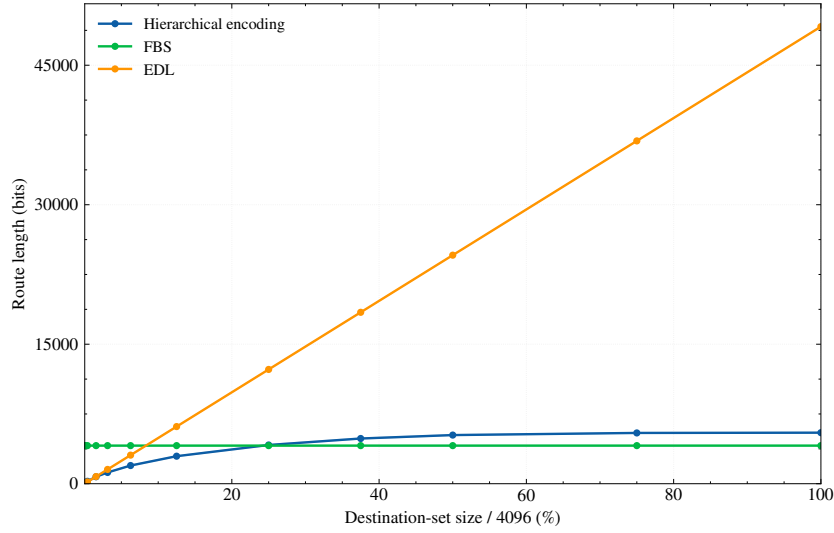


Figure 5.4: Routing-information length of the proposed hierarchical encoding compared with Flat Bit String (FBS) and Explicit Destination List (EDL) as a function of the fraction of selected destinations in a 4096-endpoint network.

More topology-aware schemes attempt to reduce this overhead by exploiting the spatial structure of the destination set. *Recursive Partitioning Multicast* (RPM), for example, recursively partitions the destination information at multicast branching points, so that each replica retains only the destinations relevant to its portion of the network [13]. This principle is closely related to the proposed scheme: in both cases, routing information becomes progressively specialized as the multicast tree is traversed. The main difference is that the proposed representation directly encodes the multicast subtree through local child-selection masks, rather than starting from an explicit global destination representation.

5.3.2 Approximate Destination Encodings

This second family reduces routing overhead by relaxing the requirement that every destination subset be represented exactly. The *Hierarchical Bit String* (HBS) proposed by Su *et al.* hierarchically represents groups of destinations and achieves routing-information scaling of $O(\log N)$, while increasing the addressing capability compared with previous symbol-based schemes [7]. The resulting representation, however, cannot encode every possible destination subset exactly, and additional endpoints included by the selected regions must filter illegal packets locally.

Such schemes therefore occupy a different point in the design space: routing information is reduced by accepting over-delivery and moving part of the destination-resolution functionality to the endpoints. This trade-off can be advantageous when local filtering is inexpensive, but it does not provide the same routing semantics as an arbitrary exact destination-set representation.

The quantitative trade-off between route compression, destination density, and spatial locality is evaluated in Chapter 8.

Table 5.1: Comparison of representative multicast-routing schemes. HBS: Hierarchical Bit String; FBS: Flat Bit String; EDL: Explicit Destination List; RPM: Recursive Partitioning Multicast. N is the number of endpoints, K the multicast fan-out, and $I(D)$ the number of internal routers in the multicast subtree induced by destination set D .

Scheme	Exact	Route size	Main trade-off
HBS [7]	No	$O(\log N)$	Filtering / over-delivery
FBS [7, 6]	Yes	$O(N)$	Large header
EDL [5]	Yes	$O(K \log N)$	Fan-out dependent
RPM [13]	Yes	Data dependent	Partitioning logic
Proposed	Yes	$O(I(D))$	Route parsing

Router Microarchitecture

This chapter presents the microarchitecture of a single router node in the proposed NoC. We begin from a top-level view of the router, describing its external interfaces and overall functionality. The design is then progressively decomposed into smaller functional blocks; for each block, its functionality and corresponding CHP specification are presented.

6.1 Node Description

Each node of the network can be modeled as a routing element with five input ports and five output ports: one pair of ports connects the router to its parent, while the remaining four connect it to its children. An incoming transmission is received through one of the input ports and forwarded toward one or more output ports, depending on whether the transmission is unicast or multicast. Each router-to-router connection is implemented as a full-duplex link composed of two independent unidirectional channels, one for each direction of communication. Each channel has a data width of 32 bits. Since multiple transmissions may arrive concurrently through different input ports, contention can arise whenever two or more messages request the same output port. Resolving such conflicts while preserving each transmission until completion is therefore one of the main responsibilities of the router.

A top-level representation of the simplified router node is shown in Fig. 6.1.

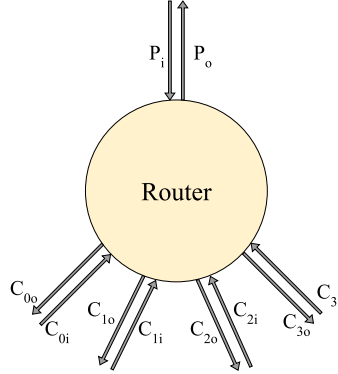


Figure 6.1: Top-level view of a router node.

6.2 Node Decomposition

The previous chapters introduced the network topology, supported transmission types, and routing scheme. Starting from this system-level description, the router can now be decomposed into smaller blocks with well-defined responsibilities, simplifying both the understanding of the architecture and its subsequent asynchronous implementation. Detailed implementations of all blocks introduced in this section are provided in Appendices A, B and C.

The decomposition is based on the following observations:

1. The first flit of each incoming transmission determines how the message must be routed. For a unicast transmission, the routing decision is derived from the *ur* field of the header flit. When a multicast route is received from the parent, the *mltc* control bit determines whether the transmission must enter the multicast datapath.
2. Messages arriving from the parent and from the children must be interpreted differently, according to the routing scheme introduced previously. The router therefore contains two distinct routing structures, referred to as the *ParentRouter* and the *ChildRouter*, which process transmissions originating from the parent and child ports, respectively.
3. Multicast transmissions require a dedicated datapath responsible for both parsing the encoded multicast route and distributing the common payload to the selected children. This block, referred to as the *Multicas-*

$tUnit$, can be requested by multiple input ports and is therefore treated as a contended resource in the same way as the router output ports.

These considerations lead to the first-level decomposition shown in Fig. 6.2. For clarity, the figure illustrates a simplified router with only two child ports, although the implemented architecture uses four. The first flit of every incoming transmission is processed by the routing block associated with its input port and directs the message either toward one of the router outputs or toward the MulticastUnit. Whenever multiple transmissions request the same destination resource, the resulting contention is resolved by an arbiter.

Arbitration is performed at message granularity: once a requester is granted access to an output port or to the MulticastUnit, ownership is maintained for the entire transmission and released only when the tail flit is observed. This prevents flits belonging to different messages from being interleaved on the same output channel.

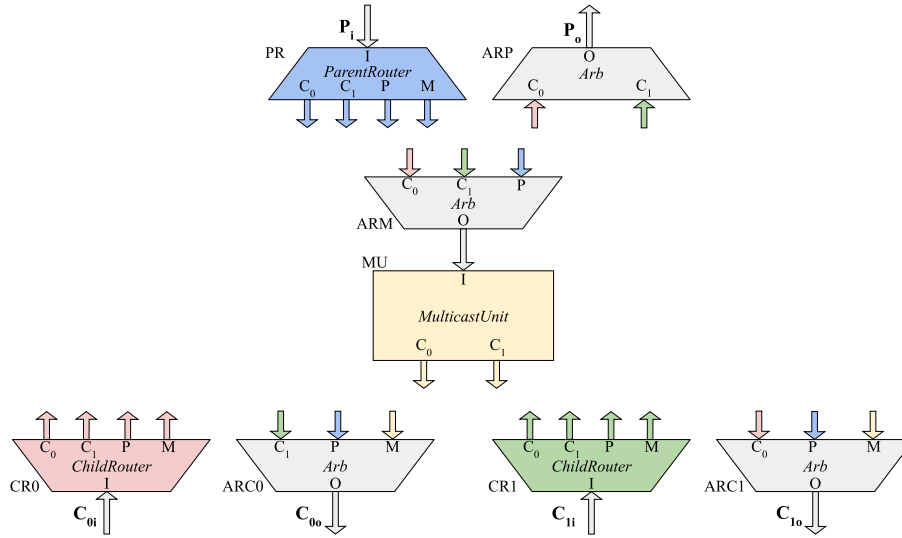


Figure 6.2: First-level decomposition of the router node. Internal channels are color-coded according to the block from which they originate. Note that, although both CR0 and CR1 include output channels toward C_0 and C_1 , respectively, these channels are not used, since a message cannot be routed back toward the branch from which it arrived. They are nevertheless retained in *ChildRouter* to preserve a uniform and modular architecture.

6.2.1 ChildRouter

A transmission entering the router from one of its child ports can either continue upward through the parent port or begin descending toward another child subtree. The latter case can arise in two different situations: the transmission may remain purely unicast, or the current router may represent the end of the LCA-directed unicast phase preceding a multicast transmission

as reported in Section 4.2.3. In the second case, the following flits contain the multicast route and, subsequently, the common payload.

The behavior of the ChildRouter is described by the CHP specification reported in Eq. (6.1). The process initially waits for the first flit of a transmission on input channel I . This flit contains the routing information required to determine the next destination of the message.

$$\begin{aligned}
 \text{ChildRouter}(I?, C_0!, C_1!, C_2!, C_3!, P!, M!) \equiv & \\
 * [I?b; & \\
 [b.ur \neq \text{USCODE} \rightarrow & \\
 [\neg b.msb \rightarrow // \text{Down} & \\
 c_h := b.(msb - 1 : msb - 2); b.ur \ll 3; & \\
 [c_h = 0 \rightarrow * [\neg b.tail \rightarrow C_0!b; I?b]; C_0!b & \\
 [c_h = 1 \rightarrow * [\neg b.tail \rightarrow C_1!b; I?b]; C_1!b & \\
 [c_h = 2 \rightarrow * [\neg b.tail \rightarrow C_2!b; I?b]; C_2!b & \\
 [c_h = 3 \rightarrow * [\neg b.tail \rightarrow C_3!b; I?b]; C_3!b & \\
] & \\
 [b.msb \rightarrow b.ur \ll 1; * [\neg b.tail \rightarrow P!b; I?b]; P!b // \text{Up} & \\
] & \\
 [b.ur = \text{USCODE} \rightarrow * [\neg b.tail \rightarrow I?b; M!b] // \text{To MulticastUnit} & \\
] & \\
] &
 \end{aligned} \tag{6.1}$$

CHP description of the ChildRouter.

The routing decision is implemented through three nested selections, after which the selected branch enters a local forwarding loop. Once an output has been selected, the remaining flits are forwarded to the same destination until the `tail` bit is observed, marking the end of the transmission.

- **$b.ur = \text{USCODE}$: transition to multicast.** The outermost selection determines whether the unicast portion of the route has terminated by comparing the unicast route field $b.ur$ with the unicast stop code, USCODE . If the two match, the current router corresponds to the end of the LCA-directed unicast phase. The current header flit is therefore consumed locally, while all subsequent flits are forwarded to the MulticastUnit through channel M . These flits contain first the encoded multicast route and then the payload shared among the selected destinations.
- **$b.ur \neq \text{USCODE}$ and $b.msb = 1$: upward unicast routing.** If USCODE is not detected, the transmission remains in unicast mode. When the most significant bit of the route field is equal to one, the message must continue upward and is therefore forwarded through the parent port P . The routing bit that has just been consumed is removed by left-shifting

the unicast route field by one position, exposing the routing information required by the next router.

- $b.ur \neq USCODE$ and $b.msb = 0$: **downward unicast routing**. When the most significant bit is equal to zero, the message must instead travel downward. The following two bits identify the destination child, selecting one of the four child ports C_0 – C_3 . Since this routing decision consumes the direction bit together with the two child-selection bits, the unicast route field is shifted by three positions before the header is forwarded. Once the destination child has been selected, the complete transmission is forwarded through that port until its tail flit is reached.

ChildRouter Decomposition

The decomposition of the *ChildRouter* follows directly from the nested selections of its CHP description. Rather than implementing the complete routing decision within a single monolithic block, each selection is mapped onto a dedicated selector responsible for one specific routing decision. The resulting architecture forms a ladder-like cascade of three stages, as shown in Fig. 6.3. The complete implementation of every block introduced in this section is provided in Appendix B.

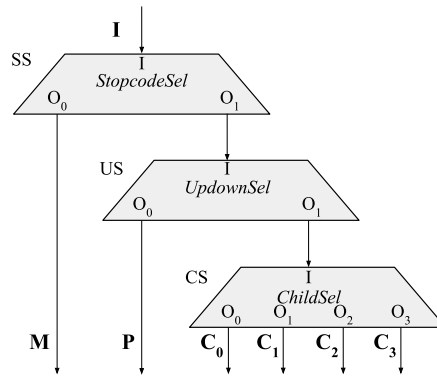


Figure 6.3: Decomposition of the *ChildRouter* into the cascaded *StopcodeSel*, *UpdownSel*, and *ChildSel* routing stages.

The first stage is the *StopcodeSel*, which examines the unicast route field and determines whether *USCODE* has been reached. If the stop code is detected, the transmission is directed through output O_0 toward the MulticastUnit. Otherwise, it is forwarded through O_1 to the next stage, the *UpdownSel*.

The *UpdownSel* determines whether the unicast transmission must continue upward or begin descending the tree. It reads the most significant bit of the *ur* field and consumes it by left-shifting the route by one position. If the bit is asserted, the transmission is sent through O_0 , which is connected to the parent output. Otherwise, it is forwarded through O_1 to the final selection stage.

The last stage, the *ChildSel*, handles downward routing. It reads the two most significant bits of the remaining *ur* field, which identify one of the four child ports, and consumes them by shifting the route field by two additional positions. The transmission is then forwarded toward the selected child.

6.2.2 ParentRouter

The structure of the *ParentRouter* is similar to that of the *ChildRouter*, although its routing behavior is more constrained. A transmission entering from the parent port can either be forwarded downward toward one of the child ports or redirected to the *MulticastUnit*. According to the routing scheme introduced in Section 4.2.2, a packet that is already descending the tree can never be routed upward again.

The behavior of the *ParentRouter* is described by the CHP description reported in Eq. (6.2). As in the *ChildRouter*, the routing logic is organized as a sequence of nested selections. The outermost selection first distinguishes between transmissions that are already in multicast mode and those that are still being interpreted as unicast traffic. If the *mltc* bit is not asserted, a second selection checks whether the unicast stop code has been reached. Only when neither multicast condition holds is the message treated as a downward unicast transmission and routed toward one of the children.

$$\begin{aligned}
& \text{ParentRouter}(I?, C_0!, C_1!, C_2!, C_3!, M!) \equiv \\
& * [I?b; \\
& \quad [\neg b.mltc \rightarrow \\
& \quad \quad [b.ur \neq \text{USCODE} \rightarrow // \text{Down} \\
& \quad \quad \quad c_h := b.(msb : msb - 1); b.ur \ll 2; \\
& \quad \quad \quad [c_h = 0 \rightarrow * [\neg b.tail \rightarrow C_0!b; I?b]; C_0!b \\
& \quad \quad \quad \parallel c_h = 1 \rightarrow * [\neg b.tail \rightarrow C_1!b; I?b]; C_1!b \\
& \quad \quad \quad \parallel c_h = 2 \rightarrow * [\neg b.tail \rightarrow C_2!b; I?b]; C_2!b \\
& \quad \quad \quad \parallel c_h = 3 \rightarrow * [\neg b.tail \rightarrow C_3!b; I?b]; C_3!b \\
& \quad \quad] \\
& \quad \quad \parallel b.ur = \text{USCODE} \rightarrow * [\neg b.tail \rightarrow I?b; M!b] // \text{To MulticastUnit} \\
& \quad] \\
& \quad \parallel b.mltc \rightarrow * [\neg b.tail \rightarrow M!b; I?b]; M!b // \text{To MulticastUnit} \\
&] \\
&]
\end{aligned} \tag{6.2}$$

CHP description of the ParentRouter.

The three possible cases are:

- **$b.mltc = 1$: ongoing multicast transmission.** If the $mltc$ control bit is asserted, the incoming message is already in multicast mode. The current flit, together with all subsequent flits belonging to the same transmission, is therefore forwarded directly to the MulticastUnit through channel M .
- **$b.mltc = 0$ and $b.ur = \text{USCODE}$: transition to multicast.** If the message is not yet in multicast mode, the unicast route field is checked against the unicast stop code USCODE . As in the ChildRouter, a match indicates the end of the LCA-directed unicast phase. The current header flit is consumed locally, while all subsequent flits are forwarded to the MulticastUnit.
- **$b.mltc = 0$ and $b.ur \neq \text{USCODE}$: downward unicast routing.** If neither multicast condition holds, the transmission is interpreted as a unicast message that must continue downward. The two most significant bits of the unicast route field identify one of the four child ports C_0 – C_3 . These bits are consumed by the current router by shifting the route field by two positions, and the complete transmission is then forwarded through the selected child port until its tail flit is observed.

ParentRouter Decomposition

The decomposition of the *ParentRouter* follows the same general approach adopted for the *ChildRouter*, namely a cascade of simpler selector blocks implementing the nested routing decisions of the CHP description. The

complete implementation of every block introduced in this section is again provided in Appendix B. The resulting architecture is shown in Fig. 6.4.

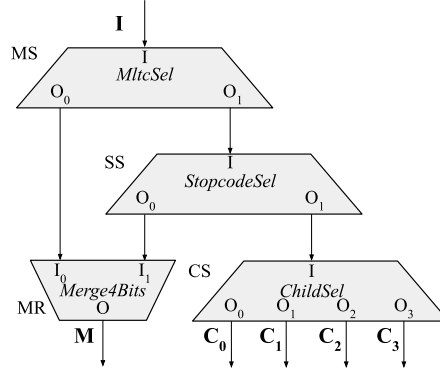


Figure 6.4: Decomposition of the ParentRouter. The *StopcodeSel* and *ChildSel* blocks are shared with the ChildRouter, while *MltcSel* identifies transmissions already operating in multicast mode. The two paths directed toward the MulticastUnit are combined through *Merge32Bits*.

The first stage is the *MltcSel*, which checks the value of the `mltc` control bit. If the bit is asserted, the incoming transmission is already in multicast mode and is therefore directed through output O_0 toward the MulticastUnit. Otherwise, it is forwarded through O_1 to the following stages.

The remaining routing decisions reuse the *StopcodeSel* and *ChildSel* blocks already introduced in the ChildRouter decomposition. The former detects the end of the LCA-directed unicast phase, while the latter selects the destination child for downward unicast traffic.

Since both *MltcSel* and *StopcodeSel* can generate traffic directed toward the MulticastUnit, their corresponding outputs must be combined into a single channel. This is achieved using a two-input merging block, denoted as *Merge32Bits*.

6.2.3 MulticastUnit

The undecomposed *MulticastUnit* implements directly the route-parsing algorithm introduced in the previous chapter, while accounting for the finite width of the physical communication channels. As in the previous architectural diagrams, the description is presented for a simplified router with two children rather than four.

Unlike the abstract parsing algorithm, which operates on a continuous route bit stream, the hardware receives and transmits one W -bit word at a time. The MulticastUnit must therefore buffer the incoming route, progressively construct the sub-routes associated with the active children, and transmit each output buffer whenever it becomes full. Once route parsing is complete, the same unit is also responsible for replicating the common payload toward all active children.

The main state variables used by the process are:

- b_i : input buffer containing the current W -bit word received from channel I ;
- $b_{o,j}$: output buffer used to construct the partial multicast route for child j ;
- p_i : input pointer, which tracks how many bits of b_i have already been consumed;
- $p_{o,j}$: output pointer, which identifies the next available position in $b_{o,j}$;
- $q_{n,j}$: number of routing masks at the current tree level that still have to be appended to the sub-route of child j ;
- $q_{n+1,j}$: number of routing masks that will have to be appended to the same sub-route at the next tree level;
- c_h : child-selection mask extracted from the first routing mask, which identifies the children participating in the multicast transmission.

The behavior of the MulticastUnit is described by the CHP description reported in Eq. (6.3).

$$\begin{aligned}
 & \text{MulticastUnit}(I?, C_0!, C_1!) \equiv \\
 & q_{n+1,0} := 0, q_{n+1,1} := 0, p_{o,0} := W - 1, p_{o,1} := W - 1; \\
 & * [I?b_i; \\
 & \quad c_h := b_i.(msb : msb - 1); b_i.mr \leq 2; \\
 & \quad q_{n,0} := c_h.msb, q_{n,1} := c_h.(msb - 1), p_i := 2, b_{o,0} := 2, b_{o,1} := 2; \\
 & \quad * [b_i.(msb : msb - 1) \neq \text{MSCODE} \rightarrow // \text{Loop 0} \\
 & \quad \quad \langle ; j : 0..1 : \\
 & \quad \quad * [q_{n,j} \neq 0 \rightarrow // \text{Loop 1} \\
 & \quad \quad \quad b_{o,j}.(p_{o,j} : p_{o,j} - 1) := b_i.(msb : msb - 1); \\
 & \quad \quad \quad q_{n,j} := q_{n,j} - 1, p_{o,j} := p_{o,j} - 2, p_i := p_i + 2; \\
 & \quad \quad \quad q_{n+1,j} := q_{n+1,j} + b_i.msb + b_i.(msb - 1); b_i.mr \leq 2; \\
 & \quad \quad \quad [p_{o,j} = 4 \rightarrow p_{o,j} := W - 1; // \text{Sending out } b_{o,j} \\
 & \quad \quad \quad \quad [\neg j \rightarrow C_0!b_{o,0}; b_{o,0} := 2 \\
 & \quad \quad \quad \quad \quad [j \rightarrow C_1!b_{o,1}; b_{o,1} := 2] \\
 & \quad \quad \quad \quad \parallel p_{o,j} \neq 4 \rightarrow \text{skip} \\
 & \quad \quad \quad]; \\
 & \quad \quad [p_i = W - 4 \rightarrow p_i := 0, I?b_i // \text{Refill of } b_i \\
 & \quad \quad \parallel p_i \neq W - 4 \rightarrow \text{skip}] \\
 & \quad \quad]; \\
 & \quad \quad q_{n,j} := q_{n+1,j}; q_{n+1,j} := 0 \\
 & \quad \rangle \\
 &]; \\
 & [p_{o,0} \neq W - 1 \rightarrow C_0!b_{o,0}, p_{o,0} := W - 1 \parallel p_{o,0} = W - 1 \rightarrow \text{skip}], // \text{Flush of } b_{o,0} \\
 & [p_{o,1} \neq W - 1 \rightarrow C_1!b_{o,1}, p_{o,1} := W - 1 \parallel p_{o,1} = W - 1 \rightarrow \text{skip}]; // \text{Flush of } b_{o,1} \\
 & * [\neg b_i.tail \rightarrow I?b_i; // \text{Dispatching the common payload} \\
 & \quad [c_h.msb \rightarrow C_0!b_i \parallel \neg c_h.msb \rightarrow \text{skip}], \\
 & \quad [c_h.(msb - 1) \rightarrow C_1!b_i \parallel \neg c_h.(msb - 1) \rightarrow \text{skip}] \\
 &] \\
 &]
 \end{aligned} \tag{6.3}$$

CHP description of the MulticastUnit.

Its operation can be divided into four main phases.

- **Initialization.** Before even beginning the loop, the next-level counters $q_{n+1,j}$ are initialized to zero, while the output pointers $p_{o,j}$ are initialized to $W - 1$, corresponding to the first available position in each output buffer. The first input word is then received and stored in b_i .

The first routing mask is stored in c_h . Its two bits determine which children participate in the multicast transmission and are consequently used to initialize the current-level counters $q_{n,0}$ and $q_{n,1}$. The multicast route field of b_i is then shifted by two positions because we consumed

the first mask, so that the next one reaches the most significant position. Since two route bits have already been consumed, the input pointer is initialized to $p_i = 2$.

At the same time, the two output buffers are initialized to the value 2, which corresponds to an empty route flit, having only the required control-field configuration with the `mltc` bit asserted.

- **Multicast route parsing.** Route parsing is performed by the outer repetition denoted as *Loop 0*, which continues until the multicast stop code *MSCODE* is observed. Within each iteration, the two children are processed sequentially through the replicated statement indexed by j .

For each child, *Loop 1* executes exactly $q_{n,j}$ times. At every iteration, the routing mask currently located at the most significant position of b_i is appended to the next available location in $b_{o,j}$. The current-level counter $q_{n,j}$ is then decremented, while the input and output pointers are advanced according to their respective conventions: p_i increases as bits are consumed from the input word, whereas $p_{o,j}$ decreases as the output buffer is filled.

The same routing mask is also used to update $q_{n+1,j}$. In particular, the number of asserted bits in the mask is added to the next-level counter, since every asserted bit corresponds to one router whose routing mask must be processed at the following level. The input route field is then shifted by two positions to expose the next mask.

Two boundary conditions must be checked during this process. If $p_{o,j} = 4$, the usable route portion of the corresponding output buffer has been filled. The buffer is therefore transmitted to the appropriate child, its pointer is reset to $W - 1$, and the buffer itself is reinitialized. Conversely, if $p_i = W - 4$, all usable routing bits in the current input word have been consumed and only the control field remains. A new word is therefore received from I , and p_i is reset.

Once $q_{n,j}$ reaches zero, all routing masks belonging to child j at the current tree level have been processed. The value accumulated in $q_{n+1,j}$ becomes the current counter for the following level, after which the next-level counter is cleared. When both children have been processed, *Loop 0* starts a new iteration and the procedure repeats for the next level.

- **Final route flush.** When *MSCODE* is observed, the complete multicast route has been parsed. The output buffers may nevertheless contain partially filled route words that have not yet been transmitted. Each buffer is therefore flushed if its pointer differs from $W - 1$. A pointer still equal to $W - 1$ indicates that the corresponding buffer is empty,

either because that child was never active or because its last full word had already been transmitted.

- **Payload dispatch.** After the multicast route has been completely distributed, the remaining incoming words correspond to the common payload. These words are replicated toward all children selected by the original mask c_h . Payload dispatch continues until the tail bit is observed, marking the end of the multicast transmission.

MulticastUnit Decomposition

The undecomposed MulticastUnit introduced in the previous section provides a compact behavioral description of the complete multicast operation, but it is not particularly suitable for a direct hardware implementation. In particular, the use of multiple pointers and arithmetic operations would result in unnecessary control complexity, area overhead, and additional latency. The objective of the decomposition is therefore to reorganize the same functionality into simpler blocks with more regular control structures. The complete implementation of every block introduced in this section is provided in Appendix C.

The key observation, as illustrated in Fig. 6.5, is that a multicast transmission consists of two distinct phases: first, the multicast route must be parsed and redistributed toward the active children; afterward, the common payload must be replicated toward those same children. The decomposed architecture therefore separates these two operations into independent datapaths. As in the previous architectural descriptions, the decomposition is illustrated for a simplified two-child configuration, while the complete implementation contains four children.

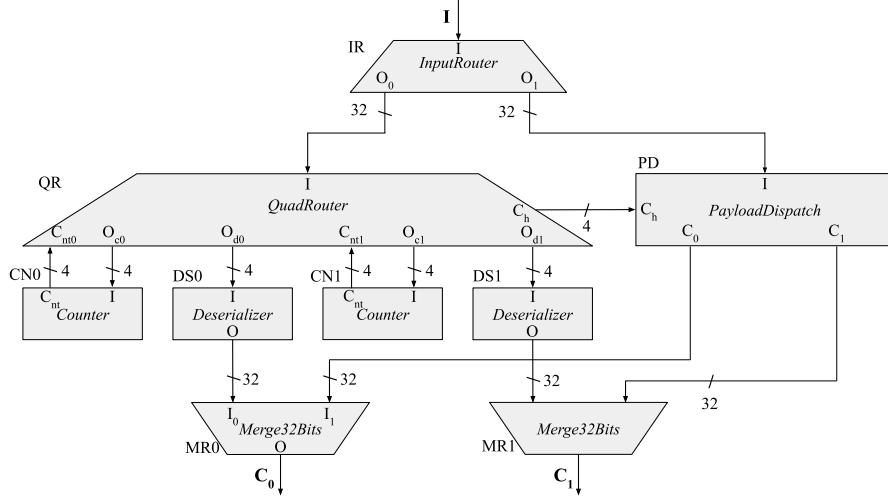


Figure 6.5: Decomposition of the MulticastUnit into separate route-parsing and payload-dispatch datapaths. The *InputRouter* selects the active datapath, while the route-parsing path serializes the incoming route, distributes its nibbles through the *QuadRouter*, accumulates the next-level counters, and reconstructs the child sub-routes through the Deserializers. For clarity, the diagram shows the simplified two-child configuration rather than the four-child structure used in the complete implementation.

- **Input routing.** The incoming 32-bit words are first processed by the *InputRouter*, whose purpose is to determine which of the two datapaths must receive the current word. At the beginning of every multicast transmission, the block forwards the input toward output O_0 , corresponding to the route-parsing datapath.

While operating in this state, the *InputRouter* checks bits $[7 : 4]$ of each incoming word, i.e. the least-significant route nibble immediately preceding the control field, for the presence of *MSCODE*. By construction of the multicast encoding, if the terminating route word contains *MSCODE*, one occurrence is guaranteed to appear in this position. Once the stop code is detected, the block switches to output O_1 , and all subsequent words are directed toward the payload datapath.

During the payload phase, the *InputRouter* monitors the *tail* bit of every incoming word. When the tail flit is observed, the multicast transmission is complete and the block returns to its initial state, ready to route the next transmission toward the route-parsing datapath.

- **Route-parsing datapath.** The first block in the route-parsing path is the *Serializer*. It receives one 32-bit word, stores it internally, and serializes its route field into 4-bit nibbles. These nibbles are emitted sequentially, from the most significant to the least significant, until the stored word

has been completely consumed. A new input word is then requested automatically.

The serialized nibbles are subsequently processed by the *QuadRouter*, which implements the core of the hierarchical parsing algorithm. For each child, the block forwards a number of nibbles determined by the corresponding current-level counter q_n . Each forwarded nibble is sent in parallel both to the corresponding *Counter*, which counts its asserted bits and accumulates the resulting value for the following tree level, and to the corresponding *Deserializer*, which sequentially appends the nibble to reconstruct the outgoing multicast route into 32-bit words. Once the current value of q_n reaches zero, the *QuadRouter* reads the accumulated next-level value from the corresponding *Counter*, which is then reset for the following accumulation cycle, and proceeds to the next child.

After all children belonging to the current tree level have been processed, the *QuadRouter* checks the current nibble for *MSCODE*. If the stop code is detected, the route-parsing phase is complete: the *QuadRouter* returns to its initialization state and simultaneously forwards *MSCODE* to all *Deserializers*, forcing them to flush any partially filled output buffers.

The *Deserializer* fills its internal word from the most significant route position toward the least significant one. Once the available route field has been filled, the completed word is automatically transmitted toward the corresponding child and the block returns to its initial state. If *MSCODE* is received before the buffer becomes full, the current partial word is flushed instead. When the internal buffer is empty, this flush operation is ignored. This preserves the behavior of the undecomposed algorithm while preventing the generation of invalid output flits containing only *MSCODE* padding.

This organization eliminates the explicit input and output pointers required by the undecomposed description. Their functionality is instead absorbed into the internal sequencing mechanisms of the *Serializer* and *Deserializers*, which cyclically traverse their respective storage elements and automatically return to their initial state after completing a word.

- **Payload-dispatch datapath.** Once *MSCODE* is detected, the *InputRouter* redirects the incoming stream toward the *PayloadDispatch* block. This block is responsible for replicating the common payload toward all children participating in the multicast transmission.

The set of active children is determined by the first routing nibble of the multicast route. At the beginning of the transmission, the *QuadRouter* forwards this nibble to the *PayloadDispatch* block, where it is stored

while the remainder of the route is being parsed. When the payload eventually arrives, the stored child-selection information is used to determine which output channels must receive each payload word.

Implementation Flow

This chapter describes the design flow used to translate the high-level router specification into a transistor-level implementation. The methodology follows the synthesis approach originally introduced by Martin for quasi-delay-insensitive asynchronous circuits and subsequently developed in the asynchronous VLSI literature [8, 14]. The same progression from Communicating Hardware Processes (CHP), through Handshaking Expansions (HSE), to Production Rule Sets (PRS) is adopted by recent asynchronous designs based on the ACT toolchain [15]. While Chapter 3 introduced the main 3 abstraction levels of an asynchronous design, this chapter describes into details how they are realized in practice to progressively refine the router from its behavioral specification to a circuit implementation.

The objective of this design flow is to make circuit correctness largely independent of physical propagation delays. A circuit is delay-insensitive if its correct operation is independent of arbitrary, finite, and positive gate and interconnect delays. Pure delay insensitivity, however, is too restrictive for general-purpose computation. The circuits developed in this work therefore belong to the broader class of *quasi-delay-insensitive* (QDI) circuits, which retain the delay-insensitive abstraction while allowing a limited set of explicitly identified relative-timing assumptions known as *isochronic forks* [8, 14, 16].

7.1 Communicating Hardware Processes

Communicating Hardware Processes (CHP) provide the highest-level representation used in the design flow. A system is described as a collection of concurrently executing processes that exchange information through communication channels using handshake-based communication. At this level, the designer specifies the computation and the required ordering and synchronization between operations without explicitly defining their lower-level implementation [8, 14].

A communication channel connects one sender to one receiver and provides both data transfer and synchronization. Unless explicit storage is introduced, the channels used in this methodology can be regarded as zero-slack channels, meaning that the corresponding send and receive form a single synchronization event. This abstraction allows functionality and concurrency to be reasoned about independently of the physical handshake protocol.

7.1.1 Top-Level Specification

The router design started from a single top-level CHP specification describing its complete externally observable behavior. This monolithic model provided an unambiguous functional reference against which all subsequent refinements could be verified.

Keeping the initial description at this level separates architectural decisions from implementation details. Packet formats and routing semantics can therefore be validated before introducing circuit-level constraints. The resulting CHP acts as the golden functional specification for the decomposed architecture described in Chapter 6.

7.1.2 Process Decomposition

A monolithic CHP description is generally unsuitable for direct circuit implementation. The next step is therefore *process decomposition*, in which the original process is partitioned into smaller concurrent processes connected by additional communication channels. This exposes concurrency, localizes state, reduces the complexity of individual circuits, and makes the resulting implementation easier to analyze and optimize, while preserving the externally observable behavior of the original specification [3].

In the proposed router, the first decomposition follows naturally from the different datapath functions. Several complex blocks are themselves recursively decomposed; for example, large arbitration and merge structures are implemented as trees of simpler two-input elements. This hierarchical decomposition substantially reduces the complexity of the subsequent HSE and PRS synthesis.

7.1.3 Bit-Sliced Datapath Decomposition: Omitted Step

A further decomposition commonly applied to asynchronous datapaths is *bit slicing*. Instead of treating an entire W -bit word as a single communication unit, the datapath is divided into smaller slices with local communication and completion-detection circuitry. This reduces the fan-in and depth of completion trees and can enable finer-grained pipelining of wide datapaths [17, 14].

For an unsliced dual-rail input, the receiver must determine that every bit has become valid before acknowledging the communication, requiring the validity of many rails to be combined through a completion tree. Bit slicing reduces the size of these completion detectors and permits less globally synchronized progress, at the cost of additional control circuitry, acknowledge signals, state, and routing overhead.

Bit slicing was not applied to the 32-bit datapaths developed in this work because its implementation and verification would have significantly increased the project effort. Consequently, wide-word completion remains on several critical paths and contributes to the elevated initiation time reported in Chapter 8. Bit slicing therefore represents an important optimization for a future implementation.

7.1.4 Functional Verification

Functional verification was performed using *actsim*, the event-driven simulator included in the ACT asynchronous design toolchain [9]. The simulator supports CHP, HSE, and PRS descriptions within the same hierarchical design and can fall back to a higher-level model when a lower-level implementation is unavailable. This enables incremental mixed-level verification, with individual processes progressively replaced by their refined implementations.

The testbenches exercised unicast and multicast communication, different source and destination combinations, packet lengths, simultaneous requests, and arbitration scenarios. At this stage, the objective was purely functional: the decomposed router had to reproduce the behavior prescribed by the original CHP specification. Timing robustness and physical delay assumptions were addressed after lowering the design to PRS.

7.2 Handshaking Expansion

Once the CHP decomposition of a block was considered satisfactory, each process was translated into a *Handshaking Expansion* (HSE). HSE is an intermediate representation between abstract CHP communication and production rules. It contains only Boolean variables, waits on Boolean conditions, and explicit signal transitions, replacing abstract channel communications with the transitions implementing the selected handshake protocols [8, 3].

For each channel, one endpoint is assigned an *active* protocol and the other a *passive* protocol. The active endpoint initiates the communication, whereas the passive endpoint waits for the corresponding request or data transition before responding. This choice determines the causal ordering of request, data, and acknowledge transitions and therefore affects the circuit synthesized at PRS level.

The circuits in this work use four-phase return-to-zero handshaking together with dual-rail data encoding. As illustrated in Fig. 7.1, each data bit is represented by two rails, C_{dt} and C_{df} : 00 represents the spacer state, 10 a logical 1, and 01 a logical 0, while 11 is forbidden. In the *active-sender/passive-receiver* configuration, the sender initiates the transfer by asserting the appropriate data rail and the receiver responds through C_a ; the sender then restores the spacer before the acknowledge is deasserted. Conversely, in the *passive-sender/active-receiver* configuration, the receiver asserts C_r to request a transfer and the sender responds by asserting the data rail encoding the transmitted value. The receiver then deasserts C_r , allowing the sender to restore the spacer and complete the cycle [3].

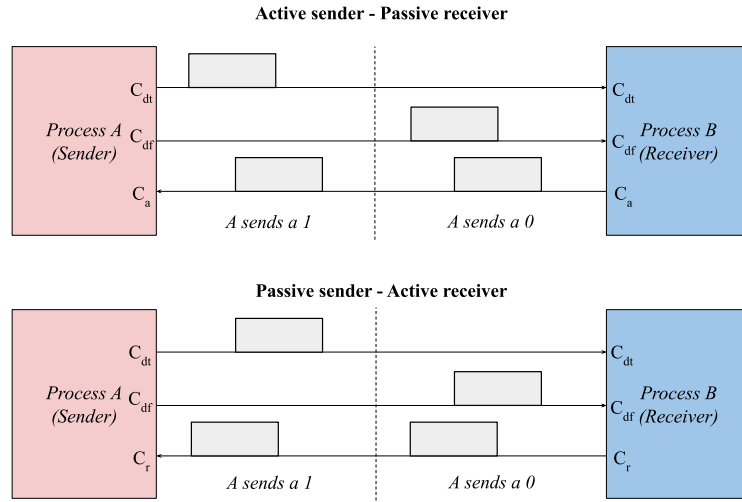


Figure 7.1: Four-phase communication of a one-bit dual-rail channel for the two protocol configurations used in this work. Top: active sender and passive receiver, where the sender initiates the transfer by asserting one of the data rails and the receiver responds through C_a . Bottom: passive sender and active receiver, where the receiver initiates the transfer through C_r and the sender responds by asserting the rail encoding the transmitted value.

7.2.1 HSE Reshuffling

The direct expansion of a CHP process into handshakes is generally not unique. Once the required ordering constraints have been identified, independent transitions can often be moved relative to one another. This transformation, known as *reshuffling*, can expose concurrency, reduce unnecessary serialization, simplify production rules, and sometimes eliminate internal state [3].

As a simple example, consider a process communicating through two four-phase channels L and R , where L is passive and R is active. As a simple

example, consider the CHP process

$$*[\bar{L} \rightarrow R; L],$$

Let l_r and l_a denote the request and acknowledge signals of L , respectively, and r_r and r_a those of R . A direct HSE can be written as

$$[l_r]; r_r \uparrow; [r_a]; r_r \downarrow; [\neg r_a]; l_a \uparrow; [\neg l_r]; l_a \downarrow.$$

A possible reshuffling is

$$[l_r]; r_r \uparrow; [r_a]; l_a \uparrow; [\neg l_r]; r_r \downarrow; [\neg r_a]; l_a \downarrow.$$

The same transitions are present in both descriptions, but their relative ordering differs. In the reshuffled version, $l_a \uparrow$ can occur before R completes its return-to-zero phase, overlapping useful work with handshake recovery while preserving the ordering required by each individual four-phase protocol.

Reshuffling cannot be performed arbitrarily because moving transitions may change the synchronization behavior and therefore the CHP semantics. In this work it was performed manually and iteratively together with PRS synthesis: when an ordering produced unnecessarily complex or unsafe rules, the HSE was revisited and an alternative legal ordering was considered.

7.2.2 State-Variable Insertion

An HSE may contain different execution points at which all existing signals have identical values even though different actions must occur next. Such states are indistinguishable from the perspective of a production-rule guard and require additional internal state. The standard solution is to introduce Boolean *state variables* that distinguish these phases of the computation [3].

Consider again the unreshuffled HSE:

$$[l_r]; r_r \uparrow; [r_a]; r_r \downarrow; [\neg r_a]; l_a \uparrow; [\neg l_r]; l_a \downarrow.$$

The circuit can reach

$$(l_r, l_a, r_r, r_a) = (1, 0, 0, 0)$$

at two different points. In one case the next transition must be $r_r \uparrow$, whereas in the other it must be $l_a \uparrow$. Since the visible signals are identical, the two situations cannot be distinguished by a production-rule guard.

Introducing a Boolean state variable x , initially low, removes the ambiguity:

$$[l_r]; r_r \uparrow; [r_a]; x \uparrow; r_r \downarrow; [\neg r_a]; l_a \uparrow; [\neg l_r]; x \downarrow; l_a \downarrow.$$

The two previously indistinguishable states are now associated with $x = 0$ and $x = 1$, respectively. State-variable insertion was used extensively in the router both to distinguish otherwise ambiguous execution phases and to simplify guards by replacing larger Boolean conditions with locally generated state signals.

The final HSE is therefore not a purely mechanical expansion of the CHP, but the result of an iterative refinement in which transition ordering and state representation are chosen to obtain a correct and practical circuit.

7.3 Production Rule Sets

The HSE is subsequently translated into a *Production Rule Set* (PRS), where each rule associates a Boolean condition with a transition of a circuit node. For example, the state-holding behavior of a node x controlled by a and b can be described by

$$a \wedge b \rightarrow x \uparrow \quad \neg a \wedge \neg b \rightarrow x \downarrow. \quad (7.1)$$

The first rule eventually drives x high when both inputs are high, while the second drives it low when both are low. When the inputs disagree, neither rule is enabled and x retains its previous value. The Boolean expressions on the left-hand side are the *guards* of the rules and provide a direct bridge between HSE behavior and the transistor networks that implement the circuit [8].

The initial guards derived from the HSE are not always sufficient. For example, if $x \uparrow$ is allowed only after an internal signal s has been asserted, $a \wedge b \rightarrow x \uparrow$ may need to be strengthened to $s \wedge a \wedge b \rightarrow x \uparrow$. The final PRS must be stable and non-interfering: an effectively enabled guard must remain valid until the corresponding transition occurs, and two complementary rules must never request opposing effective transitions simultaneously. Reset conditions are also incorporated directly into the PRS so that state variables, acknowledgement signals, and dual-rail outputs are brought to the initial state assumed by the HSE, with output rails initialized to spacer and conflicting transitions disabled during reset [3].

PRS synthesis therefore remained closely coupled to HSE development. Whenever the resulting rules exposed ambiguous states, overly complex guards, or undesirable transition ordering, the HSE was revisited through reshuffling or state-variable insertion before deriving the PRS again.

7.3.1 PRS Optimization

Even a functionally correct PRS may result in inefficient transistor networks. In particular, guards with many conjunctive terms map to long series stacks, increasing both resistance and internal capacitance; under a simplified Elmore-delay model, the resulting delay grows approximately quadratically with the number of series devices. To limit this effect, complex guards were sometimes decomposed by introducing additional state variables, trading an extra transition for shallower and faster transistor networks.

For wide dual-rail datapaths, a similar strategy is used through *completion trees*. Instead of implementing completion detection with a single high-fanin C-element, the operation is decomposed into a tree of smaller fixed-fanin C-elements. This is valid when the tree inputs remain stable until the completion event has propagated and acknowledged them [3].

These optimizations were applied where necessary, not exhaustively because of the project timeline. The implemented PRS is therefore functionally correct and suitable for transistor-level evaluation, while further state-variable insertion, operator decomposition, and completion-tree optimization could reduce critical-path delay in a future implementation.

7.3.2 Isochronic-Fork Analysis

As previously mentioned, in QDI circuits we have timing assumptions that must be met for the circuit to work, known as *isochronic-forks*. A fork occurs when a gate output fans out toward multiple downstream gates. One branch of a fork may acknowledge a transition by causing a subsequent transition on another signal, while another branch remains unacknowledged. A race can then occur if the effect of the acknowledged branch propagates through the circuit and activates a downstream guard before this guard has been disabled by the unacknowledged branch. In that case, the guard may change state while the unacknowledged branch still observes the old value, potentially violating stability. Correctness then requires the unacknowledged branch to observe the transition before the acknowledged computation can progress sufficiently far to invalidate it [16].

An isochronic-fork assumption therefore does not require equal branch delays. Instead, the unacknowledged branch must be sufficiently faster than the relevant adversarial path through the acknowledged portion of the circuit. This relative constraint can eventually be translated into placement and routing requirements.

Because no automated procedure was available in the adopted flow to extract the complete set of fork constraints, a simulation-based stress methodology was used. The final PRS was simulated with `prsim`, ACT's production-rule simulator. Normally, when a signal appears in several guards, its fanout is

treated ideally and the transition becomes visible to all consumers without independently modeling each physical branch. Explicit intermediate nodes were therefore inserted on individual branches to emulate wire delay, while randomized production-rule delays were used to stress different timing combinations over long simulations and multiple seeds.

If an unacknowledged branch became slower than the causal path acknowledging the original transition, *prsim* could expose an instability or interference. The corresponding fork was then documented and the artificial delay on that branch removed in subsequent simulations, thereby imposing the required isochronic assumption while continuing to stress the remaining fanouts.

This procedure is empirical and does not formally prove that every possible isochronic fork has been identified. Nevertheless, repeated randomized-delay simulations provide a practical way to expose the most critical races, with shorter adversarial paths naturally tending to fail first. The resulting constraints were retained for bubble reshuffling and, in a complete flow, would also constrain the physical implementation.

7.3.3 Bubble Reshuffling

A stable and non-interfering PRS is not necessarily directly implementable in static CMOS. Pull-down networks are naturally constructed from non-inverted inputs controlling nMOS transistors, whereas pull-up networks naturally require complemented inputs controlling pMOS transistors. Production rules requiring incompatible signal polarities must therefore be transformed through *bubble reshuffling* [3].

As a simple example, consider

$$a \wedge b \rightarrow y \uparrow, \quad \neg a \vee \neg b \rightarrow y \downarrow.$$

These rules implement $y = a \wedge b$, but the pull-up rule contains non-inverted inputs and is therefore not directly implementable as a pMOS network. Reversing the polarity of the output gives

$$a \wedge b \rightarrow \neg y \downarrow, \quad \neg a \vee \neg b \rightarrow \neg y \uparrow,$$

which is directly realizable as a NAND gate. Subsequent logic must then account for the fact that the physical signal represents $\neg y$. Bubble reshuffling generalizes this operation by selecting and propagating suitable signal polarities throughout the PRS.

Arbitrarily inserting inverters is unsafe in a QDI circuit. An inverter placed only on an isochronic, unacknowledged branch may itself generate unacknowledged transitions and violate PRS stability. Inverters can instead be

placed before the fork or on branches whose transitions are independently acknowledged.

For each block, bubble reshuffling therefore started from the fork analysis of Section 7.3.2. Unacknowledged branches were listed together with the relevant signal polarity and consuming guard, establishing polarity constraints that could not be violated. Signal senses were then selected globally and propagated through all rules, while remaining opposite polarities were generated only on acknowledged paths or before fanout.

Bubble reshuffling is therefore constrained by the acknowledgement structure of the asynchronous computation rather than being a conventional Boolean optimization. If no legal polarity assignment exists, the design must be revisited at HSE level through a different reshuffling or additional state variables [14].

7.4 Physical Implementation

Once the PRS has been made stable, non-interfering, correctly initialized, and CMOS-implementable, it can be mapped to an electrical circuit. The implementation performed in this thesis was carried through transistor-level netlist generation and SPICE characterization. A complete tapeout flow would subsequently continue with custom-cell generation, placement and routing, timing-constraint closure, parasitic extraction, and physical verification.

7.4.1 SPICE Netlist Generation

The final bubble-reshuffled PRS was translated into a hierarchical transistor-level SPICE netlist using `prs2net`, part of the ACT toolchain. For each driven node, the pull-down guard is mapped to an nMOS switching network and the pull-up guard to the corresponding pMOS network. State-holding nodes additionally require feedback circuitry such as a staticizer or combinational-feedback implementation. After netlist generation, the transistor dimensions of selected high-fanout drivers were manually increased to provide sufficient drive strength for their larger capacitive loads.

The generated circuits use the 65-nm Predictive Technology Model at the typical–typical corner (PTM65 TT), with $V_{DD} = 1.1$ V and a temperature of 27°C. These simulations represent the transistor-level logic implementation but not a final extracted layout; physical wire parasitics are therefore not inherently included.

7.4.2 Transistor-Level Verification and Characterization

The generated netlists were simulated with HSPICE to verify that the Boolean and handshake behavior obtained at PRS level remained correct once transistor dynamics, finite slew rates, loading, and state-holding circuitry were introduced. This stage was particularly important because the implementation effort intentionally terminated before physical layout.

Dedicated testbenches exercised the relevant input-to-output paths and operating cases of each router block. In addition to functional correctness, these simulations provided the timing and energy information required for system-level performance modeling. Characterization was performed separately for the relevant source and destination combinations to preserve path-dependent behavior.

The resulting measurements were stored in lookup tables describing the timing and energy cost of elementary router operations. These tables form the interface between transistor-level characterization and the system-level NoC simulator used in Chapter 8.

7.4.3 Custom Cell Library: Omitted Step

A complete physical implementation would next map the PRS operators to a physical cell library. This is particularly important for QDI logic because state-holding operators, generalized C-elements, arbitration structures, and custom pull-up/pull-down networks are generally absent from conventional synchronous standard-cell libraries.

Each custom cell must provide a layout consistent with its transistor network together with the views required by the place-and-route flow, and must subsequently be characterized for delay, energy, and loading. Liu and Boahen followed this approach by constructing a custom asynchronous library before physically implementing their hierarchical network in a 65-nm CMOS process [15]. This stage was not completed for the router presented in this thesis; the reported circuit results therefore correspond to pre-layout SPICE simulations.

7.4.4 Placement and Routing: Omitted Step

After cell mapping, a complete ASIC flow proceeds to placement and routing. In addition to conventional connectivity, congestion, and electrical constraints, a QDI implementation must preserve the relative-delay requirements introduced by isochronic forks. The unacknowledged branch of each critical fork must therefore be routed so that its delay satisfies the corresponding adversarial-path constraint.

Liu and Boahen provide an example of this methodology: after placement and routing, they extract a timing graph and verify the relative-timing constraints; violating unacknowledged branches are assigned increased routing priority before repeating the flow, after which a parasitic netlist is extracted and simulated in SPICE [15].

The long interconnects at the upper levels of the H-tree may additionally require physical pipelining. If a link dominates the local handshake cycle, asynchronous buffers can divide it into shorter pipeline segments. Liu and Boahen use Pre-Charge Full Buffers for this purpose in long H-tree datapaths [15].

7.4.5 Post-Layout Verification: Omitted Step

The final implementation step would extract interconnect resistance and capacitance from the routed layout and repeat transistor-level simulation with these parasitics included. Post-layout characterization is required to verify functionality and obtain realistic latency, cycle-time, and energy values, while the layout must also pass conventional design-rule and layout-versus-schematic checks.

For a QDI design, this stage additionally verifies that every isochronic-fork constraint identified at PRS level remains satisfied after actual wire lengths and loads are known. Physical implementation therefore closes the loop between the timing assumptions introduced during logical synthesis and the geometry of the fabricated circuit.

Results and Discussion

This chapter evaluates the proposed NoC at three complementary levels. First, transistor-level simulations characterize the implemented asynchronous router in terms of timing, dynamic energy, and area. Second, the extracted router parameters are used in a discrete-event simulator to study the network-level behavior of the architecture under controlled unicast and multicast traffic, including the effects of communication distance, multicast locality, destination density, and network load. Finally, application-driven SNN benchmarks evaluate the proposed multicast encoding against HBS and synchronous reference architectures using realistic communication traces.

8.1 Methodology

8.1.1 Router Characterization

The asynchronous router is characterized at transistor level using HSPICE and PTM 65-nm models at the TT corner, $V_{DD} = 1.1$ V, and 27 °C. For the relevant input–output paths and multicast fan-outs, the simulations extract traversal latency, initiation interval, and dynamic energy. Transition times are measured at $V_{DD}/2$, while dynamic energy is obtained by integrating the supply power over an isolated handshake and subtracting the corresponding quiescent contribution.

The extracted measurements are collected into a lookup table indexed by operation type, path, and multicast fan-out. The corresponding timing and energy characterization is presented in Section 8.2, and the resulting lookup table provides the router parameters used by the network simulator.

8.1.2 Network Characterization

The network characterization evaluates the proposed NoC under controlled traffic patterns using a discrete-event simulator parameterized by the router

measurements described above.

Interconnect and Link Pipelining

Physical wires are modeled separately because the transistor-level router simulations do not include extracted interconnect parasitics. For a link of length L , the resistance and capacitance per rail are assumed to be $R' = 0.20 \Omega/\mu\text{m}$ and $C' = 0.20 \text{ fF}/\mu\text{m}$, respectively. The delay of an unbuffered distributed RC wire is approximated as

$$t_{\text{wire}}(L) = \frac{1}{2} R' C' L^2, \quad (8.1)$$

while its capacitive switching energy grows linearly with L .

Long asynchronous links can be divided into shorter pipeline sections using Pre-Charge Full Buffers (PCFBs). The number of sections is chosen as the minimum value that keeps the RC delay of each section below a 2 ns budget,

$$N_{\text{seg}} = \max \left(1, \left\lceil \sqrt{\frac{t_{\text{wire}}(L)}{2 \text{ ns}}} \right\rceil \right). \quad (8.2)$$

A link is therefore divided into N_{seg} equal sections and $N_{\text{seg}} - 1$ asynchronous pipeline stages are inserted between them. With the geometry considered in this work, however, the longest tree link is 8 mm and remains below the segmentation threshold, so no additional PCFB stages are required in the reported experiments. The wire parameters are modeling assumptions rather than parasitics extracted from a placed-and-routed implementation.

Network Simulator

The router and interconnect models are integrated into a discrete-event simulator of the complete NoC. The simulator tracks flit propagation, buffering, arbitration, backpressure, and packet resource occupation. Router operations use the timing and energy parameters obtained from the SPICE-derived lookup table, while each transmission additionally incurs the delay and energy associated with its physical link. The simulator follows the actual routing protocol implemented by the router. Unicast routes are progressively consumed as packets traverse the tree, while multicast packets first reach their LCA and subsequently undergo hierarchical route parsing, branch generation, and payload replication. Communication is therefore modeled from the operations performed by the architecture rather than from an abstract hop-count approximation.

Experimental Setup

Unless otherwise specified, the simulated system contains $64 \times 64 = 4096$ endpoints with a tile pitch of $250 \mu\text{m}$. The proposed network is a depth-six quad-tree containing 1365 internal routers. Packets contain eight 32-bit data flits, corresponding to 224 useful payload bits after excluding the four control bits of each flit. The synchronous baseline used for the unicast comparisons is a 64×64 XY-routed mesh based on the Intel 80-tile router architecture [18]. The router is adapted to a 32-bit datapath and modeled as a five-stage pipeline operating at 4 GHz.

For the multicast experiments, two parameters are used to characterize the destination set. The LCA level λ is counted from the router directly above the leaves and determines the size of the multicast subtree. A level- λ subtree contains

$$C_\lambda = 4^{\lambda+1} \quad (8.3)$$

leaf endpoints. If K of these endpoints are selected as destinations, the destination density is defined as

$$\rho = \frac{K}{C_\lambda}. \quad (8.4)$$

Thus, λ controls the spatial extent of the multicast, while ρ controls how densely the corresponding subtree is populated with destinations. These conventions are illustrated in Fig. 8.1.

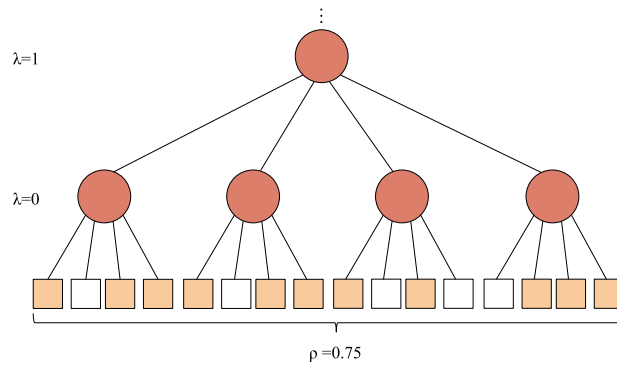


Figure 8.1: Definition of the multicast parameters λ and ρ . The LCA level λ determines the size of the destination subtree, while ρ represents the fraction of leaves within that subtree selected as destinations.

Evaluation Metrics

Packet latency, simply called "latency" in the next sections, is measured from message release until the final data flit reaches the last destination,

$$T_{\text{packet}} = \max_{d \in D} (t_{\text{last},d}) - t_{\text{release}}, \quad (8.5)$$

Dynamic energy is reported per delivered bit as

$$e_{\text{bit}} = \frac{E_{\text{dyn}}}{32N_{\text{flit,delivered}}}, \quad (8.6)$$

where E_{dyn} includes router processing, route setup, multicast replication, and modeled interconnect energy.

Throughput experiments consider the 28 useful payload bits carried by each data flit. Offered load and accepted throughput are both expressed in Gb/s/endpoint, allowing the sustainable throughput and the onset of network saturation to be identified under continuous traffic.

8.1.3 Network Benchmarks

Workloads and Spike Traces

We trained implementations of SNN architectures derived from the literature and selected to study a communication regime with high batching compatibility. The resulting inference traces are then used as application-level workloads to compare the different NoC architectures under identical neural activity and destination sets. MAP-SNN on N-MNIST consists of two polarity-specific feed-forward branches, each comprising 1156 input neurons and 400 hidden neurons, which converge onto a shared 10-neuron output layer; inference is evaluated over 15 timesteps. MAP-SNN on SHD consists of a feed-forward network with 700 input neurons, two hidden layers of 400 neurons each, and a 20-neuron output layer, evaluated over 50 timesteps. The third workload is an LSM on N-MNIST, consisting of a recurrent liquid of 4096 neurons followed by a 10-neuron readout layer and evaluated over 25 timesteps. These workloads were chosen because their dense feed-forward and recurrent connectivity produces repeated one-to-many communication patterns, creating favorable conditions for multicast batching.

During inference, the activity of every neuron is recorded for each sample and timestep, producing a communication trace that specifies the firing source, spike multiplicity, and time of occurrence. Spike multiplicity is preserved: if a neuron emits c spikes within one timestep, the trace contains c occurrences of the corresponding source address rather than collapsing them

into a single binary event. For each spike occurrence, the physical destination set is obtained by identifying the tiles containing the postsynaptic neurons connected to the source neuron. Since communication is modeled at tile granularity, multiple postsynaptic neurons located on the same tile contribute a single destination entry. The spike traces are generated independently of the NoC simulation and are reused unchanged across all compared architectures. The network experiments use a fixed set of the first 100 test samples, selected before evaluating any NoC metric, so that every architecture is exposed to exactly the same neural activity and communication demand.

Mapping and Multicast Batching

The workloads are mapped onto a common substrate of 16 physical tiles, with a target capacity of 256 logical neurons per tile. Neurons are assigned using an activity-aware mapping that orders them by decreasing activity, computed exclusively from the training set, before distributing them across the tiles assigned to each population. The mapping is frozen before test simulation and is shared by all compared NoCs.

A natural multicast batch contains spike occurrences belonging to the same sample and timestep, originating from the same source tile, and targeting exactly the same set of destination tiles. Batches never combine different timesteps or different samples. The maximum number of source addresses carried by one packet is varied as

$$B_{\max} \in \{1, 2, 4, 8, 16, 32, 64\}.$$

For a natural group containing B spike occurrences, the proposed tree generates $\lceil B/B_{\max} \rceil$ packets, whereas the HBS tree and synchronous mesh transmit one packet per spike occurrence, consistent with their modeled packet format and native operation, in which each spike is carried independently in a single flit. Source identifiers require 13 bits in these workloads; consequently, a batched packet carrying B addresses on the proposed 32-bit interface requires $\lceil 13B/28 \rceil$ payload flits in addition to the routing and control information.

Compared NoCs

We compare the proposed NoC against two reference architectures that are particularly relevant to the communication patterns considered in this work: the Hierarchical Bit String (HBS) tree proposed in [7] by Su et al. and a synchronous multicast-capable mesh characterized in the same work by Su et al. [19].

The proposed NoC is instantiated as a depth-two quaternary tree containing five routers and 16 endpoints, using the routing protocol and characterization LUT described in the previous sections.

The HBS reference is the Hierarchical Bit String architecture, which uses two 4-bit hierarchical masks for a 16-core system. Its implementation consists of four first-level switches and one second-level switch, matching the depth-two tree considered here, and was implemented in 22-nm technology. The HBS work builds on the asynchronous multicast switch introduced by Su et al. [19], whose post-layout timing and energy characterization is therefore used to parameterize the HBS routers in the simulator.

The synchronous reference is the multicast-capable switch characterized in the same work. In that study, synchronous and asynchronous switches were implemented and compared after placement and routing in the same 22-nm industrial technology. Here, the published synchronous switch parameters are used to instantiate a 4×4 XY-routed mesh with an exact 16-destination mask. Thus, while the mesh topology and routing behavior are modeled at network level, its router timing and energy parameters are taken directly from the published synchronous implementation.

All three architectures operate on the same physical mapping and spike traces. As already mentioned, the proposed tree is characterized independently using PTM 65-nm transistor models, whereas the HBS and synchronous-router parameters are derived from the 22-nm implementations described above. No technology normalization is applied; consequently, the comparison is intended primarily to study architectural communication behavior and the effect of multicast batching rather than to provide a technology-normalized PPA comparison.

Simulation and Evaluation Metrics

The recorded spike traces drive a discrete-event NoC simulation. All traffic generated within one neural-network timestep is made available to the network together, and the following timestep begins after all intended deliveries for the current timestep have completed. Contention, buffering, backpressure, router recovery, and HBS false-positive traffic are explicitly simulated.

NoC latency per inference is computed as the sum of the communication completion times of its timesteps and excludes neuronal computation, training, the biological timestep duration, and any waiting time required to form a batch. Dynamic energy includes router and interconnect activity for all simulated transmissions, while neural computation, synaptic-memory accesses, endpoint-side batching overhead, and leakage are excluded. The distribution of naturally occurring batch sizes, are additionally recorded to quantify the batching opportunities present in each workload.

8.2 Router Characterization

The transistor-level measurements provide the timing and energy parameters used by the network simulators. The complete path-dependent lookup tables are reported in Appendix D; this section highlights only the main characteristics of the implementation.

8.2.1 Timing Characteristics

The timing characterization distinguishes between forward latency L_f and initiation interval II . As will be shown in the following sections, the current implementation exhibits relatively short forward propagation paths, while the initiation interval is substantially larger. The main timing limitation is therefore associated with the recovery phase of the four-phase asynchronous pipeline rather than with the propagation of valid data through the forward datapath. After the valid-data phase, the communication signals must return to the neutral state and this transition must be acknowledged before the corresponding pipeline stage can safely participate in the next transaction [3]. Internal pipelining with Pre-Charge Full Buffers (PCFBs) reduces the length of the forward paths and allows different pipeline stages to recover concurrently, but it does not eliminate the recovery latency internal to each individual stage.

As already mentioned, in the present implementation, the 32-bit dual-rail datapaths are completion-detected as complete words. Consequently, each stage must detect the validity and neutrality of a large number of rails before completing its handshake. This is closely related to the *aggressive datapath design* problem described by Manohar [3]: globally synchronizing an N -bit channel introduces completion structures whose cycle time grows with $\log N$, whereas partitioning the datapath into independently synchronized fixed-width slices can remove this width-dependent throughput penalty. Such a bit-sliced datapath decomposition was not performed here because of project-time constraints. In addition, the final PRS was primarily optimized for correctness, CMOS implementability, and bubble reshuffling rather than exhaustively optimized for minimum cycle time. The initiation intervals reported in the following sections should therefore be interpreted as properties of the present implementation rather than as an architectural lower bound.

Unicast Routing

The unicast measurements provide the first indication of this distinction between forward latency L_f and initiation interval II . Table 8.1 summarizes the timing ranges for the different unicast paths, while the complete path-dependent characterization is reported in Appendix D.

Table 8.1: Representative timing ranges for unicast forwarding. L_f denotes forward latency and II the initiation interval.

Path	Flit type	L_f [ns]	II [ns]
$C_i \rightarrow C_o$	Header	2.291–2.611	3.497–3.512
	Data	0.822–0.825	3.221–3.247
	Tail	0.834–0.838	3.151–3.164
$C_i \rightarrow P_o$	Header	1.737–1.845	3.491–3.500
	Data	0.530–0.533	3.104–3.113
	Tail	0.537–0.539	3.149–3.159
$P_i \rightarrow C_o$	Header	2.265–2.324	2.691–2.701
	Data	0.789–0.791	2.492–2.498
	Tail	0.795–0.799	2.364–2.374

For payload flits, L_f ranges from approximately 0.53 ns for child-to-parent forwarding to 0.82 ns for child-to-child forwarding, while parent-to-child forwarding requires approximately 0.79 ns. Headers require additional time because the routing direction and output port must first be determined, but the payload data path remains short once the packet route has been established.

Multicast Payload Routing

Multicast payload routing characterizes the replication of the common payload toward the selected child outputs. Here the local fan-out F is defined as the number of child outputs active in the current router. The corresponding timing ranges are summarized in Table 8.2.

Table 8.2: Timing ranges for multicast common-payload forwarding over fan-outs $F = 1, \dots, 4$. The very small variation with F shows that local replication has little effect on the forward critical path.

Input	Flit type	L_f [ns]	II [ns]
C_i	First data flit	1.378–1.480	4.891–4.950
	Intermediate data	0.875–0.880	4.186–4.191
	Tail	0.877–0.880	4.208–4.212
P_i	First data flit	1.390–1.391	1.604–1.606
	Intermediate data	0.886–0.889	3.211–3.213
	Tail	0.887–0.890	3.233–3.240

The first common-payload flit incurs additional control overhead associated with initializing payload distribution for the current multicast transaction,

whereas intermediate flits reuse the routing state already maintained by the router. Importantly, L_f changes very little with local fan-out: replicating a payload toward four children instead of one mainly activates additional output circuitry in parallel rather than extending the forward critical path.

Multicast Route Parsing

Multicast route parsing accounts for the processing of the hierarchical route representation and the construction of the corresponding child subroutes. The route words are then divided into an initial word and one or more continuation words, as summarized in Table 8.3.

Table 8.3: Timing ranges for multicast-route parsing. For parser operations, the processing time is also used as the initiation interval in the network model.

Parser phase	II [ns]
Initial route word	32.713–74.853
Continuation route word	23.115–51.273

The initial route word initializes the multicast parser and the state required to repartition the encoded route among the child subroutes. Continuation words represent the main route-processing phase, and their processing time depends on the distribution of the four-bit masks contained in the flit. Masks assigned to the same child can be processed consecutively without changing the child currently being served, whereas masks distributed across multiple children require the parser to switch repeatedly between child-serving phases. A continuation word whose masks are concentrated on a single child therefore incurs fewer control transitions and can be processed more quickly than one whose masks are distributed across several children.

The *MSCODE* marks the end of the multicast route and may appear within any route word rather than occupying a dedicated terminator flit. Its position therefore directly affects the parsing cost: once the first zero nibble corresponding to the *MSCODE* is detected, all following zero nibbles can be skipped. A route word in which the *MSCODE* appears earlier consequently requires fewer parsing operations and can be processed more quickly than one in which the terminator appears later. The complete data-dependent timing matrices are reported in Appendix D.4.

8.2.2 Dynamic Energy

The dynamic-energy characterization follows the same functional decomposition used for timing. The main trends are that unicast energy depends primarily on the forwarding direction, multicast common-payload energy

increases with the local fan-out, and multicast-route parsing incurs a substantially larger but route-dependent cost. The complete characterization data are reported in Appendix D.

Unicast Routing

Table 8.4 summarizes the measured dynamic-energy ranges for the different unicast paths.

Table 8.4: Dynamic-energy ranges for unicast forwarding, expressed per input bit.

Path	Flit type	E [fj/bit]
$C_i \rightarrow C_o$	Header	149.291–159.311
	Data	131.409–132.185
	Tail	138.211–138.966
$C_i \rightarrow P_o$	Header	92.088–93.308
	Data	79.967–80.393
	Tail	83.295–83.601
$P_i \rightarrow C_o$	Header	137.071–141.977
	Data	120.586–120.838
	Tail	126.373–126.807

For unicast payload forwarding, child-to-parent communication is the least expensive at approximately 80 fj/bit, while parent-to-child and child-to-child forwarding require approximately 121 fj/bit and 132 fj/bit, respectively. Header energy is higher because additional routing logic is activated when the packet path is established.

Multicast Payload Routing

The dynamic energy required to forward the common multicast payload depends strongly on the local fan-out F . Table 8.5 reports the directly characterized fan-out endpoints; fan-outs two and three are interpolated in the network model.

Table 8.5: Dynamic-energy ranges for multicast common-payload forwarding. Fan-outs one and four are directly characterized; intermediate fan-outs are interpolated.

Input	Flit type	$F = 1$ [fJ/bit]	$F = 4$ [fJ/bit]
C_i	First data flit	178.237–179.688	359.932–364.211
	Intermediate data	173.998–174.177	347.575–347.876
	Tail	176.624–176.806	351.692–352.060
P_i	First data flit	183.831	365.401
	Intermediate data	179.269	353.135
	Tail	181.768	356.697

The common-payload cost increases approximately with the number of copies produced locally. For an intermediate payload flit, the measured energy rises from approximately 174–179 fJ/bit at fan-out one to approximately 348–353 fJ/bit at fan-out four. This reflects the cost of native replication: the packet traverses the shared path only once, while additional switching activity is introduced only at the router where the multicast path branches.

Multicast Route Parsing

Multicast-route parsing incurs a substantially larger dynamic-energy cost than common-payload forwarding because each route word activates the serialization, mask-processing, child-selection, counter-update, and subroute-construction logic. The corresponding energy ranges are summarized in Table 8.6.

Table 8.6: Dynamic-energy ranges for multicast-route parsing. Energy is normalized per 32-bit input route word and reported per input bit.

Parser phase	E [fJ/bit]
Initial route word	497.008–984.838
Continuation route word	343.314–923.297

The multicast-route parsing energy follows the same data-dependent activity discussed in the timing characterization. The initial route word includes the additional switching required to initialize the parser state, while continuation words exhibit a cost that depends on the amount of route-processing activity performed for the corresponding mask distribution. In addition, the *MSCODE* may appear within any route word rather than in a dedicated terminator flit. Once the first zero nibble corresponding to the *MSCODE* is encountered, subsequent zero nibbles can be skipped, reducing the amount of active parsing and therefore the associated dynamic energy.

8.2.3 Area

The circuit complexity of the router is summarized in Fig. 8.2. The generated transistor-level netlist contains 129720 MOS devices.

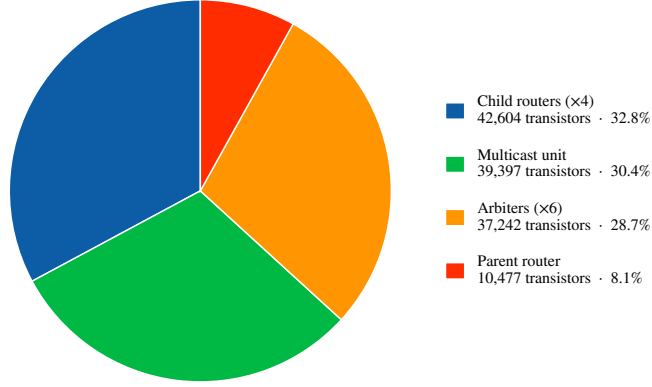


Figure 8.2: Transistor-count breakdown of one router node.

The four child-routing blocks account for 32.8% of the devices, the multicast unit for 30.4%, the arbitration network for 28.7%, and the parent-routing datapath for the remaining 8.1%. The multicast functionality therefore represents a substantial but not dominant fraction of the complete router implementation. Using the conventional gate-equivalent definition based on a two-input NAND gate, corresponding to four MOS transistors, the total count can be approximately expressed as

$$A_{\text{kGE}} \simeq \frac{N_{\text{MOS}}}{4000},$$

giving approximately 32.4 kGE for the complete router. This conversion is only a transistor-count normalization: the reported values are not post-layout silicon area and do not include transistor sizing, routing, or floorplanning effects.

8.3 Network-Level Characterization

The following experiments characterize the proposed NoC under controlled synthetic traffic patterns. Rather than reproducing a specific application workload, these experiments are intended to expose the fundamental latency, energy, throughput, and congestion behavior of the network.

8.3.1 Unicast Results

Latency

Figure 8.3 compares the latency of a message having 8 payload flits, as the destination moves horizontally away from the source at tile (0,0). In the tree, the router contribution grows approximately logarithmically with distance, producing the characteristic staircase behavior as progressively higher LCAs are reached. The longer upper-level links also introduce an increasingly visible wire-delay contribution. In contrast, the mesh latency grows linearly with Manhattan distance.

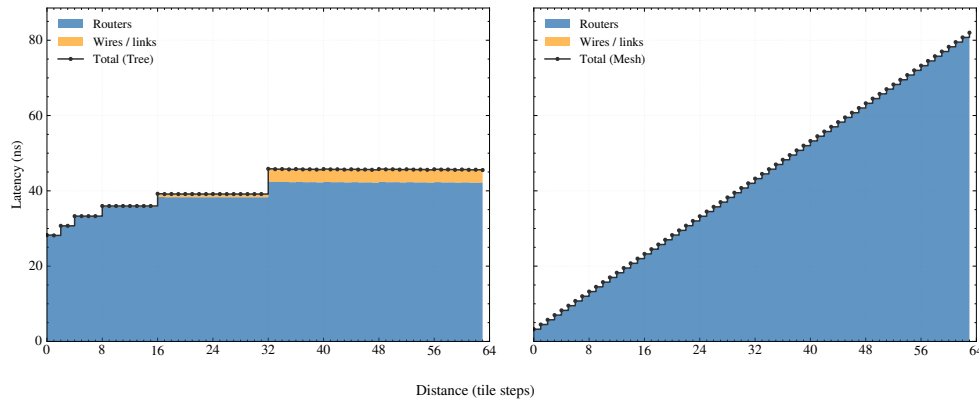


Figure 8.3: Unicast packet latency as a function of source–destination distance for the proposed tree and the synchronous mesh baseline.

At short distances the mesh is faster, mainly because the current asynchronous router has a relatively large and not yet fully optimized initiation time. However, at longer distances, the lower router count of the hierarchical topology compensates for this overhead: at 63 tile steps, the tree requires approximately 45.5 ns, compared with 82 ns for the mesh. This demonstrates that tree topologies can provide a clear latency advantage for long-distance unicast communication.

Energy

Figure 8.4 compares the dynamic energy of the tree and mesh as the destination moves horizontally away from the source at tile (0,0), separating router and interconnect contributions. As with latency, the tree exhibits a characteristic staircase behavior as progressively higher LCAs are reached. Each transition to a higher level activates a longer H-tree link, increasing the interconnect energy accordingly. In contrast, the mesh energy grows approximately linearly with Manhattan distance, since each additional tile step introduces another router and link traversal. In both architectures, the

wire contribution becomes increasingly dominant as communication distance increases.

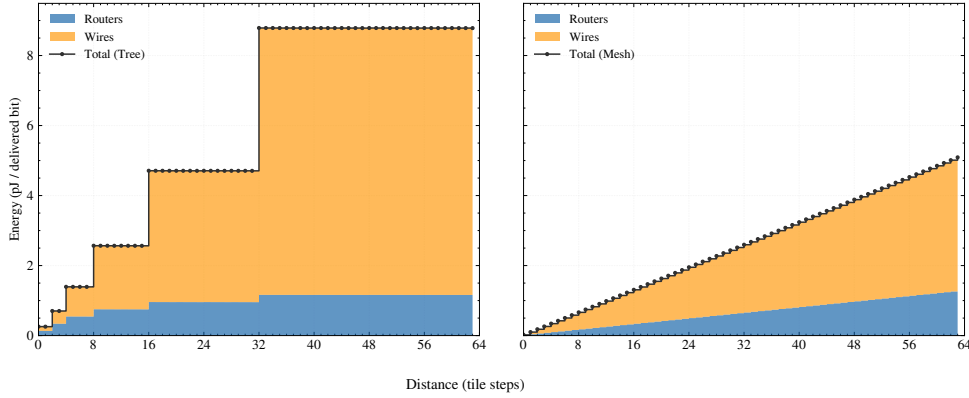


Figure 8.4: Unicast dynamic energy per transferred bit as a function of distance, separated into router and modeled interconnect contributions.

At the largest simulated distance, the tree reaches approximately 8.78 pJ/bit, of which 7.62 pJ/bit, or about 87%, is attributed to the modeled wires. The mesh grows more gradually and remains lower in absolute energy over the evaluated distance range, mainly because its links are short and uniform. This highlights an important trade-off: although the tree reduces the number of router traversals, the increasingly long upper-level links make interconnect energy the dominant contribution as higher levels of the hierarchy are reached. It should also be noted, however, that this experiment only captures the energy associated with a transmission and does not include the time dimension. As a result, it does not reflect one potential advantage of the asynchronous tree under sparse traffic: inactive regions naturally stop switching, whereas the synchronous mesh continues to incur clock-related dynamic power even during idle periods.

Throughput

Figure 8.5 evaluates the two networks under continuous global unicast traffic. At low load, accepted throughput follows the offered load in both cases. Once the network capacity is reached, backpressure causes the accepted rate to deviate from the ideal line.

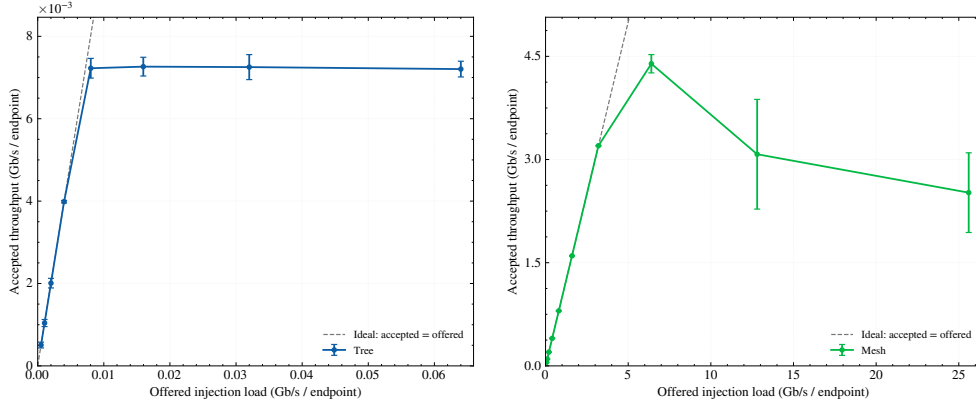


Figure 8.5: Accepted unicast throughput versus offered injection load for the proposed tree and the synchronous mesh baseline. Error bars show the 95% confidence interval across three traffic seeds.

The tree saturates at approximately 7.2 Mb/s/endpoint, corresponding to about 30 Gb/s of aggregate delivered traffic across the 4096 endpoints. This exposes a fundamental trade-off of hierarchical networks: the reduced number of routers and links also concentrates global traffic onto a relatively small set of high-level resources, making them easier to congest. The mesh distributes traffic across many more routers and therefore sustains a much higher load, reaching approximately 4.4 Gb/s/endpoint before entering a strongly congested regime. Beyond this point the current mesh model exhibits throughput degradation under overload.

8.3.2 Multicast Results

Latency

Figure 8.6 compares native multicast against repeated unicast. The left panel increases the LCA level while maintaining a destination density close to 0.8, whereas the right panel varies destination density within a fixed level-3 subtree.

8.3. Network-Level Characterization

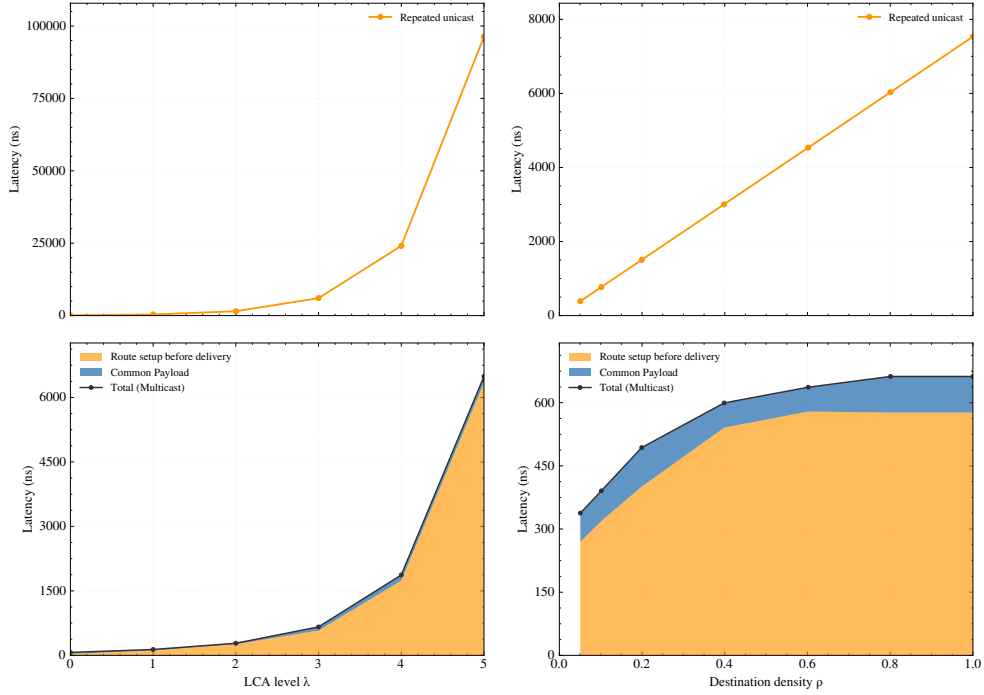


Figure 8.6: Multicast completion latency compared with repeated unicast. Left: increasing multicast-subtree size at approximately constant destination density. Right: increasing destination density for a fixed level-3 subtree.

The multicast latency grows approximately exponentially with the LCA level. Increasing λ by one expands the corresponding quad-tree subtree by a factor of four and, at approximately constant destination density, exponentially increases the number of internal nodes that must be represented in the multicast route. Since the route length grows approximately linearly with this number of nodes, the corresponding parsing overhead increases rapidly with λ . Moreover, due to the tree topology, the throughput normalized per destination effectively decreases by approximately a factor of four for each additional level that the multicast route must traverse downward. This further penalizes large values of λ and confirms that the topology is particularly well suited to locally dense multicast patterns.

As shown by the latency breakdown, multicast-route parsing is the dominant contribution to the total latency, whereas forwarding the common payload accounts for only a relatively small fraction. This is consistent with the physical characterization of the router and motivates the batching analysis presented in the following sections: when a long common payload is shared among many destinations, the route-parsing overhead can be amortized over a much larger amount of useful data.

Nevertheless, the latency growth remains substantially lower than for re-

peated unicast, thanks to payload replication inside the network. At $\lambda = 5$ and 3277 destinations, multicast completes in approximately $6.53 \mu\text{s}$, compared with $96.3 \mu\text{s}$ for repeated unicast, corresponding to a $14.8\times$ reduction.

Energy

The same trends are visible in Fig. 8.7.

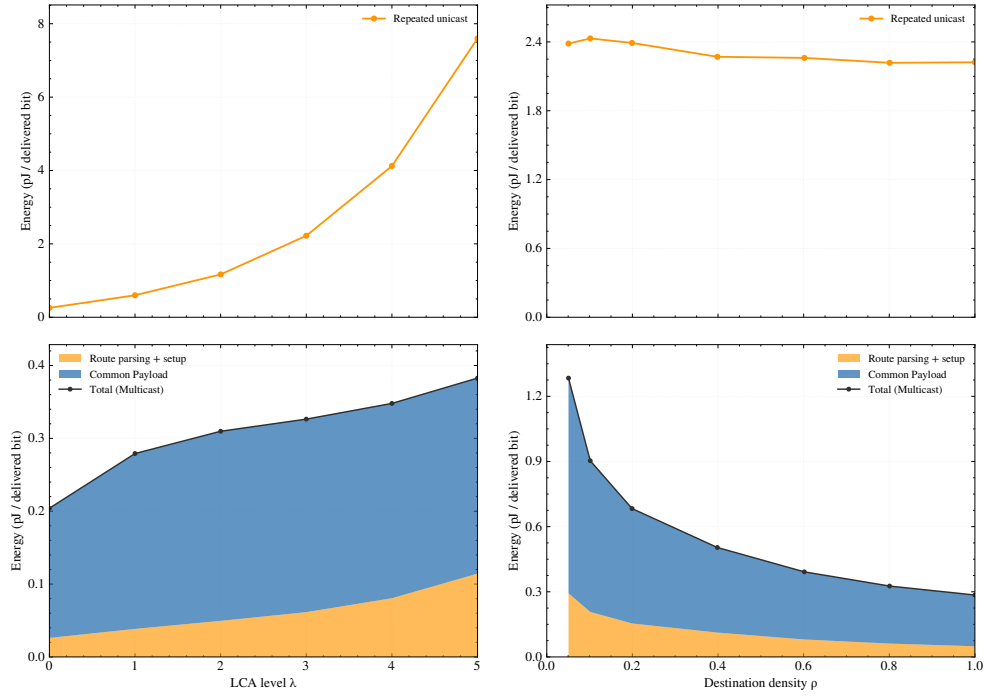


Figure 8.7: Dynamic energy per delivered bit for native multicast and repeated unicast as a function of multicast-subtree size and destination density.

Unlike latency, the dominant energy contribution comes from common-payload forwarding, while route parsing and setup account for a smaller fraction of the total. The decomposition shown in Fig. 8.7 should be interpreted purely as a breakdown of the total transaction energy: both contributions are accumulated independently and then normalized by the same number of delivered bits. It therefore does not imply that each delivered bit individually incurs both a “payload” and a “route” energy cost. Instead, the stacked areas show how much of the overall energy-per-delivered-bit metric originates from route processing and how much from forwarding the common payload. Although processing an individual route word is more energy intensive than forwarding a payload flit, the payload is replicated across multiple active branches and spans several flits, so its cumulative switching activity

dominates the total energy.

The route-parsing contribution increases with λ , since higher LCA levels require a longer multicast route and therefore more parsing activity. The payload contribution, however, remains comparatively stable because the payload length is fixed and its forwarding cost depends mainly on the number of payload flits. At the same time, increasing the destination density ρ increases the multicast fanout, allowing the same payload transmission to serve more destinations. Since the total energy is normalized by the number of delivered bits, this reduces the energy per delivered bit. This again confirms that the architecture is best suited to locally dense multicast traffic.

Compared with repeated unicast, the energy reduction reaches up to $19.8\times$, primarily thanks to in-network payload replication.

Throughput

Figure 8.8 evaluates continuous multicast traffic. Each active source broadcasts within one independent LCA subtree, corresponding to $\rho = 1$.

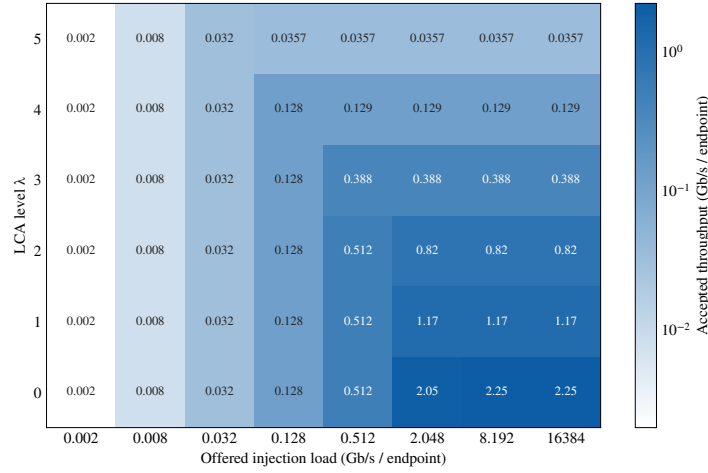


Figure 8.8: Accepted multicast throughput under continuous broadcast traffic within independent subtrees. The color scale reports delivered useful throughput per endpoint.

The sustainable throughput decreases as the multicast subtree expands. A level-0 subtree reaches approximately 2.25 Gb/s/endpoint, whereas a network-wide level-5 multicast saturates near 35.7 Mb/s/endpoint. Larger subtrees require more route parsing and progressively concentrate traffic on shared high-level resources. The experiment therefore illustrates again the central scalability trade-off of the tree: replication is efficient within localized

groups, whereas increasingly global communication reduces the effective injection rate, causing congestion.

8.4 Network Benchmarks

The application-driven benchmarks evaluate whether the multicast batching opportunities identified in the SNN communication traces can be translated into reductions in network latency and dynamic energy. Unlike the synthetic experiments of the previous section, these simulations preserve the temporal activity and connectivity of the trained neural networks. The same spike occurrences, physical mapping, and destination sets are presented to all three NoCs; only the packetization and communication mechanisms differ. In particular, the proposed tree can aggregate multiple source addresses that share the same destination set into a single multicast transaction, whereas the HBS tree and synchronous baseline transmit each multicast spike occurrence independently.

8.4.1 Naturally Occurring Multicast Batches

Before evaluating the NoC performance, we first quantify how frequently compatible spike occurrences arise naturally in the considered workloads. Figure 8.9 reports the distribution of natural-group sizes, where a natural group contains spike occurrences generated in the same sample and timestep by the same source tile and targeting exactly the same destination-tile set.

8.4. Network Benchmarks

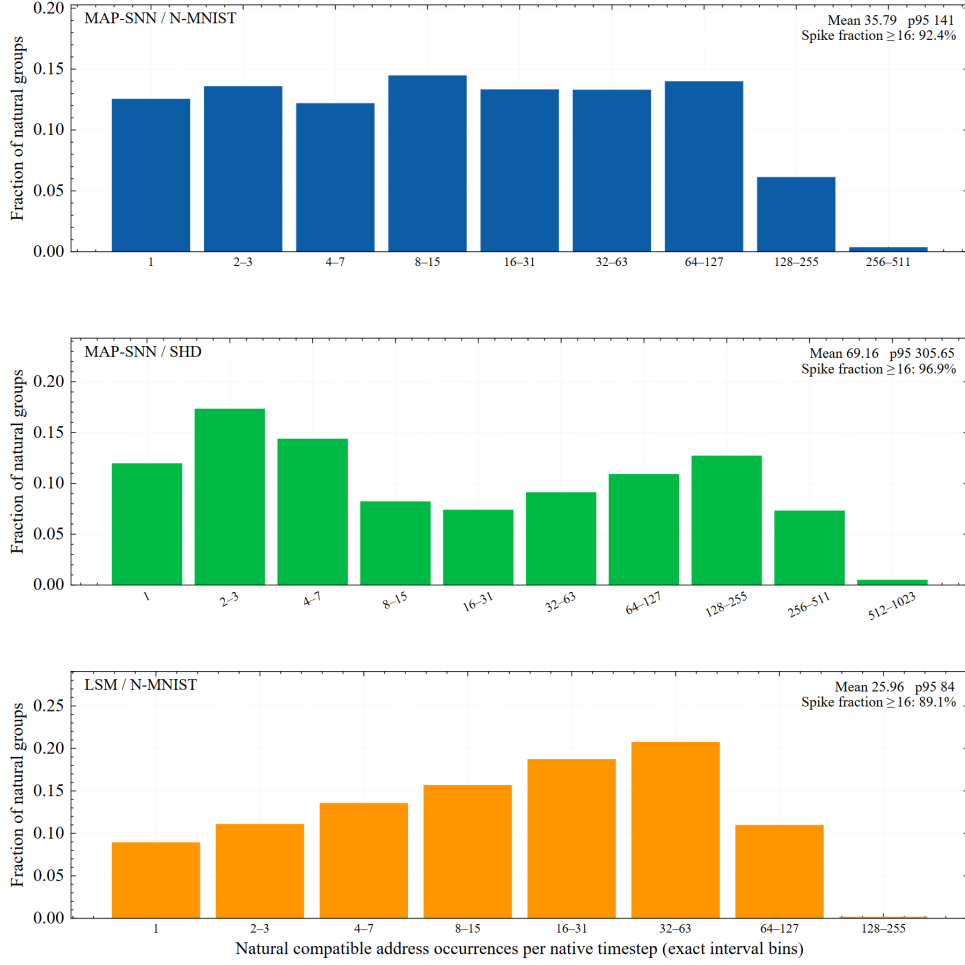


Figure 8.9: Distribution of naturally occurring multicast-group sizes for the three application workloads. A natural group contains spike occurrences from the same sample and timestep, originating from the same source tile and sharing exactly the same set of destination tiles.

Large compatible groups occur frequently in all three workloads. The mean natural-group size is 35.79 occurrences for MAP-SNN/N-MNIST, 69.16 for MAP-SNN/SHD, and 25.96 for LSM/N-MNIST. The corresponding 95th percentiles are 141, 305.65, and 84, respectively.

The group-count distribution alone does not fully capture the importance of these large batches. A group containing hundreds of addresses contributes only one entry to Fig. 8.9, despite representing substantially more network traffic than a group containing one or two addresses. Figure 8.10 therefore shows the same data weighted by the number of spike addresses contained in each group.

8.4. Network Benchmarks

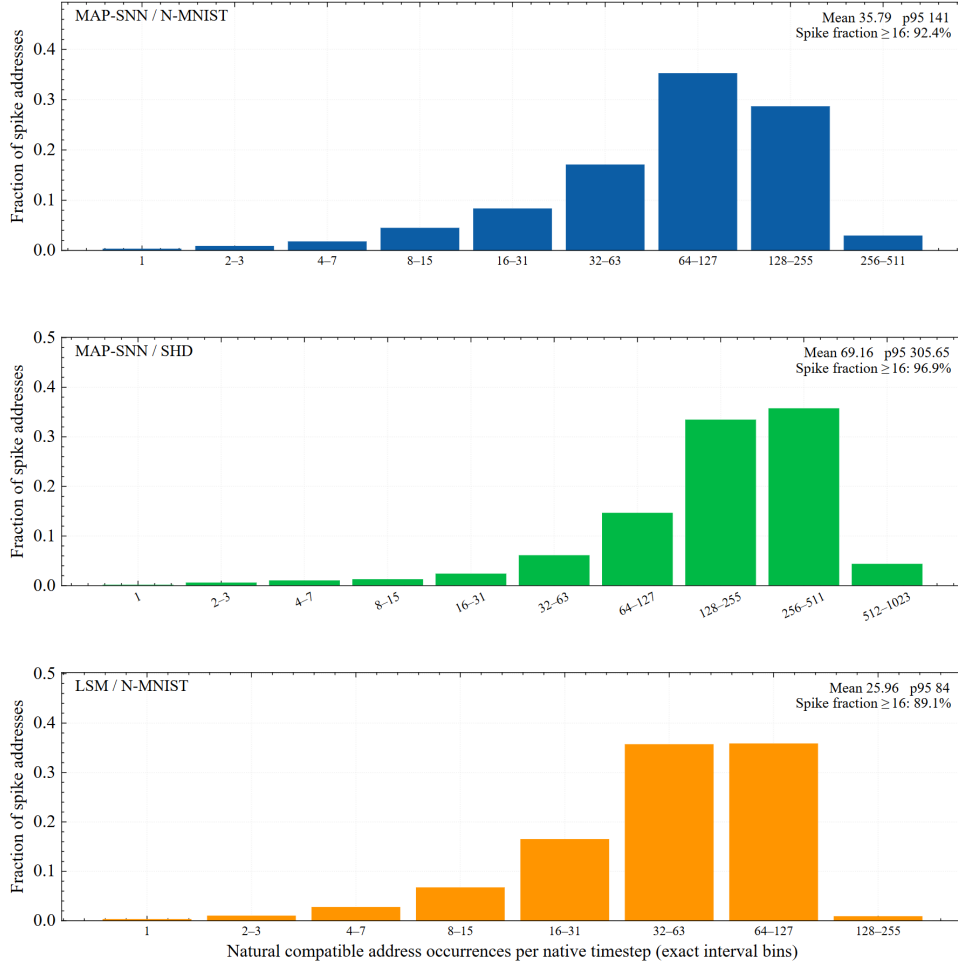


Figure 8.10: Spike-weighted distribution of naturally occurring multicast-group sizes. Each bin is weighted by the number of spike addresses carried by the corresponding natural groups and therefore represents the fraction of communication traffic associated with each group-size range.

From the traffic perspective, large groups dominate all three benchmarks. The fraction of spike addresses belonging to natural groups containing at least 16 occurrences is 92.4% for MAP-SNN/N-MNIST, 96.9% for MAP-SNN/SHD, and 89.1% for LSM/N-MNIST. Thus, although small groups are not uncommon when counting groups individually, the overwhelming majority of actual spike traffic belongs to groups large enough to benefit from batching. This distinction is important for the proposed architecture. Increasing $B_{\max}$ does not require the neural workload itself to become more synchronous or artificially combine unrelated communication. It simply allows a larger fraction of spike occurrences that are already naturally compatible to share the same multicast route and packet-level control overhead.

8.4.2 Inference Latency

Figure 8.11 shows the accumulated NoC communication latency per inference as $B_{\max}$ is increased from one to 64 addresses per packet.

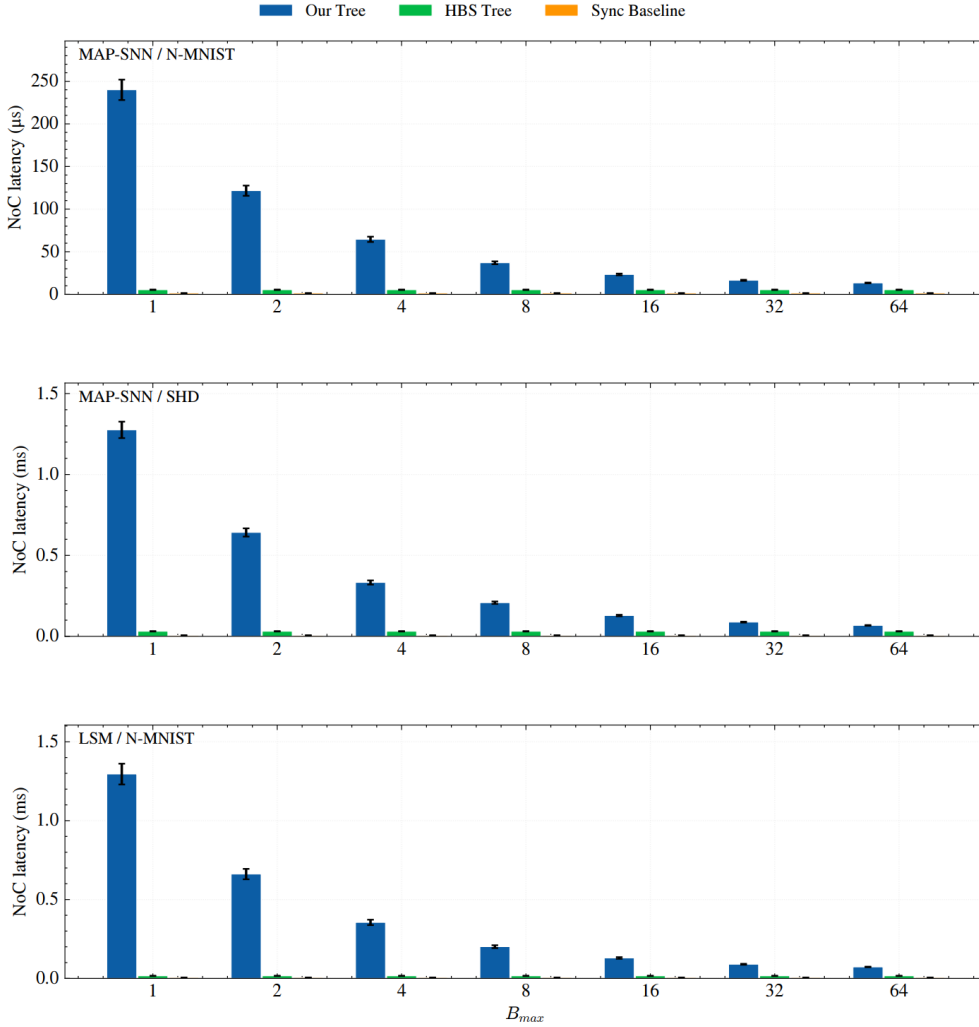


Figure 8.11: NoC communication latency per inference as a function of the maximum multicast batch size $B_{\max}$. The proposed tree can aggregate compatible source addresses into a common multicast transaction, while the HBS tree and synchronous baseline transmit individual spike occurrences independently.

When $B_{\max} = 1$, batching is effectively disabled, and every compatible source address generates a separate multicast transaction. Under this condition, the proposed tree exhibits substantially higher latency than both reference architectures. This behavior is consistent with the results presented in Section 8.3.2: each multicast transaction incurs route-processing and recovery over-

head, and the relatively large initiation interval of the current implementation becomes particularly expensive when repeated independently for every spike occurrence.

Increasing $B_{\max}$ strongly reduces this overhead. Several source addresses can now reuse one multicast destination description, so fewer packets must traverse the network and fewer multicast routes must be parsed. The latency of the proposed tree consequently decreases by more than one order of magnitude between the smallest and largest batching configurations for all three workloads. The reduction is especially pronounced at small and intermediate values of $B_{\max}$. Increasing the limit from one address to two, four, or eight removes a large number of independent multicast transactions. At larger batch sizes, the improvement progressively diminishes. This behavior follows directly from the natural-group distributions in Figs. 8.9 and 8.10: once $B_{\max}$ approaches the size of a significant fraction of the naturally occurring groups, further increases can reduce the packet count only for the remaining larger groups. In addition, the benefit of increasing the batch size is fundamentally limited by the payload cost itself. Larger batches amortize the fixed multicast-route overhead over more addresses, but this improvement is ultimately bounded by the cost of transmitting the payload flits themselves.

The HBS and synchronous baselines are essentially independent of $B_{\max}$, since batching is not applied to their packet formats in this comparison. Their latency therefore remains constant across the sweep. Even at the largest evaluated batch size, the proposed implementation remains consistently slower in absolute inference latency. This is mainly due to the relatively long initiation interval of the current router implementation, whose 32-bit datapath was not decomposed through bit slicing and whose PRS was not exhaustively optimized for cycle time. The reference routers also use published post-layout 22-nm parameters, whereas the proposed router is characterized in predictive 65-nm technology. The most relevant observation is therefore the strong latency reduction obtained within the proposed architecture as multicast setup is amortized over increasingly large naturally occurring batches.

At $B_{\max} = 64$, batching substantially reduces the latency of the proposed tree, although the current implementation remains slower than HBS for all three workloads. Table 8.7 summarizes the corresponding values. The last column reports the latency overhead of the proposed tree relative to HBS.

Table 8.7: NoC communication latency per inference at $B_{\max} = 64$. The last column reports the latency overhead of the proposed tree relative to HBS.

Workload	Our Tree	HBS Tree	Sync Baseline	ΔL
MAP-SNN / N-MNIST	13.5 μs	5.4 μs	1.36 μs	150%
MAP-SNN / SHD	68.1 μs	31.4 μs	6.2 μs	117%
LSM / N-MNIST	73.0 μs	15.9 μs	5.5 μs	359%

Thus, at $B_{\max} = 64$, the proposed tree exhibits approximately 150%, 117%, and 359% higher latency than HBS for MAP-SNN/N-MNIST, MAP-SNN/SHD, and LSM/N-MNIST, respectively.

8.4.3 Inference Energy

The effect of batching on dynamic communication energy is shown in Fig. 8.12.

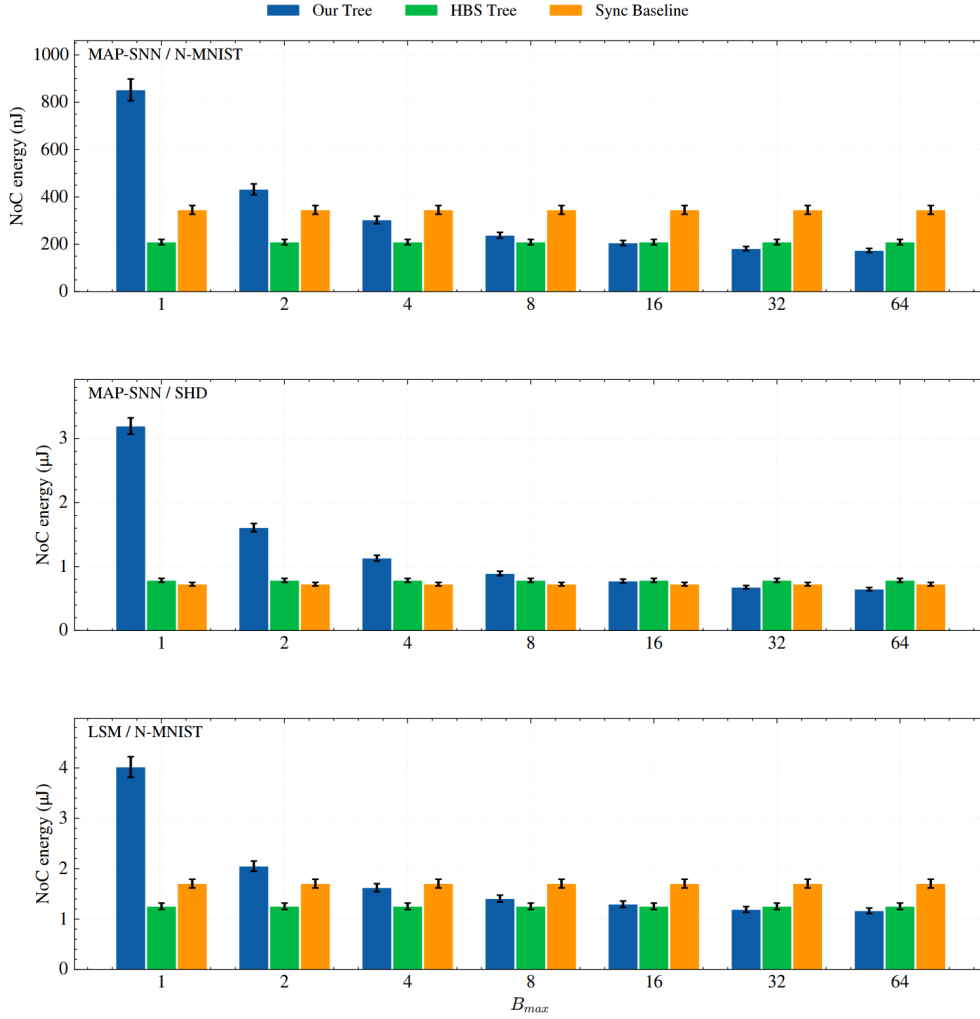


Figure 8.12: Dynamic NoC energy per inference as a function of the maximum multicast batch size $B_{\max}$. Batching allows the proposed tree to amortize route generation, parsing, and shared-path communication over multiple compatible source addresses.

The energy trend follows the same fundamental mechanism observed for latency. At $B_{\max} = 1$, every spike occurrence incurs an independent multicast transaction, repeatedly activating the route-processing datapath and paying the corresponding router and interconnect costs. The proposed tree is therefore more expensive than the reference architectures in this regime. As $B_{\max}$ increases, the number of multicast routes that must be transported and parsed decreases rapidly. Route setup, traversal of common tree segments can then be amortized across several source addresses. Consequently, the energy of the proposed tree falls monotonically with increasing $B_{\max}$ for all three workloads, while the energy of the two reference NoCs remains

constant.

The main energy advantage of the proposed encoding is that the common payload is forwarded only along branches that actually contain destinations. In contrast, the HBS representation can activate false-positive branches, causing the payload to be unnecessarily transported through routers and links that do not lead to any intended destination. At $B_{\max} = 64$, the proposed tree becomes the lowest-energy solution for all three workloads. Table 8.8 summarizes the corresponding values. To make the comparison explicit, the last column reports the ratio $E_{\text{HBS}}/E_{\text{Our}}$, i.e., how much more energy the HBS tree consumes for the same inference.

Table 8.8: Dynamic NoC energy per inference at $B_{\max} = 64$. The last column reports the energy reduction of the proposed tree relative to HBS.

Workload	Our Tree	HBS Tree	Sync Baseline	ΔE
MAP-SNN / N-MNIST	0.170 μJ	0.205 μJ	0.343 μJ	17%
MAP-SNN / SHD	0.64 μJ	0.78 μJ	0.71 μJ	18%
LSM / N-MNIST	1.15 μJ	1.24 μJ	1.70 μJ	7%

At the largest evaluated batch size, the proposed tree reduces dynamic NoC energy relative to HBS by approximately 17%, 18%, and 7% for MAP-SNN/N-MNIST, MAP-SNN/SHD, and LSM/N-MNIST, respectively. Importantly, this advantage is obtained despite the proposed router being characterized in predictive 65-nm technology, while the HBS parameters originate from a post-layout 22-nm implementation. The comparison is therefore not intended as a technology-normalized circuit-efficiency result, but rather to show that the reduction in network activity enabled by the proposed multicast encoding and batching mechanism is large enough to offset the disadvantages of a non-optimized, older-technology circuit implementation.

8.5 Discussion

In this chapter, the proposed NoC was evaluated from three complementary perspectives: transistor-level router characterization, synthetic network-level characterization, and application-driven benchmarking. Together, these evaluations provide a view of the architecture from the individual circuit blocks up to complete network behavior and realistic application workloads.

The router characterization highlighted both the strengths and the main limitations of the current circuit implementation. In particular, forward propagation remains relatively short, with unicast payload latencies between approximately 0.53 and 0.82 ns, while the initiation interval is significantly larger, reaching roughly 2.5–3.5 ns for unicast traffic and more than 4 ns for

some multicast payload paths. Multicast-route parsing is considerably more expensive, requiring 23.1–51.3 ns for continuation words and up to 74.9 ns for the initial route word. As discussed in Section 8.2, these values should not be interpreted as architectural limits. The current 32-bit dual-rail datapath was not decomposed through bit slicing because of project-time constraints, resulting in large completion-detection trees whose cycle time grows with datapath width. Moreover, the final PRS was optimized primarily for correctness, CMOS implementability, and bubble reshuffling rather than minimum cycle time. The measured performance should therefore be regarded as representative of the current implementation rather than of the ultimate performance potential of the proposed architecture, leaving significant room for circuit-level optimization.

The network-level characterization exposed the fundamental architectural trade-offs of the hierarchical topology. For unicast traffic, the logarithmic growth in router traversals makes the tree particularly attractive for sparse long-distance communication. As shown in Fig. 8.3, at the largest evaluated distance of 63 tile steps, the proposed tree completes an eight-flit transmission in approximately 45.5 ns, compared with 82 ns for the synchronous mesh, despite the relatively large initiation interval of the current router. At the same time, the experiments also exposed the main limitations of the topology. At long distances, upper-level H-tree links dominate the communication energy: at the largest simulated distance, 7.62 pJ/bit of the total 8.78 pJ/bit, or approximately 87%, is attributed to the modeled interconnect, as shown in Fig. 8.4. Furthermore, the progressive reduction in the number of available resources toward the root concentrates global traffic and causes the tree to saturate at approximately 7.2 Mb/s/endpoint, compared with roughly 4.4 Gb/s/endpoint for the mesh, as discussed in Fig. 8.5. The multicast experiments reveal the complementary operating regime in which the architecture is most effective. As discussed in Section 8.3.2, route-parsing latency grows rapidly with the LCA level because increasingly large subtrees require increasingly long hierarchical routes, whereas increasing destination density allows the same route and payload to serve a larger number of endpoints. Consequently, the proposed topology is best matched to spatially localized and dense multicast traffic. Even with the current parser overhead, native multicast already provides large gains over repeated unicast, reaching a $14.8\times$ latency reduction and up to a $19.8\times$ reduction in dynamic energy in the evaluated synthetic experiments.

Finally, the application-driven benchmarks demonstrate the main advantage of the proposed multicast encoding: its routing overhead can be amortized across multiple source addresses through batching. This aspect is particularly important because the workloads naturally provide large groups of compatible messages. As shown in Figs. 8.9 and 8.10, 92.4%, 96.9%, and 89.1% of all spike addresses in MAP-SNN/N-MNIST, MAP-SNN/SHD, and

LSM/N-MNIST, respectively, belong to natural multicast groups containing at least 16 occurrences. With $B_{\max} = 1$, the current implementation performs poorly because every source address independently pays the relatively expensive multicast-route setup and parsing cost. Increasing $B_{\max}$ progressively amortizes this fixed overhead, reducing the proposed tree latency by more than one order of magnitude between $B_{\max} = 1$ and $B_{\max} = 64$ for all three workloads, as shown in Fig. 8.11. More importantly, the same mechanism strongly reduces dynamic communication energy. At $B_{\max} = 64$, the proposed architecture reaches $0.170 \mu\text{J}$, $0.64 \mu\text{J}$, and $1.15 \mu\text{J}$ per inference for MAP-SNN/N-MNIST, MAP-SNN/SHD, and LSM/N-MNIST, respectively, outperforming HBS by approximately 17%, 18%, and 7%, as summarized in Table 8.8. These results are particularly significant because the proposed router is characterized using predictive 65-nm models and is not aggressively optimized for cycle time, whereas the HBS reference uses published post-layout 22-nm parameters. The fact that the architectural reduction in network activity is sufficient to compensate for this implementation disadvantage provides strong evidence for the effectiveness of the proposed encoding scheme when combined with batching techniques, while also suggesting that a more optimized circuit implementation could further improve its competitiveness.

Conclusion

This thesis presented a fully asynchronous quaternary-tree Network-on-Chip with native support for exact multicast communication. The proposed routing scheme represents the active multicast subtree hierarchically and progressively repartitions the route as the packet descends the tree. This allows arbitrary destination sets to be addressed without false-positive deliveries, while making the routing overhead depend on the number and spatial structure of the active destinations rather than on the total network size. A batching mechanism further allows multiple compatible source addresses to share the same multicast route, amortizing its generation, transport, and parsing cost over a longer common payload.

The evaluation showed that the main limitation of the current implementation is the cost of route parsing and the relatively large initiation interval of the 32-bit dual-rail datapath. Nevertheless, the architectural advantages remain clearly visible. In particular, batching reduces the communication latency of the proposed NoC by more than one order of magnitude between $B_{\max} = 1$ and $B_{\max} = 64$ for all evaluated SNN workloads. At $B_{\max} = 64$, the proposed architecture also reduces dynamic communication energy relative to HBS by approximately 17%, 18%, and 7% for MAP-SNN/N-MNIST, MAP-SNN/SHD, and LSM/N-MNIST, respectively. These results are obtained despite the proposed router being characterized using predictive 65-nm models, while the HBS reference is based on a post-layout 22-nm implementation.

Overall, the results support the central hypothesis of this work: exact multicast routing can remain energy-efficient when the routing information grows only with the active portion of the tree and its overhead can be amortized over sufficiently large batches. The proposed approach is therefore particularly well suited to globally sparse but locally dense communication patterns, such as those arising in event-driven neuromorphic workloads.

9.1 Limitations and Future Work

The main limitations of this work arise from the current circuit implementation and from the evaluation methodology. The 32-bit dual-rail datapath was not decomposed through bit slicing, and the final PRS was optimized primarily for correctness, CMOS implementability, and bubble reshuffling rather than minimum cycle time. In addition, the router was characterized using predictive 65-nm transistor models rather than a complete post-layout implementation, while interconnect delay and energy were modeled analytically.

Future work should focus on reducing the initiation interval and multicast-route parsing cost through bit-sliced datapath decomposition and further PRS optimization. A complete physical implementation in a modern technology node would also allow extracted interconnect parasitics, actual area, and post-layout timing and energy to be evaluated directly. At network level, further improvements could investigate strategies to mitigate congestion near the upper levels of the tree, such as the use of a fat-tree topology, and strategies to increase the batching and locality opportunities exposed by application mapping.

Bibliography

- [1] W. J. Dally and B. Towles, “Route packets, not wires: On-chip interconnection networks,” in *Proceedings of the 38th Annual Design Automation Conference*, ACM, 2001, pp. 684–689. doi: [10.1145/378239.379048](https://doi.org/10.1145/378239.379048)
- [2] L. Benini and G. De Micheli, “Networks on chips: A new SoC paradigm,” *Computer*, vol. 35, no. 1, pp. 70–78, Jan. 2002. doi: [10.1109/2.976921](https://doi.org/10.1109/2.976921)
- [3] R. Manohar, *Asynchronous VLSI systems*, Lecture notes on Asynchronous VLSI Design, 2007.
- [4] P. Merolla, J. V. Arthur, R. Alvarez-Icaza, J.-M. Bussat, and K. Boahen, “A multicast tree router for multichip neuromorphic systems,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 61, no. 3, pp. 820–833, Mar. 2014. doi: [10.1109/TCSI.2013.2284184](https://doi.org/10.1109/TCSI.2013.2284184)
- [5] N. D. Enright Jerger, L.-S. Peh, and M. H. Lipasti, “Virtual circuit tree multicasting: A case for on-chip hardware multicast support,” in *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, IEEE Computer Society, 2008, pp. 229–240. doi: [10.1109/ISCA.2008.12](https://doi.org/10.1109/ISCA.2008.12)
- [6] K. Bhardwaj and S. M. Nowick, “A continuous-time replication strategy for efficient multicast in asynchronous NoCs,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 2, pp. 350–363, Feb. 2019. doi: [10.1109/TVLSI.2018.2876856](https://doi.org/10.1109/TVLSI.2018.2876856)
- [7] Z. Su, A. Bencsik, G. Indiveri, and D. Bertozzi, “An efficient multicast addressing encoding scheme for multi-core neuromorphic processors,” in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, IEEE, May 2025. doi: [10.1109/ISCAS56072.2025.11043591](https://doi.org/10.1109/ISCAS56072.2025.11043591)
- [8] A. J. Martin, “Compiling communicating processes into delay-insensitive VLSI circuits,” *Distributed Computing*, vol. 1, no. 4, pp. 226–234, 1986. doi: [10.1007/BF01660034](https://doi.org/10.1007/BF01660034)

-
- [9] S. Ataei et al., "An open-source EDA flow for asynchronous logic," *IEEE Design & Test*, vol. 38, no. 2, pp. 27–37, Apr. 2021. doi: [10.1109/MDAT.2021.3051334](https://doi.org/10.1109/MDAT.2021.3051334)
 - [10] N. Mysore, G. Hota, S. R. Deiss, B. U. Pedroni, and G. Cauwenberghs, "Hierarchical network connectivity and partitioning for reconfigurable large-scale neuromorphic systems," *Frontiers in Neuroscience*, vol. 15, p. 797654, 2022. doi: [10.3389/fnins.2021.797654](https://doi.org/10.3389/fnins.2021.797654)
 - [11] T. Dalgaty et al., "Mosaic: In-memory computing and routing for small-world spike-based neuromorphic systems," *Nature Communications*, vol. 15, no. 1, p. 142, 2024. doi: [10.1038/s41467-023-44365-x](https://doi.org/10.1038/s41467-023-44365-x)
 - [12] S. Billaudelle et al., "Small-world communication fabrics for neuromorphic multicore-SoCs," *arXiv preprint arXiv:2608.24351*, 2026. doi: [10.48550/arXiv.2608.24351](https://doi.org/10.48550/arXiv.2608.24351)
 - [13] L. Wang, Y. Jin, H. Kim, and E. J. Kim, "Recursive partitioning multicast: A bandwidth-efficient routing for networks-on-chip," in *2009 3rd ACM/IEEE International Symposium on Networks-on-Chip (NoCS)*, IEEE, May 2009, pp. 64–73. doi: [10.1109/NOCS.2009.5071446](https://doi.org/10.1109/NOCS.2009.5071446)
 - [14] J. Sparsø and S. Furber, Eds., *Principles of Asynchronous Circuit Design: A Systems Perspective*. Springer, 2001, ISBN: 978-0-7923-7613-2. doi: [10.1007/978-1-4757-3385-3](https://doi.org/10.1007/978-1-4757-3385-3)
 - [15] L. Liu and K. Boahen, "Hierarchical event readout with asynchronous pipelined opportunistic merges," in *2025 29th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*, IEEE, 2025, pp. 108–117. doi: [10.1109/ASYNC65240.2025.00023](https://doi.org/10.1109/ASYNC65240.2025.00023)
 - [16] R. Manohar and Y. Moses, "Analyzing isochronic forks with potential causality," in *2015 21st IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*, IEEE, 2015, pp. 69–76. doi: [10.1109/ASYNC.2015.19](https://doi.org/10.1109/ASYNC.2015.19)
 - [17] A. M. Lines, "Pipelined asynchronous circuits," California Institute of Technology, Tech. Rep. CaltechCSTR:1998.cs-tr-95-21, 1998, June 1995, revised June 1998. doi: [10.7907/Z92V2D4Z](https://doi.org/10.7907/Z92V2D4Z)
 - [18] S. R. Vangal et al., "An 80-tile sub-100-w teraflops processor in 65-nm CMOS," *IEEE Journal of Solid-State Circuits*, vol. 43, no. 1, pp. 29–41, Jan. 2008. doi: [10.1109/JSSC.2007.910957](https://doi.org/10.1109/JSSC.2007.910957)
 - [19] Z. Su et al., "An ultra-low cost and multicast-enabled asynchronous NoC for neuromorphic edge computing," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 14, no. 3, pp. 409–424, Sep. 2024. doi: [10.1109/JETCAS.2024.3433427](https://doi.org/10.1109/JETCAS.2024.3433427)

Appendix A

Common Building Blocks

This appendix presents the common building blocks used throughout the design.

Notation used in the appendices. Signal names are local to each module. The variable s denotes the main initialization or operating mode, and p denotes a selected port or path. The family $f_0, f_1, \dots$ is reserved for auxiliary state variables inserted to distinguish control states or simplify PRS guards. In Manohar’s terminology, eliminating indistinguishable HSE states is *state-variable insertion*; the f naming is the convention used here. Their precise roles are stated locally; equal subscripts in different modules do not imply identical timing. In the *Sequencer* and *WordBuffer*, the family $q_0, q_1, \dots$ records progress through the handshake sequence; auxiliary control states in other blocks use $f_0, f_1, \dots$. Stored data, counters, and explicit completion-tree nodes retain their functional names. In arithmetic blocks, s_t, s_f remain the sum rails rather than a control-state encoding.

The subscripts t and f identify true and false rails. Thus o_{jtk} is the true rail of bit k on output O_j , and o_{ja} is that port’s acknowledge signal. The rail index $\rho \in \{t, f\}$ abbreviates both rails only when the stated rule or constraint applies to both. An underscore prefix, as in $_x$, names the complemented physical signal; $\neg x$ is a Boolean negation. The guards G_x^+ and G_x^- control the assertion and deassertion of x in the polarity under discussion.

In fork descriptions, P_d and P_i denote the direct branch and its competing indirect path, respectively, and $\tau(P)$ denotes a path delay. These names are local to each constraint. The labels $V(I)$ and $N(I)$ denote input-valid and all-rails-spacer events, not additional circuit signals. Replication retains the source notation: a comma denotes parallel composition and a semicolon denotes sequential composition; changing a line break does not change either.

A.1 Reg4

This block is a dual-rail 4-bit register.

A.1.1 PRS

Port W writes the register, port R reads it, and $x_{3:0}$ stores the value.

$$\begin{aligned}
w_{tk} &\rightarrow x_k \uparrow & k=0,\dots,3 \\
w_{fk} \vee r &\rightarrow x_k \downarrow & k=0,\dots,3 \\
r_r \wedge \neg x_k &\rightarrow r_{fk} \uparrow & k=0,\dots,3 \\
\neg r_r &\rightarrow r_{fk} \downarrow & k=0,\dots,3 \\
r_r \wedge x_k &\rightarrow r_{tk} \uparrow & k=0,\dots,3 \\
\neg r_r &\rightarrow r_{tk} \downarrow & k=0,\dots,3 \\
((w_{t3} \wedge x_3) \vee (w_{f3} \wedge \neg x_3)) \wedge \dots \wedge ((w_{t0} \wedge x_0) \vee (w_{f0} \wedge \neg x_0)) &\rightarrow w_a \uparrow \\
\neg w_{t3} \wedge \neg w_{f3} \wedge \dots \wedge \neg w_{t0} \wedge \neg w_{f0} &\rightarrow w_a \downarrow
\end{aligned}$$

A.1.2 Bubble Reshuffling

There are no isochronic-fork constraints for this register. We retain the data rails and stored bits in their original sense, represent w_a in complemented form, and generate the complementary views needed by the read and write guards.

$$\begin{aligned}
\neg w_{tk} &\rightarrow x_k \uparrow & k=0,\dots,3 \\
w_{fk} \vee r &\rightarrow x_k \downarrow & k=0,\dots,3 \\
\neg r_r \wedge \neg x_k &\rightarrow r_{fk} \uparrow & k=0,\dots,3 \\
r_r &\rightarrow r_{fk} \downarrow & k=0,\dots,3 \\
\neg r_r \wedge \neg x_k &\rightarrow r_{tk} \uparrow & k=0,\dots,3 \\
r_r &\rightarrow r_{tk} \downarrow & k=0,\dots,3 \\
(((w_{t3} \wedge x_3) \vee (w_{f3} \wedge \neg x_3)) \wedge \dots \wedge ((w_{t0} \wedge x_0) \vee (w_{f0} \wedge \neg x_0))) &\rightarrow \neg w_a \downarrow \\
\neg w_{t3} \wedge \neg w_{f3} \wedge \dots \wedge \neg w_{t0} \wedge \neg w_{f0} &\rightarrow \neg w_a \uparrow \\
\neg r_r &\Rightarrow r_r \\
\neg x_k &\Rightarrow x_k & k=0,\dots,3 \\
\neg w_a &\Rightarrow w_a \\
\neg w_{tk} &\Rightarrow w_{tk} & k=0,\dots,3
\end{aligned}$$

A.2 Reg8

The *Reg8* extends the *Reg4* to eight stored bits.

A.2.1 PRS

We present the PRS directly. Here, port W is the input, R is the output, and $x_{7:0}$ stores the internal value.

$$\begin{array}{ll}
w_{tk} \rightarrow x_k \uparrow & k=0,\dots,7 \\
w_{fk} \vee r \rightarrow x_k \downarrow & k=0,\dots,7 \\
r_r \wedge \neg x_k \rightarrow r_{fk} \uparrow & k=0,\dots,7 \\
\neg r_r \rightarrow r_{fk} \downarrow & k=0,\dots,7 \\
r_r \wedge x_k \rightarrow r_{tk} \uparrow & k=0,\dots,7 \\
\neg r_r \rightarrow r_{tk} \downarrow & k=0,\dots,7 \\
(((w_{t7} \wedge x_7) \vee (w_{f7} \wedge \neg x_7)) \wedge \dots \wedge ((w_{t0} \wedge x_0) \vee (w_{f0} \wedge \neg x_0))) \rightarrow w_a \uparrow \\
\neg w_{t7} \wedge \neg w_{f7} \wedge \dots \wedge \neg w_{t0} \wedge \neg w_{f0} \rightarrow w_a \downarrow
\end{array}$$

A.2.2 Bubble Reshuffling

There are no isochronic-fork constraints for this register.

We preserve the input rails, the output rails of R , and the stored bits x_k . The read request r_r and write acknowledge w_a are represented in complemented form; explicit inverters provide the other polarities where required. The resulting PRS is:

$$\begin{array}{ll}
w_{tk} \rightarrow x_k \uparrow & k=0,\dots,7 \\
w_{fk} \vee r \rightarrow x_k \downarrow & k=0,\dots,7 \\
\neg r_r \wedge \neg x_k \rightarrow r_{fk} \uparrow & k=0,\dots,7 \\
r_r \rightarrow r_{fk} \downarrow & k=0,\dots,7 \\
\neg r_r \wedge x_k \rightarrow r_{tk} \uparrow & k=0,\dots,7 \\
r_r \rightarrow r_{tk} \downarrow & k=0,\dots,7 \\
(((w_{t7} \wedge x_7) \vee (w_{f7} \wedge \neg x_7)) \wedge \dots \wedge ((w_{t0} \wedge x_0) \vee (w_{f0} \wedge \neg x_0))) \rightarrow \neg w_a \downarrow \\
\neg w_{t7} \wedge \neg w_{f7} \wedge \dots \wedge \neg w_{t0} \wedge \neg w_{f0} \rightarrow \neg w_a \uparrow
\end{array}$$

At this point, the polarities of the signals on the right-hand side are fixed. We therefore only need to adjust the guards, introducing explicit inverters whenever the opposite polarity of a polarity-fixed signal is required. Finally, an inverter is added to w_a to restore its original external polarity. The write-capture condition is implemented with a completion tree rooted at c_{all} ; its rules are given before the remaining PRS.

$$\begin{array}{ll}
((w_{tk} \wedge x_k) \vee (w_{fk} \wedge \neg x_k)) \rightarrow \neg c_k \downarrow & k=0,\dots,7 \\
\neg w_{tk} \wedge \neg w_{fk} \rightarrow \neg c_k \uparrow & k=0,\dots,7 \\
\neg c_{2k+1} \wedge \neg c_{2k} \rightarrow d_k \uparrow & k=0,\dots,3 \\
\neg c_{2k+1} \wedge \neg c_{2k} \rightarrow d_k \downarrow & k=0,\dots,3 \\
d_{2k+1} \wedge d_{2k} \rightarrow e_k \downarrow & k=0,1 \\
\neg d_{2k+1} \wedge \neg d_{2k} \rightarrow e_k \uparrow & k=0,1 \\
\neg e_1 \wedge \neg e_0 \rightarrow c_{\text{all}} \uparrow \\
e_1 \wedge e_0 \rightarrow c_{\text{all}} \downarrow
\end{array}$$

$$\begin{array}{ll}
\neg w_{tk} \rightarrow x_k \uparrow & k=0,\dots,7 \\
w_{fk} \vee r \rightarrow x_k \downarrow & k=0,\dots,7 \\
\neg r_r \wedge \neg x_k \rightarrow r_{fk} \uparrow & k=0,\dots,7 \\
r_r \rightarrow r_{fk} \downarrow & k=0,\dots,7 \\
\neg r_r \wedge \neg x_k \rightarrow r_{tk} \uparrow & k=0,\dots,7 \\
r_r \rightarrow r_{tk} \downarrow & k=0,\dots,7 \\
c_{all} \rightarrow w_a \downarrow & \\
\neg c_{all} \rightarrow w_a \uparrow & \\
\neg r_r \Rightarrow r_r & \\
\neg x_k \Rightarrow x_k & k=0,\dots,7 \\
\neg w_a \Rightarrow w_a & \\
\neg w_{tk} \Rightarrow w_{tk} & k=0,\dots,7
\end{array}$$

A.3 FullAdder

This block is a simple dual-rail asynchronous implementation of a full adder.

A.3.1 PRS

We present the PRS directly. Here, the two input bits are a and b , while c and d are the carry-in and carry-out, respectively, and s is the sum.

$$\begin{array}{l}
(a_f \wedge b_f) \vee ((a_f \vee b_f) \wedge c_f) \rightarrow d_f \uparrow \\
\neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow d_f \downarrow \\
(a_t \wedge b_t) \vee ((a_t \vee b_t) \wedge c_t) \rightarrow d_t \uparrow \\
\neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow d_t \downarrow \\
c_f \wedge ((a_t \wedge b_t) \vee (a_f \wedge b_f)) \vee c_t \wedge ((a_t \wedge b_f) \vee (a_f \wedge b_t)) \rightarrow s_f \uparrow \\
\neg c_t \wedge \neg c_f \rightarrow s_f \downarrow \\
c_t \wedge ((a_t \wedge b_t) \vee (a_f \wedge b_f)) \vee (c_f \wedge ((a_t \wedge b_f) \vee (a_f \wedge b_t))) \rightarrow s_t \uparrow \\
\neg c_t \wedge \neg c_f \rightarrow s_t \downarrow
\end{array}$$

A.3.2 Bubble Reshuffling

The *FullAdder* contains no isochronic forks, so there are no fork-critical polarity constraints. We preserve all the input rails in their original sense and implement the output rails d_t, d_f, s_t, s_f in complemented form. With this choice, all pull-down guards contain only non-negated literals and all pull-up guards contain only negated literals. Since the output rails are externally visible, their original polarities are restored before they leave the module.

$$\begin{aligned}
& (a_f \wedge b_f) \vee ((a_f \vee b_f) \wedge c_f) \rightarrow \neg d_f \downarrow \\
& \neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow \neg d_f \uparrow \\
& (a_t \wedge b_t) \vee ((a_t \vee b_t) \wedge c_t) \rightarrow \neg d_t \downarrow \\
& \neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow \neg d_t \uparrow \\
& c_f \wedge ((a_t \wedge b_t) \vee (a_f \wedge b_f)) \vee c_t \wedge ((a_t \wedge b_f) \vee (a_f \wedge b_t)) \rightarrow \neg s_f \downarrow \\
& \neg c_t \wedge \neg c_f \rightarrow \neg s_f \uparrow \\
& c_t \wedge ((a_t \wedge b_t) \vee (a_f \wedge b_f)) \vee (c_f \wedge ((a_t \wedge b_f) \vee (a_f \wedge b_t))) \rightarrow \neg s_t \downarrow \\
& \neg c_t \wedge \neg c_f \rightarrow \neg s_t \uparrow \\
& \neg d_f \Rightarrow d_f \\
& \neg d_t \Rightarrow d_t \\
& \neg s_f \Rightarrow s_f \\
& \neg s_t \Rightarrow s_t
\end{aligned}$$

A.4 HalfAdder

The block is a simple dual-rail implementation of a half adder.

A.4.1 PRS

We present the PRS directly. Here, the two input bits are a and b , while d is the carry output and s is the sum.

$$\begin{aligned}
& a_f \vee b_f \rightarrow d_f \uparrow \\
& \neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow d_f \downarrow \\
& a_t \wedge b_t \rightarrow d_t \uparrow \\
& \neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow d_t \downarrow \\
& (a_t \wedge b_t) \vee (a_f \wedge b_f) \rightarrow s_f \uparrow \\
& \neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow s_f \downarrow \\
& (a_t \wedge b_f) \vee (a_f \wedge b_t) \rightarrow s_t \uparrow \\
& \neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f \rightarrow s_t \downarrow
\end{aligned}$$

A.4.2 Bubble Reshuffling

The *HalfAdder* contains no isochronic forks, so there are no fork-critical polarity constraints. We preserve all the input rails in their original sense and implement the output rails d_t, d_f, s_t, s_f in complemented form. With this choice, all pull-down guards contain only non-negated literals and all pull-up guards contain only negated literals. Since the output rails are externally visible, their original polarities are restored before they leave the module.

$$\begin{aligned}
a_f \vee b_f &\rightarrow \neg d_f \downarrow \\
\neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f &\rightarrow \neg d_f \uparrow \\
a_t \wedge b_t &\rightarrow \neg d_t \downarrow \\
\neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f &\rightarrow \neg d_t \uparrow \\
(a_t \wedge b_t) \vee (a_f \wedge b_f) &\rightarrow \neg s_f \downarrow \\
\neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f &\rightarrow \neg s_f \uparrow \\
(a_t \wedge b_f) \vee (a_f \wedge b_t) &\rightarrow \neg s_t \downarrow \\
\neg a_t \wedge \neg a_f \wedge \neg b_t \wedge \neg b_f &\rightarrow \neg s_t \uparrow \\
\neg \neg d_f &\Rightarrow d_f \\
\neg \neg d_t &\Rightarrow d_t \\
\neg \neg s_f &\Rightarrow s_f \\
\neg \neg s_t &\Rightarrow s_t
\end{aligned}$$

A.5 FullAdder'

The *FullAdder'* is used in the counter decremter inside the *QuadRouter*. The objective is to subtract one from the current 8-bit counter value using only the counter itself as an explicit input. Since arithmetic is performed modulo 2^8 , subtracting one is equivalent to adding the constant 11111111_2 . The standard full adder can therefore be simplified by fixing the second operand b to one. As in the original *FullAdder*, c and d denote the input and output carry, respectively, while s denotes the sum output.

A.5.1 PRS

With $b = 1$, the true rail of the output carry is asserted whenever either a or c is true, whereas the false rail is asserted only when both a and c are false. Both rails of d return to the neutral state only when both a and c become invalid; including both inputs in the reset condition is necessary to avoid interference with the positive guard. The sum is true when a and c have the same value and false when they have different values. Its rails return to the neutral state once the input carry has returned to neutral.

$$\begin{aligned}
a_f \wedge c_f &\rightarrow d_f \uparrow \\
\neg a_t \wedge \neg a_f \wedge \neg c_t \wedge \neg c_f &\rightarrow d_f \downarrow \\
a_t \vee c_t &\rightarrow \neg d_t \uparrow \\
\neg a_t \wedge \neg a_f \wedge \neg c_t \wedge \neg c_f &\rightarrow \neg d_t \downarrow \\
(c_f \vee a_f) \wedge (c_t \vee a_t) &\rightarrow \neg s_f \uparrow \\
\neg c_t \wedge \neg c_f &\rightarrow \neg s_f \downarrow \\
c_t \wedge a_t \vee c_f \wedge a_f &\rightarrow \neg s_t \uparrow \\
\neg c_t \wedge \neg c_f &\rightarrow \neg s_t \downarrow
\end{aligned}$$

A.5.2 Bubble Reshuffling

To keep every non-acknowledged branch inverter-free, it is convenient to preserve all the input rails a_t, a_f, c_t, c_f in their original sense and implement all the output rails d_t, d_f, s_t, s_f in complemented form. This allows the input transitions to directly drive the corresponding pull-up and pull-down

networks, while requiring explicit inverters only at the outputs to restore their original polarities.

$$\begin{aligned}
a_f \wedge c_f &\rightarrow \neg d_f \downarrow \\
\neg a_t \wedge \neg a_f \wedge \neg c_t \wedge \neg c_f &\rightarrow \neg d_f \uparrow \\
a_t \vee c_t &\rightarrow \neg d_t \downarrow \\
\neg a_t \wedge \neg a_f \wedge \neg c_t \wedge \neg c_f &\rightarrow \neg d_t \uparrow \\
(c_f \vee a_f) \wedge (c_t \vee a_t) &\rightarrow \neg s_f \downarrow \\
\neg c_t \wedge \neg c_f &\rightarrow \neg s_f \uparrow \\
(c_t \wedge a_t) \vee (c_f \wedge a_f) &\rightarrow \neg s_t \downarrow \\
\neg c_t \wedge \neg c_f &\rightarrow \neg s_t \uparrow \\
\neg \neg d_f &\Rightarrow d_f \\
\neg \neg d_t &\Rightarrow d_t \\
\neg \neg s_f &\Rightarrow s_f \\
\neg \neg s_t &\Rightarrow s_t
\end{aligned}$$

A.6 HalfAdder'

The *HalfAdder'* implements the least significant stage of the counter decrem-
 enter. As in the *FullAdder'*, the second operand is implicitly fixed to one;
 however, since this is the least significant stage, no input carry is required.
 The block therefore receives only the dual-rail input a and produces the carry
 output d and the sum output s .

A.6.1 PRS

With the second operand fixed to one and no input carry, the carry output d
 simply follows a , while the sum output s is its complement. Both outputs
 return to the neutral state once the input rails become invalid.

$$\begin{aligned}
a_f &\rightarrow d_f \uparrow \\
\neg a_t \wedge \neg a_f &\rightarrow d_f \downarrow \\
a_t &\rightarrow d_t \uparrow \\
\neg a_t \wedge \neg a_f &\rightarrow d_t \downarrow \\
a_t &\rightarrow s_f \uparrow \\
\neg a_t \wedge \neg a_f &\rightarrow s_f \downarrow \\
a_f &\rightarrow s_t \uparrow \\
\neg a_t \wedge \neg a_f &\rightarrow s_t \downarrow
\end{aligned}$$

A.6.2 Bubble Reshuffling

To keep every non-acknowledged branch inverter-free, it is convenient to
 preserve both input rails a_t, a_f in their original sense and implement all the
 output rails d_t, d_f, s_t, s_f in complemented form. This allows the input transi-
 tions to directly drive the corresponding pull-up and pull-down networks,
 while requiring explicit inverters only at the outputs to restore their original
 polarities.

$$\begin{aligned}
& a_f \rightarrow \neg d_f \downarrow \\
& \neg a_t \wedge \neg a_f \rightarrow \neg d_f \uparrow \\
& a_t \rightarrow \neg d_t \downarrow \\
& \neg a_t \wedge \neg a_f \rightarrow \neg d_t \uparrow \\
& a_t \rightarrow \neg s_f \downarrow \\
& \neg a_t \wedge \neg a_f \rightarrow \neg s_f \uparrow \\
& a_f \rightarrow \neg s_t \downarrow \\
& \neg a_t \wedge \neg a_f \rightarrow \neg s_t \uparrow \\
& \neg \neg d_f \Rightarrow d_f \\
& \neg \neg d_t \Rightarrow d_t \\
& \neg \neg s_f \Rightarrow s_f \\
& \neg \neg s_t \Rightarrow s_t
\end{aligned}$$

A.7 Sequencer7

This block repeatedly performs seven dataless handshakes to activate the connected processes in sequence.

A.7.1 CHP

The CHP of the *Sequencer* is straightforward. The block repeatedly activates the seven channels in a fixed order, starting from A_0 and proceeding sequentially up to A_6 .

$$\begin{aligned}
\text{Sequencer7}(A_0, A_1, A_2, A_3, A_4, A_5, A_6) \equiv \\
* [A_0; A_1; A_2; A_3; A_4; A_5; A_6]
\end{aligned}$$

A.7.2 HSE

We start by deriving the HSE directly from the CHP. Each communication is implemented as a complete four-phase handshake before the following channel is activated.

$$\begin{aligned}
\text{Sequencer7} \equiv \\
& * [a_{0r} \uparrow; [a_{0a}]; a_{0r} \downarrow; [\neg a_{0a}]; \\
& \quad a_{1r} \uparrow; [a_{1a}]; a_{1r} \downarrow; [\neg a_{1a}]; \\
& \quad a_{2r} \uparrow; [a_{2a}]; a_{2r} \downarrow; [\neg a_{2a}]; \\
& \quad a_{3r} \uparrow; [a_{3a}]; a_{3r} \downarrow; [\neg a_{3a}]; \\
& \quad a_{4r} \uparrow; [a_{4a}]; a_{4r} \downarrow; [\neg a_{4a}]; \\
& \quad a_{5r} \uparrow; [a_{5a}]; a_{5r} \downarrow; [\neg a_{5a}]; \\
& \quad a_{6r} \uparrow; [a_{6a}]; a_{6r} \downarrow; [\neg a_{6a}]]
\end{aligned}$$

We can observe that, whenever a handshake is completed, the system returns to an idle state in which all handshake signals take the same values. These idle states are therefore indistinguishable from one another, even though each of them corresponds to a different position in the sequence. We consequently introduce additional state variables that uniquely identify the current position

and determine which channel must be activated next. To preserve the symmetry of the implementation as much as possible, these state variables are updated between consecutive channel handshakes.

Sequencer7 $\equiv$

```
* [a0r ↑; [a0a]; q1 ↑; q0 ↓; a0r ↓; [¬a0a];
  a1r ↑; [a1a]; q2 ↑; q1 ↓; a1r ↓; [¬a1a];
  a2r ↑; [a2a]; q3 ↑; q2 ↓; a2r ↓; [¬a2a];
  a3r ↑; [a3a]; q4 ↑; q3 ↓; a3r ↓; [¬a3a];
  a4r ↑; [a4a]; q5 ↑; q4 ↓; a4r ↓; [¬a4a];
  a5r ↑; [a5a]; q6 ↑; q5 ↓; a5r ↓; [¬a5a];
  a6r ↑; [a6a]; q0 ↑; q6 ↓; a6r ↓; [¬a6a]]
```

A.7.3 PRS

The state variables q_1 through q_6 are asserted when the corresponding acknowledge signal is high and the next state variable in the sequence is low. They are deasserted when the preceding acknowledge signal is low and the next state variable is high. The only exception is q_0 , which must be initialized high so that the first communication on A_0 can be activated after reset.

```
r ∨ (a6a ∧ ¬q1) → q0 ↑
¬a6a ∧ q1 → q0 ↓
a0a ∧ ¬q2 → q1 ↑
r ∨ (¬a0a ∧ q2) → q1 ↓
a1a ∧ ¬q3 → q2 ↑
r ∨ (¬a1a ∧ q3) → q2 ↓
a2a ∧ ¬q4 → q3 ↑
r ∨ (¬a2a ∧ q4) → q3 ↓
a3a ∧ ¬q5 → q4 ↑
r ∨ (¬a3a ∧ q5) → q4 ↓
a4a ∧ ¬q6 → q5 ↑
r ∨ (¬a4a ∧ q6) → q5 ↓
a5a ∧ ¬q0 → q6 ↑
r ∨ (¬a5a ∧ q0) → q6 ↓
```

The output requests are asserted when their corresponding state variable is high and the acknowledge signal of the preceding channel is low. They are deasserted when their corresponding state variable returns low.

$$\begin{aligned}
q_0 \wedge \neg a_{6a} &\rightarrow a_{0r} \uparrow \\
\neg q_0 &\rightarrow a_{0r} \downarrow \\
q_1 \wedge \neg a_{0a} &\rightarrow a_{1r} \uparrow \\
\neg q_1 &\rightarrow a_{1r} \downarrow \\
q_2 \wedge \neg a_{1a} &\rightarrow a_{2r} \uparrow \\
\neg q_2 &\rightarrow a_{2r} \downarrow \\
q_3 \wedge \neg a_{2a} &\rightarrow a_{3r} \uparrow \\
\neg q_3 &\rightarrow a_{3r} \downarrow \\
q_4 \wedge \neg a_{3a} &\rightarrow a_{4r} \uparrow \\
\neg q_4 &\rightarrow a_{4r} \downarrow \\
q_5 \wedge \neg a_{4a} &\rightarrow a_{5r} \uparrow \\
\neg q_5 &\rightarrow a_{5r} \downarrow \\
q_6 \wedge \neg a_{5a} &\rightarrow a_{6r} \uparrow \\
\neg q_6 &\rightarrow a_{6r} \downarrow
\end{aligned}$$

¶ An isochronic fork on a_{ja} constrains arrival at $G_{a_{v(j)r}}^+$. The two paths are

$$\begin{aligned}
P_d : a_{ja} \uparrow &\rightarrow G_{a_{v(j)r}}^+ \\
P_i : a_{ja} \uparrow &\rightarrow q_{v(j)} \uparrow \rightarrow G_{a_{v(j)r}}^+
\end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The acknowledge must inhibit the next request before the next state bit is asserted, preventing a transient true request guard. $j \in \{0, \dots, 6\}$ and $v(j) = (j + 1) \bmod 7$.

A.7.4 Bubble Reshuffling

The *Sequencer* contains an isochronic fork on each acknowledge signal a_{ja} . Therefore, we must avoid introducing an inverter on the corresponding non-acknowledged branch toward the positive guard of the next request.

$$\neg a_{ja} \rightarrow G_{a_{v(j)r}}^+$$

To preserve the original polarity of the externally visible signals, we keep a_{ja} and a_{jr} in their original sense. We also preserve all the state variables q_k in their original sense. The complemented versions of a_{ja} and q_k , as well as the complemented reset, are generated explicitly wherever required. The resulting PRS is:

$$\begin{aligned}
& \neg r \vee (\neg a_{6a} \wedge \neg q_1) \rightarrow q_0 \uparrow \\
& a_{6a} \wedge q_1 \rightarrow q_0 \downarrow \\
& \neg a_{0a} \wedge \neg q_2 \rightarrow q_1 \uparrow \\
& r \vee (\neg a_{0a} \wedge q_2) \rightarrow q_1 \downarrow \\
& \neg a_{1a} \wedge \neg q_3 \rightarrow q_2 \uparrow \\
& r \vee (\neg a_{1a} \wedge q_3) \rightarrow q_2 \downarrow \\
& \neg a_{2a} \wedge \neg q_4 \rightarrow q_3 \uparrow \\
& r \vee (\neg a_{2a} \wedge q_4) \rightarrow q_3 \downarrow \\
& \neg a_{3a} \wedge \neg q_5 \rightarrow q_4 \uparrow \\
& r \vee (\neg a_{3a} \wedge q_5) \rightarrow q_4 \downarrow \\
& \neg a_{4a} \wedge \neg q_6 \rightarrow q_5 \uparrow \\
& r \vee (\neg a_{4a} \wedge q_6) \rightarrow q_5 \downarrow \\
& \neg a_{5a} \wedge \neg q_0 \rightarrow q_6 \uparrow \\
& r \vee (\neg a_{5a} \wedge q_0) \rightarrow q_6 \downarrow \\
& \neg q_0 \wedge \neg a_{6a} \rightarrow a_{0r} \uparrow \\
& a_{0r} \rightarrow a_{0r} \downarrow \\
& \neg q_1 \wedge \neg a_{0a} \rightarrow a_{1r} \uparrow \\
& a_{1r} \rightarrow a_{1r} \downarrow \\
& \neg q_2 \wedge \neg a_{1a} \rightarrow a_{2r} \uparrow \\
& a_{2r} \rightarrow a_{2r} \downarrow \\
& \neg q_3 \wedge \neg a_{2a} \rightarrow a_{3r} \uparrow \\
& a_{3r} \rightarrow a_{3r} \downarrow \\
& \neg q_4 \wedge \neg a_{3a} \rightarrow a_{4r} \uparrow \\
& a_{4r} \rightarrow a_{4r} \downarrow \\
& \neg q_5 \wedge \neg a_{4a} \rightarrow a_{5r} \uparrow \\
& a_{5r} \rightarrow a_{5r} \downarrow \\
& \neg q_6 \wedge \neg a_{5a} \rightarrow a_{6r} \uparrow \\
& a_{6r} \rightarrow a_{6r} \downarrow \\
& \neg q_0 \Rightarrow \neg q_0 \\
& \neg q_1 \Rightarrow \neg q_1 \\
& \neg q_2 \Rightarrow \neg q_2 \\
& \neg q_3 \Rightarrow \neg q_3 \\
& \neg q_4 \Rightarrow \neg q_4 \\
& \neg q_5 \Rightarrow \neg q_5 \\
& \neg q_6 \Rightarrow \neg q_6 \\
& \neg a_{0a} \Rightarrow a_{0a} \\
& \neg a_{1a} \Rightarrow a_{1a} \\
& \neg a_{2a} \Rightarrow a_{2a} \\
& \neg a_{3a} \Rightarrow a_{3a} \\
& \neg a_{4a} \Rightarrow a_{4a} \\
& \neg a_{5a} \Rightarrow a_{5a} \\
& \neg a_{6a} \Rightarrow a_{6a} \\
& \neg r \Rightarrow r
\end{aligned}$$

A.8 Merge4Bits

This block receives two mutually exclusive dual-rail 4-bit input channels and merges them into a single output channel. Since the two inputs are guaranteed to be mutually exclusive, arbitration is not required.

A.8.1 CHP

The CHP simply probes the two input channels and forwards the pending input to the output. Since the two inputs are mutually exclusive, the selection is deterministic.

$$\begin{aligned} \text{Merge4Bits}(I_0?, I_1?, O!) &\equiv \\ &* [[\overline{I_0} \rightarrow O!I_0? \parallel \overline{I_1} \rightarrow O!I_1?]] \end{aligned}$$

A.8.2 HSE

We again start by translating the CHP directly into HSE. Repeated operations on the dual-rail data signals are expressed using vector notation.

$$\begin{aligned} \text{Merge4Bits} &\equiv \\ &* [[i_{0t3} \vee i_{0f3} \rightarrow \\ &\quad \langle, k : 3..0 : [i_{0tk} \rightarrow o_{tk} \uparrow \parallel i_{0fk} \rightarrow o_{fk} \uparrow]; [o_a]; i_{0a} \uparrow; \\ &\quad \langle, k : 3..0 : [\neg i_{0tk} \rightarrow o_{tk} \downarrow], [\neg i_{0fk} \rightarrow o_{fk} \downarrow]; [\neg o_a]; i_{0a} \downarrow \\ &\quad \parallel i_{1t3} \vee i_{1f3} \rightarrow \\ &\quad \langle, k : 3..0 : [i_{1tk} \rightarrow o_{tk} \uparrow \parallel i_{1fk} \rightarrow o_{fk} \uparrow]; [o_a]; i_{1a} \uparrow; \\ &\quad \langle, k : 3..0 : [\neg i_{1tk} \rightarrow o_{tk} \downarrow], [\neg i_{1fk} \rightarrow o_{fk} \downarrow]; [\neg o_a]; i_{1a} \downarrow \\ &\quad]] \end{aligned}$$

Since the surrounding sequencer guarantees that two data words cannot be present simultaneously, the initial selection is unnecessary at the circuit level. We can therefore decompose the HSE into two independent concurrent processes, one for each input channel.

$$\begin{aligned} \text{Merge4Bits} &\equiv \\ &* [\langle, k : 3..0 : [i_{0tk} \rightarrow o_{tk} \uparrow \parallel i_{0fk} \rightarrow o_{fk} \uparrow]; [o_a]; i_{0a} \uparrow; \\ &\quad \langle, k : 3..0 : [\neg i_{0tk} \rightarrow o_{tk} \downarrow], [\neg i_{0fk} \rightarrow o_{fk} \downarrow]; [\neg o_a]; i_{0a} \downarrow \\ &\quad] \parallel \\ &* [\langle, k : 3..0 : [i_{1tk} \rightarrow o_{tk} \uparrow \parallel i_{1fk} \rightarrow o_{fk} \uparrow]; [o_a]; i_{1a} \uparrow; \\ &\quad \langle, k : 3..0 : [\neg i_{1tk} \rightarrow o_{tk} \downarrow], [\neg i_{1fk} \rightarrow o_{fk} \downarrow]; [\neg o_a]; i_{1a} \downarrow \\ &\quad] \end{aligned}$$

A.8.3 PRS

The input acknowledge signals are asserted when the output has been acknowledged and a valid value is present on the corresponding input channel. They are deasserted when the output acknowledge returns low.

$$\begin{aligned} o_a \wedge (i_{0t3} \vee i_{0f3}) &\rightarrow i_{0a} \uparrow \\ \neg o_a &\rightarrow i_{0a} \downarrow \\ o_a \wedge (i_{1t3} \vee i_{1f3}) &\rightarrow i_{1a} \uparrow \\ \neg o_a &\rightarrow i_{1a} \downarrow \end{aligned}$$

The output rails can instead be implemented as combinational OR gates, since the two input channels are mutually exclusive.

$$\begin{aligned} i_{0fk} \vee i_{1fk} &\Rightarrow o_{fk} & k=0,\dots,3 \\ i_{0tk} \vee i_{1tk} &\Rightarrow o_{tk} & k=0,\dots,3 \end{aligned}$$

A.8.4 Bubble Reshuffling

The *Merge4Bits* contains no isochronic forks, so there are no fork-critical polarity constraints. To make every operator directly CMOS-implementable, we implement the input acknowledge signals and the output rails in complemented form, while preserving o_a and all input rails in their original sense. Since both the input acknowledge signals and the output rails are externally visible, their original polarities are restored before they leave the module. The resulting PRS is:

$$\begin{aligned} o_a \wedge (i_{0t3} \vee i_{0f3}) &\rightarrow \neg i_{0a} \downarrow \\ \neg o_a &\rightarrow \neg i_{0a} \uparrow \\ o_a \wedge (i_{1t3} \vee i_{1f3}) &\rightarrow \neg i_{1a} \downarrow \\ \neg o_a &\rightarrow \neg i_{1a} \uparrow \\ i_{0fk} \vee i_{1fk} &\rightarrow \neg o_{fk} \downarrow & k=0,\dots,3 \\ \neg i_{0fk} \wedge \neg i_{1fk} &\rightarrow \neg o_{fk} \uparrow & k=0,\dots,3 \\ i_{0tk} \vee i_{1tk} &\rightarrow \neg o_{tk} \downarrow & k=0,\dots,3 \\ \neg i_{0tk} \wedge \neg i_{1tk} &\rightarrow \neg o_{tk} \uparrow & k=0,\dots,3 \\ \neg \neg i_{0a} &\Rightarrow i_{0a} \\ \neg \neg i_{1a} &\Rightarrow i_{1a} \\ \neg \neg o_{fk} &\Rightarrow o_{fk} & k=0,\dots,3 \\ \neg \neg o_{tk} &\Rightarrow o_{tk} & k=0,\dots,3 \end{aligned}$$

A.9 Merge8Bits

This block performs the same function as the *Merge4Bits*, but operates on 8-bit dual-rail channels. Since its implementation follows directly from that of the *Merge4Bits*, we present its PRS directly.

A.9.1 PRS

The input acknowledge signals are asserted when the output has been acknowledged and a valid value is present on the corresponding input channel. They are deasserted when the output acknowledge returns low.

$$\begin{aligned} o_a \wedge (i_{0t7} \vee i_{0f7}) &\rightarrow i_{0a} \uparrow \\ \neg o_a &\rightarrow i_{0a} \downarrow \\ o_a \wedge (i_{1t7} \vee i_{1f7}) &\rightarrow i_{1a} \uparrow \\ \neg o_a &\rightarrow i_{1a} \downarrow \end{aligned}$$

The output rails can instead be implemented as combinational OR gates, since the two input channels are mutually exclusive.

$$\begin{aligned} i_{0fk} \vee i_{1fk} &\Rightarrow o_{fk} & k=0,\dots,7 \\ i_{0tk} \vee i_{1tk} &\Rightarrow o_{tk} & k=0,\dots,7 \end{aligned}$$

A.9.2 Bubble Reshuffling

The *Merge8Bits* contains no isochronic forks, so there are no fork-critical polarity constraints. To make every operator directly CMOS-implementable, we implement the input acknowledge signals and the output rails in complemented form, while preserving o_a and all input rails in their original sense. Since both the input acknowledge signals and the output rails are externally visible, their original polarities are restored before they leave the module. The resulting PRS is:

$$\begin{aligned}
o_a \wedge (i_{0t7} \vee i_{0f7}) &\rightarrow \neg i_{0a} \downarrow \\
\neg o_a &\rightarrow \neg i_{0a} \uparrow \\
o_a \wedge (i_{1t7} \vee i_{1f7}) &\rightarrow \neg i_{1a} \downarrow \\
\neg o_a &\rightarrow \neg i_{1a} \uparrow \\
i_{0fk} \vee i_{1fk} &\rightarrow \neg o_{fk} \downarrow & k=0,\dots,7 \\
\neg i_{0fk} \wedge \neg i_{1fk} &\rightarrow \neg o_{fk} \uparrow & k=0,\dots,7 \\
i_{0tk} \vee i_{1tk} &\rightarrow \neg o_{tk} \downarrow & k=0,\dots,7 \\
\neg i_{0tk} \wedge \neg i_{1tk} &\rightarrow \neg o_{tk} \uparrow & k=0,\dots,7 \\
\neg \neg i_{0a} &\Rightarrow i_{0a} \\
\neg \neg i_{1a} &\Rightarrow i_{1a} \\
\neg \neg o_{fk} &\Rightarrow o_{fk} & k=0,\dots,7 \\
\neg \neg o_{tk} &\Rightarrow o_{tk} & k=0,\dots,7
\end{aligned}$$

A.10 Merge32Bits

This block performs the same function as *Merge4Bits* on 32-bit channels. Its PRS is presented directly.

A.10.1 PRS

The input acknowledge signals are asserted when the output has been acknowledged and a valid value is present on the corresponding input channel. They are deasserted when the output acknowledge returns low.

$$\begin{aligned}
o_a \wedge (i_{0t31} \vee i_{0f31}) &\rightarrow i_{0a} \uparrow \\
\neg o_a &\rightarrow i_{0a} \downarrow \\
o_a \wedge (i_{1t31} \vee i_{1f31}) &\rightarrow i_{1a} \uparrow \\
\neg o_a &\rightarrow i_{1a} \downarrow
\end{aligned}$$

The output rails can instead be implemented as combinational OR gates, since the two input channels are mutually exclusive.

$$\begin{aligned}
i_{0fk} \vee i_{1fk} &\Rightarrow o_{fk} & k=0,\dots,31 \\
i_{0tk} \vee i_{1tk} &\Rightarrow o_{tk} & k=0,\dots,31
\end{aligned}$$

A.10.2 Bubble Reshuffling

The *Merge32Bits* contains no isochronic forks, so there are no fork-critical polarity constraints. To make every operator directly CMOS-implementable,

we implement the input acknowledge signals and the output rails in complemented form, while preserving o_a and all input rails in their original sense. Since both the input acknowledge signals and the output rails are externally visible, their original polarities are restored before they leave the module. The resulting PRS is:

$$\begin{aligned}
& o_a \wedge (i_{0t31} \vee i_{0f31}) \rightarrow \neg i_{0a} \downarrow \\
& \neg o_a \rightarrow \neg i_{0a} \uparrow \\
& o_a \wedge (i_{1t31} \vee i_{1f31}) \rightarrow \neg i_{1a} \downarrow \\
& \neg o_a \rightarrow \neg i_{1a} \uparrow \\
& i_{0fk} \vee i_{1fk} \rightarrow \neg o_{fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{0fk} \wedge \neg i_{1fk} \rightarrow \neg o_{fk} \uparrow \quad k=0,\dots,31 \\
& i_{0tk} \vee i_{1tk} \rightarrow \neg o_{tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{0tk} \wedge \neg i_{1tk} \rightarrow \neg o_{tk} \uparrow \quad k=0,\dots,31 \\
& \neg i_{0a} \Rightarrow i_{0a} \\
& \neg i_{1a} \Rightarrow i_{1a} \\
& \neg o_{fk} \Rightarrow o_{fk} \quad k=0,\dots,31 \\
& \neg o_{tk} \Rightarrow o_{tk} \quad k=0,\dots,31
\end{aligned}$$

A.11 Split4Bits

This block receives two mutually exclusive requests for data from its output channels and forwards the selected request to the input channel, effectively pulling the input data toward the requesting output.

A.11.1 CHP

The CHP is straightforward. Since requests from the two output channels are mutually exclusive, the block simply forwards the input data to whichever output is requesting it.

$$\begin{aligned}
& \text{Split4Bits}(I?, O_0!, O_1!) \equiv \\
& * [[\overline{O_0} \rightarrow O_0! I? \parallel \overline{O_1} \rightarrow O_1! I?]]
\end{aligned}$$

A.11.2 HSE

For the HSE of the *Split4Bits*, we must keep in mind that the outputs pull data from the input toward the corresponding output channel. Requests from the two output ports are mutually exclusive. Repeated operations on the dual-rail data signals are expressed using vector notation.

$$\begin{aligned}
& \text{Split4Bits} \equiv \\
& * [[o_{0r} \rightarrow i_r \uparrow; \langle , k : 3..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle; [\neg o_{0r}]; i_r \downarrow; \\
& \quad \langle , k : 3..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow] \rangle \\
& \parallel o_{1r} \rightarrow i_r \uparrow; \langle , k : 3..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle; [\neg o_{1r}]; i_r \downarrow; \\
& \quad \langle , k : 3..0 : [\neg i_{tk} \rightarrow o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{1fk} \downarrow] \rangle \\
&]]
\end{aligned}$$

A.11.3 PRS

For i_r , the implementation is purely combinational: the input request is asserted whenever either output channel requests data.

$$o_{0r} \vee o_{1r} \Rightarrow i_r$$

For the output rails, only the rails belonging to the channel currently requesting data are asserted. Each rail returns to spacer when the corresponding input rail becomes invalid.

$$\begin{array}{ll} o_{0r} \wedge i_{fk} \rightarrow o_{0fk} \uparrow & k=0,\dots,3 \\ \neg i_{fk} \rightarrow o_{0fk} \downarrow & k=0,\dots,3 \\ o_{0r} \wedge i_{tk} \rightarrow o_{0tk} \uparrow & k=0,\dots,3 \\ \neg i_{tk} \rightarrow o_{0tk} \downarrow & k=0,\dots,3 \end{array}$$

For O_1 , replace o_{0pk} with o_{1pk} and o_{0r} with o_{1r} , including complemented names.

A.11.4 Bubble Reshuffling

The *Split4Bits* contains no isochronic forks, so there are no fork-critical polarity constraints. We preserve the output requests o_{0r}, o_{1r} and all the input rails in their original sense, while implementing the input request i_r and all the output rails in complemented form. The original polarities of i_r and of the output rails are then restored before they leave the module.

$$\begin{array}{ll} \neg o_{0r} \wedge \neg o_{1r} \Rightarrow \neg i_r & \\ o_{0r} \wedge i_{fk} \rightarrow \neg o_{0fk} \downarrow & k=0,\dots,3 \\ \neg i_{fk} \rightarrow \neg o_{0fk} \uparrow & k=0,\dots,3 \\ o_{0r} \wedge i_{tk} \rightarrow \neg o_{0tk} \downarrow & k=0,\dots,3 \\ \neg i_{tk} \rightarrow \neg o_{0tk} \uparrow & k=0,\dots,3 \\ \neg \neg i_r \Rightarrow i_r & \\ \neg \neg o_{0fk} \Rightarrow o_{0fk} & k=0,\dots,3 \\ \neg \neg o_{0tk} \Rightarrow o_{0tk} & k=0,\dots,3 \end{array}$$

For O_1 , replace o_{0pk} with o_{1pk} and o_{0r} with o_{1r} , including complemented names.

A.12 Arb

The arbiter receives two or more potentially concurrent input channels, resolves any conflict by selecting a single winner, and forwards the selected input to the output. The output port remains assigned to the winning channel until the tail bit is observed, indicating that the entire message has been transmitted.

A.12.1 CHP

The CHP first arbitrates among the pending input channels and stores the identity of the selected input in p . The selected channel then retains ownership of the output until its tail flit is observed.

```

Arb( $C_0?$ ,  $C_1?$ ,  $C_2?$ ,  $C_3?$ ,  $P?$ ,  $M?$ ,  $O!$ )  $\equiv$ 
 $s := \text{true};$ 
*  $[[s \rightarrow$ 
     $\overline{C_0} \rightarrow p := 0$ 
     $\overline{C_1} \rightarrow p := 1$ 
     $\overline{C_2} \rightarrow p := 2$ 
     $\overline{C_3} \rightarrow p := 3$ 
     $\overline{P} \rightarrow p := 4$ 
     $\overline{M} \rightarrow p := 5$ 
     $]; s := \text{false}$ 
 $|| \neg s \rightarrow$ 
     $[p = 0 \rightarrow s := \overline{C_0}?.tail; O!C_0?$ 
     $|| p = 1 \rightarrow s := \overline{C_1}?.tail; O!C_1?$ 
     $|| p = 2 \rightarrow s := \overline{C_2}?.tail; O!C_2?$ 
     $|| p = 3 \rightarrow s := \overline{C_3}?.tail; O!C_3?$ 
     $|| p = 4 \rightarrow s := \overline{P}?.tail; O!P?$ 
     $|| p = 5 \rightarrow s := \overline{M}?.tail; O!M?$ 
     $]$ 
 $]]$ 

```

We can decompose this module into two-input arbiter modules, called *Arb2s*, and connect them in a tree structure, as shown in the figure. Importantly, an input cannot be acknowledged until the final output of the complete *Arb* has been acknowledged, since the input is not known to have won the complete arbitration until its request has propagated through all the arbitration stages.

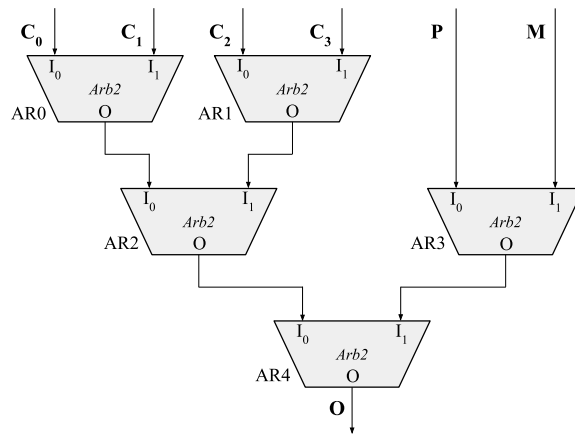


Figure A.1: Arbiter decomposition

Each of these modules has the following structure.

$$\begin{aligned}
 \text{Arb2}(I_0?, I_1?, O!) &\equiv \\
 s &:= \text{true}; \\
 * &[[s \rightarrow [\overline{I_0} \rightarrow p := \text{true} | \overline{I_1} \rightarrow p := \text{false}]; s := \text{false} \\
 &\quad \parallel \neg s \rightarrow [p \rightarrow w := \overline{I_0}?.tail; O!I_0?; s := w] \parallel \neg p \rightarrow w := \overline{I_1}?.tail; O!I_1?; s := w] \\
 &]]
 \end{aligned}$$

A.12.2 HSE

The mutex in the s branch selects an input and determines p . Its mutually exclusive grants are u for I_0 and v for I_1 . A grant remains high until its input request is withdrawn. The other grant may then rise immediately if its input is pending, so each flit waits only for the grant associated with the selected input to return low. Requiring both grants to be low could unnecessarily block the process. The stored selection p retains message ownership until the tail flit; the mutex grants track individual input requests. The mutex PRS and its required filter are presented with the circuit implementation below.

Throughout this derivation, f_0 records completion of input selection and f_1 identifies the end-of-flit state-update phase. The pair w_0, w_1 encodes the next value of s .

$$\begin{aligned}
 \text{Arb2} &\equiv \\
 * &[[s \rightarrow [u \rightarrow p \uparrow \parallel v \rightarrow p \downarrow]; s \downarrow \\
 &\quad \parallel \neg s \rightarrow \\
 &\quad [p \rightarrow \\
 &\quad [i_{0t0} \rightarrow w_1 \uparrow \parallel i_{0f0} \rightarrow w_0 \uparrow], \langle, k : 31..0 : [i_{0tk} \rightarrow o_{tk} \uparrow \parallel i_{0fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; i_{0a} \uparrow; \\
 &\quad \langle, k : 31..0 : [\neg i_{0tk} \rightarrow o_{tk} \downarrow], [\neg i_{0fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg u], [\neg o_a]; i_{0a} \downarrow; \\
 &\quad [w_0 \rightarrow w_0 \downarrow \parallel w_1 \rightarrow s \uparrow; w_1 \downarrow] \\
 &\quad \parallel \neg p \rightarrow \\
 &\quad [i_{1t0} \rightarrow w_1 \uparrow \parallel i_{1f0} \rightarrow w_0 \uparrow], \langle, k : 31..0 : [i_{1tk} \rightarrow o_{tk} \uparrow \parallel i_{1fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; i_{1a} \uparrow; \\
 &\quad \langle, k : 31..0 : [\neg i_{1tk} \rightarrow o_{tk} \downarrow], [\neg i_{1fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg v], [\neg o_a]; i_{1a} \downarrow; \\
 &\quad [w_0 \rightarrow w_0 \downarrow \parallel w_1 \rightarrow s \uparrow; w_1 \downarrow] \\
 &\quad] \\
 &]]
 \end{aligned}$$

After p is updated, the initialization state is not uniquely identified by the existing signals alone. We insert f_0 to identify this point and use it to lower s . Before either i_{0a} or i_{1a} is lowered, all internal state updates must complete. This prevents the next input from arriving while the current flit is still being finalized.

```

Arb2 ≡
* [[s → [u → p ↑ ∥ v → p ↓]; f0 ↑; s ↓
  ∥ ¬s →
    [p →
      [i0t0 → w1 ↑ ∥ i0f0 → w0 ↑], ⟨, k : 31..0 : [i0tk → otk ↑ ∥ i0fk → ofk ↑]⟩; [oa]; f0 ↓; i0a ↑;
      ⟨, k : 31..0 : [¬i0tk → otk ↓], [¬i0fk → ofk ↓]⟩; [¬u], [¬oa];
      [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]; i0a ↓
    ]
    ∥ ¬p →
      [i1t0 → w1 ↑ ∥ i1f0 → w0 ↑], ⟨, k : 31..0 : [i1tk → otk ↑ ∥ i1fk → ofk ↑]⟩; [oa]; f0 ↓; i1a ↑;
      ⟨, k : 31..0 : [¬i1tk → otk ↓], [¬i1fk → ofk ↓]⟩; [¬v], [¬oa];
      [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]; i1a ↓
    ]
  ]
]]

```

The end-of-flit update of s otherwise requires a large guard. We insert f_1 to identify this phase and simplify the rules for w_0, w_1 , and s . The selected input acknowledge is lowered only after f_1 returns low.

```

Arb2 ≡
* [[s → [u → p ↑ ∥ v → p ↓]; f0 ↑; s ↓
  ∥ ¬s →
    [p →
      [i0t0 → w1 ↑ ∥ i0f0 → w0 ↑], ⟨, k : 31..0 : [i0tk → otk ↑ ∥ i0fk → ofk ↑]⟩; [oa]; f0 ↓; i0a ↑;
      ⟨, k : 31..0 : [¬i0tk → otk ↓], [¬i0fk → ofk ↓]⟩; [¬u], [¬oa]; f1 ↑;
      [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]; f1 ↓; i0a ↓
    ]
    ∥ ¬p →
      [i1t0 → w1 ↑ ∥ i1f0 → w0 ↑], ⟨, k : 31..0 : [i1tk → otk ↑ ∥ i1fk → ofk ↑]⟩; [oa]; f0 ↓; i1a ↑;
      ⟨, k : 31..0 : [¬i1tk → otk ↓], [¬i1fk → ofk ↓]⟩; [¬v], [¬oa]; f1 ↑;
      [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]; f1 ↓; i1a ↓
    ]
  ]
]]

```

A.12.3 PRS

The variable f_0 is asserted only when s is high and p has been updated. It is deasserted only when s is low and the output acknowledge is high.

$$\begin{aligned}
& s \wedge ((u \wedge p) \vee (v \wedge \neg p)) \rightarrow f_0 \uparrow \\
& (\neg s \wedge o_a) \vee r \rightarrow f_0 \downarrow
\end{aligned}$$

The variable f_1 is asserted in a state uniquely identified by one of the input acknowledge signals being high, the output acknowledge being low, and the arbiter state variable corresponding to the selected branch being low. It is deasserted whenever both w_0 and w_1 are low. To prevent interference in the positive guard, the condition $w_0 \vee w_1$ is also included.

$$\neg o_a \wedge (i_{0a} \vee i_{1a}) \wedge (w_0 \vee w_1) \wedge ((\neg u \wedge p) \vee (\neg v \wedge \neg p)) \rightarrow f_1 \uparrow$$

$$(\neg w_0 \wedge \neg w_1) \vee r \rightarrow f_1 \downarrow$$

The variable s is asserted by reset and whenever both w_1 and f_1 are high. It is deasserted when f_0 is high. In the $\neg s$ branch, s is already low, so no additional condition is required.

$$(w_1 \wedge f_1) \vee r \rightarrow s \uparrow$$

$$f_0 \rightarrow s \downarrow$$

The variable p is modified only in the s branch, according to the result of the arbiter, except during reset. Note, however, that when data arriving on I_0 are being handled, as soon as s returns high the circuit could already begin handling a pending input on I_1 before i_{0a} has returned low. Therefore, p may be updated only when both input acknowledge signals are low.

$$(\neg i_{0a} \wedge \neg i_{1a}) \wedge (u \wedge s) \vee r \rightarrow p \uparrow$$

$$(\neg i_{0a} \wedge \neg i_{1a}) \wedge (v \wedge s) \rightarrow p \downarrow$$

we want the input acknowledge signals to change only after the internal state variables have been updated. To assert the acknowledge of the selected input, o_a must be high, either w_0 or w_1 must be asserted, and f_0 must be low. To deassert it, f_1 must be low. Since $\neg f_1$ also holds immediately after o_a returns low, the negative guards are strengthened with $\neg w_0 \wedge \neg w_1$ to prevent interference.

$$p \wedge o_a \wedge (w_1 \vee w_0) \wedge \neg f_0 \rightarrow i_{0a} \uparrow$$

$$\neg f_1 \wedge (\neg w_0 \wedge \neg w_1) \rightarrow i_{0a} \downarrow$$

$$\neg p \wedge o_a \wedge (w_1 \vee w_0) \wedge \neg f_0 \rightarrow i_{1a} \uparrow$$

$$\neg f_1 \wedge (\neg w_0 \wedge \neg w_1) \rightarrow i_{1a} \downarrow$$

The variables w_0 and w_1 are asserted in the $\neg s$ branch according to the value of the least significant bit of the selected input. They are deasserted once s has been updated and f_1 is high.

$$\neg s \wedge ((i_{0f0} \wedge p) \vee (i_{1f0} \wedge \neg p)) \rightarrow w_0 \uparrow$$

$$(\neg s \wedge f_1) \vee r \rightarrow w_0 \downarrow$$

$$\neg s \wedge ((i_{0f0} \wedge p) \vee (i_{1f0} \wedge \neg p)) \rightarrow w_1 \uparrow$$

$$(s \wedge f_1) \vee r \rightarrow w_1 \downarrow$$

Finally, we report the PRS of the Mutex, where $\neg u$ and $\neg v$ denote the complemented versions of u and v , respectively. These PRS rules alone are not sufficient to fully describe the arbiter circuit, since a metastability filter is also required when both a and b are asserted concurrently. This filter cannot be described using production rules alone; the complete arbiter implementation is therefore shown in the following figure.

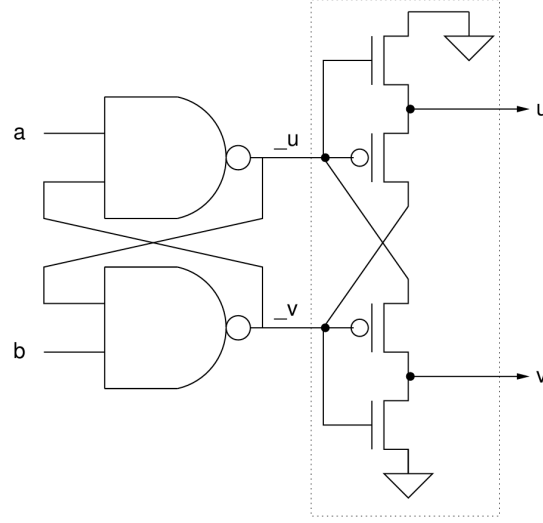


Figure A.2: Mutex circuit

In our implementation, a and b correspond to the validity of the two input channels. We therefore obtain

$$\begin{aligned}
 (i_{1f31} \vee i_{1f31}) \wedge \neg u &\rightarrow \neg v \downarrow \\
 \neg(i_{1f31} \vee i_{1f31}) \vee \neg \neg u &\rightarrow \neg v \uparrow \\
 (i_{0f31} \vee i_{0f31}) \wedge \neg v &\rightarrow \neg u \downarrow \\
 \neg(i_{0f31} \vee i_{0f31}) \vee \neg \neg v &\rightarrow \neg u \uparrow \\
 \neg \neg v &\Rightarrow v \\
 \neg \neg u &\Rightarrow u
 \end{aligned}$$

The output rails follow the corresponding rails of the input selected by p . Before being asserted, they must also wait for the circuit to enter the $\neg s$ branch, ensuring that the input selection has been completed. The negative guards distinguish between the two input branches because, while transmission from one input is being completed, new data may already be pending on the other.

$$\begin{aligned}
 \neg s \wedge ((p \wedge i_{0fk}) \vee (\neg p \wedge i_{1fk})) &\rightarrow o_{fk} \uparrow & k=0,\dots,31 \\
 (p \wedge \neg i_{0fk}) \vee (\neg p \wedge \neg i_{1fk}) &\rightarrow o_{fk} \downarrow & k=0,\dots,31 \\
 \neg s \wedge ((p \wedge i_{0tk}) \vee (\neg p \wedge i_{1tk})) &\rightarrow o_{tk} \uparrow & k=0,\dots,31 \\
 (p \wedge \neg i_{0tk}) \vee (\neg p \wedge \neg i_{1tk}) &\rightarrow o_{tk} \downarrow & k=0,\dots,31
 \end{aligned}$$

In the fork paths below, $j \in \{0, 1\}$ identifies the selected input, $k \in \{0, \dots, 31\}$ identifies a bit, $\rho \in \{t, f\}$ identifies a rail, and $\ell \in \{0, 1\}$ identifies a next-state rail. The notation $g_0 = u$ and $g_1 = v$ associates each existing grant with its input; it does not introduce new circuit signals.

¶ An isochronic fork on f_1 constrains arrival at $G_{i_{ja}}^-$. The two paths are

$$\begin{aligned} P_d : f_1 \uparrow \rightarrow G_{i_{ja}}^- \\ P_i : f_1 \uparrow \rightarrow w_\ell \downarrow \rightarrow G_{i_{ja}}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The completion phase must inhibit input-acknowledge release before the next-state rails are cleared.

The same fork also constrains arrival at G_s^+ . The two paths are

$$\begin{aligned} P_d : f_1 \downarrow \rightarrow G_s^+ \\ P_i : f_1 \downarrow \rightarrow i_{ja} \downarrow \rightarrow i_{jt0} \uparrow \rightarrow w_1 \uparrow \rightarrow G_s^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. Completion must be inactive at the start-state guard before a new tail bit is captured.

¶ An isochronic fork on p constrains arrival at $G_{o_{pk}}^+$. The two paths are

$$\begin{aligned} P_d : p \rightarrow G_{o_{pk}}^+ \\ P_i : p \rightarrow f_0 \uparrow \rightarrow s \downarrow \rightarrow G_{o_{pk}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The selected input must reach the output guard before forwarding is enabled.

The same fork also constrains arrival at $G_{w_\ell}^+$. The two paths are

$$\begin{aligned} P_d : p \downarrow \rightarrow G_{w_\ell}^+ \\ P_i : p \downarrow \rightarrow f_0 \uparrow \rightarrow s \downarrow \rightarrow G_{w_\ell}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The next-state capture guard must observe the updated selection before it is re-enabled.

¶ An isochronic fork on i_{ja} constrains arrival at $G_{f_1}^+$. The two paths are

$$\begin{aligned} P_d : i_{ja} \downarrow \rightarrow G_{f_1}^+ \\ P_i : i_{ja} \downarrow \rightarrow i_{j\rho 0} \uparrow \rightarrow w_\ell \uparrow \rightarrow G_{f_1}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The old input acknowledge must no longer qualify completion when the next input sets a next-state rail. Here $(\rho, \ell) \in \{(f, 0), (t, 1)\}$.

¶ An isochronic fork on g_j constrains arrival at $G_{f_0}^+$. The two paths are

$$\begin{aligned} P_d : g_j \downarrow \rightarrow G_{f_0}^+ \\ P_i : g_j \downarrow \rightarrow f_1 \uparrow \rightarrow s \uparrow \rightarrow G_{f_0}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The old grant must be withdrawn at the selection-completion guard before the next arbitration phase begins.

¶ An isochronic fork on i_{jf0} constrains arrival at $G_{w_\ell}^+$. The two paths are

$$\begin{aligned} P_d : i_{jf0} \downarrow \rightarrow G_{w_\ell}^+ \\ P_i : i_{jf0} \downarrow \rightarrow o_{f0} \downarrow \rightarrow o_a \downarrow \rightarrow f_1 \uparrow \rightarrow G_{w_\ell}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The data-dependent assertion guard must be disabled before the corresponding clearing condition can arrive.

¶ An isochronic fork on o_a constrains arrival at $G_{f_1}^+$. The two paths are

$$\begin{aligned} P_d : o_a \uparrow \rightarrow G_{f_1}^+ \\ P_i : o_a \uparrow \rightarrow i_{ja} \uparrow \rightarrow G_{f_1}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit completion detection before the selected input acknowledge can enable it.

A.12.4 Bubble Reshuffling

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned} \neg f_1 &\rightarrow G_{i_{ja}}^- \\ f_1 &\rightarrow G_s^+ \\ p &\rightarrow G_{o_{pk}}^+ \\ \neg p &\rightarrow G_{o_{pk}}^+ \\ p &\rightarrow G_{w_\ell}^+ \\ \neg p &\rightarrow G_{w_\ell}^+ \\ i_{ja} &\rightarrow G_{f_1}^+ \\ g_j &\rightarrow G_{f_0}^+ \\ i_{jfo} &\rightarrow G_{w_\ell}^+ \\ \neg o_a &\rightarrow G_{f_1}^+ \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve f_1, f_0, o_a, p , together with the input data rails, in their original sense, while implementing $s, i_{0a}, i_{1a}, w_0, w_1, u, v$ and the internal output rails in complemented form.

$$\begin{aligned}
& \neg s \wedge ((\neg u \wedge p) \vee (\neg v \wedge \neg p)) \rightarrow f_0 \uparrow \\
& (s \wedge o_a) \vee r \rightarrow f_0 \downarrow \\
& \neg o_a \wedge (\neg i_{0a} \vee \neg i_{1a}) \wedge (\neg w_0 \vee \neg w_1) \wedge ((u \wedge p) \vee (v \wedge \neg p)) \rightarrow f_1 \uparrow \\
& (w_0 \wedge w_1) \vee r \rightarrow f_1 \downarrow \\
& (\neg w_1 \wedge f_1) \vee r \rightarrow s \downarrow \\
& f_0 \rightarrow s \uparrow \\
& (i_{0a} \wedge i_{1a}) \wedge (\neg u \wedge \neg s) \vee r \rightarrow p \uparrow \\
& (i_{0a} \wedge i_{1a}) \wedge (\neg v \wedge \neg s) \rightarrow p \downarrow \\
& p \wedge o_a \wedge (\neg w_1 \vee \neg w_0) \wedge \neg f_0 \rightarrow i_{0a} \downarrow \\
& \neg f_1 \wedge (w_0 \wedge w_1) \rightarrow i_{0a} \uparrow \\
& \neg p \wedge o_a \wedge (\neg w_1 \vee \neg w_0) \wedge \neg f_0 \rightarrow i_{1a} \downarrow \\
& \neg f_1 \wedge (w_0 \wedge w_1) \rightarrow i_{1a} \uparrow \\
& s \wedge ((i_{0f0} \wedge p) \vee (i_{1f0} \wedge \neg p)) \rightarrow w_0 \downarrow \\
& (s \wedge f_1) \vee r \rightarrow w_0 \uparrow \\
& s \wedge ((i_{0f0} \wedge p) \vee (i_{1f0} \wedge \neg p)) \rightarrow w_1 \downarrow \\
& (\neg s \wedge f_1) \vee r \rightarrow w_1 \uparrow \\
& (i_{1t31} \vee i_{1f31}) \wedge u \rightarrow v \downarrow \\
& \neg(i_{1t31} \vee i_{1f31}) \vee \neg u \rightarrow v \uparrow \\
& (i_{0t31} \vee i_{0f31}) \wedge v \rightarrow u \downarrow \\
& \neg(i_{0t31} \vee i_{0f31}) \vee \neg v \rightarrow u \uparrow \\
& s \wedge ((p \wedge i_{0fk}) \vee (\neg p \wedge i_{1fk})) \rightarrow o_{fk} \downarrow \quad k=0,\dots,31 \\
& (p \wedge \neg i_{0fk}) \vee (\neg p \wedge \neg i_{1fk}) \rightarrow o_{fk} \uparrow \quad k=0,\dots,31 \\
& s \wedge ((p \wedge i_{0tk}) \vee (\neg p \wedge i_{1tk})) \rightarrow o_{tk} \downarrow \quad k=0,\dots,31 \\
& (p \wedge \neg i_{0tk}) \vee (\neg p \wedge \neg i_{1tk}) \rightarrow o_{tk} \uparrow \quad k=0,\dots,31
\end{aligned}$$

At this point, the fork-critical polarities are fixed. We therefore propagate the remaining bubbles through the guards, introducing explicit inverters only where the opposite sense of a polarity-fixed signal is still required. Finally, inverters are added to the input acknowledge signals and output rails to restore their original external polarities.

$$\begin{aligned}
& \neg s \wedge ((\neg u \wedge \neg p) \vee (\neg v \wedge \neg p)) \rightarrow f_0 \uparrow \\
& (s \wedge o_a) \vee r \rightarrow f_0 \downarrow \\
& \neg o_a \wedge (\neg i_{0a} \vee \neg i_{1a}) \wedge (\neg w_0 \vee \neg w_1) \wedge ((\neg u \wedge \neg p) \vee (\neg v \wedge \neg p)) \rightarrow f_1 \uparrow \\
& (w_0 \wedge w_1) \vee r \rightarrow f_1 \downarrow \\
& (w_1 \wedge f_1) \vee r \rightarrow s \downarrow \\
& \neg f_0 \rightarrow s \uparrow \\
& (\neg i_{0a} \wedge \neg i_{1a}) \wedge (\neg u \wedge \neg s) \vee \neg r \rightarrow p \uparrow \\
& (i_{0a} \wedge i_{1a}) \wedge (v \wedge s) \rightarrow p \downarrow \\
& p \wedge o_a \wedge (w_1 \vee w_0) \wedge \neg f_0 \rightarrow i_{0a} \downarrow \\
& \neg f_1 \wedge (\neg w_0 \wedge \neg w_1) \rightarrow i_{0a} \uparrow \\
& \neg p \wedge o_a \wedge (w_1 \vee w_0) \wedge \neg f_0 \rightarrow i_{1a} \downarrow \\
& \neg f_1 \wedge (\neg w_0 \wedge \neg w_1) \rightarrow i_{1a} \uparrow \\
& s \wedge ((i_{0f0} \wedge p) \vee (i_{1f0} \wedge \neg p)) \rightarrow w_0 \downarrow \\
& (\neg s \wedge \neg f_1) \vee \neg r \rightarrow w_0 \uparrow \\
& s \wedge ((i_{0f0} \wedge p) \vee (i_{1f0} \wedge \neg p)) \rightarrow w_1 \downarrow \\
& (\neg s \wedge \neg f_1) \vee \neg r \rightarrow w_1 \uparrow \\
& (i_{1f31} \vee i_{1f31}) \wedge u \rightarrow v \downarrow \\
& (\neg i_{1f31} \wedge \neg i_{1f31}) \vee \neg u \rightarrow v \uparrow \\
& (i_{0f31} \vee i_{0f31}) \wedge v \rightarrow u \downarrow \\
& (\neg i_{0f31} \wedge \neg i_{0f31}) \vee \neg v \rightarrow u \uparrow \\
& s \wedge ((p \wedge i_{0fk}) \vee (\neg p \wedge i_{1fk})) \rightarrow o_{fk} \downarrow & k=0,\dots,31 \\
& (\neg p \wedge \neg i_{0fk}) \vee (\neg p \wedge \neg i_{1fk}) \rightarrow o_{fk} \uparrow & k=0,\dots,31 \\
& s \wedge ((p \wedge i_{0tk}) \vee (\neg p \wedge i_{1tk})) \rightarrow o_{tk} \downarrow & k=0,\dots,31 \\
& (\neg p \wedge \neg i_{0tk}) \vee (\neg p \wedge \neg i_{1tk}) \rightarrow o_{tk} \uparrow & k=0,\dots,31 \\
& \neg f_0 \Rightarrow \neg f_0 \\
& \neg f_1 \Rightarrow \neg f_1 \\
& \neg s \Rightarrow s \\
& \neg p \Rightarrow \neg p \\
& \neg i_{0a} \Rightarrow i_{0a} \\
& \neg i_{1a} \Rightarrow i_{1a} \\
& \neg w_0 \Rightarrow w_0 \\
& \neg w_1 \Rightarrow w_1 \\
& \neg v \Rightarrow v \\
& \neg u \Rightarrow u \\
& \neg r \Rightarrow r \\
& \neg o_{fk} \Rightarrow o_{fk} & k=0,\dots,31 \\
& \neg o_{tk} \Rightarrow o_{tk} & k=0,\dots,31
\end{aligned}$$

Appendix B

Sub-Router Building Blocks

This appendix presents the building blocks used to implement the *ChildRouter* and *ParentRouter*.

The signal, auxiliary-state, guard, and path-delay conventions of Appendix A apply throughout.

B.1 StopcodeSel

This block is responsible for routing a message to either O_0 or O_1 , depending on whether its *ur* field contains the *USCODE*.

B.1.1 CHP

$$\begin{aligned} \text{StopcodeSel}(I?, O_0!, O_1!) \equiv & \\ & * [I?b; \\ & \quad [b.ur = \text{USCODE} \rightarrow *[\neg b.tail \rightarrow I?b; O_0!b] \\ & \quad \parallel b.ur \neq \text{USCODE} \rightarrow *[\neg b.tail \rightarrow O_1!b; I?b]; O_1!b \\ &]] \end{aligned}$$

We replace the nested loops with explicit control state and remove the data buffer.

$$\begin{aligned} \text{StopcodeSel}(I?, O_0!, O_1!) \equiv & \\ s := \text{true}; & \\ * [[s \rightarrow p := \overline{I?.ur} = \text{USCODE}; [p \rightarrow I? \parallel \neg p \rightarrow \text{skip}]; s := \text{false} & \\ \parallel \neg s \rightarrow [p \rightarrow w := \overline{I?.tail}; O_0!I?; s := w] \parallel \neg p \rightarrow w := \overline{I?.tail}; O_1!I?; s := w] & \\]] & \end{aligned}$$

B.1.2 HSE

We introduce two variables, w_0 and w_1 , to encode the next value of s , allowing s to remain stable throughout the execution of the selected branch. We also use the state of w_0 and w_1 to determine whether the condition evaluation has completed. Finally, w_0 and w_1 can be updated while the input data rails are being propagated to the output, reducing the complexity of the resulting PRS. Repeated operations on the dual-rail data signals are expressed using vector notation.

The auxiliary variables introduced below are f_0 for header-selection completion, f_1 for header-input return to spacer, and f_2 for the body-flit acknowledge phase. The pair w_0, w_1 encodes the next value of s .

```

StopcodeSel  $\equiv$ 
* [[s  $\rightarrow$ 
  [it31  $\wedge$  if30  $\wedge$   $\dots$   $\wedge$  if13  $\rightarrow$  p  $\uparrow$   $\parallel$  if31  $\vee$  it30  $\vee$   $\dots$   $\vee$  it13  $\rightarrow$  p  $\downarrow$ ];
  [p  $\rightarrow$  ia  $\uparrow$ ; [ $\neg$ it31  $\wedge$   $\neg$ if31  $\wedge$   $\dots$   $\wedge$   $\neg$ it0  $\wedge$   $\neg$ if0]; ia  $\downarrow$   $\parallel$   $\neg$ p  $\rightarrow$  skip]; s  $\downarrow$ 
 $\parallel$   $\neg$ s  $\rightarrow$ 
  [p  $\rightarrow$ 
    [it0  $\rightarrow$  w1  $\uparrow$   $\parallel$  if0  $\rightarrow$  w0  $\uparrow$ ],  $\langle, k : 31..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle$ ; [o0a]; ia  $\uparrow$ ;
     $\langle, k : 31..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow] \rangle$ ; [ $\neg$ o0a]; ia  $\downarrow$ ;
    [w0  $\rightarrow$  w0  $\downarrow$   $\parallel$  w1  $\rightarrow$  s  $\uparrow$ ; w1  $\downarrow$ ]
   $\parallel$   $\neg$ p  $\rightarrow$ 
    [it0  $\rightarrow$  w1  $\uparrow$   $\parallel$  if0  $\rightarrow$  w0  $\uparrow$ ],  $\langle, k : 31..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle$ ; [o1a]; ia  $\uparrow$ ;
     $\langle, k : 31..0 : [\neg i_{tk} \rightarrow o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{1fk} \downarrow] \rangle$ ; [ $\neg$ o1a]; ia  $\downarrow$ ;
    [w0  $\rightarrow$  w0  $\downarrow$   $\parallel$  w1  $\rightarrow$  s  $\uparrow$ ; w1  $\downarrow$ ]
  ]
]]

```

The portions of the two branches immediately following the output handshake have the same structure. We can therefore move these operations outside the selection.

```

StopcodeSel  $\equiv$ 
* [[s  $\rightarrow$ 
  [it31  $\wedge$  if30  $\wedge$   $\dots$   $\wedge$  if13  $\rightarrow$  p  $\uparrow$   $\parallel$  if31  $\vee$  it30  $\vee$   $\dots$   $\vee$  it13  $\rightarrow$  p  $\downarrow$ ];
  [p  $\rightarrow$  ia  $\uparrow$ ; [ $\neg$ it31  $\wedge$   $\neg$ if31  $\wedge$   $\dots$   $\wedge$   $\neg$ it0  $\wedge$   $\neg$ if0]; ia  $\downarrow$   $\parallel$   $\neg$ p  $\rightarrow$  skip]; s  $\downarrow$ 
 $\parallel$   $\neg$ s  $\rightarrow$ 
  [p  $\rightarrow$  [it0  $\rightarrow$  w1  $\uparrow$   $\parallel$  if0  $\rightarrow$  w0  $\uparrow$ ],  $\langle, k : 31..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle$ 
   $\parallel$   $\neg$ p  $\rightarrow$  [it0  $\rightarrow$  w1  $\uparrow$   $\parallel$  if0  $\rightarrow$  w0  $\uparrow$ ],  $\langle, k : 31..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle$ 
  ]; [o0a  $\vee$  o1a]; ia  $\uparrow$ ;  $\langle, k : 31..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow, o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow, o_{1fk} \downarrow] \rangle$ ;
  [ $\neg$ o0a  $\wedge$   $\neg$ o1a]; ia  $\downarrow$ ; [w0  $\rightarrow$  w0  $\downarrow$   $\parallel$  w1  $\rightarrow$  s  $\uparrow$ ; w1  $\downarrow$ ]
]]

```

We move the final state update before $i_a \downarrow$, so that the next input cannot arrive before the current update completes. The auxiliary state f_0 records completion of the header decision. In the stop-code branch, f_1 additionally records that the discarded header has returned to spacer. Both states are needed: $p = 0$ bypasses header discard, whereas $p = 1$ must wait for its completion.

StopcodeSel $\equiv$

```
* [[s →
  [it31 ∧ if30 ∧ ⋯ ∧ if13 → p ↑ ∥ if31 ∨ it30 ∨ ⋯ ∨ it13 → p ↓]; f0 ↑;
  [p → ia ↑; [¬it31 ∧ ¬if31 ∧ ⋯ ∧ ¬it0 ∧ ¬if0]; f1 ↑; ia ↓ ∥ ¬p → skip]; s ↓
  ∥ ¬s →
  [p → [it0 → w1 ↑ ∥ if0 → w0 ↑]; ⟨, k : 31..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩
  ∥ ¬p → [it0 → w1 ↑ ∥ if0 → w0 ↑]; ⟨, k : 31..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩
  ]; [o0a ∨ o1a]; f0 ↓; f1 ↓; ia ↑; ⟨, k : 31..0 : [¬itk → o0tk ↓, o1tk ↓], [¬ifk → o0fk ↓, o1fk ↓]⟩;
  [¬o0a ∧ ¬o1a]; [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]; ia ↓
  ]]
```

Finally, we introduce the state variable f_2 , which is asserted immediately before $i_a \uparrow$ and deasserted immediately before $i_a \downarrow$ in the $\neg s$ branch. Its purpose is to simplify the production rules for i_a , whose positive and negative guards both contain conditions associated with the s and $\neg s$ branches.

StopcodeSel $\equiv$

```
* [[s →
  [it31 ∧ if30 ∧ ⋯ ∧ if13 → p ↑ ∥ if31 ∨ it30 ∨ ⋯ ∨ it13 → p ↓]; f0 ↑;
  [p → ia ↑; [¬it31 ∧ ¬if31 ∧ ⋯ ∧ ¬it0 ∧ ¬if0]; f1 ↑; ia ↓ ∥ ¬p → skip]; s ↓
  ∥ ¬s →
  [p → [it0 → w1 ↑ ∥ if0 → w0 ↑]; ⟨, k : 31..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩
  ∥ ¬p → [it0 → w1 ↑ ∥ if0 → w0 ↑]; ⟨, k : 31..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩
  ]; [o0a ∨ o1a]; f0 ↓; f1 ↓; f2 ↑; ia ↑; ⟨, k : 31..0 : [¬itk → o0tk ↓, o1tk ↓], [¬ifk → o0fk ↓, o1fk ↓]⟩;
  [¬o0a ∧ ¬o1a]; [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]; f2 ↓; ia ↓
  ]]
```

B.1.3 PRS

The variable f_0 is asserted only when s is high and p has been updated according to the input route field. It is deasserted only when s is low and one of the two output acknowledge signals is high.

$$\begin{aligned} s \wedge (((i_{t31} \wedge i_{f30} \wedge \dots \wedge i_{f13}) \wedge p) \vee ((i_{f31} \vee i_{t30} \vee \dots \vee i_{t13}) \wedge \neg p)) &\rightarrow f_0 \uparrow \\ \neg s \wedge (o_{0a} \vee o_{1a}) \vee r &\rightarrow f_0 \downarrow \end{aligned}$$

The variable f_1 is asserted when the input has returned to spacer while f_0 is high, and is deasserted whenever f_0 is low. We avoid using i_a in the positive

guard because it does not uniquely identify the program state in which f_1 must be asserted.

$$\begin{aligned} f_0 \wedge (\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}) &\rightarrow f_1 \uparrow \\ \neg f_0 \vee r &\rightarrow f_1 \downarrow \end{aligned}$$

The variable f_2 is asserted when one of the output acknowledge signals is high, either w_0 or w_1 is asserted, and both f_0 and f_1 are low. It is deasserted when both output acknowledge signals and both w_0 and w_1 are low.

$$\begin{aligned} (o_{0a} \vee o_{1a}) \wedge (w_1 \vee w_0) \wedge \neg f_0 \wedge \neg f_1 &\rightarrow f_2 \uparrow \\ (\neg o_{0a} \wedge \neg o_{1a}) \wedge (\neg w_1 \wedge \neg w_0) \vee r &\rightarrow f_2 \downarrow \end{aligned}$$

The variable s is asserted by reset and whenever it is updated by w_1 . It is deasserted only in the initialization branch, since it is already low in the other branch. Depending on the value of p , the transition may also need to wait for $f_1 \wedge \neg i_a$.

$$\begin{aligned} w_1 \wedge i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow s \uparrow \\ f_0 \wedge ((f_1 \wedge \neg i_a) \vee \neg p) &\rightarrow s \downarrow \end{aligned}$$

The variable p is modified only in the s branch, before f_0 is asserted, according to the ur field of the input. Reset initializes p high.

$$\begin{aligned} \neg f_0 \wedge (i_{t31} \wedge i_{f30} \wedge \dots \wedge i_{f13} \wedge s) \vee r &\rightarrow p \uparrow \\ \neg f_0 \wedge (i_{f31} \vee i_{t30} \vee \dots \vee i_{t13}) \wedge s &\rightarrow p \downarrow \end{aligned}$$

The condition for asserting i_a differs between the s and $\neg s$ branches. In the s branch, p and f_0 must both be high, whereas in the $\neg s$ branch, f_2 must be high. The negative guard similarly combines conditions from both branches. Since $s \wedge f_0 \wedge p$ may still hold when i_a must be deasserted, $\neg f_1$ is added to the positive guard to prevent interference. Moreover, instead of using $\neg s \wedge \neg f_2$ in the negative guard, we use $\neg f_0 \wedge \neg f_2$, since s may already have returned high before i_a is lowered.

$$\begin{aligned} (\neg s \wedge f_2) \vee (s \wedge f_0 \wedge p \wedge \neg f_1) &\rightarrow i_a \uparrow \\ (\neg f_0 \wedge \neg f_2) \vee (s \wedge f_1) \vee r &\rightarrow i_a \downarrow \end{aligned}$$

The variables w_0 and w_1 are asserted in the $\neg s$ branch according to the value of the least significant input bit. They are deasserted in a state identified by i_a being high, both output acknowledge signals being low, and s having already been updated.

$$\begin{aligned} \neg s \wedge i_{f0} &\rightarrow w_0 \uparrow \\ \neg s \wedge i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow w_0 \downarrow \\ \neg s \wedge i_{t0} &\rightarrow w_1 \uparrow \\ s \wedge i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow w_1 \downarrow \end{aligned}$$

All output rails follow the corresponding input rails, but they may only be asserted after p has selected the appropriate output and the circuit has entered the $\neg s$ branch.

$$\begin{array}{ll}
 p \wedge \neg s \wedge i_{fk} \rightarrow o_{0fk} \uparrow & k=0,\dots,31 \\
 \neg i_{fk} \rightarrow o_{0fk} \downarrow & k=0,\dots,31 \\
 p \wedge \neg s \wedge i_{tk} \rightarrow o_{0tk} \uparrow & k=0,\dots,31 \\
 \neg i_{tk} \rightarrow o_{0tk} \downarrow & k=0,\dots,31
 \end{array}$$

The rules for O_1 use $o_{1\rho k}$ in place of $o_{0\rho k}$ and $\neg p$ in place of p ; all other signals and ranges are unchanged.

In the fork paths below, $j \in \{0, 1\}$, $k \in \{0, \dots, 31\}$, and $\rho \in \{t, f\}$. Indices refer to the connected port and rail on each path.

¶ An isochronic fork on f_0 constrains arrival at $G_{f_1}^+$. The two paths are

$$\begin{array}{l}
 P_d : f_0 \downarrow \rightarrow G_{f_1}^+ \\
 P_i : f_0 \downarrow \rightarrow f_2 \uparrow \rightarrow i_a \uparrow \rightarrow N(I) \rightarrow G_{f_1}^+
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The header-completion guard must be disabled before the producer is allowed to return the input to spacer.

¶ An isochronic fork on s constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{array}{l}
 P_d : s \uparrow \rightarrow G_{o_{j\rho k}}^+ \\
 P_i : s \uparrow \rightarrow w_1 \downarrow \rightarrow f_2 \downarrow \rightarrow i_a \downarrow \rightarrow i_{\rho k} \uparrow \rightarrow G_{o_{j\rho k}}^+
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Header mode must disable body forwarding before the next input arrives.

The same fork also constrains arrival at $G_{w_0}^+$. The two paths are

$$\begin{array}{l}
 P_d : s \uparrow \rightarrow G_{w_0}^+ \\
 P_i : s \uparrow \rightarrow w_1 \downarrow \rightarrow f_2 \downarrow \rightarrow i_a \downarrow \rightarrow i_{f_0} \uparrow \rightarrow G_{w_0}^+
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Header mode must disable body-state capture before the next input can enable it.

¶ An isochronic fork on p constrains arrival at G_s^- . The two paths are

$$\begin{array}{l}
 P_d : p \downarrow \rightarrow G_s^- \\
 P_i : p \downarrow \rightarrow f_0 \uparrow \rightarrow G_s^-
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The mode-update guard must observe the new selection before selection completion can qualify it.

The same fork also constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{array}{l}
 P_d : p \downarrow \rightarrow G_{o_{j\rho k}}^+ \\
 P_i : p \downarrow \rightarrow f_0 \uparrow \rightarrow s \downarrow \rightarrow G_{o_{j\rho k}}^+
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The output guard must observe the selected port before forwarding is enabled.

The same fork also constrains arrival at $G_{i_a}^+$. The two paths are

$$\begin{array}{l}
 P_d : p \downarrow \rightarrow G_{i_a}^+ \\
 P_i : p \downarrow \rightarrow f_0 \uparrow \rightarrow G_{i_a}^+
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The input-acknowledge guard must observe the new selection before selection completion can enable it.

¶ An isochronic fork on i_a constrains arrival at G_s^+ . The two paths are

$$\begin{aligned} P_d : i_a \downarrow \rightarrow G_s^+ \\ P_i : i_a \downarrow \rightarrow i_{t0} \uparrow \rightarrow w_1 \uparrow \rightarrow G_s^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The start-state guard must stop using the old input acknowledge before the next tail bit is captured.

The same fork also constrains arrival at G_{w0}^- . The two paths are

$$\begin{aligned} P_d : i_a \downarrow \rightarrow G_{w0}^- \\ P_i : i_a \downarrow \rightarrow i_{f0} \uparrow \rightarrow G_{w0}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The clearing guard must be disabled before a new non-tail input enables the assertion guard.

¶ An isochronic fork on $i_{\rho h}$ constrains arrival at G_{f1}^+ . The two paths are

$$\begin{aligned} P_d : i_{\rho h} \uparrow \rightarrow G_{f1}^+ \\ P_i : i_{\rho h} \uparrow \rightarrow f_0 \uparrow \rightarrow G_{f1}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. Here $h \in \{13, \dots, 31\}$ and the source is an asserted route rail that participates in the current header decision. That rail must invalidate the spacer test before the decision asserts f_0 . The lower control and payload bits do not feed f_0 ; the PRS still checks all input rails when detecting return to spacer.

¶ An isochronic fork on o_{ja} constrains arrival at G_s^+ . The two paths are

$$\begin{aligned} P_d : o_{ja} \uparrow \rightarrow G_s^+ \\ P_i : o_{ja} \uparrow \rightarrow f_2 \uparrow \rightarrow i_a \uparrow \rightarrow G_s^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit the start-state update before the input acknowledge can qualify it.

The same fork also constrains arrival at G_{w0}^- . The two paths are

$$\begin{aligned} P_d : o_{ja} \uparrow \rightarrow G_{w0}^- \\ P_i : o_{ja} \uparrow \rightarrow f_2 \uparrow \rightarrow i_a \uparrow \rightarrow G_{w0}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit clearing of the non-tail state before the input acknowledge rises.

B.1.4 Bubble Reshuffling

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned}
 f_0 &\rightarrow G_{f_1}^+ \\
 \neg s &\rightarrow G_{o_{0\rho k}}^+ \\
 \neg s &\rightarrow G_{o_{1\rho k}}^+ \\
 \neg s &\rightarrow G_{w_0}^+ \\
 \neg p &\rightarrow G_s^- \\
 p &\rightarrow G_{o_{0\rho k}}^+ \\
 \neg p &\rightarrow G_{o_{1\rho k}}^+ \\
 p &\rightarrow G_{i_a}^+ \\
 i_a &\rightarrow G_s^+ \\
 i_a &\rightarrow G_{w_0}^- \\
 \neg i_{ph} &\rightarrow G_{f_1}^+ \quad h=13,\dots,31; \rho=t,f \\
 \neg o_{ja} &\rightarrow G_s^+ \\
 \neg o_{ja} &\rightarrow G_{w_0}^-
 \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve f_1 , s , p , o_{0a} , o_{1a} , as well as the input data rails and the output rails of o_1 , in their original sense, while implementing w_0 , i_a and the output rails of o_0 in complemented form.

Notice, however, that this polarity assignment gives the wrong sense for both $\neg s \rightarrow G_{w_0}^+$ and $\neg s \rightarrow G_{o_{0\rho k}}^+$, since w_0 and the rails of o_0 are complemented while s is not. The two forks share a common acknowledged path toward the guard $s \wedge i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r \rightarrow w_1 \uparrow$. We can therefore complement w_1 as well, so that the required inversion is shared by the three branches and lies on an acknowledged path.

A second conflict involves $\neg p \rightarrow G_s^-$. In this case, the acknowledged branch leads toward the positive guard of f_0 , whose polarity can still be chosen freely. We resolve the conflict by preserving f_0 in its original sense.

For the f_0 fork toward $G_{f_1}^+$, the physical input is the shared complementary view $\neg f_0$: $G_{f_1}^+$ uses $\neg f_0$, while the body-flit state $\neg f_2$ also depends on $\neg f_0$. The signal f_2 is implemented in complemented form, as shown below.

$$\begin{aligned}
& s \wedge (((i_{t31} \wedge i_{f30} \wedge \dots \wedge i_{f13}) \wedge p) \vee ((i_{f31} \vee i_{t30} \vee \dots \vee i_{t13}) \wedge \neg p)) \rightarrow f_0 \uparrow \\
& \neg s \wedge (o_{0a} \vee o_{1a}) \vee r \rightarrow f_0 \downarrow \\
& \neg f_0 \wedge (\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}) \rightarrow f_1 \uparrow \\
& \neg f_0 \vee r \rightarrow f_1 \downarrow \\
& (o_{0a} \vee o_{1a}) \wedge (\neg w_1 \vee \neg w_0) \wedge \neg f_0 \wedge \neg f_1 \rightarrow \neg f_2 \downarrow \\
& (\neg o_{0a} \wedge \neg o_{1a}) \wedge (\neg w_1 \wedge \neg w_0) \vee r \rightarrow \neg f_2 \uparrow \\
& \neg w_1 \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r \rightarrow s \uparrow \\
& f_0 \wedge ((p \wedge f_1 \wedge i_a) \vee \neg p) \rightarrow s \downarrow \\
& \neg f_0 \wedge (\neg i_{t31} \wedge \neg i_{f30} \wedge \dots \wedge \neg i_{f13} \wedge \neg s) \vee \neg r \rightarrow p \uparrow \\
& \neg f_0 \wedge (i_{f31} \vee i_{t30} \vee \dots \vee i_{t13}) \wedge s \rightarrow p \downarrow \\
& (\neg s \wedge \neg \neg f_2) \vee (s \wedge f_0 \wedge p \wedge \neg f_1) \rightarrow i_a \downarrow \\
& (\neg f_0 \wedge \neg f_2) \vee (s \wedge f_1) \vee r \rightarrow i_a \uparrow \\
& \neg s \wedge i_{f0} \rightarrow w_0 \downarrow \\
& \neg s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r \rightarrow w_0 \uparrow \\
& \neg s \wedge i_{t0} \rightarrow w_1 \downarrow \\
& s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r \rightarrow w_1 \uparrow \\
& p \wedge \neg s \wedge i_{fk} \rightarrow o_{0fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow o_{0fk} \uparrow \quad k=0,\dots,31 \\
& p \wedge \neg s \wedge i_{tk} \rightarrow o_{0tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow o_{0tk} \uparrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg s \wedge i_{fk} \rightarrow o_{1fk} \uparrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow o_{1fk} \downarrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg s \wedge i_{tk} \rightarrow o_{1tk} \uparrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow o_{1tk} \downarrow \quad k=0,\dots,31
\end{aligned}$$

At this point, the polarities of the signals on the right-hand side are fixed. We therefore adjust the guards, introducing explicit inverters wherever the opposite polarity of a polarity-fixed signal is required. An inverter is also added to each output rail of o_0 and to the input acknowledge to restore their original external polarities. Finally, the large completion conditions required by f_1 , p , and f_0 are decomposed using completion trees.

$$\begin{aligned}
 & ((i_{t(4k+3)} \vee i_{f(4k+3)}) \wedge \cdots \wedge (i_{t4k} \vee i_{f4k})) \rightarrow \neg c_k \downarrow & k=0,\dots,7 \\
 & (\neg i_{t(4k+3)} \wedge \neg i_{f(4k+3)} \wedge \cdots \wedge \neg i_{t4k} \wedge \neg i_{f4k}) \rightarrow \neg c_k \uparrow & k=0,\dots,7 \\
 & \neg c_{2k+1} \wedge \neg c_{2k} \rightarrow d_k \uparrow & k=0,\dots,3 \\
 & c_{2k+1} \wedge c_{2k} \rightarrow d_k \downarrow & k=0,\dots,3 \\
 & d_{2k+1} \wedge d_{2k} \rightarrow e_k \downarrow & k=0,1 \\
 & \neg d_{2k+1} \wedge \neg d_{2k} \rightarrow e_k \uparrow & k=0,1 \\
 & \neg e_1 \wedge \neg e_0 \rightarrow c \uparrow \\
 & e_1 \wedge e_0 \rightarrow c \downarrow \\
 & i_{t31} \wedge i_{f30} \wedge \cdots \wedge i_{f27} \rightarrow \neg g_3 \downarrow \\
 & \neg i_{t31} \wedge \neg i_{f31} \wedge \cdots \wedge \neg i_{t27} \wedge \neg i_{f27} \rightarrow \neg g_3 \uparrow \\
 & i_{f26} \wedge \cdots \wedge i_{f22} \rightarrow \neg g_2 \downarrow \\
 & \neg i_{t26} \wedge \neg i_{f26} \wedge \cdots \wedge \neg i_{t22} \wedge \neg i_{f22} \rightarrow \neg g_2 \uparrow \\
 & i_{f21} \wedge \cdots \wedge i_{f17} \rightarrow \neg g_1 \downarrow \\
 & \neg i_{t21} \wedge \neg i_{f21} \wedge \cdots \wedge \neg i_{t17} \wedge \neg i_{f17} \rightarrow \neg g_1 \uparrow \\
 & i_{f16} \wedge \cdots \wedge i_{f13} \rightarrow \neg g_0 \downarrow \\
 & \neg i_{t16} \wedge \neg i_{f16} \wedge \cdots \wedge \neg i_{t13} \wedge \neg i_{f13} \rightarrow \neg g_0 \uparrow \\
 & \neg g_{2k+1} \wedge \neg g_{2k} \rightarrow h_k \uparrow & k=0,1 \\
 & g_{2k+1} \wedge g_{2k} \rightarrow h_k \downarrow & k=0,1 \\
 & h_1 \wedge h_0 \rightarrow \neg z \downarrow \\
 & \neg h_1 \wedge \neg h_0 \rightarrow \neg z \uparrow \\
 & (\neg i_{f31} \vee \neg i_{f30} \vee \cdots \vee \neg i_{t27}) \rightarrow u_3 \uparrow \\
 & i_{t31} \wedge i_{f31} \wedge \cdots \wedge i_{t27} \wedge i_{f27} \rightarrow u_3 \downarrow \\
 & (\neg i_{t26} \vee \cdots \vee \neg i_{t22}) \rightarrow u_2 \uparrow \\
 & i_{t26} \wedge i_{f26} \wedge \cdots \wedge i_{t22} \wedge i_{f22} \rightarrow u_2 \downarrow \\
 & (\neg i_{t21} \vee \cdots \vee \neg i_{t17}) \rightarrow u_1 \uparrow \\
 & i_{t21} \wedge i_{f21} \wedge \cdots \wedge i_{t17} \wedge i_{f17} \rightarrow u_1 \downarrow \\
 & (\neg i_{t16} \vee \cdots \vee \neg i_{t13}) \rightarrow u_0 \uparrow \\
 & i_{t16} \wedge i_{f16} \wedge \cdots \wedge i_{t13} \wedge i_{f13} \rightarrow u_0 \downarrow \\
 & u_{2k+1} \vee u_{2k} \rightarrow \neg x_k \downarrow & k=0,1 \\
 & \neg u_{2k+1} \wedge \neg u_{2k} \rightarrow \neg x_k \uparrow & k=0,1 \\
 & \neg x_1 \vee \neg x_0 \rightarrow y \uparrow \\
 & x_1 \wedge x_0 \rightarrow y \downarrow
 \end{aligned}$$

$$\begin{aligned}
& \neg s \wedge (((\neg z) \wedge \neg p) \vee ((\neg x_1 \vee \neg x_0) \wedge \neg p)) \rightarrow f_0 \uparrow \\
& s \wedge (o_{0a} \vee o_{1a}) \vee r \rightarrow f_0 \downarrow \\
& \neg f_0 \wedge \neg c \rightarrow f_1 \uparrow \\
& \neg f_0 \vee r \rightarrow f_1 \downarrow \\
& (o_{0a} \vee o_{1a}) \wedge (w_1 \vee w_0) \wedge \neg f_0 \wedge \neg f_1 \rightarrow \neg f_2 \downarrow \\
& (\neg o_{0a} \wedge \neg o_{1a}) \wedge (\neg w_1 \wedge \neg w_0) \vee \neg r \rightarrow \neg f_2 \uparrow \\
& \neg w_1 \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee \neg r \rightarrow s \uparrow \\
& f_0 \wedge ((p \wedge f_1 \wedge i_a) \vee \neg p) \rightarrow s \downarrow \\
& \neg f_0 \wedge \neg z \wedge \neg s \vee \neg r \rightarrow p \uparrow \\
& \neg f_0 \wedge y \wedge s \rightarrow p \downarrow \\
& (\neg s \wedge f_2) \vee (s \wedge f_0 \wedge p \wedge \neg f_1) \rightarrow i_a \downarrow \\
& (\neg f_0 \wedge \neg f_2) \vee (\neg s \wedge \neg f_1) \vee \neg r \rightarrow i_a \uparrow \\
& s \wedge i_{f0} \rightarrow w_0 \downarrow \\
& \neg s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee \neg r \rightarrow w_0 \uparrow \\
& s \wedge i_{t0} \rightarrow w_1 \downarrow \\
& \neg s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee \neg r \rightarrow w_1 \uparrow \\
& p \wedge s \wedge i_{fk} \rightarrow o_{0fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow o_{0fk} \uparrow \quad k=0,\dots,31 \\
& p \wedge s \wedge i_{tk} \rightarrow o_{0tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow o_{0tk} \uparrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg s \wedge \neg i_{fk} \rightarrow o_{1fk} \uparrow \quad k=0,\dots,31 \\
& i_{fk} \rightarrow o_{1fk} \downarrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg s \wedge \neg i_{tk} \rightarrow o_{1tk} \uparrow \quad k=0,\dots,31 \\
& i_{tk} \rightarrow o_{1tk} \downarrow \quad k=0,\dots,31 \\
& \neg f_0 \Rightarrow \neg f_0 \\
& \neg f_1 \Rightarrow \neg f_1 \\
& \neg f_2 \Rightarrow f_2 \\
& \neg s \Rightarrow s \\
& \neg p \Rightarrow \neg p \\
& \neg i_a \Rightarrow i_a \\
& \neg w_0 \Rightarrow w_0 \\
& \neg w_1 \Rightarrow w_1 \\
& \neg i_{fk} \Rightarrow i_{fk} \quad k=0,\dots,31 \\
& \neg i_{tk} \Rightarrow i_{tk} \quad k=0,\dots,31 \\
& \neg r \Rightarrow r \\
& \neg o_{0a} \Rightarrow o_{0a} \\
& \neg o_{0fk} \Rightarrow o_{0fk} \quad k=0,\dots,31 \\
& \neg o_{0tk} \Rightarrow o_{0tk} \quad k=0,\dots,31 \\
& \neg o_{1a} \Rightarrow o_{1a}
\end{aligned}$$

B.2 UpdownSel

This block is responsible for routing a message to either O_0 or O_1 according to the value of the most significant bit of the ur field. Before forwarding the first flit, the ur field is shifted by one bit so that the routing information required at the current level is removed.

B.2.1 CHP

$$\begin{aligned}
&\text{UpdownSel}(I?, O_0!, O_1!) \equiv \\
&\quad * [I?b; p := b.msb; b.ur < 1; \\
&\quad \quad [p \rightarrow *[\neg b.tail \rightarrow O_0!b; I?b]; O_0!b \\
&\quad \quad \parallel \neg p \rightarrow *[\neg b.tail \rightarrow O_1!b; I?b]; O_1!b \\
&\quad]]
\end{aligned}$$

We replace the nested loops with explicit control state and remove the data buffer. The constant shift can be implemented using simple wiring. Moreover, to simplify the logic, we assume that every unicast message contains at least one additional flit after the header, since the tail bit is not checked in the initialization branch.

$$\begin{aligned}
&\text{UpdownSel}(I?, O_0!, O_1!) \equiv \\
&\quad s := \text{true}; \\
&\quad * [[s \rightarrow p := \overline{I?}.msb; \\
&\quad \quad [p \rightarrow O_0!(\overline{I?}.(30 : 13), 0, \overline{I?}.(12 : 0)); I?] \parallel \neg p \rightarrow O_1!(\overline{I?}.(30 : 13), 0, \overline{I?}.(12 : 0)); I?]; \\
&\quad \quad s := \text{false} \\
&\quad \parallel \neg s \rightarrow [p \rightarrow w := \overline{I?}.tail; O_0!I?; s := w] \parallel \neg p \rightarrow w := \overline{I?}.tail; O_1!I?; s := w] \\
&\quad]]
\end{aligned}$$

B.2.2 HSE

We introduce two variables, w_0 and w_1 , to encode the next value of s , allowing s to remain stable throughout the execution of the selected branch. Their state also indicates whether the condition evaluation has completed. Finally, w_0 and w_1 can be updated while the input data are being propagated to the output, reducing the complexity of the resulting PRS. Repeated operations on the dual-rail data signals are expressed using vector notation.

The auxiliary variables introduced below are f_0 for header-selection completion and f_1 for the end-of-flit update phase. The pair w_0, w_1 encodes the next value of s .

UpdownSel $\equiv$

```

* [[s →
  [it31 → p ↑ ∥ if31 → p ↓];
  [p →
    ⟨, k : 31..14 : [it(k-1) → o0tk ↑ ∥ if(k-1) → o0fk ↑]⟩, o0f13 ↑,
    ⟨, k : 12..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩; [o0a]; ia ↑;
    ⟨, k : 31..14 : [¬it(k-1) → o0tk ↓], [¬if(k-1) → o0fk ↓]⟩, o0f13 ↓,
    ⟨, k : 12..0 : [¬itk → o0tk ↓], [¬ifk → o0fk ↓]⟩; [¬o0a]; ia ↓
  ] ∥ ¬p →
    ⟨, k : 31..14 : [it(k-1) → o1tk ↑ ∥ if(k-1) → o1fk ↑]⟩, o1f13 ↑,
    ⟨, k : 12..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩; [o1a]; ia ↑;
    ⟨, k : 31..14 : [¬it(k-1) → o1tk ↓], [¬if(k-1) → o1fk ↓]⟩, o1f13 ↓,
    ⟨, k : 12..0 : [¬itk → o1tk ↓], [¬ifk → o1fk ↓]⟩; [¬o1a]; ia ↓
  ]; s ↓
] ∥ ¬s →
  [p →
    [it0 → w1 ↑ ∥ if0 → w0 ↑], ⟨, k : 31..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩; [o0a]; ia ↑;
    ⟨, k : 31..0 : [¬itk → o0tk ↓], [¬ifk → o0fk ↓]⟩; [¬o0a]; ia ↓;
    [w0 → s ↓; w0 ↓ ∥ w1 → s ↑; w1 ↓]
  ] ∥ ¬p →
    [it0 → w1 ↑ ∥ if0 → w0 ↑], ⟨, k : 31..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩; [o1a]; ia ↑;
    ⟨, k : 31..0 : [¬itk → o1tk ↓], [¬ifk → o1fk ↓]⟩; [¬o1a]; ia ↓;
    [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]
  ]
]]

```

The portions of the two p branches immediately following the output handshake have the same structure, both when s is high and when it is low. We can therefore move these operations outside the selection.

UpdownSel $\equiv$

* [[$s \rightarrow$

$[i_{t31} \rightarrow p \uparrow \parallel i_{f31} \rightarrow p \downarrow];$

$[p \rightarrow$

$\langle, k : 31..14 : [i_{t(k-1)} \rightarrow o_{0tk} \uparrow \parallel i_{f(k-1)} \rightarrow o_{0fk} \uparrow] \rangle, o_{0f13} \uparrow,$

$\langle, k : 12..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle$

$\parallel \neg p \rightarrow$

$\langle, k : 31..14 : [i_{t(k-1)} \rightarrow o_{1tk} \uparrow \parallel i_{f(k-1)} \rightarrow o_{1fk} \uparrow] \rangle, o_{1f13} \uparrow,$

$\langle, k : 12..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle$

$]; [o_{0a} \vee o_{1a}]; i_a \uparrow;$

$\langle, k : 31..14 : [\neg i_{t(k-1)} \rightarrow o_{0tk} \downarrow, o_{1tk} \downarrow], [\neg i_{f(k-1)} \rightarrow o_{0fk} \downarrow, o_{1fk} \downarrow] \rangle, o_{0f13} \downarrow, o_{1f13} \downarrow,$

$\langle, k : 12..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow, o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow, o_{1fk} \downarrow] \rangle; [\neg o_{0a} \wedge \neg o_{1a}]; i_a \downarrow; s \downarrow$

$\parallel \neg s \rightarrow$

$[p \rightarrow [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle, k : 31..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle$

$\parallel \neg p \rightarrow [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle, k : 31..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle$

$]; [o_{0a} \vee o_{1a}]; i_a \uparrow; \langle, k : 31..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow, o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow, o_{1fk} \downarrow] \rangle;$

$[\neg o_{0a} \wedge \neg o_{1a}]; i_a \downarrow; [w_0 \rightarrow w_0 \downarrow \parallel w_1 \rightarrow s \uparrow; w_1 \downarrow]$

$]]$

We move the body-phase state update before $i_a \downarrow$. The auxiliary state f_0 identifies completion of the header decision; f_1 identifies the subsequent end-of-flit update. The input acknowledge is released only after s , w_0 , and w_1 reach their required states. This prevents the next input from arriving while the current iteration is still completing.

UpdownSel $\equiv$

```

* [[s →
  [it31 → p ↑ ∥ if31 → p ↓]; f0 ↑;
  [p →
    ⟨, k : 31..14 : [it(k-1) → o0tk ↑ ∥ if(k-1) → o0fk ↑]⟩, o0f13 ↑,
    ⟨, k : 12..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩
  ∥ ¬p →
    ⟨, k : 31..14 : [it(k-1) → o1tk ↑ ∥ if(k-1) → o1fk ↑]⟩, o1f13 ↑,
    ⟨, k : 12..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩
  ]; [o0a ∨ o1a]; ia ↑;
  ⟨, k : 31..14 : [¬it(k-1) → o0tk ↓, o1tk ↓], [¬if(k-1) → o0fk ↓, o1fk ↓]⟩, o0f13 ↓, o1f13 ↓,
  ⟨, k : 12..0 : [¬itk → o0tk ↓, o1tk ↓], [¬ifk → o0fk ↓, o1fk ↓]⟩;
  [¬o0a ∧ ¬o1a]; f1 ↑; s ↓; f1 ↓; ia ↓
  ∥ ¬s →
    [p → [it0 → w1 ↑ ∥ if0 → w0 ↑], ⟨, k : 31..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩
    ∥ ¬p → [it0 → w1 ↑ ∥ if0 → w0 ↑], ⟨, k : 31..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩
  ]; [o0a ∨ o1a]; f0 ↓; ia ↑; ⟨, k : 31..0 : [¬itk → o0tk ↓, o1tk ↓], [¬ifk → o0fk ↓, o1fk ↓]⟩;
  [¬o0a ∧ ¬o1a]; f1 ↑; [w0 → w0 ↓ ∥ w1 → s ↑; w1 ↓]; f1 ↓; ia ↓
  ]]

```

B.2.3 PRS

The variable f_0 is asserted only when s is high and p has been updated according to the input MSB. It is deasserted only when s is low and one of the two output acknowledge signals is high.

$$\begin{aligned}
s \wedge ((i_{t31} \wedge p) \vee (i_{f31} \wedge \neg p)) &\rightarrow f_0 \uparrow \\
\neg s \wedge (o_{0a} \vee o_{1a}) \vee r &\rightarrow f_0 \downarrow
\end{aligned}$$

The variable f_1 is asserted when i_a is high, both output acknowledge signals are low, and the state required to complete the current branch has been reached. It is deasserted after the corresponding updates of s , w_0 , and w_1 have completed. The guards are strengthened to prevent interference between the two transitions.

$$\begin{aligned}
i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \wedge ((s \wedge f_0) \vee (\neg s \wedge (w_0 \vee w_1))) &\rightarrow f_1 \uparrow \\
(\neg w_1 \wedge \neg w_0) \wedge ((f_0 \wedge \neg s) \vee \neg f_0) \vee r &\rightarrow f_1 \downarrow
\end{aligned}$$

The variable s is asserted by reset and whenever the next state encoded by w_1 is selected. It is deasserted when the header phase has completed, as indicated by f_0 and f_1 .

$$\begin{aligned}
w_1 \wedge f_1 \vee r &\rightarrow s \uparrow \\
f_0 \wedge f_1 &\rightarrow s \downarrow
\end{aligned}$$

The variable p is modified only in the s branch according to the most significant bit of the input, except during reset.

$$\begin{aligned} & (i_{t31} \wedge s) \vee r \rightarrow p \uparrow \\ & i_{f31} \wedge s \rightarrow p \downarrow \end{aligned}$$

i_a must change only after the internal state variables have been updated. Its positive guard always requires one of the output acknowledge signals to be high. In the $\neg s$ branch, it additionally waits for f_0 to be low and for either w_0 or w_1 to be asserted. Its negative guard requires f_1 to be low and distinguishes the completion conditions of the two branches to avoid interference with the positive guard.

$$\begin{aligned} & (o_{0a} \vee o_{1a}) \wedge ((\neg s \wedge (\neg f_0 \wedge (w_1 \vee w_0))) \vee s) \rightarrow i_a \uparrow \\ & \neg f_1 \wedge (\neg w_0 \wedge \neg w_1) \wedge ((f_0 \wedge \neg s) \vee \neg f_0) \vee r \rightarrow i_a \downarrow \end{aligned}$$

The variables w_0 and w_1 are asserted in the $\neg s$ branch according to the least significant input bit. They are deasserted once f_1 is high and s has been updated.

$$\begin{aligned} & \neg s \wedge i_{f0} \rightarrow w_0 \uparrow \\ & \neg s \wedge f_1 \vee r \rightarrow w_0 \downarrow \\ & \neg s \wedge i_{t0} \rightarrow w_1 \uparrow \\ & s \wedge f_1 \vee r \rightarrow w_1 \downarrow \end{aligned}$$

Output bit 13 is different because its shifted value is always zero. Therefore, the true rail can only be asserted in the unshifted case. The false rail can instead be asserted either from the corresponding input rail in the unshifted case or directly during the shifted header phase. The condition $\neg i_a$ prevents interference between the positive and negative guards.

$$\begin{aligned} & p \wedge ((i_{f13} \wedge \neg s) \vee (\neg i_a \wedge f_0 \wedge s)) \rightarrow o_{0f13} \uparrow \\ & (\neg i_{f13} \wedge \neg s) \vee (i_a \wedge s) \rightarrow o_{0f13} \downarrow \\ & p \wedge i_{t13} \wedge \neg s \rightarrow o_{0t13} \uparrow \\ & (\neg i_{t13} \wedge \neg s) \vee r \rightarrow o_{0t13} \downarrow \\ & O_1: \text{ same rules, with } p \mapsto \neg p. \end{aligned}$$

The output bits from 31 down to 14 contain the shifted input when s is high and a direct copy of the corresponding input when s is low. In the shifted case, the assertion also waits for f_0 to guarantee that the output port selected by p is stable.

$$\begin{aligned} & p \wedge ((i_{fk} \wedge \neg s) \vee (i_{f(k-1)} \wedge s \wedge f_0)) \rightarrow o_{0fk} \uparrow & k=14, \dots, 31 \\ & (\neg i_{fk} \wedge \neg s) \vee (\neg i_{f(k-1)} \wedge s) \rightarrow o_{0fk} \downarrow & k=14, \dots, 31 \\ & p \wedge ((i_{tk} \wedge \neg s) \vee (i_{t(k-1)} \wedge s \wedge f_0)) \rightarrow o_{0tk} \uparrow & k=14, \dots, 31 \\ & (\neg i_{tk} \wedge \neg s) \vee (\neg i_{t(k-1)} \wedge s) \rightarrow o_{0tk} \downarrow & k=14, \dots, 31 \end{aligned}$$

The rules for O_1 use o_{1pk} in place of o_{0pk} and $\neg p$ in place of p , including complemented output names.

The output bits from 12 down to 0 always follow their corresponding input bits. During the shifted header phase, their assertion still waits for f_0 to guarantee that the output selection is stable.

$$\begin{array}{ll}
 p \wedge i_{fk} \wedge (\neg s \vee (s \wedge f_0)) \rightarrow o_{0fk} \uparrow & k=0,\dots,12 \\
 \neg i_{fk} \rightarrow o_{0fk} \downarrow & k=0,\dots,12 \\
 p \wedge i_{tk} \wedge (\neg s \vee (s \wedge f_0)) \rightarrow o_{0tk} \uparrow & k=0,\dots,12 \\
 \neg i_{tk} \rightarrow o_{0tk} \downarrow & k=0,\dots,12 \\
 O_1: \text{ same rules, with } p \mapsto \neg p.
 \end{array}$$

In the fork paths below, $j \in \{0, 1\}$ and $\rho \in \{t, f\}$. Output-bit indices follow the three ranges defined above: 14 : 31, bit 13, and 0 : 12. A data-path constraint applies only where the displayed guard actually contains the stated dependency.

¶¶ An isochronic fork on f_1 constrains arrival at $G_{i_a}^-$. The two paths are

$$\begin{array}{l}
 P_d : f_1 \uparrow \rightarrow G_{i_a}^- \\
 P_i : f_1 \uparrow \rightarrow s \downarrow \rightarrow G_{i_a}^-
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Header completion must inhibit acknowledge release before the mode change qualifies that release.

The same fork also constrains arrival at $G_{i_a}^-$. The two paths are

$$\begin{array}{l}
 P_d : f_1 \uparrow \rightarrow G_{i_a}^- \\
 P_i : f_1 \uparrow \rightarrow w_0 \downarrow \rightarrow G_{i_a}^-
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Completion must inhibit acknowledge release before the non-tail state is cleared.

The same fork also constrains arrival at $G_{i_a}^-$. The two paths are

$$\begin{array}{l}
 P_d : f_1 \uparrow \rightarrow G_{i_a}^- \\
 P_i : f_1 \uparrow \rightarrow s \uparrow \rightarrow w_1 \downarrow \rightarrow G_{i_a}^-
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Completion must inhibit acknowledge release before the tail-state update qualifies that release.

The same fork also constrains arrival at $G_{w_0}^-$. The two paths are

$$\begin{array}{l}
 P_d : f_1 \downarrow \rightarrow G_{w_0}^- \\
 P_i : f_1 \downarrow \rightarrow i_a \downarrow \rightarrow i_{f0} \uparrow \rightarrow G_{w_0}^+
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The non-tail clearing guard must be disabled before the next non-tail input can assert the state.

¶¶ An isochronic fork on s constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{array}{l}
 P_d : s \downarrow \rightarrow G_{o_{j\rho k}}^+ \\
 P_i : s \downarrow \rightarrow f_1 \downarrow \rightarrow i_a \downarrow \rightarrow i_{\rho k} \uparrow \rightarrow G_{o_{j\rho k}}^+
 \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The next body word must not be interpreted using the previous header shift. For a shifted header rail, use its connected

input index $k - 1$ instead of k . The source also specifies the corresponding mode-arrival condition before a new header.

The same fork also constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{aligned} P_d &: s \uparrow \rightarrow G_{o_{j\rho k}}^+ \\ P_i &: s \uparrow \rightarrow w_1 \downarrow \rightarrow f_1 \downarrow \rightarrow i_a \downarrow \rightarrow i_{\rho k} \uparrow \rightarrow G_{o_{j\rho k}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. Header mode must reach the output guards before the next input is released. The connected input index is $k - 1$ for shifted header rails and k for unshifted rails; the constant zero rail has no data-input dependency.

The same fork also constrains arrival at G_p^σ . The two paths are

$$\begin{aligned} P_d &: s \downarrow \rightarrow G_p^\sigma \\ P_i &: s \downarrow \rightarrow f_1 \downarrow \rightarrow i_a \downarrow \rightarrow i_{\rho 31} \uparrow \rightarrow G_p^\sigma \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. Header selection must be disabled before the next body MSB can affect the selected port. Here $(\rho, \sigma) \in \{(t, +), (f, -)\}$.

¶¶ An isochronic fork on p constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{aligned} P_d &: p \downarrow \rightarrow G_{o_{j\rho k}}^+ \\ P_i &: p \downarrow \rightarrow f_0 \uparrow \rightarrow G_{o_{j\rho k}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The selected port must reach each header-output guard before header forwarding is enabled. The same condition applies to $p \uparrow$.

¶¶ An isochronic fork on i_a constrains arrival at $G_{f_1}^+$. The two paths are

$$\begin{aligned} P_d &: i_a \downarrow \rightarrow G_{f_1}^+ \\ P_i &: i_a \downarrow \rightarrow i_{f0} \uparrow \rightarrow w_0 \uparrow \rightarrow G_{f_1}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The withdrawn input acknowledge must disable completion before the next body word sets its state. The corresponding tail path replaces i_{f0}, w_0 with i_{t0}, w_1 .

¶¶ An isochronic fork on o_{ja} constrains arrival at $G_{f_1}^+$. The two paths are

$$\begin{aligned} P_d &: o_{ja} \uparrow \rightarrow G_{f_1}^+ \\ P_i &: o_{ja} \uparrow \rightarrow i_a \uparrow \rightarrow G_{f_1}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit completion detection before the input acknowledge rises.

The same fork also constrains arrival at $G_{f_0}^-$. The two paths are

$$\begin{aligned} P_d &: o_{ja} \downarrow \rightarrow G_{f_0}^- \\ P_i &: o_{ja} \downarrow \rightarrow f_1 \uparrow \rightarrow s \downarrow \rightarrow G_{f_0}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The old output acknowledge must stop qualifying the body-phase clearing guard before header mode ends.

B.2.4 Bubble Reshuffling

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned}
 \neg f_1 &\rightarrow G_{i_a}^- \\
 f_1 &\rightarrow G_{w_0}^- \\
 s &\rightarrow G_{o_{ipk}}^+ \\
 \neg s &\rightarrow G_{o_{ipk}}^+ \\
 s &\rightarrow G_p^+ \\
 s &\rightarrow G_p^- \\
 p &\rightarrow G_{o_{opk}}^+ \\
 \neg p &\rightarrow G_{o_{opk}}^+ \\
 i_a &\rightarrow G_{f_1}^+ \\
 \neg o_{ja} &\rightarrow G_{f_1}^+ \\
 o_{ja} &\rightarrow G_{f_0}^-
 \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve $s, f_0, p, f_1, o_{0a}, o_{1a}$ in their original sense, while implementing w_1, i_a and the output rails of o_0 and o_1 in complemented form. We preserve w_0 in its original sense. This keeps its data-dependent rule in its natural polarity, while the complemented sense required by some guards can be generated explicitly after the fork-critical polarities have been fixed. We also preserve the reset and input data rails in their original sense. The resulting PRS is:

$$\begin{aligned}
& s \wedge ((i_{t31} \wedge p) \vee (i_{f31} \wedge \neg p)) \rightarrow f_0 \uparrow \\
& \neg s \wedge (o_{0a} \vee o_{1a}) \vee r \rightarrow f_0 \downarrow \\
& \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \wedge ((s \wedge f_0) \vee (\neg s \wedge (w_0 \vee \neg w_1))) \rightarrow f_1 \uparrow \\
& (\neg w_1 \wedge \neg w_0) \wedge ((f_0 \wedge \neg s) \vee \neg f_0) \vee r \rightarrow f_1 \downarrow \\
& \neg w_1 \wedge f_1 \vee r \rightarrow s \uparrow \\
& f_0 \wedge f_1 \rightarrow s \downarrow \\
& (i_{t31} \wedge s) \vee r \rightarrow p \uparrow \\
& i_{f31} \wedge s \rightarrow p \downarrow \\
& (o_{0a} \vee o_{1a}) \wedge ((\neg s \wedge (\neg f_0 \wedge (\neg w_1 \vee w_0))) \vee s) \rightarrow i_a \downarrow \\
& \neg f_1 \wedge (\neg w_0 \wedge \neg w_1) \wedge ((f_0 \wedge \neg s) \vee \neg f_0) \vee r \rightarrow i_a \uparrow \\
& \neg s \wedge i_{f0} \rightarrow w_0 \uparrow \\
& \neg s \wedge f_1 \vee r \rightarrow w_0 \downarrow \\
& \neg s \wedge i_{t0} \rightarrow \neg w_1 \downarrow \\
& s \wedge f_1 \vee r \rightarrow \neg w_1 \uparrow \\
& p \wedge ((i_{f13} \wedge \neg s) \vee (i_a \wedge f_0 \wedge s)) \rightarrow o_{0f13} \downarrow \\
& (\neg i_{f13} \wedge \neg s) \vee (\neg i_a \wedge s) \vee r \rightarrow o_{0f13} \uparrow \\
& p \wedge ((i_{fk} \wedge \neg s) \vee (i_{f(k-1)} \wedge s \wedge f_0)) \rightarrow o_{0fk} \downarrow \quad k=14,\dots,31 \\
& (\neg i_{fk} \wedge \neg s) \vee (\neg i_{f(k-1)} \wedge s) \rightarrow o_{0fk} \uparrow \quad k=14,\dots,31 \\
& p \wedge i_{fk} \wedge (\neg s \vee (s \wedge f_0)) \rightarrow o_{0fk} \downarrow \quad k=0,\dots,12 \\
& \neg i_{fk} \rightarrow o_{0fk} \uparrow \quad k=0,\dots,12 \\
& p \wedge i_{t13} \wedge \neg s \rightarrow o_{0t13} \downarrow \\
& (\neg i_{t13} \wedge \neg s) \vee r \rightarrow o_{0t13} \uparrow \\
& p \wedge ((i_{tk} \wedge \neg s) \vee (i_{t(k-1)} \wedge s \wedge f_0)) \rightarrow o_{0tk} \downarrow \quad k=14,\dots,31 \\
& (\neg i_{tk} \wedge \neg s) \vee (\neg i_{t(k-1)} \wedge s) \rightarrow o_{0tk} \uparrow \quad k=14,\dots,31 \\
& p \wedge i_{tk} \wedge (\neg s \vee (s \wedge f_0)) \rightarrow o_{0tk} \downarrow \quad k=0,\dots,12 \\
& \neg i_{tk} \rightarrow o_{0tk} \uparrow \quad k=0,\dots,12
\end{aligned}$$

O_1 : same rules, with $p \mapsto \neg p$.

At this point, the polarities of the signals on the right-hand side are fixed. We therefore adjust the guards, introducing explicit inverters wherever the opposite polarity of a polarity-fixed signal is required. Finally, an inverter is added to each output rail and to the input acknowledge to restore their original polarities before they are observed by the surrounding blocks.

$$\begin{aligned}
& \neg s \wedge ((\neg i_{t31} \wedge \neg p) \vee (\neg i_{f31} \wedge \neg p)) \rightarrow f_0 \uparrow \\
& s \wedge (o_{0a} \vee o_{1a}) \vee r \rightarrow f_0 \downarrow \\
& \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \wedge ((\neg s \wedge \neg f_0) \vee (\neg s \wedge (\neg w_0 \vee \neg w_1))) \rightarrow f_1 \uparrow \\
& (\neg w_1 \wedge \neg w_0) \wedge ((f_0 \wedge s) \vee \neg f_0) \vee r \rightarrow f_1 \downarrow \\
& (\neg w_1 \wedge \neg f_1) \vee \neg r \rightarrow s \uparrow \\
& f_0 \wedge f_1 \rightarrow s \downarrow \\
& (\neg i_{t31} \wedge \neg s) \vee \neg r \rightarrow p \uparrow \\
& i_{f31} \wedge s \rightarrow p \downarrow \\
& (o_{0a} \vee o_{1a}) \wedge ((s \wedge (\neg f_0 \wedge (w_1 \vee w_0))) \vee s) \rightarrow i_a \downarrow \\
& \neg f_1 \wedge (\neg w_0 \wedge \neg w_1) \wedge ((\neg f_0 \wedge \neg s) \vee \neg f_0) \vee \neg r \rightarrow i_a \uparrow \\
& \neg s \wedge \neg i_{f0} \rightarrow w_0 \uparrow \\
& s \wedge f_1 \vee r \rightarrow w_0 \downarrow \\
& s \wedge i_{t0} \rightarrow w_1 \downarrow \\
& (\neg s \wedge \neg f_1) \vee \neg r \rightarrow w_1 \uparrow \\
& p \wedge ((i_{f13} \wedge s) \vee (i_a \wedge f_0 \wedge s)) \rightarrow o_{0f13} \downarrow \\
& (\neg i_{f13} \wedge \neg s) \vee (\neg i_a \wedge \neg s) \vee \neg r \rightarrow o_{0f13} \uparrow \\
& p \wedge ((i_{fk} \wedge s) \vee (i_{f(k-1)} \wedge s \wedge f_0)) \rightarrow o_{0fk} \downarrow \quad k=14,\dots,31 \\
& (\neg i_{fk} \wedge \neg s) \vee (\neg i_{f(k-1)} \wedge \neg s) \rightarrow o_{0fk} \uparrow \quad k=14,\dots,31 \\
& p \wedge i_{fk} \wedge (s \vee (s \wedge f_0)) \rightarrow o_{0fk} \downarrow \quad k=0,\dots,12 \\
& \neg i_{fk} \rightarrow o_{0fk} \uparrow \quad k=0,\dots,12 \\
& p \wedge i_{t13} \wedge s \rightarrow o_{0t13} \downarrow \\
& (\neg i_{t13} \wedge \neg s) \vee \neg r \rightarrow o_{0t13} \uparrow \\
& p \wedge ((i_{tk} \wedge s) \vee (i_{t(k-1)} \wedge s \wedge f_0)) \rightarrow o_{0tk} \downarrow \quad k=14,\dots,31 \\
& (\neg i_{tk} \wedge \neg s) \vee (\neg i_{t(k-1)} \wedge \neg s) \rightarrow o_{0tk} \uparrow \quad k=14,\dots,31 \\
& p \wedge i_{tk} \wedge (s \vee (s \wedge f_0)) \rightarrow o_{0tk} \downarrow \quad k=0,\dots,12 \\
& \neg i_{tk} \rightarrow o_{0tk} \uparrow \quad k=0,\dots,12 \\
& \neg f_0 \Rightarrow \neg f_0 \\
& \neg f_1 \Rightarrow \neg f_1 \\
& \neg s \Rightarrow s \\
& \neg p \Rightarrow \neg p \\
& \neg i_a \Rightarrow i_a \\
& \neg w_0 \Rightarrow w_0 \\
& \neg w_1 \Rightarrow w_1 \\
& \neg i_{f0} \Rightarrow i_{f0} \\
& \neg i_{f31} \Rightarrow i_{f31} \\
& \neg i_{t31} \Rightarrow i_{t31} \\
& \neg r \Rightarrow r \\
& \neg o_{0fk} \Rightarrow o_{0fk} \quad k=0,\dots,31 \\
& \neg o_{0tk} \Rightarrow o_{0tk} \quad k=0,\dots,31
\end{aligned}$$

O_1 : $p \mapsto \neg p$; apply the output-name substitution to the inverter definitions as well.

B.3 ChildSel

This block routes the message to one of its four output ports based on the two most significant bits of the header. Once these bits have been read and consumed, the *ur* field is shifted left by two positions.

B.3.1 CHP

$$\begin{aligned}
\text{ChildSel}(I?, O_0!, O_1!, O_2!, O_3!) \equiv & \\
& * [I?b; p := b.(msb : msb - 1); b.ur << 2; \\
& \quad [p = 0 \rightarrow *[\neg b.tail \rightarrow O_0!b; I?b]; O_0!b \\
& \quad \parallel p = 1 \rightarrow *[\neg b.tail \rightarrow O_1!b; I?b]; O_1!b \\
& \quad \parallel p = 2 \rightarrow *[\neg b.tail \rightarrow O_2!b; I?b]; O_2!b \\
& \quad \parallel p = 3 \rightarrow *[\neg b.tail \rightarrow O_3!b; I?b]; O_3!b \\
&]]
\end{aligned}$$

The first flit is used to determine the child port to which the message must be routed. In particular, the two most significant bits of the *ur* field are extracted and stored in *p*, after which the *ur* field is shifted two positions to the left so that the routing information consumed at the current level is removed. The message is then routed to one of the four output ports according to the value of *p*.

This operation can be naturally decomposed into a tree of three *UpdownSel* blocks, which have already been designed. The upper *UpdownSel* examines the current MSB of the *ur* field and shifts the header by one position. If the MSB is one, the message is directed toward the subtree containing child ports *O*₂ and *O*₃; otherwise, it is directed toward the subtree containing *O*₀ and *O*₁. The new MSB of the shifted header is then examined by the corresponding lower *UpdownSel*, which routes the message to the final child port. Therefore, the two levels of *UpdownSel* blocks together consume the two routing bits required by the *ChildSel*, as shown in the figure.

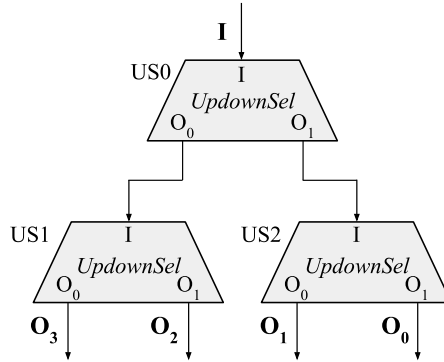


Figure B.1: ChildSel decomposition

B.4 MltcSel

This block is responsible for routing a message to either O_0 or O_1 depending on the value of its *mltc* bit.

B.4.1 CHP

$$\begin{aligned} \text{MltcSel}(I?, O_0!, O_1!) \equiv & \\ & * [I?b; \\ & \quad [b.mltc \rightarrow *[\neg b.tail \rightarrow O_0!b; I?b]; O_0!b \\ & \quad \parallel \neg b.mltc \rightarrow *[\neg b.tail \rightarrow O_1!b; I?b]; O_1!b \\ &]] \end{aligned}$$

We replace the nested loops with explicit control state and remove the data buffer.

$$\begin{aligned} \text{MltcSel}(I?, O_0!, O_1!) \equiv & \\ & s := \text{true}; \\ & * [[s \rightarrow p := \overline{I?.mltc}; s := \text{false} \\ & \quad \parallel \neg s \rightarrow [p \rightarrow w := \overline{I?.tail}; O_0!I?; s := w] \parallel \neg p \rightarrow w := \overline{I?.tail}; O_1!I?; s := w] \\ &]] \end{aligned}$$

B.4.2 HSE

We introduce two variables, w_0 and w_1 , to encode the next value of s , allowing s to remain stable throughout the execution of the selected branch. Their state also indicates whether the condition evaluation has completed. Finally, w_0 and w_1 can be updated while the input data rails are being propagated to the output, reducing the complexity of the resulting PRS. Repeated operations on the dual-rail data signals are expressed using vector notation.

The auxiliary variable f_0 , introduced below, records completion of header selection. The pair w_0, w_1 encodes the next value of s .

$$\begin{aligned}
\text{MltcSel} \equiv & \\
& * [[s \rightarrow [i_{t1} \rightarrow p \uparrow \parallel i_{f1} \rightarrow p \downarrow]; s \downarrow \\
& \quad \parallel \neg s \rightarrow \\
& \quad [p \rightarrow \\
& \quad \quad [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle; [o_{0a}]; i_a \uparrow; \\
& \quad \quad \langle , k : 31..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow] \rangle; [\neg o_{0a}]; i_a \downarrow; \\
& \quad \quad [w_0 \rightarrow s \downarrow; w_0 \downarrow \parallel w_1 \rightarrow s \uparrow; w_1 \downarrow] \\
& \quad \parallel \neg p \rightarrow \\
& \quad \quad [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle; [o_{1a}]; i_a \uparrow; \\
& \quad \quad \langle , k : 31..0 : [\neg i_{tk} \rightarrow o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{1fk} \downarrow] \rangle; [\neg o_{1a}]; i_a \downarrow; \\
& \quad \quad [w_0 \rightarrow w_0 \downarrow \parallel w_1 \rightarrow s \uparrow; w_1 \downarrow] \\
& \quad] \\
&]]
\end{aligned}$$

The portions of the two branches immediately following the output handshake have the same structure. We can therefore move these operations outside the selection.

$$\begin{aligned}
\text{MltcSel} \equiv & \\
& * [[s \rightarrow [i_{t1} \rightarrow p \uparrow \parallel i_{f1} \rightarrow p \downarrow]; s \downarrow \\
& \quad \parallel \neg s \rightarrow \\
& \quad [p \rightarrow [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle \\
& \quad \parallel \neg p \rightarrow [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle \\
& \quad]; [o_{0a} \vee o_{1a}]; i_a \uparrow; \langle , k : 31..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow, o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow, o_{1fk} \downarrow] \rangle; \\
& \quad [\neg o_{0a} \wedge \neg o_{1a}]; i_a \downarrow; [w_0 \rightarrow w_0 \downarrow \parallel w_1 \rightarrow s \uparrow; w_1 \downarrow] \\
&]]
\end{aligned}$$

Driving s low directly in the initialization branch is not possible because the state reached after updating p is not unique unless s is included in its own guard. We therefore introduce the additional state variable f_0 . We also ensure that i_a is lowered, and therefore that a new input can be requested, only after all the internal state variables have been updated. This prevents new input data from being received or the outputs from being updated while the current iteration is still being completed.

$$\begin{aligned}
\text{MltcSel} \equiv & \\
& * [[s \rightarrow [i_{t1} \rightarrow p \uparrow \parallel i_{f1} \rightarrow p \downarrow]; f_0 \uparrow; s \downarrow \\
& \quad \parallel \neg s \rightarrow \\
& \quad [p \rightarrow [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{0tk} \uparrow \parallel i_{fk} \rightarrow o_{0fk} \uparrow] \rangle \\
& \quad \parallel \neg p \rightarrow [i_{t0} \rightarrow w_1 \uparrow \parallel i_{f0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{1tk} \uparrow \parallel i_{fk} \rightarrow o_{1fk} \uparrow] \rangle \\
& \quad]; [o_{0a} \vee o_{1a}]; f_0 \downarrow; i_a \uparrow; \langle , k : 31..0 : [\neg i_{tk} \rightarrow o_{0tk} \downarrow, o_{1tk} \downarrow], [\neg i_{fk} \rightarrow o_{0fk} \downarrow, o_{1fk} \downarrow] \rangle; \\
& \quad [\neg o_{0a} \wedge \neg o_{1a}]; [w_0 \rightarrow w_0 \downarrow \parallel w_1 \rightarrow s \uparrow; w_1 \downarrow]; i_a \downarrow \\
&]]
\end{aligned}$$

B.4.3 PRS

The variable f_0 is asserted only when s is high and p has been updated according to the *mltc* bit. It is deasserted only when s is low and one of the two output acknowledge signals is high.

$$\begin{aligned} s \wedge ((i_{f1} \wedge p) \vee (i_{f1} \wedge \neg p)) &\rightarrow f_0 \uparrow \\ \neg s \wedge (o_{0a} \vee o_{1a}) \vee r &\rightarrow f_0 \downarrow \end{aligned}$$

The variable s is asserted by reset and whenever the next state encoded by w_1 is selected. The corresponding program state is identified by i_a being high and both output acknowledge signals being low. It is deasserted when f_0 is high.

$$\begin{aligned} w_1 \wedge i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow s \uparrow \\ f_0 \rightarrow s &\downarrow \end{aligned}$$

The variable p is modified only in the s branch according to the *mltc* bit of the input, except during reset.

$$\begin{aligned} (i_{f1} \wedge s) \vee r &\rightarrow p \uparrow \\ i_{f1} \wedge s &\rightarrow p \downarrow \end{aligned}$$

i_a must change only after w_0 and w_1 have been updated. To assert it, one of the two output acknowledge signals must be high, either w_0 or w_1 must be asserted, and f_0 must be low. To deassert it, both output acknowledge signals and both w_0 and w_1 must be low.

$$\begin{aligned} (o_{0a} \vee o_{1a}) \wedge (w_1 \vee w_0) \wedge \neg f_0 &\rightarrow i_a \uparrow \\ (\neg o_{0a} \wedge \neg o_{1a}) \wedge (\neg w_1 \wedge \neg w_0) &\rightarrow i_a \downarrow \end{aligned}$$

The variables w_0 and w_1 are asserted in the $\neg s$ branch according to the value of the least significant input bit. They are deasserted once i_a is high, both output acknowledge signals are low, and s has been updated.

$$\begin{aligned} \neg s \wedge i_{f0} &\rightarrow w_0 \uparrow \\ \neg s \wedge i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow w_0 \downarrow \\ \neg s \wedge i_{f0} &\rightarrow w_1 \uparrow \\ s \wedge i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow w_1 \downarrow \end{aligned}$$

The output rails of both ports follow the corresponding input rails. Before being asserted, however, they must wait for p to select the correct output port and for the circuit to enter the $\neg s$ branch.

$$\begin{aligned} p \wedge \neg s \wedge i_{fk} &\rightarrow o_{0fk} \uparrow & k=0,\dots,31 \\ \neg i_{fk} &\rightarrow o_{0fk} \downarrow & k=0,\dots,31 \\ p \wedge \neg s \wedge i_{tk} &\rightarrow o_{0tk} \uparrow & k=0,\dots,31 \\ \neg i_{tk} &\rightarrow o_{0tk} \downarrow & k=0,\dots,31 \\ O_1: &\text{ same rules, with } p \mapsto \neg p. \end{aligned}$$

In the fork paths below, $j \in \{0, 1\}$, $k \in \{0, \dots, 31\}$, $\rho \in \{t, f\}$, and $\ell \in \{0, 1\}$.

¶ An isochronic fork on s constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{aligned} P_d : s \uparrow &\rightarrow G_{o_{j\rho k}}^+ \\ P_1 : s \uparrow &\rightarrow w_1 \downarrow \rightarrow i_a \downarrow \rightarrow i_{\rho k} \uparrow \rightarrow G_{o_{j\rho k}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The next header must not be forwarded under the previous body-phase selection.

The same fork also constrains arrival at $G_{w_0}^+$. The two paths are

$$\begin{aligned} P_d : s &\uparrow \rightarrow G_{w_0}^+ \\ P_i : s &\uparrow \rightarrow w_1 \downarrow \rightarrow i_a \downarrow \rightarrow i_{f_0} \uparrow \rightarrow G_{w_0}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The next header must not be captured as a body-phase non-tail state.

¶ An isochronic fork on p constrains arrival at $G_{o_{jpk}}^+$. The two paths are

$$\begin{aligned} P_d : p &\rightarrow G_{o_{jpk}}^+ \\ P_i : p &\rightarrow f_0 \uparrow \rightarrow s \downarrow \rightarrow G_{o_{jpk}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The selected output must reach the data guards before forwarding is enabled.

¶ An isochronic fork on i_a constrains arrival at G_s^+ . The two paths are

$$\begin{aligned} P_d : i_a &\downarrow \rightarrow G_s^+ \\ P_i : i_a &\downarrow \rightarrow i_{t_0} \uparrow \rightarrow w_1 \uparrow \rightarrow G_s^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The start-state guard must stop using the old acknowledge before a new tail bit is captured.

¶ An isochronic fork on o_{ja} constrains arrival at G_s^+ . The two paths are

$$\begin{aligned} P_d : o_{ja} &\uparrow \rightarrow G_s^+ \\ P_i : o_{ja} &\uparrow \rightarrow i_a \uparrow \rightarrow G_s^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit the start-state update before the input acknowledge can qualify it.

The same fork also constrains arrival at $G_{w_\ell}^-$. The two paths are

$$\begin{aligned} P_d : o_{ja} &\uparrow \rightarrow G_{w_\ell}^- \\ P_i : o_{ja} &\uparrow \rightarrow i_a \uparrow \rightarrow G_{w_\ell}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The next-state clearing guards must observe the output acknowledge before the input acknowledge rises.

B.4.4 Bubble Reshuffling

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned}
\neg s &\rightarrow G_{o_{0\rho k}}^+ \\
\neg s &\rightarrow G_{o_{1\rho k}}^+ \\
\neg s &\rightarrow G_{w_0}^+ \\
p &\rightarrow G_{o_{0\rho k}}^+ \\
\neg p &\rightarrow G_{o_{1\rho k}}^+ \\
i_a &\rightarrow G_s^+ \\
\neg o_{ja} &\rightarrow G_s^+ \\
\neg o_{ja} &\rightarrow G_{w_0}^- \\
\neg o_{ja} &\rightarrow G_{w_1}^-
\end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve s , f_0 , p , o_{0a} , o_{1a} , together with the input rails and the output rails of o_1 , in their original sense, while implementing w_0 , w_1 , i_a and the output rails of o_0 in complemented form.

$$\begin{aligned}
s \wedge ((i_{t1} \wedge p) \vee (i_{f1} \wedge \neg p)) &\rightarrow f_0 \uparrow \\
\neg s \wedge (o_{0a} \vee o_{1a}) \vee r &\rightarrow f_0 \downarrow \\
\neg w_1 \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow s \uparrow \\
f_0 &\rightarrow s \downarrow \\
(i_{t1} \wedge s) \vee r &\rightarrow p \uparrow \\
i_{f1} \wedge s &\rightarrow p \downarrow \\
(o_{0a} \vee o_{1a}) \wedge (\neg w_1 \vee \neg w_0) \wedge \neg f_0 &\rightarrow i_a \downarrow \\
(\neg o_{0a} \wedge \neg o_{1a}) \wedge (\neg w_1 \wedge \neg w_0) &\rightarrow i_a \uparrow \\
\neg s \wedge i_{f0} &\rightarrow w_0 \downarrow \\
\neg s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow w_0 \uparrow \\
\neg s \wedge i_{t0} &\rightarrow w_1 \downarrow \\
s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee r &\rightarrow w_1 \uparrow \\
p \wedge \neg s \wedge i_{fk} &\rightarrow o_{0fk} \downarrow & k=0,\dots,31 \\
\neg i_{fk} &\rightarrow o_{0fk} \uparrow & k=0,\dots,31 \\
p \wedge \neg s \wedge i_{tk} &\rightarrow o_{0tk} \downarrow & k=0,\dots,31 \\
\neg i_{tk} &\rightarrow o_{0tk} \uparrow & k=0,\dots,31 \\
\neg p \wedge \neg s \wedge i_{fk} &\rightarrow o_{1fk} \uparrow & k=0,\dots,31 \\
\neg i_{fk} &\rightarrow o_{1fk} \downarrow & k=0,\dots,31 \\
\neg p \wedge \neg s \wedge i_{tk} &\rightarrow o_{1tk} \uparrow & k=0,\dots,31 \\
\neg i_{tk} &\rightarrow o_{1tk} \downarrow & k=0,\dots,31
\end{aligned}$$

At this point, the fork-critical polarities are fixed. We therefore propagate the remaining bubbles through the guards, introducing explicit inverters whenever the opposite polarity of a polarity-fixed signal is required. Finally, an inverter is added to the input acknowledge and to each output rail of o_0 to restore their original polarities before they are observed by the surrounding blocks.

$$\begin{aligned}
& \neg s \wedge ((\neg i_{t1} \wedge \neg p) \vee (\neg i_{f1} \wedge \neg p)) \rightarrow f_0 \uparrow \\
& s \wedge (o_{0a} \vee o_{1a}) \vee r \rightarrow f_0 \downarrow \\
& \neg w_1 \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee \neg r \rightarrow s \uparrow \\
& f_0 \rightarrow s \downarrow \\
& (\neg i_{t1} \wedge \neg s) \vee \neg r \rightarrow p \uparrow \\
& i_{f1} \wedge s \rightarrow p \downarrow \\
& (o_{0a} \vee o_{1a}) \wedge (w_1 \vee w_0) \wedge \neg f_0 \rightarrow i_a \downarrow \\
& (\neg o_{0a} \wedge \neg o_{1a}) \wedge (\neg w_1 \wedge \neg w_0) \rightarrow i_a \uparrow \\
& s \wedge i_{f0} \rightarrow w_0 \downarrow \\
& \neg s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee \neg r \rightarrow w_0 \uparrow \\
& s \wedge i_{t0} \rightarrow w_1 \downarrow \\
& \neg s \wedge \neg i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \vee \neg r \rightarrow w_1 \uparrow \\
& p \wedge s \wedge i_{fk} \rightarrow o_{0fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow o_{0fk} \uparrow \quad k=0,\dots,31 \\
& p \wedge s \wedge i_{tk} \rightarrow o_{0tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow o_{0tk} \uparrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg s \wedge \neg i_{fk} \rightarrow o_{1fk} \uparrow \quad k=0,\dots,31 \\
& i_{fk} \rightarrow o_{1fk} \downarrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg s \wedge \neg i_{tk} \rightarrow o_{1tk} \uparrow \quad k=0,\dots,31 \\
& i_{tk} \rightarrow o_{1tk} \downarrow \quad k=0,\dots,31 \\
& \neg f_0 \Rightarrow \neg f_0 \\
& \neg s \Rightarrow s \\
& \neg p \Rightarrow p \\
& \neg i_a \Rightarrow i_a \\
& \neg w_0 \Rightarrow w_0 \\
& \neg w_1 \Rightarrow w_1 \\
& \neg i_{fk} \Rightarrow i_{fk} \quad k=0,\dots,31 \\
& \neg i_{tk} \Rightarrow i_{tk} \quad k=0,\dots,31 \\
& \neg r \Rightarrow r \\
& \neg o_{0fk} \Rightarrow o_{0fk} \quad k=0,\dots,31 \\
& \neg o_{0tk} \Rightarrow o_{0tk} \quad k=0,\dots,31
\end{aligned}$$

Appendix C

Multicast Unit Building Blocks

This appendix presents the internal building blocks of the *MulticastUnit*.

The signal, auxiliary-state, guard, and path-delay conventions of Appendix A apply throughout.

C.1 InputRouter

The *InputRouter* directs each incoming 32-bit word either to the route-parsing datapath or to the payload-distribution datapath, switching between the two according to the detected *MSCODE* and returning to its initial state when the tail flit is received.

C.1.1 CHP

$$\begin{aligned} \text{InputRouter}(I?, O_0!, O_1!) &\equiv \\ p &:= \text{true}; \\ * [[p \rightarrow p &:= \neg(\overline{I?}.(7:4) = \text{MSCODE}); O_0!I? \parallel \neg p \rightarrow p &:= \overline{I?}.tail; O_1!I?]] \end{aligned}$$

We introduce w to store the future value of p . This is useful when writing the HSE, since p cannot be updated before the selected branch has completed.

$$\begin{aligned} \text{InputRouter}(I?, O_0!, O_1!) &\equiv \\ p &:= \text{true}; \\ * [[p \rightarrow w &:= \neg(\overline{I?}.(7:4) = \text{MSCODE}); O_0!I?; p &:= w \parallel \neg p \rightarrow w &:= \overline{I?}.tail; O_1!I?; p &:= w]] \end{aligned}$$

C.1.2 HSE

We introduce two variables, w_0 and w_1 , to encode the future value of p . Their values can be updated while the input data are being propagated to the output, so no sequential dependency is required between these operations.

Repeated operations on the dual-rail data signals are expressed using vector notation.

Here w_0 encodes the next value $p = 0$ and w_1 encodes $p = 1$. The auxiliary variable f_0 , introduced below, identifies the end-of-flit update phase.

```
InputRouter ≡
* [[p →
  [(if7 ∧ if6 ∧ if5 ∧ if4) → w0 ↑ ∥ (it7 ∨ it6 ∨ it5 ∨ it4) → w1 ↑],
  ⟨, k : 31..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩; [o0a]; ia ↑;
  ⟨, k : 31..0 : [¬itk → o0tk ↓, [¬ifk → o0fk ↓]⟩; [¬o0a]; ia ↓; [w0 → p ↓; w0 ↓ ∥ w1 → w1 ↓]
  ∥ ¬p →
  [if0 → w0 ↑ ∥ it0 → w1 ↑], ⟨, k : 31..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩; [o1a]; ia ↑;
  ⟨, k : 31..0 : [¬itk → o1tk ↓, [¬ifk → o1fk ↓]⟩; [¬o1a]; ia ↓; [w0 → w0 ↓ ∥ w1 → p ↑; w1 ↓]
]]
```

The portions of the two branches immediately following the output update have the same structure. We can therefore move them outside the selection.

```
InputRouter ≡
* [[p →
  [(if7 ∧ if6 ∧ if5 ∧ if4) → w0 ↑ ∥ (it7 ∨ it6 ∨ it5 ∨ it4) → w1 ↑],
  ⟨, k : 31..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩
  ∥ ¬p → [if0 → w0 ↑ ∥ it0 → w1 ↑], ⟨, k : 31..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩
  ]; [o0a ∨ o1a]; ia ↑; ⟨, k : 31..0 : [¬itk → o0tk ↓, o1tk ↓, [¬ifk → o0fk ↓, o1fk ↓]⟩; [¬o0a ∧ ¬o1a];
  ia ↓; [w0 → p ↓; w0 ↓ ∥ w1 → p ↑; w1 ↓]
]
```

Since the update of p is now outside the selection, it can be moved to a portion of the program characterized by a unique state, before i_a is deasserted. This guarantees that no new input can be received before the internal state has been updated. To simplify the resulting PRS, we introduce the state variable f_0 , which is asserted when i_a is high while both output acknowledge signals are low.

```
InputRouter ≡
* [[p →
  [(if7 ∧ if6 ∧ if5 ∧ if4) → w0 ↑ ∥ (it7 ∨ it6 ∨ it5 ∨ it4) → w1 ↑],
  ⟨, k : 31..0 : [itk → o0tk ↑ ∥ ifk → o0fk ↑]⟩
  ∥ ¬p → [if0 → w0 ↑ ∥ it0 → w1 ↑], ⟨, k : 31..0 : [itk → o1tk ↑ ∥ ifk → o1fk ↑]⟩
  ]; [o0a ∨ o1a]; ia ↑; ⟨, k : 31..0 : [¬itk → o0tk ↓, o1tk ↓, [¬ifk → o0fk ↓, o1fk ↓]⟩; [¬o0a ∧ ¬o1a];
  f0 ↑; [w0 → p ↓; w0 ↓ ∥ w1 → p ↑; w1 ↓]; f0 ↓; ia ↓
]
```

C.1.3 PRS

The variable f_0 is asserted when i_a is high, both output acknowledge signals are low, and either w_1 or w_0 is high. It is deasserted when both write variables have returned low.

$$\begin{aligned} i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \wedge (w_1 \vee w_0) &\rightarrow f_0 \uparrow \\ \neg w_1 \wedge \neg w_0 &\rightarrow f_0 \downarrow \end{aligned}$$

To update p , we wait until f_0 is high and then set its value according to w_1 and w_0 . The rail w_1 encodes the next value $p = 1$, and w_0 encodes $p = 0$. Reset also initializes p high.

$$\begin{aligned} w_1 \wedge f_0 \vee r &\rightarrow p \uparrow \\ w_0 \wedge f_0 &\rightarrow p \downarrow \end{aligned}$$

The input acknowledge is asserted whenever one of the output acknowledge signals is high and either w_1 or w_0 is asserted. It is deasserted only after both write variables have returned low and f_0 has been deasserted.

$$\begin{aligned} (w_1 \vee w_0) \wedge (o_{0a} \vee o_{1a}) &\rightarrow i_a \uparrow \\ (\neg w_1 \wedge \neg w_0) \wedge \neg f_0 &\rightarrow i_a \downarrow \end{aligned}$$

The following rules describe w_1 and w_0 . They are asserted by two complementary conditions that depend on the current value of p . They are deasserted only when f_0 is high and p has been updated to the required value.

$$\begin{aligned} ((i_{f0} \wedge \neg p) \vee ((i_{f7} \wedge i_{f6} \wedge i_{f5} \wedge i_{f4}) \wedge p)) &\rightarrow w_0 \uparrow \\ \neg p \wedge f_0 \vee r &\rightarrow w_0 \downarrow \\ ((i_{t0} \wedge \neg p) \vee ((i_{t7} \vee i_{t6} \vee i_{t5} \vee i_{t4}) \wedge p)) &\rightarrow w_1 \uparrow \\ p \wedge f_0 \vee r &\rightarrow w_1 \downarrow \end{aligned}$$

The output rails follow the corresponding input rails and are enabled according to the value of p .

$$\begin{aligned} p \wedge i_{fk} &\rightarrow o_{0fk} \uparrow & k=0,\dots,31 \\ \neg i_{fk} &\rightarrow o_{0fk} \downarrow & k=0,\dots,31 \\ p \wedge i_{tk} &\rightarrow o_{0tk} \uparrow & k=0,\dots,31 \\ \neg i_{tk} &\rightarrow o_{0tk} \downarrow & k=0,\dots,31 \end{aligned}$$

The rules for O_1 use $o_{1\rho k}$ in place of $o_{0\rho k}$ and $\neg p$ in place of p ; all other signals and ranges are unchanged.

In the fork paths below, $j \in \{0, 1\}$, $k \in \{0, \dots, 31\}$, $\rho \in \{t, f\}$, and $\ell \in \{0, 1\}$. The event $V(I)$ refers to the valid input condition used by the active branch, not to an additional validity signal.

¶ An isochronic fork on f_0 constrains arrival at $G_{w_\ell}^-$. The two paths are

$$\begin{aligned} P_d : f_0 \downarrow &\rightarrow G_{w_\ell}^- \\ P_i : f_0 \downarrow &\rightarrow i_a \downarrow \rightarrow V(I) \rightarrow G_{w_\ell}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The clearing guard must be disabled before new input data can assert the corresponding next-state rail.

The same fork also constrains arrival at G_p^σ . The two paths are

$$\begin{aligned} P_d &: f_0 \downarrow \rightarrow G_p^\sigma \\ P_i &: f_0 \downarrow \rightarrow i_a \downarrow \rightarrow V(I) \rightarrow w_\ell \uparrow \rightarrow G_p^\sigma \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The selection-update guard must stop using the old completion phase before a new next-state value is captured. Here $(\ell, \sigma) \in \{(0, -), (1, +)\}$.

The same fork also constrains arrival at $G_{i_a}^-$. The two paths are

$$\begin{aligned} P_d &: f_0 \uparrow \rightarrow G_{i_a}^- \\ P_i &: f_0 \uparrow \rightarrow w_\ell \downarrow \rightarrow G_{i_a}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The input-acknowledge guard must observe completion high before the next-state rails are cleared.

¶ An isochronic fork on p constrains arrival at $G_{o_{jk}}^+$. The two paths are

$$\begin{aligned} P_d &: p \rightarrow G_{o_{jk}}^+ \\ P_i &: p \rightarrow w_\ell \downarrow \rightarrow i_a \downarrow \rightarrow i_{\rho k} \uparrow \rightarrow G_{o_{jk}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The updated route/payload selection must reach the output guards before the next input arrives.

¶ An isochronic fork on o_{ja} constrains arrival at $G_{f_0}^+$. The two paths are

$$\begin{aligned} P_d &: o_{ja} \uparrow \rightarrow G_{f_0}^+ \\ P_i &: o_{ja} \uparrow \rightarrow i_a \uparrow \rightarrow G_{f_0}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit completion detection before the input acknowledge can enable it.

C.1.4 Bubble Reshuffling

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned} f_0 &\rightarrow G_{w_1}^- \\ f_0 &\rightarrow G_{w_0}^- \\ f_0 &\rightarrow G_p^+ \\ f_0 &\rightarrow G_p^- \\ \neg f_0 &\rightarrow G_{i_a}^- \\ p &\rightarrow G_{o_{\rho k}}^+ \\ \neg p &\rightarrow G_{o_{1\rho k}}^+ \\ \neg o_{ja} &\rightarrow G_{f_0}^+ \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve $p, f_0, i_a, o_{0a}, o_{1a}$, together with the input rails and the output rails of o_1 , in their original sense, while implementing w_1, w_0 and the output rails of o_0 in complemented form. Whenever the opposite sense of a forked signal is

required, the corresponding inverter is shared with an acknowledged branch so that its output transition is itself acknowledged. The resulting PRS is:

$$\begin{aligned}
& i_a \wedge (\neg o_{0a} \wedge \neg o_{1a}) \wedge (\neg w_1 \vee \neg w_0) \rightarrow f_0 \uparrow \\
& \neg w_1 \wedge \neg w_0 \rightarrow f_0 \downarrow \\
& (\neg w_1 \wedge f_0) \vee r \rightarrow p \uparrow \\
& \neg w_0 \wedge f_0 \rightarrow p \downarrow \\
& (\neg w_1 \vee \neg w_0) \wedge (o_{0a} \vee o_{1a}) \rightarrow i_a \uparrow \\
& (\neg w_1 \wedge \neg w_0) \wedge \neg f_0 \rightarrow i_a \downarrow \\
& ((i_{f0} \wedge \neg p) \vee ((i_{f7} \wedge i_{f6} \wedge i_{f5} \wedge i_{f4}) \wedge p)) \rightarrow \neg w_0 \downarrow \\
& (\neg p \wedge f_0) \vee r \rightarrow \neg w_0 \uparrow \\
& ((i_{t0} \wedge \neg p) \vee ((i_{t7} \vee i_{t6} \vee i_{t5} \vee i_{t4}) \wedge p)) \rightarrow \neg w_1 \downarrow \\
& (p \wedge f_0) \vee r \rightarrow \neg w_1 \uparrow \\
& p \wedge i_{fk} \rightarrow \neg o_{0fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow \neg o_{0fk} \uparrow \quad k=0,\dots,31 \\
& p \wedge i_{tk} \rightarrow \neg o_{0tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow \neg o_{0tk} \uparrow \quad k=0,\dots,31 \\
& \neg p \wedge i_{fk} \rightarrow o_{1fk} \uparrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow o_{1fk} \downarrow \quad k=0,\dots,31 \\
& \neg p \wedge i_{tk} \rightarrow o_{1tk} \uparrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow o_{1tk} \downarrow \quad k=0,\dots,31
\end{aligned}$$

At this point, the fork-critical polarities are fixed. We therefore propagate the remaining bubbles through the guards. The complemented senses required at shared forks are generated before the split, with their transitions acknowledged through the corresponding acknowledged branches. The output rails of o_0 are inverted again to restore their original external polarity.

$$\begin{aligned}
& \neg i_a \wedge \neg o_{0a} \wedge \neg o_{1a} \wedge (\neg w_1 \vee \neg w_0) \rightarrow f_0 \uparrow \\
& w_1 \wedge w_0 \rightarrow f_0 \downarrow \\
& (\neg w_1 \wedge \neg f_0) \vee \neg r \rightarrow p \uparrow \\
& w_0 \wedge f_0 \rightarrow p \downarrow \\
& (\neg w_1 \vee \neg w_0) \wedge (\neg o_{0a} \vee \neg o_{1a}) \rightarrow i_a \uparrow \\
& w_1 \wedge w_0 \wedge \neg f_0 \rightarrow i_a \downarrow \\
& ((i_{f0} \wedge \neg p) \vee ((i_{f7} \wedge i_{f6} \wedge i_{f5} \wedge i_{f4}) \wedge p)) \rightarrow w_0 \downarrow \\
& (\neg p \wedge \neg f_0) \vee \neg r \rightarrow w_0 \uparrow \\
& ((i_{t0} \wedge \neg p) \vee ((i_{t7} \vee i_{t6} \vee i_{t5} \vee i_{t4}) \wedge p)) \rightarrow w_1 \downarrow \\
& (\neg p \wedge \neg f_0) \vee \neg r \rightarrow w_1 \uparrow \\
& p \wedge i_{fk} \rightarrow o_{0fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow o_{0fk} \uparrow \quad k=0,\dots,31 \\
& p \wedge i_{tk} \rightarrow o_{0tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow o_{0tk} \uparrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg i_{fk} \rightarrow o_{1fk} \uparrow \quad k=0,\dots,31 \\
& i_{fk} \rightarrow o_{1fk} \downarrow \quad k=0,\dots,31 \\
& \neg p \wedge \neg i_{tk} \rightarrow o_{1tk} \uparrow \quad k=0,\dots,31 \\
& i_{tk} \rightarrow o_{1tk} \downarrow \quad k=0,\dots,31 \\
& \neg f_0 \Rightarrow \neg f_0 \\
& \neg p \Rightarrow \neg p \\
& \neg i_a \Rightarrow \neg i_a \\
& \neg w_0 \Rightarrow w_0 \\
& \neg i_{fk} \Rightarrow i_{fk} \quad k=0,\dots,31 \\
& \neg i_{tk} \Rightarrow i_{tk} \quad k=0,\dots,31 \\
& \neg r \Rightarrow r \\
& \neg o_{0a} \Rightarrow o_{0a} \\
& \neg o_{0fk} \Rightarrow o_{0fk} \quad k=0,\dots,31 \\
& \neg o_{0tk} \Rightarrow o_{0tk} \quad k=0,\dots,31 \\
& \neg o_{1a} \Rightarrow o_{1a}
\end{aligned}$$

C.2 Serializer

The *Serializer* receives a 32-bit input word and sequentially forwards seven 4-bit slices, from the most significant to the least significant. The final nibble, corresponding to bits [3 : 0], is omitted because it contains only flags whose values can be inferred internally by the multicast unit.

$$\begin{aligned}
& \text{Serializer}(I?, O!) \equiv \\
& * [O!\overline{I?}.(31 : 28); O!\overline{I?}.(27 : 24); O!\overline{I?}.(23 : 20); O!\overline{I?}.(19 : 16); \\
& \quad O!\overline{I?}.(15 : 12); O!\overline{I?}.(11 : 8); O!\overline{I?}.(7 : 4); I \\
&]
\end{aligned}$$

To decompose this unit, we use a gated structure in which a *Sequencer* enables one 4-bit slice after another. These gates are called *QSlicer* blocks, since each one forwards a quadruplet of the input word. Their outputs are then combined by a *MergeTree* composed of smaller *Merge4Bits* blocks. Since individual portions of a channel cannot be read independently, a *WordBuffer* first receives and stores the complete 32-bit input word, allowing the seven slices to be consumed separately.

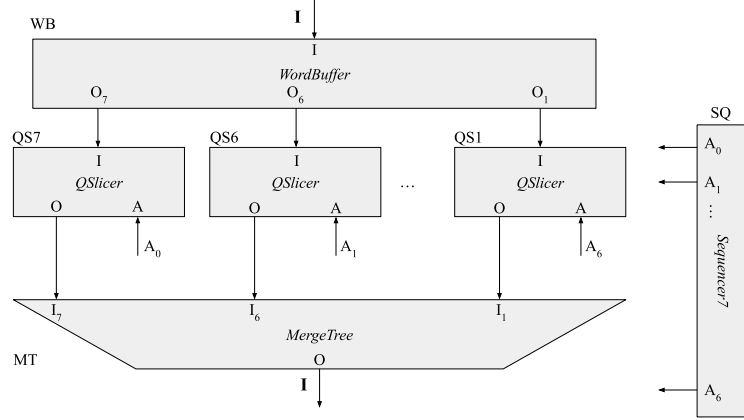


Figure C.1: Serializer decomposition

C.2.1 CHP

WordBuffer

The *WordBuffer* stores the complete input word and exposes its seven relevant quadruplets through independent output channels. The input is acknowledged only after all seven slices have been consumed, as required by the surrounding system.

$$\begin{aligned} \text{WordBuffer}(I, O_7, O_6, O_5, O_4, O_3, O_2, O_1) \equiv & \\ * [[\bar{I}]; b := \bar{I}?; & \\ O_7!b.(31 : 28), O_6!b.(27 : 24), O_5!b.(23 : 20), O_4!b.(19 : 16), & \\ O_3!b.(15 : 12), O_2!b.(11 : 8), O_1!b.(7 : 4); I & \\] & \end{aligned}$$

QSlicer

The *QSlicer* waits for the enable signal generated by the *Sequencer* and then forwards the 4-bit slice connected to its input.

$$\begin{aligned} \text{QSlicer}(I?, A, O!) \equiv & \\ * [[\bar{A}]; O!I?; A] & \end{aligned}$$

C.2.2 HSE

WordBuffer

Inside the *WordBuffer*, the complete input word is stored using one state variable for each input bit.

The stored word is $b_{31:0}$. The validity state f_0 records capture of that word, while q_j records service of output slice O_j , for $j \in \{1, \dots, 7\}$. These functional state variables retain their names.

$$\begin{aligned} \text{WordBuffer}(I, O_7, O_6, O_5, O_4, O_3, O_2, O_1) \equiv \\ * [\langle, k : 31..0 : [i_{tk} \rightarrow b_k \uparrow \parallel i_{fk} \rightarrow b_k \downarrow] \rangle; \\ \langle, j : 7..1 : \\ (\langle, k : 3..0 : [b_{4j+k} \rightarrow o_{jtk} \uparrow \parallel \neg b_{4j+k} \rightarrow o_{jfk} \uparrow] \rangle; [o_{ja}]; \langle, k : 3..0 : o_{jtk} \downarrow, o_{jfk} \downarrow \rangle; [\neg o_{ja}]) \\ \rangle; i_a \uparrow; [\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; i_a \downarrow \\] \end{aligned}$$

We first need to determine when all seven parallel output handshakes have completed, so that the input can be acknowledged and a new word can subsequently be stored. At the same time, each set of output rails must remain at spacer after its corresponding communication has completed. We therefore introduce seven state variables, $q_7, \dots, q_1$, which record the completion of the individual output handshakes. In addition, we introduce the state variable f_0 to simplify the PRS of i_a and to indicate whether the buffer currently contains a valid captured word.

$$\begin{aligned} \text{WordBuffer}(I, O_7, O_6, O_5, O_4, O_3, O_2, O_1) \equiv \\ * [\langle, k : 31..0 : [i_{tk} \rightarrow b_k \uparrow \parallel i_{fk} \rightarrow b_k \downarrow] \rangle; f_0 \uparrow; \\ \langle, j : 7..1 : \\ (\langle, k : 3..0 : [b_{4j+k} \rightarrow o_{jtk} \uparrow \parallel \neg b_{4j+k} \rightarrow o_{jfk} \uparrow] \rangle; [o_{ja}]; q_j \uparrow; \langle, k : 3..0 : o_{jtk} \downarrow, o_{jfk} \downarrow \rangle; [\neg o_{ja}]) \\ \rangle; i_a \uparrow; [\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; \langle, j : 7..1 : q_j \downarrow \rangle; f_0 \downarrow; i_a \downarrow \\] \end{aligned}$$

QSlicer

$$\begin{aligned} \text{QSlicer} \equiv \\ * [[a_r]; \langle, k : 3..0 : [i_{tk} \rightarrow o_{tk} \uparrow \parallel i_{fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; i_a \uparrow; a_a \uparrow; [\neg a_r]; \\ \langle, k : 3..0 : [\neg i_{tk} \rightarrow o_{tk} \downarrow], [\neg i_{fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg o_a]; i_a \downarrow; a_a \downarrow \\] \end{aligned}$$

C.2.3 PRS

WordBuffer

The PRS below describes the state variable f_0 , which is asserted once every bit stored in the buffer matches the corresponding valid input rail. To deassert f_0 , all input rails must have returned to spacer and all output-handshake state variables must have been cleared.

$$\begin{aligned} ((i_{t31} \wedge b_{31}) \vee (i_{f31} \wedge \neg b_{31})) \wedge \dots \wedge ((i_{t0} \wedge b_0) \vee (i_{f0} \wedge \neg b_0)) \rightarrow f_0 \uparrow \\ (\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}) \wedge (\neg q_7 \wedge \dots \wedge \neg q_1) \vee r \rightarrow f_0 \downarrow \end{aligned}$$

Each state variable q_j records whether its corresponding output handshake has been acknowledged. It is asserted by the output acknowledge and cleared when i_a is asserted.

$$\begin{array}{ll} o_{ja} \rightarrow q_j \uparrow & j=1,\dots,7 \\ i_a \vee r \rightarrow q_j \downarrow & j=1,\dots,7 \end{array}$$

The input acknowledge i_a is asserted once the complete output sequence has finished. Since O_1 is always the final slice to be served, it is sufficient to wait for q_1 to be high and o_{1a} to have returned low. The acknowledge is deasserted once f_0 returns low.

$$\begin{array}{l} q_1 \wedge \neg o_{1a} \rightarrow i_a \uparrow \\ \neg f_0 \rightarrow i_a \downarrow \end{array}$$

The stored buffer bits are updated according to the input rails only while f_0 is low, indicating that the previous iteration has fully completed.

$$\begin{array}{ll} \neg f_0 \wedge i_{tk} \rightarrow b_k \uparrow & k=0,\dots,31 \\ (\neg f_0 \wedge i_{fk}) \vee r \rightarrow b_k \downarrow & k=0,\dots,31 \end{array}$$

The output rails follow the corresponding portions of the stored word while the buffer contains valid data. A rail is enabled only while f_0 is high, the corresponding slice has not yet been served, and i_a is still low. Once the corresponding q_j is asserted, the rails return to spacer.

$$\begin{array}{ll} \neg q_j \wedge f_0 \wedge \neg i_a \wedge \neg b_{4j+k} \rightarrow o_{jfk} \uparrow & j=1,\dots,7; k=0,\dots,3 \\ q_j \vee r \rightarrow o_{jfk} \downarrow & j=1,\dots,7; k=0,\dots,3 \\ \neg q_j \wedge f_0 \wedge \neg i_a \wedge b_{4j+k} \rightarrow o_{jtk} \uparrow & j=1,\dots,7; k=0,\dots,3 \\ q_j \vee r \rightarrow o_{jtk} \downarrow & j=1,\dots,7; k=0,\dots,3 \end{array}$$

In the fork paths below, $j \in \{1, \dots, 7\}$ identifies the slice, $k \in \{0, \dots, 3\}$ identifies a bit within that slice, and $\rho \in \{t, f\}$. The associated buffer bit is b_{4j+k} .

¶ An isochronic fork on f_0 constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{array}{l} P_d : f_0 \downarrow \rightarrow G_{o_{j\rho k}}^+ \\ P_i : f_0 \downarrow \rightarrow i_a \downarrow \rightarrow G_{o_{j\rho k}}^+ \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Captured-word validity must disable the output rails before the input acknowledge returns low.

¶ An isochronic fork on i_a constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{array}{l} P_d : i_a \uparrow \rightarrow G_{o_{j\rho k}}^+ \\ P_i : i_a \uparrow \rightarrow q_j \downarrow \rightarrow G_{o_{j\rho k}}^+ \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The input acknowledge must disable the output rails before the served-slice states are cleared.

¶ An isochronic fork on b_{4j+k} constrains arrival at $G_{o_{j\rho k}}^+$. The two paths are

$$\begin{array}{l} P_d : b_{4j+k} \rightarrow G_{o_{j\rho k}}^+ \\ P_i : b_{4j+k} \rightarrow f_0 \uparrow \rightarrow G_{o_{j\rho k}}^+ \end{array}$$

We require $\tau(P_d) < \tau(P_1)$. A captured buffer bit must reach its output guard before capture completion enables that output.

¶ An isochronic fork on o_{1a} constrains arrival at $G_{q_1}^+$. The two paths are

$$\begin{aligned} P_d : o_{1a} \downarrow &\rightarrow G_{q_1}^+ \\ P_1 : o_{1a} \downarrow &\rightarrow i_a \uparrow \rightarrow G_{q_1}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_1)$. The last output acknowledge must disable its served-slice assertion guard before the input acknowledge clears that state. O_1 is the last served slice; the other q_j are cleared by the same input acknowledge after their handshakes have already completed.

QSlicer

The input acknowledge i_a directly follows the output acknowledge o_a .

$$o_a \Rightarrow i_a$$

The acknowledge a_a of the enable channel directly follows i_a .

$$i_a \Rightarrow a_a$$

The output rails are enabled by the request a_r .

$$\begin{aligned} i_{fk} \wedge a_r &\rightarrow o_{fk} \uparrow & k=0,\dots,3 \\ \neg i_{fk} &\rightarrow o_{fk} \downarrow & k=0,\dots,3 \\ i_{tk} \wedge a_r &\rightarrow o_{tk} \uparrow & k=0,\dots,3 \\ \neg i_{tk} &\rightarrow o_{tk} \downarrow & k=0,\dots,3 \end{aligned}$$

C.2.4 Bubble Reshuffling

WordBuffer

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned} f_0 &\rightarrow G_{o_{ipk}}^+ \\ \neg i_a &\rightarrow G_{o_{ipk}}^+ \\ b_{4j+k} &\rightarrow G_{o_{jik}}^+ \\ \neg b_{4j+k} &\rightarrow G_{o_{jfk}}^+ \\ o_{1a} &\rightarrow G_{q_1}^+ \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve the output acknowledge signals, the input rails, and the false output rails in their original sense, while implementing $q_7, \dots, q_1, f_0, i_a$, and the true output rails in complemented form. The polarities of the buffer state variables can then be chosen. We preserve $b_{31}, \dots, b_0$ in their original sense, since this is compatible with both $b_{4j+k} \rightarrow G_{o_{jik}}^+$ and $\neg b_{4j+k} \rightarrow G_{o_{jfk}}^+$ under the selected output polarities.

$$\begin{aligned}
& ((i_{t31} \wedge b_{31}) \vee (i_{f31} \wedge \neg b_{31})) \wedge \dots \wedge ((i_{t0} \wedge b_0) \vee (i_{f0} \wedge \neg b_0)) \rightarrow \neg f_0 \downarrow \\
& (\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}) \wedge (\neg q_7 \wedge \dots \wedge \neg q_1) \vee r \rightarrow \neg f_0 \uparrow \\
& o_{ja} \rightarrow \neg q_j \downarrow \quad j=1,\dots,7 \\
& \neg i_a \vee r \rightarrow \neg q_j \uparrow \quad j=1,\dots,7 \\
& \neg q_1 \wedge \neg o_{1a} \rightarrow i_a \downarrow \\
& \neg f_0 \rightarrow i_a \uparrow \\
& \neg f_0 \wedge i_{tk} \rightarrow b_k \uparrow \quad k=0,\dots,31 \\
& (\neg f_0 \wedge i_{fk}) \vee r \rightarrow b_k \downarrow \quad k=0,\dots,31 \\
& \neg q_j \wedge \neg f_0 \wedge i_a \wedge \neg b_{4j+k} \rightarrow o_{jfk} \uparrow \quad j=1,\dots,7; k=0,\dots,3 \\
& \neg q_j \vee r \rightarrow o_{jfk} \downarrow \quad j=1,\dots,7; k=0,\dots,3 \\
& \neg q_j \wedge \neg f_0 \wedge i_a \wedge b_{4j+k} \rightarrow o_{jtk} \downarrow \quad j=1,\dots,7; k=0,\dots,3 \\
& \neg q_j \vee r \rightarrow o_{jtk} \uparrow \quad j=1,\dots,7; k=0,\dots,3
\end{aligned}$$

At this point, the fork-critical polarities are fixed. We therefore propagate the remaining bubbles through the guards, introducing explicit inverters wherever the opposite polarity of a polarity-fixed signal is required. The input acknowledge and the true output rails are inverted again to restore their original external polarities. Since the positive guard of f_0 would otherwise be excessively large, we decompose its completion condition using a completion tree.

$$\begin{aligned}
& ((i_{t(4k+3)} \wedge b_{4k+3}) \vee (i_{f(4k+3)} \wedge \neg b_{4k+3})) \wedge \dots \wedge ((i_{t4k} \wedge b_{4k}) \vee (i_{f4k} \wedge \neg b_{4k})) \\
& \rightarrow \neg c_k \downarrow \quad k=0,\dots,7 \\
& (\neg i_{t(4k+3)} \wedge \neg i_{f(4k+3)} \wedge \dots \wedge \neg i_{t4k} \wedge \neg i_{f4k}) \rightarrow \neg c_k \uparrow \quad k=0,\dots,7 \\
& \neg c_{2k+1} \wedge \neg c_{2k} \rightarrow d_k \uparrow \quad k=0,\dots,3 \\
& c_{2k+1} \wedge c_{2k} \rightarrow d_k \downarrow \quad k=0,\dots,3 \\
& d_{2k+1} \wedge d_{2k} \rightarrow e_k \downarrow \quad k=0,1 \\
& \neg d_{2k+1} \wedge \neg d_{2k} \rightarrow e_k \uparrow \quad k=0,1 \\
& \neg e_1 \wedge \neg e_0 \rightarrow c \uparrow \\
& e_1 \wedge e_0 \rightarrow c \downarrow \\
& c \rightarrow \neg f_0 \downarrow \\
& \neg c \wedge (\neg q_7 \wedge \dots \wedge \neg q_1) \vee \neg r \rightarrow \neg f_0 \uparrow \\
& o_{ja} \rightarrow \neg q_j \downarrow \quad j=1,\dots,7 \\
& \neg i_a \vee \neg r \rightarrow \neg q_j \uparrow \quad j=1,\dots,7 \\
& q_1 \wedge o_{1a} \rightarrow i_a \downarrow \\
& \neg f_0 \rightarrow i_a \uparrow \\
& \neg f_0 \wedge \neg i_{tk} \rightarrow b_k \uparrow \quad k=0,\dots,31 \\
& (\neg f_0 \wedge i_{fk}) \vee r \rightarrow b_k \downarrow \quad k=0,\dots,31 \\
& \neg q_j \wedge \neg f_0 \wedge \neg i_a \wedge \neg b_{4j+k} \rightarrow o_{jfk} \uparrow \quad j=1,\dots,7; k=0,\dots,3 \\
& q_j \vee r \rightarrow o_{jfk} \downarrow \quad j=1,\dots,7; k=0,\dots,3 \\
& \neg q_j \wedge f_0 \wedge i_a \wedge b_{4j+k} \rightarrow o_{jtk} \downarrow \quad j=1,\dots,7; k=0,\dots,3 \\
& \neg q_j \vee \neg r \rightarrow o_{jtk} \uparrow \quad j=1,\dots,7; k=0,\dots,3 \\
& \neg f_0 \Rightarrow f_0 \\
& \neg q_j \Rightarrow q_j \quad j=1,\dots,7 \\
& \neg i_a \Rightarrow i_a \\
& \neg b_k \Rightarrow b_k \quad k=0,\dots,31 \\
& \neg i_{tk} \Rightarrow i_{tk} \quad k=0,\dots,31 \\
& \neg r \Rightarrow r \\
& \neg o_{1a} \Rightarrow o_{1a} \\
& \neg o_{jtk} \Rightarrow o_{jtk} \quad j=1,\dots,7; k=0,\dots,3
\end{aligned}$$

QSlicer

The *QSlicer* contains no isochronic forks, and therefore no fork-critical polarity constraints are present. We preserve o_a , i_a , a_a , a_r , and the input rails in their original sense, while implementing the output rails in complemented form. Since the output rails are externally visible, their original polarity is restored before they leave the module.

$$\begin{array}{ll}
o_a \Rightarrow i_a & \\
i_a \Rightarrow a_a & \\
i_{fk} \wedge a_r \rightarrow .o_{fk} \downarrow & k=0,\dots,3 \\
\neg i_{fk} \rightarrow .o_{fk} \uparrow & k=0,\dots,3 \\
i_{tk} \wedge a_r \rightarrow .o_{tk} \downarrow & k=0,\dots,3 \\
\neg i_{tk} \rightarrow .o_{tk} \uparrow & k=0,\dots,3 \\
\neg .o_{fk} \Rightarrow o_{fk} & k=0,\dots,3 \\
\neg .o_{tk} \Rightarrow o_{tk} & k=0,\dots,3
\end{array}$$

C.3 QuadRouter

C.3.1 CHP

This block receives the quadruplets that encode the multicast route. The first quadruplet initializes the four counters q_i to either zero or one, thereby starting the routing process. The same quadruplet is also forwarded to the *PayloadDispatch* to indicate which children must receive the payload. Once initialization is complete, the children are served level by level. Starting from child 0, each child receives a number of quadruplets equal to the value stored in its counter, while the corresponding count for the next level is accumulated in parallel. After all four children have been served, the updated values are fetched from the *Counters*, which simultaneously reset their internal accumulators, and the procedure is repeated for the next level. The process terminates after the final level has been completed, when an *MSCODE* is observed at the end of the child sequence.

```

QuadRouter( $I?$ ,  $C_{nt0}?$ ,  $C_{nt1}?$ ,  $C_{nt2}?$ ,  $C_{nt3}?$ ,  $O_{c0}!$ ,  $O_{c1}!$ ,  $O_{c2}!$ ,  $O_{c3}!$ ,  $O_{d0}!$ ,  $O_{d1}!$ ,  $O_{d2}!$ ,  $O_{d3}!$ ,  $C_h!$ )  $\equiv$ 
 $s := \text{true};$ 
 $* [[s \rightarrow O_{d0}!MSCODE, O_{d1}!MSCODE, O_{d2}!MSCODE, O_{d3}!MSCODE; I?b;$ 
 $\quad [b \neq MSCODE \rightarrow s := \text{false}, C_h!b, (q_0, q_1, q_2, q_3) := b; I?b] \parallel b = MSCODE \rightarrow \text{skip}]$ 
 $\quad \parallel \neg s \rightarrow$ 
 $\quad \quad * [q_0 \neq 0 \rightarrow O_{c0}!b, O_{d0}!b, q_0 := q_0 - 1; I?b]; * [q_1 \neq 0 \rightarrow O_{c1}!b, O_{d1}!b, q_1 := q_1 - 1; I?b];$ 
 $\quad \quad * [q_2 \neq 0 \rightarrow O_{c2}!b, O_{d2}!b, q_2 := q_2 - 1; I?b]; * [q_3 \neq 0 \rightarrow O_{c3}!b, O_{d3}!b, q_3 := q_3 - 1; I?b];$ 
 $\quad \quad s := (b = MSCODE), C_{nt0}?q_0, C_{nt1}?q_1, C_{nt2}?q_2, C_{nt3}?q_3$ 
 $]]$ 

```

We now decompose the program using a *Sequencer7* and split it into smaller processes that can operate in parallel, as follows. In this case, not all seven sequencer stages are required; however, rather than designing a separate sequencer, we reuse the existing block. To prevent deadlock, each unused

request output is directly connected to its corresponding acknowledge input, so that the unused stages are immediately skipped.

```

QuadRouter( $I?$ ,  $C_{nt0}?$ ,  $C_{nt1}?$ ,  $C_{nt2}?$ ,  $C_{nt3}?$ ,  $O_{c0}!$ ,  $O_{c1}!$ ,  $O_{c2}!$ ,  $O_{c3}!$ ,  $O_{d0}!$ ,  $O_{d1}!$ ,  $O_{d2}!$ ,  $O_{d3}!$ ,  $C_h!$ )  $\equiv$ 
 $s := \text{true};$ 
*  $[[\overline{A_0} \rightarrow$ 
     $[s \rightarrow O_{d0}!MSCODE, O_{d1}!MSCODE, O_{d2}!MSCODE, O_{d3}!MSCODE; I?b;$ 
     $[b \neq MSCODE \rightarrow s := \text{false}, C_h!b, (q_0, q_1, q_2, q_3) := b; I?b] [b = MSCODE \rightarrow \text{skip}]$ 
     $\neg s \rightarrow A_0$ 
 $]$ 
 $]] \parallel$ 
*  $[[\overline{A_1} \rightarrow [q_0 \neq 0 \rightarrow O_{c0}!b, O_{d0}!b, q_0 := q_0 - 1; I?b] [q_0 = 0 \rightarrow A_1]]] \parallel$ 
*  $[[\overline{A_2} \rightarrow [q_1 \neq 0 \rightarrow O_{c1}!b, O_{d1}!b, q_1 := q_1 - 1; I?b] [q_1 = 0 \rightarrow A_2]]] \parallel$ 
*  $[[\overline{A_3} \rightarrow [q_2 \neq 0 \rightarrow O_{c2}!b, O_{d2}!b, q_2 := q_2 - 1; I?b] [q_2 = 0 \rightarrow A_3]]] \parallel$ 
*  $[[\overline{A_4} \rightarrow [q_3 \neq 0 \rightarrow O_{c3}!b, O_{d3}!b, q_3 := q_3 - 1; I?b] [q_3 = 0 \rightarrow A_4]]] \parallel$ 
*  $[[\overline{A_5} \rightarrow s := (b = MSCODE), C_{nt0}?q_0, C_{nt1}?q_1, C_{nt2}?q_2, C_{nt3}?q_3; A_5]] \parallel$ 
*  $[A_0; A_1; A_2; A_3; A_4; A_5]$ 

```

The intermediate decomposition still shares b , the counters q_i , and the mode state s between processes. We remove the shared data buffer by inspecting the pending input directly. The update of s moves to the initialization process. Each child process now fetches its next counter value before completing its sequencer handshake, eliminating the separate A_5 stage.

```

QuadRouter( $I?$ ,  $C_{nt0}?$ ,  $C_{nt1}?$ ,  $C_{nt2}?$ ,  $C_{nt3}?$ ,  $O_{c0}!$ ,  $O_{c1}!$ ,  $O_{c2}!$ ,  $O_{c3}!$ ,  $O_{d0}!$ ,  $O_{d1}!$ ,  $O_{d2}!$ ,  $O_{d3}!$ ,  $C_h!$ )  $\equiv$ 
 $s := \text{true};$ 
*  $[[\overline{A_0} \rightarrow$ 
     $[\overline{I?} = MSCODE \rightarrow$ 
     $O_{d0}!MSCODE, O_{d1}!MSCODE, O_{d2}!MSCODE, O_{d3}!MSCODE, I; s := \text{true}$ 
     $\neg \overline{I?} \neq MSCODE \rightarrow [s \rightarrow (q_0, q_1, q_2, q_3) := \overline{I?}, C_h!I?; s := \text{false}] \neg s \rightarrow \text{skip}; A_0$ 
 $]$ 
 $]] \parallel$ 
*  $[[\overline{A_1} \rightarrow [q_0 \neq 0 \rightarrow I?b_0; O_{c0}!b_0, O_{d0}!b_0, q_0 := q_0 - 1] [q_0 = 0 \rightarrow C_{nt0}?q_0; A_1]]] \parallel$ 
*  $[[\overline{A_2} \rightarrow [q_1 \neq 0 \rightarrow I?b_1; O_{c1}!b_1, O_{d1}!b_1, q_1 := q_1 - 1] [q_1 = 0 \rightarrow C_{nt1}?q_1; A_2]]] \parallel$ 
*  $[[\overline{A_3} \rightarrow [q_2 \neq 0 \rightarrow I?b_2; O_{c2}!b_2, O_{d2}!b_2, q_2 := q_2 - 1] [q_2 = 0 \rightarrow C_{nt2}?q_2; A_3]]] \parallel$ 
*  $[[\overline{A_4} \rightarrow [q_3 \neq 0 \rightarrow I?b_3; O_{c3}!b_3, O_{d3}!b_3, q_3 := q_3 - 1] [q_3 = 0 \rightarrow C_{nt3}?q_3; A_4]]] \parallel$ 
*  $[A_0; A_1; A_2; A_3; A_4]$ 

```

Now the buffer b_i used inside each process serving the children has a completely different meaning, since it is just a local storage element used to send the same data to both O_{ci} and O_{di} . We are now left with only one buffer for each q_i , which is written both during stage A_0 , where it is initialized, and within the process corresponding to its own child. To handle this cleanly, each q_i is stored in a standalone standard register with a *Merge8Bits* connected to

its input. We prefer this solution over a multi-port register implementation because it avoids the need to design additional custom blocks and keeps the register PRS simpler. The final setup for the q register is shown in the following figure, where both *SUpdate* and *ChildServer* are schematized to display only the channels relevant to q .

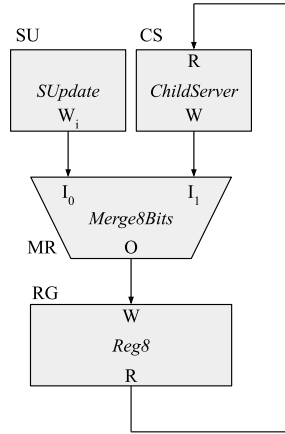
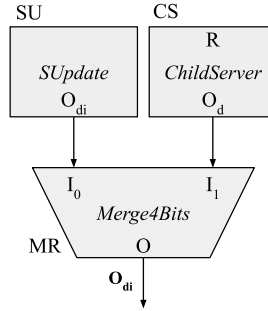


Figure C.2: q register setup

A similar issue arises for O_{di} , which is written from multiple points in the design. We solve it by using a *Merge4Bits* to merge the different accesses, as shown below.

Figure C.3: O_{di} merging scheme

Moreover, I is read from multiple points in the design. A standard solution would be to use a *SplitTree* with active output ports, so that data can be pulled from the input of the tree toward the output that requests it. However, this not only introduces additional logic, but in our case also requires an extra block at the root of the tree to convert the passive handshake generated by the requester into an active handshake that waits for the input data to arrive. Since the processes accessing I are mutually exclusive, we can instead use a shared bus, reducing both logic and design complexity.

SUpdate

After these considerations, the module can be decomposed into two new processes. The first, called *SUpdate*, corresponds to the portion activated by A_0 . The assignment $(q_0, q_1, q_2, q_3) := \bar{I}?$ must be replaced by parallel handshakes on four independent write ports. We therefore introduce a local state-holding variable b and denote the four write ports of the q_i registers by W_i .

$$\begin{aligned}
& \text{SUpdate}(I?, A, O_{d0}!, O_{d1}!, O_{d2}!, O_{d3}!, W_0!, W_1!, W_2!, W_3!, C_h!) \equiv \\
& s := \text{true}; \\
& * [[\overline{A} \rightarrow \\
& \quad [\overline{I?} = \text{MSCODE} \rightarrow \\
& \quad \quad O_{d0}!\text{MSCODE}, O_{d1}!\text{MSCODE}, O_{d2}!\text{MSCODE}, O_{d3}!\text{MSCODE}, I; s := \text{true} \\
& \quad \parallel \overline{I?} \neq \text{MSCODE} \rightarrow \\
& \quad \quad [s \rightarrow I?b; W_0!b.\text{msb}, W_1!b.(\text{msb} - 1), W_2!b.(\text{msb} - 2), W_3!b.(\text{msb} - 3), C_h!b; s := \text{false} \\
& \quad \quad \parallel \neg s \rightarrow \text{skip} \\
& \quad \quad]; A \\
& \quad] \\
&]]
\end{aligned}$$

ChildServer

The process responsible for sending data to a given child is called *ChildServer*. Within this process, we introduce a local storage element to store the value read from the corresponding q register, which we simply call q . The port used to write the updated value back to the register is called W , while the port used to read the current value is called R .

$$\begin{aligned}
& \text{ChildServer}(I?, A, O_d!, O_c!, W!, C_{nt}?, R?) \equiv \\
& * [[\overline{A} \rightarrow R?q; [q \neq 0 \rightarrow I?b; O_c!b, O_d!b, W!(q - 1) \parallel q = 0 \rightarrow W!C_{nt}?; A]]]
\end{aligned}$$

C.3.2 HSE

SUpdate

For *SUpdate*, we begin by writing its HSE directly from the CHP.

The auxiliary variables introduced below are f_0 for the stop-code broadcast branch, f_1 for the initial nonzero-mask branch, and f_2 for input capture. The variable s retains its initialization-mode meaning. Indexed output and write ports use $j \in \{0, \dots, 3\}$.

SUpdate $\equiv$

```

* [[ar];
  [if3 ∧ ⋯ ∧ if0 →
    ⟨, j : 0..3 : (⟨, n : 3..0 : odjfn ↑); [odja]; ⟨, n : 3..0 : odjfn ↓); [¬odja]]⟩; ia ↑;
    [¬if3 ∧ ⋯ ∧ ¬if0]; ia ↓; s ↑
  ] it3 ∨ ⋯ ∨ it0 →
    [s →
      ⟨, n : 3..0 : [itn → bn ↑ ] ifn → bn ↓]; ia ↑; [¬it3 ∧ ¬if3 ∧ ⋯ ∧ ¬it0 ∧ ¬if0]; ia ↓;
      (⟨, j : 0..3 : (⟨, n : 7..1 : wjfn ↑), [b3-j → wjt0 ↑ ] ¬b3-j → wjf0 ↑]; [wja];
        ⟨, n : 7..1 : wjfn ↓), wjt0 ↓, wjf0 ↓; [¬wja]
      )
    ), ⟨, n : 3..0 : [bn → chtn ↑ ] ¬bn → chfn ↑]; [cha]; ⟨, n : 3..0 : chtn ↓, chfn ↓]; [¬cha];
    s ↓
  ] ¬s → skip
]; aa ↑; [¬ar]; aa ↓
]]

```

We interleave the phases of the four O_{dj} handshakes, for $j \in \{0, \dots, 3\}$, and do the same for the four W_j handshakes and C_h in the other branch. Each group can then use one common completion sequence. The auxiliary states f_0 and f_1 identify the stop-code branch and the nonzero initialization branch, respectively. The state f_2 records input capture and returns low when the input rails return to spacer.

SUpdate $\equiv$

```

* [[ar];
  [if3 ∧ ⋯ ∧ if0 → f0 ↑; ⟨, j : 0..3 : ⟨, n : 3..0 : odjfn ↑⟩]; [od0a ∧ od1a ∧ od2a ∧ od3a];
    ⟨, j : 0..3 : ⟨, n : 3..0 : odjfn ↓⟩]; [¬od0a ∧ ¬od1a ∧ ¬od2a ∧ ¬od3a]; ia ↑; [¬if3 ∧ ⋯ ∧ ¬if0];
    ia ↓; s ↑; f0 ↓
  ] it3 ∨ ⋯ ∨ it0 →
    [s →
      f1 ↑; ⟨, n : 3..0 : [itn → bn ↑ ] ifn → bn ↓]; f2 ↑;
      ia ↑; [¬it3 ∧ ¬if3 ∧ ⋯ ∧ ¬it0 ∧ ¬if0]; f2 ↓; ia ↓;
      ⟨, j : 0..3 : (⟨, n : 7..1 : wjfn ↑), [b3-j → wjt0 ↑ ] ¬b3-j → wjf0 ↑);
      ⟨, n : 3..0 : [bn → chtn ↑ ] ¬bn → chfn ↑]; [(w0a ∧ w1a ∧ w2a ∧ w3a) ∧ cha];
      ⟨, j : 0..3 : (⟨, n : 7..1 : wjfn ↓), wjt0 ↓, wjf0 ↓), ⟨, n : 3..0 : chtn ↓, chfn ↓];
      [(¬w0a ∧ ¬w1a ∧ ¬w2a ∧ ¬w3a) ∧ ¬cha]; s ↓
    ] ¬s → skip
]; f1 ↓; aa ↑; [¬ar]; aa ↓
]]

```

The stop-code branch must wait for all four o_{dja} signals to return low. We place $s \uparrow$ and $f_0 \downarrow$ after that wait, allowing $i_a \downarrow$ to check only that the false

input rails i_{fn} have returned low, for $n \in \{0, \dots, 3\}$. Even when s is already high, $f_0 \downarrow$ still records completion of the output handshakes.

In the nonzero branch, $s \downarrow$ records completion of the write-valid phase on $W_0, \dots, W_3$ and C_h . We postpone $i_a \downarrow$ until the branch ends, preventing the next input from arriving too early.

We move $a_a \uparrow$ immediately after $f_1 \uparrow$ to simplify its positive guard. The $\neg s$ alternative also asserts a_a , so both alternatives complete the sequencer handshake.

```

SUpdate  $\equiv$ 
*  $[[a_r];$ 
   $[i_{f3} \wedge \dots \wedge i_{f0} \rightarrow f_0 \uparrow; \langle j : 0..3 : \langle n : 3..0 : o_{djfn} \uparrow \rangle; [o_{d0a} \wedge o_{d1a} \wedge o_{d2a} \wedge o_{d3a}]; i_a \uparrow;$ 
     $\langle j : 0..3 : \langle n : 3..0 : o_{djfn} \downarrow \rangle; [\neg o_{d0a} \wedge \neg o_{d1a} \wedge \neg o_{d2a} \wedge \neg o_{d3a}];$ 
     $s \uparrow; f_0 \downarrow; [\neg i_{f3} \wedge \dots \wedge \neg i_{f0}]; i_a \downarrow$ 
   $\parallel i_{t3} \vee \dots \vee i_{t0} \rightarrow$ 
     $[s \rightarrow f_1 \uparrow; a_a \uparrow; \langle n : 3..0 : [i_{tn} \rightarrow b_n \uparrow \parallel i_{fn} \rightarrow b_n \downarrow] \rangle;$ 
       $f_2 \uparrow; i_a \uparrow; [\neg i_{t3} \wedge \neg i_{f3} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; f_2 \downarrow;$ 
       $\langle j : 0..3 : (\langle n : 7..1 : w_{jfn} \uparrow \rangle, [b_{3-j} \rightarrow w_{jt0} \uparrow \parallel \neg b_{3-j} \rightarrow w_{jfo} \uparrow]) \rangle,$ 
       $\langle n : 3..0 : [b_n \rightarrow c_{htn} \uparrow \parallel \neg b_n \rightarrow c_{hfn} \uparrow]; [(w_{0a} \wedge w_{1a} \wedge w_{2a} \wedge w_{3a}) \wedge c_{ha}]; s \downarrow;$ 
       $\langle j : 0..3 : (\langle n : 7..1 : w_{jfn} \downarrow \rangle, w_{jt0} \downarrow, w_{jfo} \downarrow) \rangle, \langle n : 3..0 : c_{htn} \downarrow, c_{hfn} \downarrow \rangle;$ 
       $[(\neg w_{0a} \wedge \neg w_{1a} \wedge \neg w_{2a} \wedge \neg w_{3a}) \wedge \neg c_{ha}]; f_1 \downarrow$ 
     $\parallel \neg s \rightarrow a_a \uparrow$ 
   $]; [\neg a_r]; a_a \downarrow; i_a \downarrow$ 
 $]]$ 

```

ChildServer

When writing the HSE of the *ChildServer*, we introduce two internal channels for the decremter: S_i carries the input operand and S_o carries the result. Since an 8-bit decrement can be implemented by adding the two's-complement constant FF_{16} , we directly specialize the PRS of the *FullAdder* and *HalfAdder* instead of explicitly driving a constant operand.

The auxiliary variables introduced below are f_0 for the nonzero-count branch, f_1 for the zero-count branch, f_2 for capture progress, and f_3 for return-to-spacer progress. The stored counter value q and data nibble b remain distinct from these control states.

ChildServer $\equiv$

* $[[a_r]; r_r \uparrow; \langle, k : 7..0 : [r_{tk} \rightarrow q_k \uparrow \parallel r_{fk} \rightarrow q_k \downarrow] \rangle; r_r \downarrow; [\neg r_{t7} \wedge \neg r_{f7} \wedge \dots \wedge \neg r_{t0} \wedge \neg r_{f0}];$
 $[q_7 \vee \dots \vee q_0 \rightarrow$
 $\langle, n : 3..0 : [i_{tn} \rightarrow b_n \uparrow \parallel i_{fn} \rightarrow b_n \downarrow] \rangle; i_a \uparrow; [\neg i_{t3} \wedge \neg i_{f3} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; i_a \downarrow;$
 $((\langle, n : 3..0 : [b_n \rightarrow o_{ctn} \uparrow \parallel \neg b_n \rightarrow o_{cfn} \uparrow] \rangle; [o_{ca}]; \langle, k : 3..0 : o_{ctk} \downarrow, o_{cfk} \downarrow \rangle; [\neg o_{ca}]),$
 $(\langle, n : 3..0 : [b_n \rightarrow o_{dtn} \uparrow \parallel \neg b_n \rightarrow o_{dfn} \uparrow] \rangle; [o_{da}]; \langle, k : 3..0 : o_{dtk} \downarrow, o_{dfk} \downarrow \rangle; [\neg o_{da}]))$
 $\rangle,$
 $((\langle, k : 7..0 : [q_k \rightarrow s_{itk} \uparrow \parallel \neg q_k \rightarrow s_{ifk} \uparrow] \rangle; \langle, k : 7..0 : [s_{otk} \rightarrow w_{tk} \uparrow \parallel s_{ofk} \rightarrow w_{fk} \uparrow] \rangle; [w_a];$
 $\langle, k : 7..0 : s_{itk} \downarrow, s_{ifk} \downarrow \rangle; \langle, k : 7..0 : [\neg s_{otk} \rightarrow w_{tk} \downarrow], [\neg s_{ofk} \rightarrow w_{fk} \downarrow] \rangle; [\neg w_a]$
 $\rangle)$
 $\parallel \neg q_7 \wedge \dots \wedge \neg q_0 \rightarrow c_{ntr} \uparrow; \langle, k : 7..0 : [c_{nttk} \rightarrow w_{tk} \uparrow \parallel c_{ntfk} \rightarrow w_{fk} \uparrow] \rangle; [w_a]; c_{ntr} \downarrow;$
 $\langle, k : 7..0 : [\neg c_{nttk} \rightarrow w_{tk} \downarrow], [\neg c_{ntfk} \rightarrow w_{fk} \downarrow] \rangle; [\neg w_a]; a_a \uparrow; [\neg a_r]; a_a \downarrow$
 $]]$

The capture and spacer conditions for R and I would produce large guards. We introduce f_2 to record capture of q and f_3 to record return of the register-read rails to spacer. The branch states f_0 and f_1 distinguish nonzero and zero counts. In the nonzero branch, $f_2 \downarrow$ and $f_3 \downarrow$ can then record capture of b and return of the input rails to spacer, respectively.

In the $q = 0$ branch, we move $a_a \uparrow$ immediately after $[w_a]$. It then records completion of the valid phase of $W!C_{nt}?$.

Finally, we reshuffle and interleave the phases of O_d and O_c so that the completion of both handshakes can be detected using a single state variable. This simplification is refined in the next step.

ChildServer $\equiv$

* $[[a_r]; r_r \uparrow; \langle, k : 7..0 : [r_{tk} \rightarrow q_k \uparrow \parallel r_{fk} \rightarrow q_k \downarrow] \rangle; f_2 \uparrow;$
 $r_r \downarrow; [\neg r_{t7} \wedge \neg r_{f7} \wedge \dots \wedge \neg r_{t0} \wedge \neg r_{f0}]; f_3 \uparrow;$
 $[q_7 \vee \dots \vee q_0 \rightarrow$
 $f_0 \uparrow; \langle, n : 3..0 : [i_{tn} \rightarrow b_n \uparrow \parallel i_{fn} \rightarrow b_n \downarrow] \rangle; f_2 \downarrow;$
 $i_a \uparrow; [\neg i_{t3} \wedge \neg i_{f3} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; f_3 \downarrow; i_a \downarrow;$
 $((\langle, n : 3..0 : [b_n \rightarrow o_{ctn} \uparrow, o_{dtn} \uparrow \parallel \neg b_n \rightarrow o_{cfn} \uparrow, o_{dfn} \uparrow] \rangle; [o_{ca} \wedge o_{da}];$
 $\langle, k : 3..0 : o_{ctk} \downarrow, o_{cfk} \downarrow \rangle, \langle, k : 3..0 : o_{dtk} \downarrow, o_{dfk} \downarrow \rangle; [\neg o_{ca} \wedge \neg o_{da}])$
 $\rangle,$
 $((\langle, k : 7..0 : [q_k \rightarrow s_{itk} \uparrow \parallel \neg q_k \rightarrow s_{ifk} \uparrow] \rangle; \langle, k : 7..0 : [s_{otk} \rightarrow w_{tk} \uparrow \parallel s_{ofk} \rightarrow w_{fk} \uparrow] \rangle; [w_a];$
 $\langle, k : 7..0 : s_{itk} \downarrow, s_{ifk} \downarrow \rangle; \langle, k : 7..0 : [\neg s_{otk} \rightarrow w_{tk} \downarrow], [\neg s_{ofk} \rightarrow w_{fk} \downarrow] \rangle; [\neg w_a]$
 $\rangle; f_0 \downarrow$
 $\parallel \neg q_7 \wedge \dots \wedge \neg q_0 \rightarrow$
 $f_1 \uparrow; c_{ntr} \uparrow; \langle, k : 7..0 : [c_{nttk} \rightarrow w_{tk} \uparrow \parallel c_{ntfk} \rightarrow w_{fk} \uparrow] \rangle; [w_a]; a_a \uparrow; f_1 \downarrow, f_2 \downarrow, f_3 \downarrow; c_{ntr} \downarrow;$
 $\langle, k : 7..0 : [\neg c_{nttk} \rightarrow w_{tk} \downarrow], [\neg c_{ntfk} \rightarrow w_{fk} \downarrow] \rangle; [\neg w_a]; [\neg a_r]; a_a \downarrow$
 $]]$

We next combine completion of the counter update and the two data-output handshakes. After interleaving the phases on O_d and O_c , we include the write handshake in the same sequence. We place $f_0 \downarrow$ after $[o_{ca} \wedge o_{da} \wedge w_a]$; the condition $\neg f_0 \wedge \neg o_{ca} \wedge \neg o_{da} \wedge \neg w_a$ then identifies the end of this branch.

ChildServer $\equiv$

```

* [[ar]; rr ↑; ⟨, k : 7..0 : [rtk → qk ↑ ∥ rfk → qk ↓]⟩; f2 ↑;
  rr ↓; [¬rt7 ∧ ¬rf7 ∧ ⋯ ∧ ¬rt0 ∧ ¬rf0]; f3 ↑;
  [q7 ∨ ⋯ ∨ q0 →
    f0 ↑; ⟨, n : 3..0 : [itn → bn ↑ ∥ ifn → bn ↓]⟩; f2 ↓;
    ia ↑; [¬it3 ∧ ¬if3 ∧ ⋯ ∧ ¬it0 ∧ ¬if0]; f3 ↓; ia ↓;
    ⟨, n : 3..0 : [bn → octn ↑, odtn ↑ ∥ ¬bn → ocfn ↑, odfn ↑]⟩,
    ⟨, k : 7..0 : [qk → sitk ↑ ∥ ¬qk → sifk ↑]⟩; ⟨, k : 7..0 : [sotk → wtk ↑ ∥ sofk → wfk ↑]⟩;
    [oca ∧ oda ∧ wa]; f0 ↓;
    ⟨, k : 3..0 : octk ↓, ocfk ↓⟩, ⟨, k : 3..0 : odtk ↓, odfk ↓⟩, ⟨, k : 7..0 : sitk ↓, sifk ↓⟩;
    ⟨, k : 7..0 : [¬sotk → wtk ↓], [¬sofk → wfk ↓]⟩; [¬oca ∧ ¬oda ∧ ¬wa]
  ∥ ¬q7 ∧ ⋯ ∧ ¬q0 →
    f1 ↑; cntr ↑; ⟨, k : 7..0 : [cnttk → wtk ↑ ∥ cntfk → wfk ↑]⟩; [wa]; aa ↑; f1 ↓, f2 ↓, f3 ↓; cntr ↓;
    ⟨, k : 7..0 : [¬cnttk → wtk ↓], [¬cntfk → wfk ↓]⟩; [¬wa]; [¬ar]; aa ↓
  ]]
```

C.3.3 PRS

SUpdate

To assert f_0 , the input bits must be equal to the *MSCODE* and a_r must be high. To deassert it, all the acknowledge signals of the O_{dj} channels must be low, while both i_a and s must be high. To assert f_1 , a_r must still be high, the input bits must be different from the *MSCODE*, and s must be high. To deassert it, we must wait for all the acknowledge signals of the W channels and C_h to become low, while also checking for $\neg s$. Note, however, that the condition for asserting f_0 may still hold when f_0 needs to be deasserted if the input rails are slow to return to the neutral state. Therefore, we must strengthen the positive guard of f_0 with $\neg i_a$.

$$\begin{aligned}
& a_r \wedge i_{f3} \wedge \dots \wedge i_{f0} \wedge \neg i_a \rightarrow f_0 \uparrow \\
& i_a \wedge s \wedge (\neg o_{d0a} \wedge \neg o_{d1a} \wedge \neg o_{d2a} \wedge \neg o_{d3a}) \vee r \rightarrow f_0 \downarrow \\
& a_r \wedge s \wedge (i_{t3} \vee \dots \vee i_{t0}) \rightarrow f_1 \uparrow \\
& \neg s \wedge (\neg w_{0a} \wedge \neg w_{1a} \wedge \neg w_{2a} \wedge \neg w_{3a} \wedge \neg c_{ha}) \vee r \rightarrow f_1 \downarrow
\end{aligned}$$

f_2 is high when f_1 is high and the buffer b has captured the data on I , and it goes low when all the input rails return to the neutral state.

$$\begin{aligned}
& f_1 \wedge ((i_{t3} \wedge b_3) \vee (i_{f3} \wedge \neg b_3)) \wedge \dots \wedge ((i_{t0} \wedge b_0) \vee (i_{f0} \wedge \neg b_0)) \rightarrow f_2 \uparrow \\
& \neg i_{t3} \wedge \neg i_{f3} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0} \rightarrow f_2 \downarrow
\end{aligned}$$

s is asserted in the first branch when all the acknowledge signals of the O_{dj} channels have gone low and i_a is high. It is deasserted when all the acknowledge signals of the W channels and C_h have gone high.

$$\begin{aligned} f_0 \wedge i_a \wedge (\neg o_{d0a} \wedge \neg o_{d1a} \wedge \neg o_{d2a} \wedge \neg o_{d3a}) \vee r &\rightarrow s \uparrow \\ w_{0a} \wedge w_{1a} \wedge w_{2a} \wedge w_{3a} \wedge c_{ha} &\rightarrow s \downarrow \end{aligned}$$

i_a is asserted either when f_2 is high or when all the acknowledge signals of the O_{dj} channels have gone high. It is deasserted either when f_1 is low in the $\neq$ *MSCODE* branch and a_a is low, or when f_0 is low and the false rails of I have returned to the neutral state in the *MSCODE* branch.

$$\begin{aligned} f_2 \vee (o_{d0a} \wedge o_{d1a} \wedge o_{d2a} \wedge o_{d3a}) &\rightarrow i_a \uparrow \\ \neg f_0 \wedge \neg f_1 \wedge \neg a_a \wedge (\neg i_{f3} \wedge \neg i_{f2} \wedge \neg i_{f1} \wedge \neg i_{f0}) \vee r &\rightarrow i_a \downarrow \end{aligned}$$

a_a is asserted when a_r is high and either f_1 is high, indicating that we are coming from the s branch, or s is low while an input different from the *MSCODE* is still pending, indicating that the selection was skipped through the $\neg s$ branch. It is deasserted when both a_r and f_1 are low.

$$\begin{aligned} a_r \wedge (f_1 \vee (\neg s \wedge (i_{t3} \vee i_{t2} \vee i_{t1} \vee i_{t0}))) &\rightarrow a_a \uparrow \\ \neg f_1 \wedge \neg a_r \vee r &\rightarrow a_a \downarrow \end{aligned}$$

The buffer bits b_n capture the input while a_a and s are high. Reset initializes them to zero so that uninitialized values do not propagate through the circuit.

$$\begin{aligned} a_a \wedge s \wedge i_{tn} &\rightarrow b_n \uparrow & n=0,\dots,3 \\ (a_a \wedge s \wedge i_{fn}) \vee r &\rightarrow b_n \downarrow & n=0,\dots,3 \end{aligned}$$

For the rails of C_h , we use similar rules, but in this case every bit of the channel is driven. Note, however, that the positive and negative guards could be true at the same time. Therefore, we must strengthen the positive guard with s , since it is the only variable that changes between the two corresponding states in the program.

$$\begin{aligned} s \wedge \neg f_2 \wedge i_a \wedge f_1 \wedge \neg b_n &\rightarrow c_{hfn} \uparrow & n=0,\dots,3 \\ \neg s \vee r &\rightarrow c_{hfn} \downarrow & n=0,\dots,3 \\ s \wedge \neg f_2 \wedge i_a \wedge f_1 \wedge b_n &\rightarrow c_{htn} \uparrow & n=0,\dots,3 \\ \neg s \vee r &\rightarrow c_{htn} \downarrow & n=0,\dots,3 \end{aligned}$$

For the rails driving the multiple W channels, the seven MSBs are always false, while the LSB is either true or false depending on b . The rails are driven to a valid state when the f_1 branch is selected, f_2 is low, and i_a is high, and they return to the neutral state when s goes low. Note, however, that the positive and negative guards could be true at the same time. Therefore, we must strengthen the positive guard with s , since it is the only variable that changes between the two corresponding states in the program.

$$\begin{aligned} s \wedge \neg f_2 \wedge i_a \wedge f_1 \wedge \neg b_{3-j} &\rightarrow w_{jf0} \uparrow & j=0,\dots,3 \\ \neg s \vee r &\rightarrow w_{jf0} \downarrow & j=0,\dots,3 \\ s \wedge \neg f_2 \wedge i_a \wedge f_1 &\rightarrow w_{jfn} \uparrow & j=0,\dots,3; n=1,\dots,7 \\ \neg s \vee r &\rightarrow w_{jfn} \downarrow & j=0,\dots,3; n=1,\dots,7 \\ s \wedge \neg f_2 \wedge i_a \wedge f_1 \wedge b_{3-j} &\rightarrow w_{jt0} \uparrow & j=0,\dots,3 \\ \neg s \vee r &\rightarrow w_{jt0} \downarrow & j=0,\dots,3 \\ r &\rightarrow w_{jtn} \downarrow & j=0,\dots,3; n=1,\dots,7 \end{aligned}$$

For the rails of O_{dj} , we only change the state of the false rails, which are asserted when f_0 is high and deasserted when i_a is high, while the true rails are kept low at all times. Note however that f_0 can be true at the same time as when i_a is also true, so we need to strengthen the positive guard with $\neg i_a$.

$$\begin{array}{ll} \neg i_a \wedge f_0 \rightarrow o_{djfn} \uparrow & j, n = 0, \dots, 3 \\ i_a \vee r \rightarrow o_{djfn} \downarrow & j, n = 0, \dots, 3 \\ r \rightarrow o_{djfn} \downarrow & j, n = 0, \dots, 3 \end{array}$$

In the fork paths below, $j \in \{0, \dots, 3\}$, $n \in \{0, \dots, 3\}$, and $\rho \in \{t, f\}$. The placeholder x denotes one driven write rail: $w_{j\rho\ell}$ with $\ell \in \{0, \dots, 7\}$, or w_{jt0} . It does not denote the permanently low true rails of write bits 1–7.

¶ An isochronic fork on f_0 constrains arrival at $G_{o_{djfn}}^+$. The two paths are

$$\begin{array}{l} P_d : f_0 \downarrow \rightarrow G_{o_{djfn}}^+ \\ P_i : f_0 \downarrow \rightarrow i_a \downarrow \rightarrow G_{o_{djfn}}^+ \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The zero-broadcast branch must be inactive before the input acknowledge returns low.

¶ An isochronic fork on f_1 constrains arrival at $G_{i_a}^-$. The two paths are

$$\begin{array}{l} P_d : f_1 \uparrow \rightarrow G_{i_a}^- \\ P_i : f_1 \uparrow \rightarrow f_2 \uparrow \rightarrow G_{i_a}^+ \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Entering the capture branch must disable acknowledge release before the captured input is acknowledged.

¶ An isochronic fork on f_2 constrains arrival at G_x^+ . The two paths are

$$\begin{array}{l} P_d : f_2 \uparrow \rightarrow G_x^+ \\ P_i : f_2 \uparrow \rightarrow i_a \uparrow \rightarrow G_x^+ \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Capture completion must inhibit output assertion before the input acknowledge can qualify it.

The same fork also constrains arrival at $G_{c_{h\rho n}}^+$. The two paths are

$$\begin{array}{l} P_d : f_2 \uparrow \rightarrow G_{c_{h\rho n}}^+ \\ P_i : f_2 \uparrow \rightarrow i_a \uparrow \rightarrow G_{c_{h\rho n}}^+ \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. Capture completion must inhibit output assertion before the input acknowledge can qualify it.

¶ An isochronic fork on i_a constrains arrival at G_x^+ . The two paths are

$$\begin{array}{l} P_d : i_a \downarrow \rightarrow G_x^+ \\ P_i : i_a \downarrow \rightarrow V(I) \rightarrow f_1 \uparrow \rightarrow G_x^+ \end{array}$$

We require $\tau(P_d) < \tau(P_i)$. The old input acknowledge must stop qualifying output assertion before the next input selects the capture branch.

The same fork also constrains arrival at $G_{c_{hpn}}^+$. The two paths are

$$\begin{aligned} P_d : i_a \downarrow \rightarrow G_{c_{hpn}}^+ \\ P_i : i_a \downarrow \rightarrow V(I) \rightarrow f_1 \uparrow \rightarrow G_{c_{hpn}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The old input acknowledge must stop qualifying output assertion before the next input selects the capture branch.

¶ An isochronic fork on a_{wr} constrains arrival at $G_{f_1}^-$. The two paths are

$$\begin{aligned} P_d : a_{wr} \uparrow \rightarrow G_{f_1}^- \\ P_i : a_{wr} \uparrow \rightarrow s \downarrow \rightarrow G_{f_1}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The write-phase release guard must observe each acknowledge before the mode change can qualify it. $a_{wr} \in \{w_{0a}, w_{1a}, w_{2a}, w_{3a}, c_{ha}\}$ is notation for one of the five existing signals, not an extra signal.

¶ An isochronic fork on o_{dja} constrains arrival at $G_{f_0}^-$. The two paths are

$$\begin{aligned} P_d : o_{dja} \uparrow \rightarrow G_{f_0}^- \\ P_i : o_{dja} \uparrow \rightarrow i_a \uparrow \rightarrow G_{f_0}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The zero-broadcast branch must not be released until all four output acknowledge signals have returned low.

The same fork also constrains arrival at G_s^+ . The two paths are

$$\begin{aligned} P_d : o_{dja} \uparrow \rightarrow G_s^+ \\ P_i : o_{dja} \uparrow \rightarrow i_a \uparrow \rightarrow G_s^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. Each output acknowledge must inhibit the start-state update before the input acknowledge rises. This extends the branch shared by f_0 and i_a ; in physical polarity, the direct effect precedes $i_a \downarrow$.

ChildServer

f_0 is asserted when f_3 is high and q is not equal to zero, and it is deasserted after $[o_{ca} \wedge o_{da} \wedge w_a]$. Conversely, f_1 is asserted when f_3 is high and q is equal to zero, and it is deasserted when a_a becomes high. However, in the f_1 branch, a_a goes high while its positive guard is still true. Therefore, we must strengthen the positive guard with $\neg a_a$.

$$\begin{aligned} f_3 \wedge (q_7 \vee \dots \vee q_0) &\rightarrow f_0 \uparrow \\ (o_{ca} \wedge o_{da} \wedge w_a) \vee r &\rightarrow f_0 \downarrow \\ \neg a_a \wedge f_3 \wedge (\neg q_7 \wedge \dots \wedge \neg q_0) &\rightarrow f_1 \uparrow \\ a_a \vee r &\rightarrow f_1 \downarrow \end{aligned}$$

We set f_2 high once q has captured the data from R , and low once b has stored the data received on I or when a_a is high. Since the conditions indicating completion of these two captures may overlap, we strengthen the positive

guard with r_r and the negative guard with f_0 to prevent f_2 from being deasserted prematurely.

$$\begin{aligned} r_r \wedge ((r_{t7} \wedge q_7) \vee (r_{f7} \wedge \neg q_7)) \wedge \dots \wedge ((r_{t0} \wedge q_0) \vee (r_{f0} \wedge \neg q_0)) &\rightarrow f_2 \uparrow \\ f_0 \wedge ((i_{t3} \wedge b_3) \vee (i_{f3} \wedge \neg b_3)) \wedge \dots \wedge ((i_{t0} \wedge b_0) \vee (i_{f0} \wedge \neg b_0)) \vee a_a \vee r &\rightarrow f_2 \downarrow \end{aligned}$$

We set f_3 high when f_2 is high and all the rails of R have returned to the neutral state. We set it low when f_2 is low and all the input rails have returned to the neutral state, or when a_a is high. Note, however, that a_a may still be high when the positive guard is true. Therefore, we must strengthen the positive guard by adding $\neg a_a$.

$$\begin{aligned} f_2 \wedge \neg a_a \wedge (\neg r_{t7} \wedge \neg r_{f7} \wedge \dots \wedge \neg r_{t0} \wedge \neg r_{f0}) &\rightarrow f_3 \uparrow \\ \neg f_2 \wedge (\neg i_{t3} \wedge \neg i_{f3} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}) \vee a_a \vee r &\rightarrow f_3 \downarrow \end{aligned}$$

The value of q is updated exclusively from R .

$$\begin{aligned} r_{tk} &\rightarrow q_k \uparrow & k=0,\dots,7 \\ r_{fk} \vee r &\rightarrow q_k \downarrow & k=0,\dots,7 \end{aligned}$$

To assert i_a , we must wait for f_2 to become low while f_0 is high, whereas i_a is deasserted when f_3 becomes low. The guards would otherwise overlap. We therefore strengthen the positive guard with f_3 , the signal that distinguishes the assertion and deassertion states.

$$\begin{aligned} f_3 \wedge f_0 \wedge \neg f_2 &\rightarrow i_a \uparrow \\ \neg f_3 &\rightarrow i_a \downarrow \end{aligned}$$

a_a is asserted after w_a becomes high, only if we are in the branch of f_1 , and it is deasserted once both w_a and a_r have gone low.

$$\begin{aligned} w_a \wedge f_1 &\rightarrow a_a \uparrow \\ \neg w_a \wedge \neg a_r &\rightarrow a_a \downarrow \end{aligned}$$

c_{ntr} is asserted when f_1 is high, and it is deasserted when f_3 , f_2 , and f_1 are all low.

$$\begin{aligned} f_1 &\rightarrow c_{ntr} \uparrow \\ \neg f_3 \wedge \neg f_2 \wedge \neg f_1 &\rightarrow c_{ntr} \downarrow \end{aligned}$$

The read request r_r is asserted when a_r is high and the previous branch has completed. Because a_r may remain high across several internal iterations, we also require $f_2 = 0$. The nonzero branch must release f_0 and all three acknowledge signals o_{ca} , o_{da} , w_a ; the zero-count branch must release a_a . Capture completion, recorded by $f_2 \uparrow$, deasserts r_r .

$$\begin{aligned} a_r \wedge \neg f_2 \wedge \neg a_a \wedge \neg f_0 \wedge \neg o_{ca} \wedge \neg o_{da} \wedge \neg w_a &\rightarrow r_r \uparrow \\ f_2 \vee r &\rightarrow r_r \downarrow \end{aligned}$$

The value of b is updated exclusively from I , and only while both f_0 and f_2 are high.

$$\begin{aligned} f_0 \wedge f_2 \wedge i_{tk} &\rightarrow b_k \uparrow & k=0,\dots,3 \\ (f_0 \wedge f_2 \wedge i_{fk}) \vee r &\rightarrow b_k \downarrow & k=0,\dots,3 \end{aligned}$$

The write-output rails of W combine the corresponding rails of C_{nt} and S_o . Each output is their logical OR and returns low once both source rails are low.

$$\begin{aligned} c_{ntfk} \vee s_{ofk} &\Rightarrow w_{fk} & k=0,\dots,7 \\ c_{nttk} \vee s_{otk} &\Rightarrow w_{tk} & k=0,\dots,7 \end{aligned}$$

The input to the adder S_i , the data on channel O_c , and the data on channel O_d are all set once the f_0 branch has been selected and both f_3 and i_a have gone low. They are deasserted once f_0 goes low.

$$\begin{aligned} (f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg q_k &\rightarrow s_{ifk} \uparrow & k=0,\dots,7 \\ \neg f_0 &\rightarrow s_{ifk} \downarrow & k=0,\dots,7 \\ (f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge q_k &\rightarrow s_{itk} \uparrow & k=0,\dots,7 \\ \neg f_0 &\rightarrow s_{itk} \downarrow & k=0,\dots,7 \\ (f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg b_k &\rightarrow o_{dfk} \uparrow & k=0,\dots,3 \\ \neg f_0 &\rightarrow o_{dfk} \downarrow & k=0,\dots,3 \\ (f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge b_k &\rightarrow o_{dtk} \uparrow & k=0,\dots,3 \\ \neg f_0 &\rightarrow o_{dtk} \downarrow & k=0,\dots,3 \end{aligned}$$

For O_c , replace o_{dpk} with o_{cpk} , including complemented names.

Finally, we construct the decrement logic, which takes an 8-bit input word and automatically subtracts one. This operation can be implemented by adding 1111111_2 , which corresponds to -1 in two's complement representation. To do so, we build a ripple-carry adder using the *HalfAdder'* and *FullAdder'* blocks, as shown in Figure C.4.

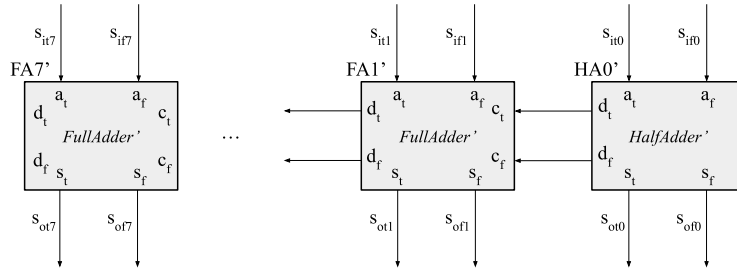


Figure C.4: Decrement logic implemented as a ripple-carry adder using *HalfAdder'* and *FullAdder'* blocks.

In the fork paths below, $k \in \{0, \dots, 7\}$ indexes counter bits, $n \in \{0, \dots, 3\}$ indexes data-nibble bits, and $\rho \in \{t, f\}$. Here $r_{\rho k}$ denotes a rail of the register-read channel, not the reset signal r .

¶ An isochronic fork on f_0 constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : f_0 \uparrow \rightarrow G_{r_r}^+ \\ P_i : f_0 \uparrow \rightarrow f_2 \downarrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The forwarding branch must inhibit a second register read before the capture-phase state changes.

¶ An isochronic fork on f_3 constrains arrival at $G_{s_{ipk}}^+$. The two paths are

$$\begin{aligned} P_d : f_3 \uparrow \rightarrow G_{s_{ipk}}^+ \\ P_i : f_3 \uparrow \rightarrow f_0 \uparrow \rightarrow G_{s_{ipk}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The spacer-phase state must inhibit output assertion before the selected branch can enable it.

The same fork also constrains arrival at $G_{o_{d\rho n}}^+$. The two paths are

$$\begin{aligned} P_d : f_3 \uparrow \rightarrow G_{o_{d\rho n}}^+ \\ P_i : f_3 \uparrow \rightarrow f_0 \uparrow \rightarrow G_{o_{d\rho n}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The spacer-phase state must inhibit output assertion before the selected branch can enable it.

The same fork also constrains arrival at $G_{o_{c\rho n}}^+$. The two paths are

$$\begin{aligned} P_d : f_3 \uparrow \rightarrow G_{o_{c\rho n}}^+ \\ P_i : f_3 \uparrow \rightarrow f_0 \uparrow \rightarrow G_{o_{c\rho n}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The spacer-phase state must inhibit output assertion before the selected branch can enable it.

¶ An isochronic fork on i_a constrains arrival at $G_{s_{ipk}}^+$. The two paths are

$$\begin{aligned} P_d : i_a \uparrow \rightarrow G_{s_{ipk}}^+ \\ P_i : i_a \uparrow \rightarrow N(I) \rightarrow f_3 \downarrow \rightarrow G_{s_{ipk}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The input acknowledge must reach the output guard before input return-to-spacer clears the spacer-phase state. The output handshake begins only after both f_3 and i_a have returned low.

The same fork also constrains arrival at $G_{o_{c\rho n}}^+$. The two paths are

$$\begin{aligned} P_d : i_a \uparrow \rightarrow G_{o_{c\rho n}}^+ \\ P_i : i_a \uparrow \rightarrow N(I) \rightarrow f_3 \downarrow \rightarrow G_{o_{c\rho n}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The input acknowledge must reach the output guard before input return-to-spacer clears the spacer-phase state. The output handshake begins only after both f_3 and i_a have returned low.

The same fork also constrains arrival at $G_{o_{\rho n}}^+$. The two paths are

$$\begin{aligned} P_d : i_a \uparrow \rightarrow G_{o_{\rho n}}^+ \\ P_i : i_a \uparrow \rightarrow N(I) \rightarrow f_3 \downarrow \rightarrow G_{o_{\rho n}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The input acknowledge must reach the output guard before input return-to-spacer clears the spacer-phase state. The output handshake begins only after both f_3 and i_a have returned low.

¶ An isochronic fork on a_r constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : a_r \downarrow \rightarrow G_{r_r}^+ \\ P_i : a_r \downarrow \rightarrow a_a \downarrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The withdrawn sequencer request must disable another register read before the sequencer acknowledge returns low.

¶ An isochronic fork on $i_{\rho n}$ constrains arrival at $G_{f_3}^-$. The two paths are

$$\begin{aligned} P_d : i_{\rho n} \uparrow \rightarrow G_{f_3}^- \\ P_i : i_{\rho n} \uparrow \rightarrow f_2 \downarrow \rightarrow G_{f_3}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The received input rail must invalidate the spacer condition before input-capture completion can qualify it.

¶ An isochronic fork on $r_{\rho k}$ constrains arrival at $G_{f_3}^+$. The two paths are

$$\begin{aligned} P_d : r_{\rho k} \uparrow \rightarrow G_{f_3}^+ \\ P_i : r_{\rho k} \uparrow \rightarrow f_2 \uparrow \rightarrow G_{f_3}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The received read-data rail must invalidate the spacer condition before capture completion enables that condition.

¶ An isochronic fork on w_a constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : w_a \uparrow \rightarrow G_{r_r}^+ \\ P_i : w_a \uparrow \rightarrow f_0 \downarrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The write acknowledge must inhibit another register read before the forwarding branch is released.

The same fork also constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : w_a \uparrow \rightarrow G_{r_r}^+ \\ P_i : w_a \uparrow \rightarrow a_a \uparrow \rightarrow f_2 \downarrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The write acknowledge must inhibit another register read while the zero-count branch completes.

C.3.4 Bubble Reshuffling

SUpdate

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{array}{ll}
f_0 \rightarrow G_{0djfn}^+ & j,n=0,\dots,3 \\
\neg f_1 \rightarrow G_{i_a}^- & \\
\neg f_2 \rightarrow G_{w_{j\ell}}^+ & j=0,\dots,3; \ell=0,\dots,7 \\
\neg f_2 \rightarrow G_{w_{j0}}^+ & j=0,\dots,3 \\
\neg f_2 \rightarrow G_{c_{h\rho n}}^+ & n=0,\dots,3; \rho=t,f \\
i_a \rightarrow G_{w_{j\ell}}^+ & j=0,\dots,3; \ell=0,\dots,7 \\
i_a \rightarrow G_{w_{j0}}^+ & j=0,\dots,3 \\
i_a \rightarrow G_{c_{h\rho n}}^+ & n=0,\dots,3; \rho=t,f \\
\neg c_{ha} \rightarrow G_{f_1}^- & \\
\neg w_{ja} \rightarrow G_{f_1}^- & j=0,\dots,3 \\
\neg o_{dja} \rightarrow G_{f_0}^- & j=0,\dots,3 \\
\neg o_{dja} \rightarrow G_s^+ & j=0,\dots,3
\end{array}$$

We implement f_0 and i_a in complemented form, while preserving f_1 , f_2 , s , a_a , the input rails, and the buffer bits b_n . The acknowledges o_{dja} and w_{ja} , for $j = 0, \dots, 3$, and c_{ha} retain their original sense, as do the data rails of O_{dj} , W_j , and C_h . The resulting PRS is:

$$\begin{array}{ll}
a_r \wedge i_{f3} \wedge i_{f2} \wedge i_{f1} \wedge i_{f0} \wedge \neg i_a \rightarrow \neg f_0 \downarrow & \\
(\neg i_a \wedge s \wedge (\neg o_{d0a} \wedge \neg o_{d1a} \wedge \neg o_{d2a} \wedge \neg o_{d3a})) \vee r \rightarrow \neg f_0 \uparrow & \\
a_r \wedge s \wedge (i_{t3} \vee i_{t2} \vee i_{t1} \vee i_{t0}) \rightarrow f_1 \uparrow & \\
(\neg s \wedge (\neg w_{0a} \wedge \neg w_{1a} \wedge \neg w_{2a} \wedge \neg w_{3a} \wedge \neg c_{ha})) \vee r \rightarrow f_1 \downarrow & \\
f_1 \wedge ((i_{t3} \wedge b_3) \vee (i_{f3} \wedge \neg b_3)) \wedge \dots \wedge ((i_{t0} \wedge b_0) \vee (i_{f0} \wedge \neg b_0)) \rightarrow f_2 \uparrow & \\
\neg i_{t3} \wedge \neg i_{f3} \wedge \neg i_{t2} \wedge \neg i_{f2} \wedge \neg i_{t1} \wedge \neg i_{f1} \wedge \neg i_{t0} \wedge \neg i_{f0} \rightarrow f_2 \downarrow & \\
(\neg f_0 \wedge \neg i_a \wedge \neg o_{d0a} \wedge \neg o_{d1a} \wedge \neg o_{d2a} \wedge \neg o_{d3a}) \vee r \rightarrow s \uparrow & \\
w_{0a} \wedge w_{1a} \wedge w_{2a} \wedge w_{3a} \wedge c_{ha} \rightarrow s \downarrow & \\
f_2 \vee (o_{d0a} \wedge o_{d1a} \wedge o_{d2a} \wedge o_{d3a}) \rightarrow \neg i_a \downarrow & \\
(\neg f_0 \wedge \neg f_1 \wedge \neg a_a \wedge \neg i_{f3} \wedge \neg i_{f2} \wedge \neg i_{f1} \wedge \neg i_{f0}) \vee r \rightarrow \neg i_a \uparrow & \\
a_r \wedge (f_1 \vee (\neg s \wedge (i_{t3} \vee i_{t2} \vee i_{t1} \vee i_{t0}))) \rightarrow a_a \uparrow & \\
(\neg f_1 \wedge \neg a_r) \vee r \rightarrow a_a \downarrow & \\
a_a \wedge s \wedge i_{tn} \rightarrow b_n \uparrow & n=0,\dots,3 \\
(a_a \wedge s \wedge i_{fn}) \vee r \rightarrow b_n \downarrow & n=0,\dots,3 \\
s \wedge \neg f_2 \wedge \neg i_a \wedge f_1 \wedge \neg b_n \rightarrow c_{hfn} \uparrow & n=0,\dots,3 \\
\neg s \vee r \rightarrow c_{hfn} \downarrow & n=0,\dots,3 \\
s \wedge \neg f_2 \wedge \neg i_a \wedge f_1 \wedge b_n \rightarrow c_{htn} \uparrow & n=0,\dots,3 \\
\neg s \vee r \rightarrow c_{htn} \downarrow & n=0,\dots,3 \\
s \wedge \neg f_2 \wedge \neg i_a \wedge f_1 \wedge \neg b_{3-j} \rightarrow w_{j\ell 0} \uparrow & j=0,\dots,3 \\
\neg s \vee r \rightarrow w_{j\ell 0} \downarrow & j=0,\dots,3 \\
s \wedge \neg f_2 \wedge \neg i_a \wedge f_1 \rightarrow w_{jfn} \uparrow & j=0,\dots,3; n=1,\dots,7 \\
\neg s \vee r \rightarrow w_{jfn} \downarrow & j=0,\dots,3; n=1,\dots,7 \\
s \wedge \neg f_2 \wedge \neg i_a \wedge f_1 \wedge b_{3-j} \rightarrow w_{jt0} \uparrow & j=0,\dots,3 \\
\neg s \vee r \rightarrow w_{jt0} \downarrow & j=0,\dots,3 \\
r \rightarrow w_{jtn} \downarrow & j=0,\dots,3; n=1,\dots,7 \\
i_a \wedge \neg f_0 \rightarrow o_{djfn} \uparrow & j,n=0,\dots,3 \\
\neg i_a \vee r \rightarrow o_{djfn} \downarrow & j,n=0,\dots,3 \\
r \rightarrow o_{djtn} \downarrow & j,n=0,\dots,3
\end{array}$$

At this point, the polarities of the signals on the right-hand side are fixed. We therefore propagate the remaining bubbles through the guards, introducing the complemented senses required by the pull-up and pull-down networks. The completion condition used to update f_2 is decomposed using a completion tree. Since f_1 and the W acknowledge signals are intentionally preserved in their original sense, $\neg w_{ja}$ is generated explicitly for the negative guard of f_1 . Finally, f_0 and i_a are restored wherever their original polarity is required.

$$\begin{array}{ll}
((i_{tn} \wedge b_n) \vee (i_{fn} \wedge \neg b_n)) \rightarrow \neg x_n \downarrow & n=0,\dots,3 \\
\neg i_{tn} \wedge \neg i_{fn} \rightarrow \neg x_n \uparrow & n=0,\dots,3 \\
\neg x_{2n+1} \wedge \neg x_{2n} \rightarrow y_n \uparrow & n=0,1 \\
x_{2n+1} \wedge x_{2n} \rightarrow y_n \downarrow & n=0,1 \\
y_1 \wedge y_0 \rightarrow z \downarrow & \\
\neg y_1 \wedge \neg y_0 \rightarrow z \uparrow &
\end{array}$$

$$\begin{aligned}
& a_r \wedge i_{f3} \wedge i_{f2} \wedge i_{f1} \wedge i_{f0} \wedge \neg i_a \rightarrow \neg f_0 \downarrow \\
& (\neg i_a \wedge \neg s \wedge (\neg o_{d0a} \wedge \neg o_{d1a} \wedge \neg o_{d2a} \wedge \neg o_{d3a})) \vee \neg r \rightarrow \neg f_0 \uparrow \\
& \neg a_r \wedge \neg s \wedge (\neg i_{t3} \vee \neg i_{t2} \vee \neg i_{t1} \vee \neg i_{t0}) \rightarrow f_1 \uparrow \\
& (\neg s \wedge \neg w_{0a} \wedge \neg w_{1a} \wedge \neg w_{2a} \wedge \neg w_{3a} \wedge \neg c_{ha}) \vee r \rightarrow f_1 \downarrow \\
& f_1 \wedge \neg z \rightarrow f_2 \uparrow \\
& z \rightarrow f_2 \downarrow \\
& (\neg f_0 \wedge \neg i_a \wedge \neg o_{d0a} \wedge \neg o_{d1a} \wedge \neg o_{d2a} \wedge \neg o_{d3a}) \vee \neg r \rightarrow s \uparrow \\
& w_{0a} \wedge w_{1a} \wedge w_{2a} \wedge w_{3a} \wedge c_{ha} \rightarrow s \downarrow \\
& f_2 \vee (o_{d0a} \wedge o_{d1a} \wedge o_{d2a} \wedge o_{d3a}) \rightarrow \neg i_a \downarrow \\
& (\neg f_0 \wedge \neg f_1 \wedge \neg a_a \wedge \neg i_{f3} \wedge \neg i_{f2} \wedge \neg i_{f1} \wedge \neg i_{f0}) \vee \neg r \rightarrow \neg i_a \uparrow \\
& \neg a_r \wedge (\neg f_1 \vee (\neg s \wedge (\neg i_{t3} \vee \neg i_{t2} \vee \neg i_{t1} \vee \neg i_{t0}))) \rightarrow a_a \uparrow \\
& (\neg f_1 \wedge \neg a_r) \vee r \rightarrow a_a \downarrow \\
& \neg a_a \wedge \neg s \wedge \neg i_{tn} \rightarrow b_n \uparrow & n=0,\dots,3 \\
& (a_a \wedge s \wedge i_{fn}) \vee r \rightarrow b_n \downarrow & n=0,\dots,3 \\
& \neg s \wedge \neg f_2 \wedge \neg i_a \wedge \neg f_1 \wedge \neg b_n \rightarrow c_{hfn} \uparrow & n=0,\dots,3 \\
& s \vee r \rightarrow c_{hfn} \downarrow & n=0,\dots,3 \\
& \neg s \wedge \neg f_2 \wedge \neg i_a \wedge \neg f_1 \wedge \neg b_n \rightarrow c_{htn} \uparrow & n=0,\dots,3 \\
& s \vee r \rightarrow c_{htn} \downarrow & n=0,\dots,3 \\
& \neg s \wedge \neg f_2 \wedge \neg i_a \wedge \neg f_1 \wedge \neg b_{3-j} \rightarrow w_{jfo} \uparrow & j=0,\dots,3 \\
& s \vee r \rightarrow w_{jfo} \downarrow & j=0,\dots,3 \\
& \neg s \wedge \neg f_2 \wedge \neg i_a \wedge \neg f_1 \rightarrow w_{jfn} \uparrow & j=0,\dots,3; n=1,\dots,7 \\
& s \vee r \rightarrow w_{jfn} \downarrow & j=0,\dots,3; n=1,\dots,7 \\
& \neg s \wedge \neg f_2 \wedge \neg i_a \wedge \neg f_1 \wedge \neg b_{3-j} \rightarrow w_{jto} \uparrow & j=0,\dots,3 \\
& s \vee r \rightarrow w_{jto} \downarrow & j=0,\dots,3 \\
& r \rightarrow w_{jtn} \downarrow & j=0,\dots,3; n=1,\dots,7 \\
& \neg i_a \wedge \neg f_0 \rightarrow o_{djfn} \uparrow & j,n=0,\dots,3 \\
& i_a \vee r \rightarrow o_{djfn} \downarrow & j,n=0,\dots,3 \\
& r \rightarrow o_{djtn} \downarrow & j,n=0,\dots,3 \\
& \neg f_0 \Rightarrow f_0 \\
& \neg f_1 \Rightarrow \neg f_1 \\
& \neg s \Rightarrow s \\
& \neg i_a \Rightarrow i_a \\
& \neg a_a \Rightarrow a_a \\
& \neg b_n \Rightarrow b_n & n=0,\dots,3 \\
& \neg b_n \Rightarrow b_n & n=0,\dots,3 \\
& \neg a_r \Rightarrow a_r \\
& \neg c_{ha} \Rightarrow c_{ha} \\
& \neg i_{fn} \Rightarrow i_{fn} & n=0,\dots,3 \\
& \neg i_{tn} \Rightarrow i_{tn} & n=0,\dots,3 \\
& \neg r \Rightarrow r \\
& \neg w_{ja} \Rightarrow w_{ja} & j=0,\dots,3
\end{aligned}$$

ChildServer

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned}
\neg f_0 &\rightarrow G_{r_r}^+ \\
\neg f_3 &\rightarrow G_{s_{ipk}}^+ & k=0,\dots,7; \rho=t,f \\
\neg f_3 &\rightarrow G_{o_{d\rho n}}^+ & n=0,\dots,3; \rho=t,f \\
\neg f_3 &\rightarrow G_{o_{c\rho n}}^+ & n=0,\dots,3; \rho=t,f \\
\neg i_a &\rightarrow G_{s_{ipk}}^+ & k=0,\dots,7; \rho=t,f \\
\neg i_a &\rightarrow G_{o_{c\rho n}}^+ & n=0,\dots,3; \rho=t,f \\
\neg i_a &\rightarrow G_{o_{d\rho n}}^+ & n=0,\dots,3; \rho=t,f \\
a_r &\rightarrow G_{r_r}^+ \\
\neg i_{\rho n} &\rightarrow G_{f_3}^- & n=0,\dots,3; \rho=t,f \\
\neg r_{\rho k} &\rightarrow G_{f_3}^+ & k=0,\dots,7; \rho=t,f \\
\neg w_a &\rightarrow G_{r_r}^+
\end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve f_3 , a_r , w_a , r_r , a_a , as well as the input rails and the rails of R , S_i , O_c and O_d , in their original sense, while implementing i_a , f_0 in complemented form. The remaining internal signals f_2 , f_1 , c_{ntr} , q , b are preserved in their original sense. The resulting PRS is:

$$\begin{aligned}
& f_3 \wedge (q_7 \vee \dots \vee q_0) \rightarrow \neg f_0 \downarrow \\
& (o_{ca} \wedge o_{da} \wedge w_a) \vee r \rightarrow \neg f_0 \uparrow \\
& \neg a_a \wedge f_3 \wedge (\neg q_7 \wedge \dots \wedge \neg q_0) \rightarrow f_1 \uparrow \\
& a_a \vee r \rightarrow f_1 \downarrow \\
& r_r \wedge ((r_{t7} \wedge q_7) \vee (r_{f7} \wedge \neg q_7)) \wedge \dots \wedge ((r_{t0} \wedge q_0) \vee (r_{f0} \wedge \neg q_0)) \rightarrow f_2 \uparrow \\
& (\neg \neg f_0 \wedge ((i_{t3} \wedge b_3) \vee (i_{f3} \wedge \neg b_3)) \wedge \dots \wedge ((i_{t0} \wedge b_0) \vee (i_{f0} \wedge \neg b_0))) \vee a_a \vee r \rightarrow f_2 \downarrow \\
& f_2 \wedge \neg a_a \wedge (\neg r_{t7} \wedge \neg r_{f7} \wedge \dots \wedge \neg r_{t0} \wedge \neg r_{f0}) \rightarrow f_3 \uparrow \\
& (\neg f_2 \wedge (\neg i_{t3} \wedge \neg i_{f3} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0})) \vee a_a \vee r \rightarrow f_3 \downarrow \\
& r_{tk} \rightarrow q_k \uparrow \quad k=0,\dots,7 \\
& r_{fk} \vee r \rightarrow q_k \downarrow \quad k=0,\dots,7 \\
& f_3 \wedge \neg \neg f_0 \wedge \neg f_2 \rightarrow \neg i_a \downarrow \\
& \neg f_3 \rightarrow \neg i_a \uparrow \\
& w_a \wedge f_1 \rightarrow a_a \uparrow \\
& \neg w_a \wedge \neg a_r \rightarrow a_a \downarrow \\
& f_1 \rightarrow c_{ntr} \uparrow \\
& \neg f_3 \wedge \neg f_2 \wedge \neg f_1 \rightarrow c_{ntr} \downarrow \\
& a_r \wedge \neg f_2 \wedge \neg a_a \wedge \neg f_0 \wedge \neg o_{ca} \wedge \neg o_{da} \wedge \neg w_a \rightarrow r_r \uparrow \\
& f_2 \vee r \rightarrow r_r \downarrow \\
& \neg \neg f_0 \wedge f_2 \wedge i_{tk} \rightarrow b_k \uparrow \quad k=0,\dots,3 \\
& (\neg \neg f_0 \wedge f_2 \wedge i_{fk}) \vee r \rightarrow b_k \downarrow \quad k=0,\dots,3 \\
& c_{ntfk} \vee s_{ofk} \Rightarrow w_{fk} \quad k=0,\dots,7 \\
& c_{nttk} \vee s_{otk} \Rightarrow w_{tk} \quad k=0,\dots,7 \\
& (\neg \neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg q_k \rightarrow s_{ifk} \uparrow \quad k=0,\dots,7 \\
& \neg f_0 \rightarrow s_{ifk} \downarrow \quad k=0,\dots,7 \\
& (\neg \neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge q_k \rightarrow s_{itk} \uparrow \quad k=0,\dots,7 \\
& \neg f_0 \rightarrow s_{itk} \downarrow \quad k=0,\dots,7 \\
& (\neg \neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg b_k \rightarrow o_{dfk} \uparrow \quad k=0,\dots,3 \\
& \neg f_0 \rightarrow o_{dfk} \downarrow \quad k=0,\dots,3 \\
& (\neg \neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge b_k \rightarrow o_{dtk} \uparrow \quad k=0,\dots,3 \\
& \neg f_0 \rightarrow o_{dtk} \downarrow \quad k=0,\dots,3
\end{aligned}$$

O_C : same rules, with $o_{d\rho k} \mapsto o_{c\rho k}$.

At this point, the polarities of the signals on the right-hand side are fixed. We therefore adjust the guards, introducing explicit inverters wherever the opposite polarity of a polarity-fixed signal is required. The large completion conditions used to update f_2 and f_3 are decomposed using completion trees. Finally, inverters are added wherever the original polarity of a complemented signal is required.

$((\neg r_{tk} \wedge \neg q_k) \vee (\neg r_{fk} \wedge \neg q_k)) \rightarrow x_k \uparrow$	$k=0,\dots,7$
$r_{tk} \wedge r_{fk} \rightarrow x_k \downarrow$	$k=0,\dots,7$
$x_{2k+1} \wedge x_{2k} \rightarrow y_k \downarrow$	$k=0,\dots,3$
$\neg x_{2k+1} \wedge \neg x_{2k} \rightarrow y_k \uparrow$	$k=0,\dots,3$
$\neg y_{2k+1} \wedge \neg y_{2k} \rightarrow z_k \uparrow$	$k=0,1$
$y_{2k+1} \wedge y_{2k} \rightarrow z_k \downarrow$	$k=0,1$
$z_1 \wedge z_0 \rightarrow u \downarrow$	
$\neg z_1 \wedge \neg z_0 \rightarrow u \uparrow$	
$((i_{tk} \wedge b_k) \vee (i_{fk} \wedge b_k)) \rightarrow h_k \downarrow$	$k=0,\dots,3$
$\neg i_{tk} \wedge \neg i_{fk} \rightarrow h_k \uparrow$	$k=0,\dots,3$
$\neg h_{2k+1} \wedge \neg h_{2k} \rightarrow l_k \uparrow$	$k=0,1$
$h_{2k+1} \wedge h_{2k} \rightarrow l_k \downarrow$	$k=0,1$
$\neg r_{tk} \wedge \neg r_{fk} \rightarrow e_k \uparrow$	$k=0,\dots,7$
$r_{tk} \vee r_{fk} \rightarrow e_k \downarrow$	$k=0,\dots,7$
$e_{2k+1} \wedge e_{2k} \rightarrow c_{R,k} \downarrow$	$k=0,\dots,3$
$\neg e_{2k+1} \wedge \neg e_{2k} \rightarrow c_{R,k} \uparrow$	$k=0,\dots,3$
$\neg c_{R,2k+1} \wedge \neg c_{R,2k} \rightarrow g_k \uparrow$	$k=0,1$
$c_{R,2k+1} \wedge c_{R,2k} \rightarrow g_k \downarrow$	$k=0,1$
$g_1 \wedge g_0 \rightarrow m \downarrow$	
$\neg g_1 \wedge \neg g_0 \rightarrow m \uparrow$	
$i_{tk} \wedge i_{fk} \rightarrow v_k \downarrow$	$k=0,\dots,3$
$\neg i_{tk} \vee \neg i_{fk} \rightarrow v_k \uparrow$	$k=0,\dots,3$
$\neg v_{2k+1} \wedge \neg v_{2k} \rightarrow t_k \uparrow$	$k=0,1$
$v_{2k+1} \wedge v_{2k} \rightarrow t_k \downarrow$	$k=0,1$

$$\begin{aligned}
& f_3 \wedge (q_7 \vee \dots \vee q_0) \rightarrow \neg f_0 \downarrow \\
& (\neg o_{ca} \wedge \neg o_{da} \wedge \neg w_a) \vee \neg r \rightarrow \neg f_0 \uparrow \\
& \neg a_a \wedge \neg f_3 \wedge (\neg q_7 \wedge \dots \wedge \neg q_0) \rightarrow f_1 \uparrow \\
& a_a \vee r \rightarrow f_1 \downarrow \\
& \neg r_r \wedge \neg u \rightarrow f_2 \uparrow \\
& (f_0 \wedge l_1 \wedge l_0) \vee a_a \vee r \rightarrow f_2 \downarrow \\
& \neg f_2 \wedge \neg a_a \wedge \neg m \rightarrow f_3 \uparrow \\
& (\neg f_2 \wedge t_1 \wedge t_0) \vee a_a \vee r \rightarrow f_3 \downarrow \\
& \neg x_{tk} \rightarrow q_k \uparrow & k=0,\dots,7 \\
& r_{fk} \vee r \rightarrow q_k \downarrow & k=0,\dots,7 \\
& f_3 \wedge f_0 \wedge \neg f_2 \rightarrow i_a \downarrow \\
& \neg f_3 \rightarrow i_a \uparrow \\
& \neg w_a \wedge \neg f_1 \rightarrow a_a \uparrow \\
& w_a \wedge a_r \rightarrow a_a \downarrow \\
& \neg f_1 \rightarrow c_{ntr} \uparrow \\
& \neg f_3 \wedge \neg f_2 \wedge \neg f_1 \rightarrow c_{ntr} \downarrow \\
& \neg a_r \wedge \neg f_2 \wedge \neg a_a \wedge \neg f_0 \wedge \neg o_{ca} \wedge \neg o_{da} \wedge \neg w_a \rightarrow r_r \uparrow \\
& f_2 \vee r \rightarrow r_r \downarrow \\
& \neg f_0 \wedge \neg f_2 \wedge \neg i_{tk} \rightarrow b_k \uparrow & k=0,\dots,3 \\
& (f_0 \wedge f_2 \wedge i_{fk}) \vee r \rightarrow b_k \downarrow & k=0,\dots,3 \\
& c_{ntfk} \vee s_{ofk} \Rightarrow w_{fk} & k=0,\dots,7 \\
& c_{nttk} \vee s_{otk} \Rightarrow w_{tk} & k=0,\dots,7 \\
& (\neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg q_k \rightarrow s_{ifk} \uparrow & k=0,\dots,7 \\
& \neg f_0 \rightarrow s_{ifk} \downarrow & k=0,\dots,7 \\
& (\neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg q_k \rightarrow s_{itk} \uparrow & k=0,\dots,7 \\
& \neg f_0 \rightarrow s_{itk} \downarrow & k=0,\dots,7 \\
& (\neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg b_k \rightarrow o_{dfk} \uparrow & k=0,\dots,3 \\
& \neg f_0 \rightarrow o_{dfk} \downarrow & k=0,\dots,3 \\
& (\neg f_0 \wedge \neg f_3 \wedge \neg i_a) \wedge \neg b_k \rightarrow o_{dtk} \uparrow & k=0,\dots,3 \\
& \neg f_0 \rightarrow o_{dtk} \downarrow & k=0,\dots,3 \\
& \neg f_0 \Rightarrow f_0 \\
& \neg f_1 \Rightarrow \neg f_1 \\
& \neg f_2 \Rightarrow \neg f_2 \\
& \neg f_3 \Rightarrow \neg f_3 \\
& \neg q_k \Rightarrow \neg q_k & k=0,\dots,7 \\
& \neg i_a \Rightarrow i_a \\
& \neg r_r \Rightarrow r_r \\
& \neg b_k \Rightarrow b_k & k=0,\dots,3 \\
& \neg a_r \Rightarrow a_r \\
& \neg i_{fk} \Rightarrow i_{fk} & k=0,\dots,3 \\
& \neg i_{tk} \Rightarrow i_{tk} & k=0,\dots,3 \\
& \neg r \Rightarrow r \\
& \neg r_{fk} \Rightarrow r_{fk} & k=0,\dots,7 \\
& \neg r_{tk} \Rightarrow r_{tk} & k=0,\dots,7 \\
& \neg w_a \Rightarrow w_a \\
& \neg o_{ca} \Rightarrow o_{ca} \\
& \neg o_{da} \Rightarrow o_{da}
\end{aligned}$$

$O_C: o_{dpk} \mapsto o_{cpk}$, including the corresponding inverter definitions.

C.4 Counter

C.4.1 CHP

$$\begin{aligned} \text{Counter}(I?, C_{nt}!) &\equiv \\ q &:= 0; \\ * [[\bar{I} \rightarrow I?b; q := q + b.msb + b.(msb - 1) + b.(msb - 2) + b.(msb - 3) \parallel \bar{C}_{nt} \rightarrow C_{nt}!q; q := 0]] \end{aligned}$$

However, we can avoid using the buffer b by directly computing the sum from the input channel.

$$\begin{aligned} \text{Counter}(I?, C_{nt}!) &\equiv \\ q &:= 0; \\ * [[\bar{I} \rightarrow q := q + \bar{I}.msb + \bar{I}.(msb - 1) + \bar{I}.(msb - 2) + \bar{I}.(msb - 3); I \parallel \bar{C}_{nt} \rightarrow C_{nt}!q; q := 0]] \end{aligned}$$

This implementation requires two standard building blocks: an adder and a register. Their implementations are well known and documented in the literature, so we only present the register CHP here and their relevant PRSs later. The arithmetic block computes the updated value. The register is accessed through write channel W and read channel R ; the requester-side handshakes are shown explicitly in the HSE. The CHP can therefore be decomposed as follows.

$$\begin{aligned} \text{Counter}(I?, C_{nt}!) &\equiv \\ q &:= 0; \\ * [[\bar{I} \rightarrow W!(R? + \bar{I}.msb + \bar{I}.(msb - 1) + \bar{I}.(msb - 2) + \bar{I}.(msb - 3)); I \parallel \bar{C}_{nt} \rightarrow C_{nt}!R?; W!0]] \end{aligned}$$

C.4.2 HSE

For clarity, we will refer to C_{nt} simply as C , while keeping in mind that it is a passive channel. For the stated configuration of 1024 final-level nodes and 4096 leaves, the stated counter bound is 256. The implementation below uses an 8-bit accumulator. The four-bit bus S_a feeds the population-count tree. Its three-bit result $j_{2:0}$ is zero-extended onto the eight-bit bus $S_{a'}$, which feeds the first ripple-carry operand. The second operand S_b carries the stored count, and S_o carries the updated count.

Once valid data are available on I , the two adder inputs are updated in parallel. The read request r_r must return low before the new sum is written through W , since simultaneously reading and writing the register could create interference in its internal state. We therefore assert i_a only after receiving the write acknowledge w_a , which also guarantees that the adder output has been consumed.

The counter-read branch follows the same principle. The consumer raises c_r to request the current counter value, and its withdrawal is used to terminate the register read before the register is reset to zero.

The auxiliary variables introduced below are f_0 for accumulation, f_1 for counter read/reset, and f_2 for read-channel return to spacer. In particular, the control flag f_0 is distinct from the bit index k .

Counter $\equiv$

```
* [[it3 ∨ if3 →
  ⟨, k : 3..0 : [itk → satk ↑ ∥ ifk → safk ↑]⟩,
  (rr ↑; ⟨, k : 7..0 : [rtk → sbtk ↑ ∥ rfk → sbfk ↑]⟩; rr ↓);
  ⟨, k : 7..0 : [sotk → wtk ↑ ∥ sofk → wfk ↑]⟩; [wa]; ia ↑;
  ⟨, k : 3..0 : [¬itk → satk ↓], [¬ifk → safk ↓]⟩, ⟨, k : 7..0 : [¬rtk → sbtk ↓], [¬rfk → sbfk ↓]⟩;
  ⟨, k : 7..0 : [¬sotk → wtk ↓], [¬sofk → wfk ↓]⟩; [¬wa]; ia ↓
  ∥ cr → rr ↑; ⟨, k : 7..0 : [rtk → ctk ↑ ∥ rfk → cfk ↑]⟩; [¬cr]; rr ↓;
  ⟨, k : 7..0 : [¬rtk → ctk ↓], [¬rfk → cfk ↓]⟩;
  ⟨, k : 7..0 : wfk ↑⟩; [wa]; ⟨, k : 7..0 : wfk ↓⟩; [¬wa]
]]
```

The counter-read branch clears the register only after its read handshake completes. We insert f_1 before $r_r \uparrow$ and clear it after $[w_a]$ to identify this branch. The state f_2 records that all counter-output rails have returned to spacer, so this wide condition need not be repeated in every write guard. It is cleared when f_1 returns low.

The state f_0 identifies accumulation. It is asserted when an input is pending and $\neg f_1 \wedge \neg w_a$ confirms that no counter-reset write remains in progress.

Counter $\equiv$

```
* [[it3 ∨ if3 → f0 ↑;
  ⟨, n : 3..0 : [itn → satn ↑ ∥ ifn → safn ↑]⟩,
  (rr ↑; ⟨, n : 7..0 : [rtn → sbtn ↑ ∥ rfn → sbfn ↑]⟩; rr ↓);
  ⟨, n : 7..0 : [sotn → wtn ↑ ∥ sofn → wfn ↑]⟩; [wa]; ia ↑;
  ⟨, n : 3..0 : [¬itn → satn ↓], [¬ifn → safn ↓]⟩, ⟨, n : 7..0 : [¬rtn → sbtn ↓], [¬rfn → sbfn ↓]⟩;
  ⟨, n : 7..0 : [¬sotn → wtn ↓], [¬sofn → wfn ↓]⟩; [¬wa]; f0 ↓; ia ↓
  ∥ cr → f1 ↑; rr ↑; ⟨, n : 7..0 : [rtn → ctn ↑ ∥ rfn → cfn ↑]⟩; [¬cr]; rr ↓;
  ⟨, n : 7..0 : [¬rtn → ctn ↓], [¬rfn → cfn ↓]⟩; f2 ↑; ⟨, n : 7..0 : wfn ↑⟩; [wa]; f1 ↓; f2 ↓;
  ⟨, n : 7..0 : wfn ↓⟩; [¬wa]
]]
```

C.4.3 PRS

The variable f_0 is asserted when a new input arrives and the second branch has completed any possible register-reset operation. We therefore additionally require $\neg f_1 \wedge \neg w_a$. It is deasserted once w_a is low while i_a is still high.

$$\begin{aligned} (i_{t3} \vee i_{f3}) \wedge (\neg w_a \wedge \neg f_1) &\rightarrow f_0 \uparrow \\ (\neg w_a \wedge i_a) \vee r &\rightarrow f_0 \downarrow \end{aligned}$$

The variable f_1 is asserted when c_r becomes high and is deasserted once the register-reset write has been acknowledged.

$$\begin{aligned} c_r &\rightarrow f_1 \uparrow \\ w_a \vee r &\rightarrow f_1 \downarrow \end{aligned}$$

The variable f_2 is asserted once all rails of C have returned to spacer while f_1 is high, indicating that the counter-read branch is active. Since the output rails are also at spacer immediately after f_1 is asserted, the positive guard additionally requires c_r to be low. The variable is deasserted once f_1 returns low.

$$\begin{aligned} f_1 \wedge \neg c_r \wedge \neg c_{t7} \wedge \neg c_{f7} \wedge \dots \wedge \neg c_{t0} \wedge \neg c_{f0} &\rightarrow f_2 \uparrow \\ \neg f_1 &\rightarrow f_2 \downarrow \end{aligned}$$

In the first branch, i_a is asserted by w_a and deasserted by $\neg f_0$. To prevent i_a from being asserted in the second branch, the positive guard also checks the validity of one bit of S_a , since this condition cannot occur in the counter-read branch.

$$\begin{aligned} (s_{at3} \vee s_{af3}) \wedge w_a &\rightarrow i_a \uparrow \\ \neg f_0 \vee r &\rightarrow i_a \downarrow \end{aligned}$$

r_r is asserted whenever either f_0 or f_1 indicates that a register read is required. In the second branch, the positive guard is strengthened with c_r , since f_1 remains high both when r_r must be asserted and when it must subsequently be deasserted. In the first branch, the positive guard also checks that S_b is still at spacer and that i_a is low, preventing a second read from being initiated after the computation has already completed.

$$\begin{aligned} (f_1 \wedge c_r) \vee (\neg i_a \wedge \neg s_{bt7} \wedge \neg s_{bf7} \wedge f_0) &\rightarrow r_r \uparrow \\ (f_1 \wedge \neg c_r) \vee ((s_{bt7} \vee s_{bf7}) \wedge \dots \wedge (s_{bt0} \vee s_{bf0})) \vee r &\rightarrow r_r \downarrow \end{aligned}$$

The rails of C follow the corresponding rails of the register read channel R , but they can only be asserted while c_r is high.

$$\begin{aligned} c_r \wedge r_{fk} &\rightarrow c_{fk} \uparrow & k=0,\dots,7 \\ \neg r_{fk} &\rightarrow c_{fk} \downarrow & k=0,\dots,7 \\ c_r \wedge r_{tk} &\rightarrow c_{tk} \uparrow & k=0,\dots,7 \\ \neg r_{tk} &\rightarrow c_{tk} \downarrow & k=0,\dots,7 \end{aligned}$$

In the first branch, the write channel W follows the adder output S_o , whereas in the second branch it is driven to zero. Therefore, the true rails of W follow the corresponding true rails of S_o , while the false rails can be asserted either by S_o or by the state variable f_2 . In the first branch, both true and false rails must wait until r_r is low before being asserted, preventing simultaneous read and write operations on the register.

$$\begin{aligned} (s_{ofk} \wedge \neg r_r) \vee f_2 &\rightarrow w_{fk} \uparrow & k=0,\dots,7 \\ \neg s_{ofk} \wedge \neg f_2 &\rightarrow w_{fk} \downarrow & k=0,\dots,7 \\ s_{otk} \wedge \neg r_r &\rightarrow w_{tk} \uparrow & k=0,\dots,7 \\ \neg s_{otk} &\rightarrow w_{tk} \downarrow & k=0,\dots,7 \end{aligned}$$

Regarding the adder implementation, we use a tree adder to sum the four input bits and an 8-bit ripple-carry adder to accumulate the result into the

current counter value, as shown in Figure C.5. The *FullAdder* and *HalfAdder* blocks are standard components whose implementations were presented previously.

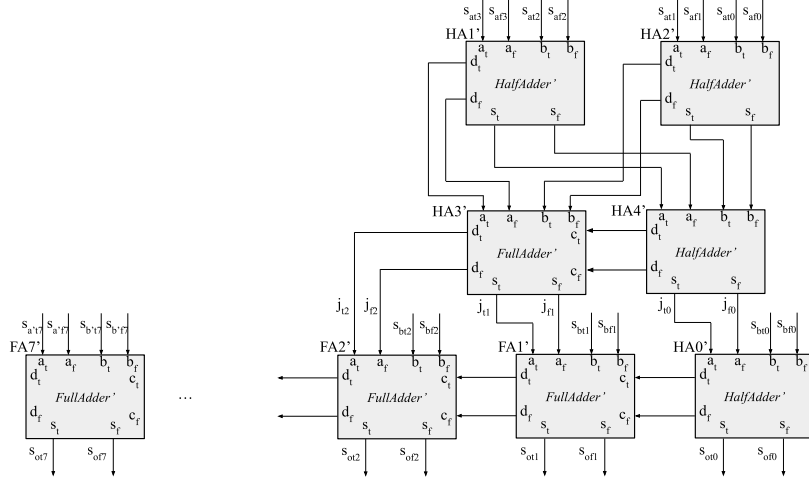


Figure C.5: Counter adder: S_a feeds the four-bit population-count tree, whose result is zero-extended onto S_a' at the ripple-carry input. S_b supplies the stored count.

The tree result $j_{2:0}$ is wired to bits 2 : 0 of S_a' . Bits 7 : 3 provide zero extension: their true rails remain low, and their false rails become valid when all three result bits are valid.

$$\begin{aligned}
 j_{fk} &\Rightarrow s_{a'fk} & k=0,1,2 \\
 (j_{i2} \vee j_{f2}) \wedge (j_{i1} \vee j_{f1}) \wedge (j_{i0} \vee j_{f0}) \wedge \neg r &\rightarrow s_{a'fk} \uparrow & k=3,\dots,7 \\
 (\neg j_{i2} \wedge \neg j_{f2} \wedge \neg j_{i1} \wedge \neg j_{f1} \wedge \neg j_{i0} \wedge \neg j_{f0}) \vee r &\rightarrow s_{a'fk} \downarrow & k=3,\dots,7 \\
 j_{tk} &\Rightarrow s_{a'tk} & k=0,1,2 \\
 r \vee \neg r &\rightarrow s_{a'tk} \downarrow & k=3,\dots,7
 \end{aligned}$$

The four bit pairs of S_a follow the corresponding input rails, but they can only be asserted once f_0 is high.

$$\begin{aligned}
 i_{fk} \wedge f_0 &\rightarrow s_{afk} \uparrow & k=0,\dots,3 \\
 \neg i_{fk} &\rightarrow s_{afk} \downarrow & k=0,\dots,3 \\
 i_{tk} \wedge f_0 &\rightarrow s_{atk} \uparrow & k=0,\dots,3 \\
 \neg i_{tk} &\rightarrow s_{atk} \downarrow & k=0,\dots,3
 \end{aligned}$$

The second adder input, S_b , follows the current value read from the register. Again, its rails can only be asserted while f_0 is high. During the return-to-spacer phase, i_a ensures that the computation has already completed before the rails are cleared.

$$\begin{aligned}
 r_{fk} \wedge f_0 &\rightarrow s_{bfk} \uparrow & k=0,\dots,7 \\
 (\neg r_{fk} \wedge i_a) \vee r &\rightarrow s_{bfk} \downarrow & k=0,\dots,7 \\
 r_{tk} \wedge f_0 &\rightarrow s_{btk} \uparrow & k=0,\dots,7 \\
 (\neg r_{tk} \wedge i_a) \vee r &\rightarrow s_{btk} \downarrow & k=0,\dots,7
 \end{aligned}$$

In the fork paths below, $\rho \in \{t, f\}$. Register and accumulated-value bits use indices $0, \dots, 7$; the raw input nibble uses $0, \dots, 3$.

¶ An isochronic fork on f_0 constrains arrival at $G_{s_{\rho k}}^+$. The two paths are

$$\begin{aligned} P_d : f_0 \downarrow \rightarrow G_{s_{\rho k}}^+ \\ P_i : f_0 \downarrow \rightarrow i_a \downarrow \rightarrow i_{\rho k} \uparrow \rightarrow G_{s_{\rho k}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The accumulation state must disable the first adder-input guards before the next input arrives. Here $k \in \{0, \dots, 3\}$, as in the raw-input rules above.

The same fork also constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : f_0 \downarrow \rightarrow G_{r_r}^+ \\ P_i : f_0 \downarrow \rightarrow i_a \downarrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The old accumulation state must not qualify a new register read when the input acknowledge returns low.

¶ An isochronic fork on i_a constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : i_a \uparrow \rightarrow G_{r_r}^+ \\ P_i : i_a \uparrow \rightarrow s_{b07} \downarrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The input acknowledge must inhibit another register read before the monitored adder-input bit returns to spacer.

The same fork also constrains arrival at $G_{f_0}^-$. The two paths are

$$\begin{aligned} P_d : i_a \downarrow \rightarrow G_{f_0}^- \\ P_i : i_a \downarrow \rightarrow i_{\rho 3} \uparrow \rightarrow G_{f_0}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The clearing guard must stop using the old acknowledge before a new input enables accumulation.

¶ An isochronic fork on c_r constrains arrival at $G_{f_2}^+$. The two paths are

$$\begin{aligned} P_d : c_r \uparrow \rightarrow G_{f_2}^+ \\ P_i : c_r \uparrow \rightarrow f_1 \uparrow \rightarrow G_{f_2}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The counter-read request must inhibit spacer completion before the read branch is enabled.

¶ An isochronic fork on w_a constrains arrival at $G_{f_0}^-$. The two paths are

$$\begin{aligned} P_d : w_a \uparrow \rightarrow G_{f_0}^- \\ P_i : w_a \uparrow \rightarrow i_a \uparrow \rightarrow G_{f_0}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The write acknowledge must inhibit clearing of the accumulation state before the input acknowledge rises.

The same fork also constrains arrival at $G_{f_0}^+$. The two paths are

$$\begin{aligned} P_d : w_a \uparrow \rightarrow G_{f_0}^+ \\ P_i : w_a \uparrow \rightarrow f_1 \downarrow \rightarrow G_{f_0}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The write acknowledge must inhibit accumulation before the register-reset branch is released.

C.4.4 Bubble Reshuffling

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned} f_0 &\rightarrow G_{s_{apk}}^+ \\ f_0 &\rightarrow G_{r_r}^+ \\ \neg i_a &\rightarrow G_{r_r}^+ \\ i_a &\rightarrow G_{f_0}^- \\ \neg c_r &\rightarrow G_{f_2}^+ \\ \neg w_a &\rightarrow G_{f_0}^+ \\ \neg w_a &\rightarrow G_{f_0}^- \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve w_a, f_1, f_2, c_r , together with the input rails and the rails of S_a , in their original sense, while implementing f_0, i_a, r_r in complemented form. For the remaining signals, we preserve the rails of S_b and C in their original sense, while implementing the rails of the write channel W in complemented form. The resulting PRS is:

$$\begin{aligned}
& (i_{t3} \vee i_{f3}) \wedge (\neg w_a \wedge \neg f_1) \rightarrow \neg f_0 \downarrow \\
& (\neg w_a \wedge \neg i_a) \vee r \rightarrow \neg f_0 \uparrow \\
& c_r \rightarrow f_1 \uparrow \\
& w_a \vee r \rightarrow f_1 \downarrow \\
& f_1 \wedge \neg c_r \wedge \neg c_{t7} \wedge \neg c_{f7} \wedge \dots \wedge \neg c_{t0} \wedge \neg c_{f0} \rightarrow f_2 \uparrow \\
& \neg f_1 \rightarrow f_2 \downarrow \\
& (s_{at3} \vee s_{af3}) \wedge w_a \rightarrow i_a \downarrow \\
& \neg f_0 \vee r \rightarrow i_a \uparrow \\
& (f_1 \wedge c_r) \vee (i_a \wedge \neg s_{bt7} \wedge \neg s_{bf7} \wedge \neg f_0) \rightarrow r_r \downarrow \\
& (f_1 \wedge \neg c_r) \vee ((s_{bt7} \vee s_{bf7}) \wedge \dots \wedge (s_{bt0} \vee s_{bf0})) \vee r \rightarrow r_r \uparrow \\
& c_r \wedge r_{fk} \rightarrow c_{fk} \uparrow \quad k=0,\dots,7 \\
& \neg r_{fk} \rightarrow c_{fk} \downarrow \quad k=0,\dots,7 \\
& c_r \wedge r_{tk} \rightarrow c_{tk} \uparrow \quad k=0,\dots,7 \\
& \neg r_{tk} \rightarrow c_{tk} \downarrow \quad k=0,\dots,7 \\
& (s_{ofk} \wedge r_r) \vee f_2 \rightarrow \neg w_{fk} \downarrow \quad k=0,\dots,7 \\
& \neg s_{ofk} \wedge \neg f_2 \rightarrow \neg w_{fk} \uparrow \quad k=0,\dots,7 \\
& s_{otk} \wedge r_r \rightarrow \neg w_{tk} \downarrow \quad k=0,\dots,7 \\
& \neg s_{otk} \rightarrow \neg w_{tk} \uparrow \quad k=0,\dots,7 \\
& j_{fk} \Rightarrow s_{a'fk} \quad k=0,1,2 \\
& (j_{t2} \vee j_{f2}) \wedge (j_{t1} \vee j_{f1}) \wedge (j_{t0} \vee j_{f0}) \wedge \neg r \rightarrow s_{a'fk} \uparrow \quad k=3,\dots,7 \\
& (\neg j_{t2} \wedge \neg j_{f2} \wedge \neg j_{t1} \wedge \neg j_{f1} \wedge \neg j_{t0} \wedge \neg j_{f0}) \vee r \rightarrow s_{a'fk} \downarrow \quad k=3,\dots,7 \\
& j_{tk} \Rightarrow s_{a'tk} \quad k=0,1,2 \\
& r \vee \neg r \rightarrow s_{a'tk} \downarrow \quad k=3,\dots,7 \\
& i_{fk} \wedge \neg f_0 \rightarrow s_{afk} \uparrow \quad k=0,\dots,3 \\
& \neg i_{fk} \rightarrow s_{afk} \downarrow \quad k=0,\dots,3 \\
& i_{tk} \wedge \neg f_0 \rightarrow s_{atk} \uparrow \quad k=0,\dots,3 \\
& \neg i_{tk} \rightarrow s_{atk} \downarrow \quad k=0,\dots,3 \\
& r_{fk} \wedge \neg f_0 \rightarrow s_{bfk} \uparrow \quad k=0,\dots,7 \\
& (\neg r_{fk} \wedge \neg i_a) \vee r \rightarrow s_{bfk} \downarrow \quad k=0,\dots,7 \\
& r_{tk} \wedge \neg f_0 \rightarrow s_{btk} \uparrow \quad k=0,\dots,7 \\
& (\neg r_{tk} \wedge \neg i_a) \vee r \rightarrow s_{btk} \downarrow \quad k=0,\dots,7
\end{aligned}$$

At this point, the fork-critical polarities are fixed. We propagate the remaining bubbles through the guards, introducing explicit inverters wherever the opposite polarity of a polarity-fixed signal is required. The large completion conditions associated with the validity of S_b and the return of C to spacer are implemented using completion trees.

$(\neg s_{btk} \vee \neg s_{bfk}) \rightarrow x_k \uparrow$	$k=0,\dots,7$
$s_{btk} \wedge s_{bfk} \rightarrow x_k \downarrow$	$k=0,\dots,7$
$x_{2k+1} \wedge x_{2k} \rightarrow y_k \downarrow$	$k=0,\dots,3$
$\neg x_{2k+1} \wedge \neg x_{2k} \rightarrow y_k \uparrow$	$k=0,\dots,3$
$\neg y_{2k+1} \wedge \neg y_{2k} \rightarrow z_k \uparrow$	$k=0,1$
$y_{2k+1} \wedge y_{2k} \rightarrow z_k \downarrow$	$k=0,1$
$z_1 \wedge z_0 \rightarrow h \downarrow$	
$\neg z_1 \wedge \neg z_0 \rightarrow h \uparrow$	
$\neg c_{tk} \wedge \neg c_{fk} \rightarrow d_k \uparrow$	$k=0,\dots,7$
$c_{tk} \vee c_{fk} \rightarrow d_k \downarrow$	$k=0,\dots,7$
$d_{2k+1} \wedge d_{2k} \rightarrow e_k \downarrow$	$k=0,\dots,3$
$\neg d_{2k+1} \wedge \neg d_{2k} \rightarrow e_k \uparrow$	$k=0,\dots,3$
$\neg e_{2k+1} \wedge \neg e_{2k} \rightarrow g_k \uparrow$	$k=0,1$
$e_{2k+1} \wedge e_{2k} \rightarrow g_k \downarrow$	$k=0,1$
$g_1 \wedge g_0 \rightarrow m \downarrow$	
$\neg g_1 \wedge \neg g_0 \rightarrow m \uparrow$	

$$\begin{aligned}
& (i_{t3} \vee i_{f3}) \wedge (\neg w_a \wedge \neg f_1) \rightarrow \neg f_0 \downarrow \\
& (\neg w_a \wedge \neg i_a) \vee \neg r \rightarrow \neg f_0 \uparrow \\
& \neg c_r \rightarrow f_1 \uparrow \\
& w_a \vee r \rightarrow f_1 \downarrow \\
& \neg f_1 \wedge \neg c_r \wedge \neg m \rightarrow f_2 \uparrow \\
& \neg f_1 \rightarrow f_2 \downarrow \\
& (s_{at3} \vee s_{af3}) \wedge w_a \rightarrow i_a \downarrow \\
& \neg f_0 \vee \neg r \rightarrow i_a \uparrow \\
& (f_1 \wedge c_r) \vee (i_a \wedge s_{bt7} \wedge s_{bf7} \wedge f_0) \rightarrow r_r \downarrow \\
& (\neg f_1 \wedge \neg c_r) \vee \neg h \vee \neg r \rightarrow r_r \uparrow \\
& \neg c_r \wedge \neg r_{fk} \rightarrow c_{fk} \uparrow & k=0,\dots,7 \\
& r_{fk} \rightarrow c_{fk} \downarrow & k=0,\dots,7 \\
& \neg c_r \wedge \neg r_{tk} \rightarrow c_{tk} \uparrow & k=0,\dots,7 \\
& r_{tk} \rightarrow c_{tk} \downarrow & k=0,\dots,7 \\
& (s_{ofk} \wedge r_r) \vee f_2 \rightarrow w_{fk} \downarrow & k=0,\dots,7 \\
& \neg s_{ofk} \wedge \neg f_2 \rightarrow w_{fk} \uparrow & k=0,\dots,7 \\
& s_{otk} \wedge r_r \rightarrow w_{tk} \downarrow & k=0,\dots,7 \\
& \neg s_{otk} \rightarrow w_{tk} \uparrow & k=0,\dots,7 \\
& j_{fk} \Rightarrow s_{a'fk} & k=0,1,2 \\
& (j_{t2} \vee j_{f2}) \wedge (j_{t1} \vee j_{f1}) \wedge (j_{t0} \vee j_{f0}) \wedge r \rightarrow s_{a'fk} \uparrow & k=3,\dots,7 \\
& (\neg j_{t2} \wedge \neg j_{f2} \wedge \neg j_{t1} \wedge \neg j_{f1} \wedge \neg j_{t0} \wedge \neg j_{f0}) \vee r \rightarrow s_{a'fk} \downarrow & k=3,\dots,7 \\
& j_{tk} \Rightarrow s_{a'tk} & k=0,1,2 \\
& r \vee r \rightarrow s_{a'tk} \downarrow & k=3,\dots,7 \\
& \neg i_{fk} \wedge \neg f_0 \rightarrow s_{afk} \uparrow & k=0,\dots,3 \\
& i_{fk} \rightarrow s_{afk} \downarrow & k=0,\dots,3 \\
& \neg i_{tk} \wedge \neg f_0 \rightarrow s_{atk} \uparrow & k=0,\dots,3 \\
& i_{tk} \rightarrow s_{atk} \downarrow & k=0,\dots,3 \\
& \neg r_{fk} \wedge \neg f_0 \rightarrow s_{bfk} \uparrow & k=0,\dots,7 \\
& (r_{fk} \wedge i_a) \vee r \rightarrow s_{bfk} \downarrow & k=0,\dots,7 \\
& \neg r_{tk} \wedge \neg f_0 \rightarrow s_{btk} \uparrow & k=0,\dots,7 \\
& (r_{tk} \wedge i_a) \vee r \rightarrow s_{btk} \downarrow & k=0,\dots,7 \\
& \neg f_0 \Rightarrow f_0 \\
& \neg f_1 \Rightarrow \neg f_1 \\
& \neg i_a \Rightarrow i_a \\
& \neg r_r \Rightarrow r_r \\
& \neg c_r \Rightarrow c_r \\
& \neg i_{fk} \Rightarrow i_{fk} & k=0,\dots,3 \\
& \neg i_{tk} \Rightarrow i_{tk} & k=0,\dots,3 \\
& \neg j_{fk} \Rightarrow j_{fk} & k=0,1,2 \\
& \neg j_{tk} \Rightarrow j_{tk} & k=0,1,2 \\
& \neg r \Rightarrow r \\
& \neg r_{fk} \Rightarrow r_{fk} & k=0,\dots,7 \\
& \neg r_{tk} \Rightarrow r_{tk} & k=0,\dots,7 \\
& \neg w_a \Rightarrow w_a \\
& \neg w_{fk} \Rightarrow w_{fk} & k=0,\dots,7 \\
& \neg w_{tk} \Rightarrow w_{tk} & k=0,\dots,7 \\
& \neg s_{bfk} \Rightarrow s_{bfk} & k=0,\dots,7 \\
& \neg s_{btk} \Rightarrow s_{btk} & k=0,\dots,7
\end{aligned}$$

C.5 Deserializer

The *Deserializer* reconstructs a 32-bit output word from a sequence of incoming data quadruplets. In addition to performing the serialization reversal, the block must handle the case in which a multicast route terminates before the output word has been completely filled. Route termination is indicated by the reception of an *MSCODE*, which causes the remaining portion of the word to be filled with zeros before the completed word is forwarded.

C.5.1 CHP

The incoming quadruplets are sequentially stored in the next available positions of the 32-bit output word. Since an *MSCODE* may be received before all positions have been filled, each incoming value is checked before being stored. If an *MSCODE* is detected after at least one useful quadruplet has already been received, the remaining positions are left at zero and the partially filled word is completed by flushing the buffer.

The input channel *I* is acknowledged only once the flushing operation has completed. However, the *QuadRouter* sends the *MSCODE* independently of whether the current output word is empty or partially filled. Therefore, if an *MSCODE* is received while the buffer is still empty, no flushing operation is required and the input is acknowledged directly.

Finally, once the filling sequence reaches b_1 , the pending value on *I* can be read unconditionally. If this value is an *MSCODE*, storing it in b_1 is equivalent to leaving the corresponding portion of the output word empty, so no additional special case is required.

```

Deserializer(I?, O!)  $\equiv$ 
 $b_7 := 0, b_6 := 0, b_5 := 0, b_4 := 0, b_3 := 0, b_2 := 0, b_1 := 0, b_0 := 2;$ 
* [[ $\overline{I} \neq \text{MSCODE} \rightarrow I?b_7; [\overline{I} \neq \text{MSCODE} \rightarrow I?b_6 \parallel \overline{I} = \text{MSCODE} \rightarrow \text{skip}];$ 
 $[\overline{I} \neq \text{MSCODE} \rightarrow I?b_5 \parallel \overline{I} = \text{MSCODE} \rightarrow \text{skip}];$ 
 $[\overline{I} \neq \text{MSCODE} \rightarrow I?b_4 \parallel \overline{I} = \text{MSCODE} \rightarrow \text{skip}];$ 
 $[\overline{I} \neq \text{MSCODE} \rightarrow I?b_3 \parallel \overline{I} = \text{MSCODE} \rightarrow \text{skip}];$ 
 $[\overline{I} \neq \text{MSCODE} \rightarrow I?b_2 \parallel \overline{I} = \text{MSCODE} \rightarrow \text{skip}]; I?b_1; O!(b_7, b_6, b_5, b_4, b_3, b_2, b_1, b_0);$ 
 $b_7 := 0, b_6 := 0, b_5 := 0, b_4 := 0, b_3 := 0, b_2 := 0, b_1 := 0$ 
 $\parallel \overline{I} = \text{MSCODE} \rightarrow I$ 
]]

```

We now decompose this block into a set of smaller components. The core idea is to use a set of buffer-management blocks, called *QBuffers*, connected in a token ring to enforce the correct filling sequence. A *QBuffer* remains idle until it receives a communication on its left channel, *L*. When this happens, the *QBuffer* requests a new quadruplet of data. Once the data have been

received and the local buffer has been filled, the token is forwarded to the next block through the right channel, R , while the buffered value is made available on the output channel. To handle flushing, we introduce a block called *InputAnalyzer*. All incoming data are first sent to this module, which checks whether there is a request for data from the token ring and, if so, waits for an input flit and examines its value. If the received quadruplet corresponds to an *MSCODE*, the block enters a flushing mode in which it returns zeros whenever its output port requests new data. Once the buffer-filling sequence has completed, the R channel of $QB0$ is connected directly to the *InputAnalyzer*, which, if necessary, exits flushing mode and restarts the loop by sending a request to the left port of $QB7$. The outputs of the buffers are concatenated by a block called *Concat*. Finally, the input channel is distributed to all the modules through a *SplitTree*. The *SplitTree* is implemented as a tree of *Split4Bits* blocks, with one *Split4Bits* omitted at the last level. The rightmost O_1 channel at the second level is connected directly to $QB1$. The tree pulls data from the output of the *InputAnalyzer* toward the consumer currently requesting it.

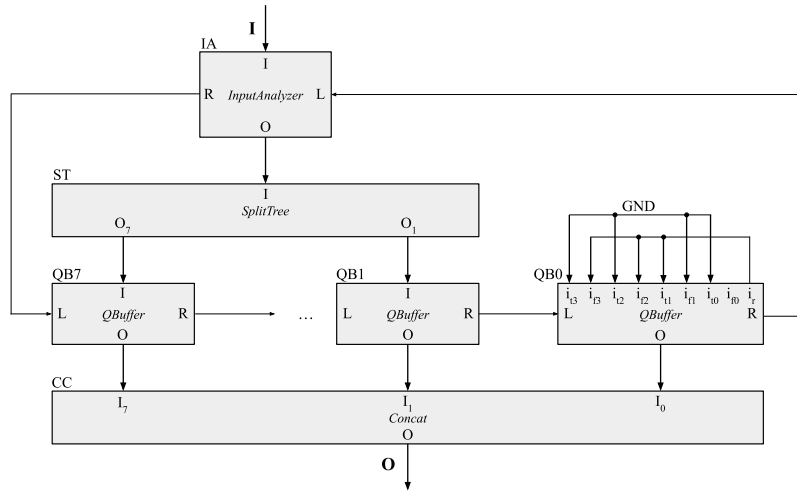


Figure C.6: Deserializer decomposition

QBuffer

The *QBuffer* waits for the token arriving on the left port, requests data to fill the buffer, and then propagates the token on R while placing the buffered value on the output. Note that the handshake on R must wait for $I?b$, whereas this is not required for the handshake on L . We therefore choose to complete the communication on L before receiving the data, in order to avoid deadlocks with the *InputAnalyzer*.

$$\begin{aligned} \text{QBuffer}(I?, O!, L, R) \equiv \\ * [[\bar{L} \rightarrow L; I?b; R; O!b]] \end{aligned}$$

Concat

The *Concat* block instead takes the outputs of the *QBuffers* as inputs and concatenates them into a single 32-bit output word.

$$\begin{aligned} \text{Concat}(I_7?, I_6?, I_5?, I_4?, I_3?, I_2?, I_1?, I_0?, O!) \equiv \\ * [O!(I_7?, I_6?, I_5?, I_4?, I_3?, I_2?, I_1?, I_0?)] \end{aligned}$$

InputAnalyzer

The *InputAnalyzer* checks whether the *SplitTree* is requesting data and then waits for an incoming flit. If the received input corresponds to the *MSCODE*, the module enters a flushing mode, represented by the state variable s being high. In this mode, for every request received on the output port, the module returns a 0 without acknowledging the input. To decide whether it should enter flushing mode, the module must know whether at least one useful quadruplet has already been sent. We therefore introduce an additional variable, v , which is asserted when the first flit enters an empty buffer. Once *QB0* has completed, a communication is received on L ; at that point, if the module was in flushing mode, it acknowledges I , resets s , and also resets v . Only after these operations have been completed does it send a request on channel R to restart the token ring. Finally, note that, in order to start the token ring, the block must perform a communication on R before entering the main loop.

$$\begin{aligned} \text{InputAnalyzer}(I?, O!, R, L) \equiv \\ v := \text{false}; s := \text{false}; \\ R; \\ * [[\bar{O} \rightarrow \\ \quad [\bar{I} \neq \text{MSCODE} \rightarrow O!I?, v := \text{true} \\ \quad \parallel \bar{I} = \text{MSCODE} \wedge v \rightarrow O!\text{MSCODE}, s := \text{true} \\ \quad \parallel \bar{I} = \text{MSCODE} \wedge \neg v \rightarrow I \\ \quad] \\ \quad \parallel \bar{L} \rightarrow [s \rightarrow I; s := \text{false} \parallel \neg s \rightarrow \text{skip}], v := \text{false}; L; R \\]] \end{aligned}$$

C.5.2 HSE

QBuffer

For the *QBuffer*, we begin by writing the HSE directly from the CHP, without any reshuffling. For the internal buffer, we use the previously defined *Reg4*.

To avoid a naming conflict between the register read port R and the token-ring channel R of the *QBuffer*, we locally rename the register write and read ports from W and R to P and Q , respectively. Therefore, P corresponds to the write port of the register, while Q corresponds to its read port. Moreover, since I is an active input channel for this module, the signal i_r must be asserted to request data from the input.

The auxiliary variables introduced below are f_0 for token receipt, f_1 for input capture, and f_2 for token forwarding/output service. They do not store the data value denoted by b in the CHP; that value is held by the internal *Reg4*.

$$\begin{aligned} \text{QBuffer} \equiv & \\ & * [[I_r]; l_a \uparrow; [\neg l_r]; l_a \downarrow; i_r \uparrow; \langle, k : 3..0 : [i_{tk} \rightarrow p_{tk} \uparrow \parallel i_{fk} \rightarrow p_{fk} \uparrow] \rangle; [p_a]; i_r \downarrow; \\ & \quad \langle, k : 3..0 : [\neg i_{tk} \rightarrow p_{tk} \downarrow], [\neg i_{fk} \rightarrow p_{fk} \downarrow] \rangle; [\neg p_a]; r_r \uparrow; [r_a]; r_r \downarrow; [\neg r_a]; q_r \uparrow; \\ & \quad \langle, k : 3..0 : [q_{tk} \rightarrow o_{tk} \uparrow \parallel q_{fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; \\ & \quad q_r \downarrow; \langle, k : 3..0 : [\neg q_{tk} \rightarrow o_{tk} \downarrow], [\neg q_{fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg o_a] \\ &] \end{aligned}$$

At this point, we have four handshakes executed in sequence, and after each of them the program returns to a state that would otherwise be indistinguishable from the others. Note that we cannot reshuffle L , because, in the case of *QB7*, the communication on L must be fully completed before new data are requested. Indeed, the source of L , namely the *InputAnalyzer*, cannot send any new output data until its communication on R has completed; otherwise, the system would deadlock. We therefore introduce state variables that are asserted between consecutive handshakes and keep track of the current state of the program. We denote them by f_0 , f_1 , and f_2 .

At this point, a further issue is that the module must also wait for the output sequence to complete. Since this is not guaranteed by the *InputAnalyzer* when it restarts the loop, we must ensure that i_r is asserted only when the buffer is known to be empty.

$$\begin{aligned} \text{QBuffer} \equiv & \\ & * [[I_r]; l_a \uparrow; [\neg l_r], f_0 \uparrow; l_a \downarrow; i_r \uparrow; \langle, k : 3..0 : [i_{tk} \rightarrow p_{tk} \uparrow \parallel i_{fk} \rightarrow p_{fk} \uparrow] \rangle; [p_a]; f_1 \uparrow, f_0 \downarrow; i_r \downarrow; \\ & \quad \langle, k : 3..0 : [\neg i_{tk} \rightarrow p_{tk} \downarrow], [\neg i_{fk} \rightarrow p_{fk} \downarrow] \rangle; [\neg p_a]; r_r \uparrow; [r_a]; f_1 \downarrow, f_2 \uparrow; r_r \downarrow; [\neg r_a]; q_r \uparrow; \\ & \quad \langle, k : 3..0 : [q_{tk} \rightarrow o_{tk} \uparrow \parallel q_{fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; f_2 \downarrow; q_r \downarrow; \\ & \quad \langle, k : 3..0 : [\neg q_{tk} \rightarrow o_{tk} \downarrow], [\neg q_{fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg o_a] \\ &] \end{aligned}$$

Concat

The HSE of the *Concat* block is straightforward, and its PRS will consist only of a collection of wires.

Concat $\equiv$

$$\begin{aligned}
 & * [\langle , j : 7..0 : \langle , k : 3..0 : [i_{jtk} \rightarrow o_{t(4j+k)} \uparrow \parallel i_{jfk} \rightarrow o_{f(4j+k)} \uparrow] \rangle ; [o_a]; i_{7a} \uparrow, \dots, i_{0a} \uparrow; \\
 & \quad \langle , j : 7..0 : \langle , k : 3..0 : [\neg i_{jtk} \rightarrow o_{t(4j+k)} \downarrow], [\neg i_{jfk} \rightarrow o_{f(4j+k)} \downarrow] \rangle ; [\neg o_a]; i_{7a} \downarrow, \dots, i_{0a} \downarrow \\
 &]
 \end{aligned}$$

InputAnalyzer

We begin by deriving the *InputAnalyzer* HSE from its CHP.

The main mode state is s : $s = 1$ records flushing. The variable v records whether the current output word is nonempty. The auxiliary variables introduced below are f_0 for stop-code detection, f_1 for normal-data forwarding, and f_2 for token-ring restart.

InputAnalyzer $\equiv$

$$\begin{aligned}
 & * [[o_r \rightarrow \\
 & \quad [i_{t3} \vee \dots \vee i_{t0} \rightarrow (\langle , k : 3..0 : [i_{tk} \rightarrow o_{tk} \uparrow \parallel i_{fk} \rightarrow o_{fk} \uparrow] \rangle ; [\neg o_r]; i_a \uparrow; \\
 & \quad \quad \langle , k : 3..0 : [\neg i_{tk} \rightarrow o_{tk} \downarrow], [\neg i_{fk} \rightarrow o_{fk} \downarrow] \rangle ; i_a \downarrow \\
 & \quad \quad), v \uparrow \\
 & \quad \parallel (i_{f3} \wedge \dots \wedge i_{f0}) \wedge v \rightarrow (o_{f3} \uparrow, \dots, o_{f0} \uparrow; [\neg o_r]; o_{f3} \downarrow, \dots, o_{f0} \downarrow), s \uparrow \\
 & \quad \parallel (i_{f3} \wedge \dots \wedge i_{f0}) \wedge \neg v \rightarrow i_a \uparrow; [\neg i_{f3} \wedge \dots \wedge \neg i_{f0}]; i_a \downarrow \\
 & \quad] \\
 & \parallel l_r \rightarrow v \downarrow, [s \rightarrow i_a \uparrow; [\neg i_{f3} \wedge \dots \wedge \neg i_{f0}]; i_a \downarrow; s \downarrow \parallel \neg s \rightarrow \text{skip}]; \\
 & \quad l_a \uparrow; [\neg l_r]; l_a \downarrow; r_r \uparrow; [r_a]; r_r \downarrow; [\neg r_a] \\
 &]]
 \end{aligned}$$

Although $v \uparrow$ can in principle occur in parallel with the other operations, we want v to change before the output rails return to spacer and the module can potentially receive another request for data from the output. We therefore move the assertion of v to immediately before the assertion of i_a . The same reasoning applies to the second branch of the o_r selection, where we move the assertion of s to immediately after $[\neg o_r]$.

InputAnalyzer $\equiv$

$$\begin{aligned}
 & * [[o_r \rightarrow \\
 & \quad [i_{t3} \vee \dots \vee i_{t0} \rightarrow \\
 & \quad \quad \langle , k : 3..0 : [i_{tk} \rightarrow o_{tk} \uparrow \parallel i_{fk} \rightarrow o_{fk} \uparrow] \rangle ; [\neg o_r]; v \uparrow; i_a \uparrow; \\
 & \quad \quad \langle , k : 3..0 : [\neg i_{tk} \rightarrow o_{tk} \downarrow], [\neg i_{fk} \rightarrow o_{fk} \downarrow] \rangle ; i_a \downarrow \\
 & \quad \parallel (i_{f3} \wedge \dots \wedge i_{f0}) \wedge v \rightarrow o_{f3} \uparrow, \dots, o_{f0} \uparrow; [\neg o_r]; s \uparrow; o_{f3} \downarrow, \dots, o_{f0} \downarrow \\
 & \quad \parallel (i_{f3} \wedge \dots \wedge i_{f0}) \wedge \neg v \rightarrow i_a \uparrow; [\neg i_{f3} \wedge \dots \wedge \neg i_{f0}]; i_a \downarrow \\
 & \quad] \\
 & \parallel l_r \rightarrow v \downarrow, [s \rightarrow i_a \uparrow; [\neg i_{f3} \wedge \dots \wedge \neg i_{f0}]; i_a \downarrow; s \downarrow \parallel \neg s \rightarrow \text{skip}]; \\
 & \quad l_a \uparrow; [\neg l_r]; l_a \downarrow; r_r \uparrow; [r_a]; r_r \downarrow; [\neg r_a] \\
 &]]
 \end{aligned}$$

To simplify the production rules for i_a , we introduce a state variable f_0 , which is asserted whenever the input is equal to the *MSCODE* and there is a pull request from the output, and it is deasserted once the input becomes invalid. To simplify the positive guards of the output rails, we introduce a state variable f_1 , which is asserted whenever the output port is requesting data and the input is not an *MSCODE*, and is deasserted when all the output rails have returned low. Finally, we introduce a state variable f_2 , which is asserted during the handshake on L , so that we can identify a unique state of the program in which to assert r_r .

```

InputAnalyzer  $\equiv$ 
* [[  $o_r \rightarrow$ 
    [  $i_{t3} \vee \dots \vee i_{t0} \rightarrow f_1 \uparrow; \langle , k : 3..0 : [i_{tk} \rightarrow o_{tk} \uparrow \parallel i_{fk} \rightarrow o_{fk} \uparrow] \rangle; [\neg o_r]; v \uparrow; i_a \uparrow;$ 
       $\langle , k : 3..0 : [\neg i_{tk} \rightarrow o_{tk} \downarrow], [\neg i_{fk} \rightarrow o_{fk} \downarrow] \rangle; f_1 \downarrow; i_a \downarrow$ 
       $\parallel (i_{f3} \wedge \dots \wedge i_{f0}) \wedge v \rightarrow f_0 \uparrow; o_{f3} \uparrow, \dots, o_{f0} \uparrow; [\neg o_r]; s \uparrow; o_{f3} \downarrow, \dots, o_{f0} \downarrow$ 
       $\parallel (i_{f3} \wedge \dots \wedge i_{f0}) \wedge \neg v \rightarrow f_0 \uparrow; i_a \uparrow; [\neg i_{f3} \wedge \dots \wedge \neg i_{f0}]; f_0 \downarrow; i_a \downarrow$ 
    ]
     $\parallel l_r \rightarrow v \downarrow; [s \rightarrow i_a \uparrow; [\neg i_{f3} \wedge \dots \wedge \neg i_{f0}]; f_0 \downarrow; i_a \downarrow; s \downarrow \parallel \neg s \rightarrow \text{skip}];$ 
       $f_2 \uparrow; l_a \uparrow; [\neg l_r]; l_a \downarrow; r_r \uparrow; [r_a], f_2 \downarrow; r_r \downarrow; [\neg r_a]$ 
  ]]

```

C.5.3 PRS

QBuffer

The state variable f_0 is asserted once l_a becomes high and is deasserted once p_a becomes high.

$$\begin{aligned} l_a &\rightarrow f_0 \uparrow \\ p_a \vee r &\rightarrow f_0 \downarrow \end{aligned}$$

The state variable f_1 is asserted once p_a becomes high and is deasserted once r_a becomes high.

$$\begin{aligned} p_a &\rightarrow f_1 \uparrow \\ r_a \vee r &\rightarrow f_1 \downarrow \end{aligned}$$

The state variable f_2 is asserted once r_a becomes high and is deasserted once o_a becomes high.

$$\begin{aligned} r_a &\rightarrow f_2 \uparrow \\ o_a \vee r &\rightarrow f_2 \downarrow \end{aligned}$$

To assert i_r , we require f_0 to be high and l_a to be low, whereas deassertion requires f_0 to be low and f_1 to be high.

$$\begin{aligned} f_0 \wedge \neg l_a &\rightarrow i_r \uparrow \\ (\neg f_0 \wedge f_1) \vee r &\rightarrow i_r \downarrow \end{aligned}$$

To assert l_a , we require l_r to be high and the buffer to be empty, which is true when both f_2 and o_a are low. The signal is deasserted once l_r has returned low while f_0 is high.

$$\begin{aligned} l_r \wedge \neg f_2 \wedge \neg o_a \wedge \neg r &\rightarrow l_a \uparrow \\ (\neg l_r \wedge f_0) \vee r &\rightarrow l_a \downarrow \end{aligned}$$

Likewise, q_r is asserted when f_2 is high and r_a is low, ensuring that the handshake on R has already completed. It is deasserted once f_2 returns low.

$$\begin{aligned} f_2 \wedge \neg r_a &\rightarrow q_r \uparrow \\ \neg f_2 &\rightarrow q_r \downarrow \end{aligned}$$

Similarly, r_r is asserted when f_1 is high and p_a is low, ensuring that the input data have already been captured. It is deasserted once f_1 becomes low and f_2 goes high.

$$\begin{aligned} f_1 \wedge \neg p_a &\rightarrow r_r \uparrow \\ (f_2 \wedge \neg f_1) \vee r &\rightarrow r_r \downarrow \end{aligned}$$

The input rails of the internal register are implemented as simple wires connected directly to the corresponding input rails of the module.

$$\begin{aligned} i_{fk} &\Rightarrow p_{fk} & k=0,\dots,3 \\ i_{tk} &\Rightarrow p_{tk} & k=0,\dots,3 \end{aligned}$$

Similarly, the output rails of the module are implemented as simple wires connected directly to the corresponding output rails of the register.

$$\begin{aligned} q_{fk} &\Rightarrow o_{fk} & k=0,\dots,3 \\ q_{tk} &\Rightarrow o_{tk} & k=0,\dots,3 \end{aligned}$$

¶ An isochronic fork on l_a constrains arrival at $G_{i_r}^+$. The two paths are

$$\begin{aligned} P_d : l_a \uparrow &\rightarrow G_{i_r}^+ \\ P_i : l_a \uparrow &\rightarrow f_0 \uparrow \rightarrow G_{i_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The token acknowledge must inhibit a data request before the token-received state rises.

¶ An isochronic fork on p_a constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : p_a \uparrow &\rightarrow G_{r_r}^+ \\ P_i : p_a \uparrow &\rightarrow f_1 \uparrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The register-write acknowledge must inhibit token forwarding before the captured-data state rises.

¶ An isochronic fork on r_a constrains arrival at $G_{q_r}^+$. The two paths are

$$\begin{aligned} P_d : r_a \uparrow &\rightarrow G_{q_r}^+ \\ P_i : r_a \uparrow &\rightarrow f_2 \uparrow \rightarrow G_{q_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The outgoing-token acknowledge must inhibit a register read before the output-phase state rises.

¶ An isochronic fork on o_a constrains arrival at $G_{l_a}^+$. The two paths are

$$\begin{aligned} P_d : o_a \uparrow \rightarrow G_{l_a}^+ \\ P_i : o_a \uparrow \rightarrow f_2 \downarrow \rightarrow G_{l_a}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit the next token acceptance before the occupied-buffer state is cleared.

Concat

Each input acknowledge is simply connected to the output acknowledge.

$$o_a \Rightarrow i_{ja} \quad j=0,\dots,7$$

Similarly, each output rail is directly connected to the corresponding input rail. Each input I_j contributes four consecutive bits to the 32-bit output word.

$$\begin{aligned} i_{jfk} &\Rightarrow o_{f(4j+k)} & j=0,\dots,7; k=0,\dots,3 \\ i_{jtk} &\Rightarrow o_{t(4j+k)} & j=0,\dots,7; k=0,\dots,3 \end{aligned}$$

InputAnalyzer

The state variable f_0 is asserted when all input bits correspond to the *MSCODE* and there is a pull request from the output. It is deasserted once all false input rails have returned low.

$$\begin{aligned} o_r \wedge i_{f3} \wedge i_{f2} \wedge i_{f1} \wedge i_{f0} &\rightarrow f_0 \uparrow \\ (\neg i_{f3} \wedge \neg i_{f2} \wedge \neg i_{f1} \wedge \neg i_{f0}) \vee r &\rightarrow f_0 \downarrow \end{aligned}$$

The state variable f_1 is asserted when the input is not an *MSCODE* and there is a request for data on the output. It is deasserted once all output rails have returned to spacer and i_a has already been asserted.

$$\begin{aligned} o_r \wedge (i_{t3} \vee i_{t2} \vee i_{t1} \vee i_{t0}) &\rightarrow f_1 \uparrow \\ (i_a \wedge \neg o_{t3} \wedge \neg o_{f3} \wedge \dots \wedge \neg o_{t0} \wedge \neg o_{f0}) \vee r &\rightarrow f_1 \downarrow \end{aligned}$$

The state variable f_2 is asserted only when l_r is high, v has already returned low, and s is low. It is deasserted when r_r becomes high.

$$\begin{aligned} l_r \wedge \neg v \wedge \neg s &\rightarrow f_2 \uparrow \\ r_r \vee r &\rightarrow f_2 \downarrow \end{aligned}$$

The state variable s is asserted only when f_0 is high, indicating that the input is the *MSCODE*, while v is high and o_r is low. It is deasserted in the l_r branch once both i_a and f_0 have returned low.

$$\begin{aligned} f_0 \wedge v \wedge \neg o_r &\rightarrow s \uparrow \\ (l_r \wedge \neg i_a \wedge \neg f_0) \vee r &\rightarrow s \downarrow \end{aligned}$$

The input acknowledge i_a is asserted in three cases: during the l_r branch when s , l_r , and f_0 are high; in the normal-data branch when v and f_1 are high and o_r has returned low; and when an *MSCODE* is received while the buffer is still empty, corresponding to $\neg v \wedge f_0$.

The condition $v \wedge f_1$ may become true before the output rails have been updated, since v may still be high from the previous state. We therefore additionally require $\neg o_r$. Similarly, $s \wedge l_r$ may remain true when i_a must be deasserted, so this term is additionally qualified by f_0 .

$$\begin{aligned} (s \wedge l_r \wedge f_0) \vee (v \wedge f_1 \wedge \neg o_r) \vee (\neg v \wedge f_0) &\rightarrow i_a \uparrow \\ \neg f_0 \wedge \neg f_1 &\rightarrow i_a \downarrow \end{aligned}$$

The state variable v is asserted only when o_r is low while f_1 is high. It is deasserted when a request is received on L or when the circuit is reset. Since f_1 is driven low only after the communication with the output has completed, l_r may become high while f_1 is still asserted. Therefore, the positive guard is strengthened by additionally requiring l_r to be low.

$$\begin{aligned} \neg l_r \wedge \neg o_r \wedge f_1 &\rightarrow v \uparrow \\ l_r \vee r &\rightarrow v \downarrow \end{aligned}$$

The acknowledge l_a is asserted when f_2 and l_r are both high. It is deasserted when l_r returns low. Since l_r becomes low while f_2 is still high, the positive guard must explicitly include l_r to avoid interference.

$$\begin{aligned} f_2 \wedge l_r &\rightarrow l_a \uparrow \\ \neg l_r \vee r &\rightarrow l_a \downarrow \end{aligned}$$

The request r_r is asserted when f_2 is high and both l_a and l_r are low. It is deasserted when r_a is high and f_2 has returned low. The token ring must also be initialized after reset, so r_r is additionally asserted by r .

$$\begin{aligned} (f_2 \wedge \neg l_a \wedge \neg l_r) \vee r &\rightarrow r_r \uparrow \\ r_a \wedge \neg f_2 &\rightarrow r_r \downarrow \end{aligned}$$

The true output rails are asserted when f_1 is high and the corresponding true input rail is high, and they return low when the corresponding input rail returns low. The false output rails follow the corresponding false input rails during normal operation, but can also be asserted while flushing by the condition $o_r \wedge v \wedge f_0$. During flushing, they are deasserted once s is asserted and o_r has returned low.

$$\begin{aligned} (f_1 \wedge i_{fk}) \vee (o_r \wedge v \wedge f_0) &\rightarrow o_{fk} \uparrow & k=0,\dots,3 \\ \neg i_{fk} \vee (s \wedge \neg o_r) &\rightarrow o_{fk} \downarrow & k=0,\dots,3 \\ f_1 \wedge i_{tk} &\rightarrow o_{tk} \uparrow & k=0,\dots,3 \\ \neg i_{tk} &\rightarrow o_{tk} \downarrow & k=0,\dots,3 \end{aligned}$$

In the fork paths below, $k \in \{0, \dots, 3\}$ and $\rho \in \{t, f\}$. The event $N_f(I)$ means that all four false input rails have returned low.

¶ An isochronic fork on f_1 constrains arrival at $G_{o_{\rho k}}^+$. The two paths are

$$\begin{aligned} P_d : f_1 \downarrow &\rightarrow G_{o_{\rho k}}^+ \\ P_i : f_1 \downarrow &\rightarrow i_a \downarrow \rightarrow i_{\rho k} \uparrow \rightarrow G_{o_{\rho k}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The normal-data branch must disable its output contribution before a new input arrives.

¶ An isochronic fork on i_a constrains arrival at $G_{f_1}^-$. The two paths are

$$\begin{aligned} P_d : i_a \downarrow \rightarrow G_{f_1}^- \\ P_i : i_a \downarrow \rightarrow i_{tk} \uparrow \rightarrow G_{f_1}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The normal-data clearing guard must be disabled before the next nonzero input can assert that branch.

The same fork also constrains arrival at G_s^- . The two paths are

$$\begin{aligned} P_d : i_a \uparrow \rightarrow G_s^- \\ P_i : i_a \uparrow \rightarrow N_f(I) \rightarrow f_0 \downarrow \rightarrow G_s^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The input acknowledge must inhibit flushing-state release before the stop-code input is withdrawn.

¶ An isochronic fork on l_a constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : l_a \uparrow \rightarrow G_{r_r}^+ \\ P_i : l_a \uparrow \rightarrow l_r \downarrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The token acknowledge must reach the restart guard before the requester withdraws the token request.

¶ An isochronic fork on i_{fk} constrains arrival at $G_{o_{fk}}^-$. The two paths are

$$\begin{aligned} P_d : i_{fk} \uparrow \rightarrow G_{o_{fk}}^- \\ P_i : i_{fk} \uparrow \rightarrow f_0 \uparrow \rightarrow G_{o_{fk}}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The zero-input rail must disable the output clearing guard before stop-code detection enables zero padding.

¶ An isochronic fork on l_r constrains arrival at $G_{f_2}^+$. The two paths are

$$\begin{aligned} P_d : l_r \downarrow \rightarrow G_{f_2}^+ \\ P_i : l_r \downarrow \rightarrow l_a \downarrow \rightarrow r_r \uparrow \rightarrow G_{f_2}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The restart-state assertion guard must observe token-request withdrawal before the restart request clears that state.

The same fork also constrains arrival at $G_{r_r}^+$. The two paths are

$$\begin{aligned} P_d : l_r \uparrow \rightarrow G_{r_r}^+ \\ P_i : l_r \uparrow \rightarrow f_2 \uparrow \rightarrow l_a \uparrow \rightarrow G_{r_r}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. A pending incoming token must inhibit the restart request before the restart state can enable it.

¶ An isochronic fork on o_r constrains arrival at G_v^+ . The two paths are

$$\begin{aligned} P_d : o_r \uparrow \rightarrow G_v^+ \\ P_i : o_r \uparrow \rightarrow f_1 \uparrow \rightarrow G_v^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output request must inhibit the nonempty-state update before the normal-data branch is enabled.

The same fork also constrains arrival at $G_{i_a}^+$. The two paths are

$$\begin{aligned} P_d : o_r \uparrow \rightarrow G_{i_a}^+ \\ P_i : o_r \uparrow \rightarrow f_1 \uparrow \rightarrow G_{i_a}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output request must inhibit input acknowledge before the normal-data branch is enabled.

The same fork also constrains arrival at G_s^+ . The two paths are

$$\begin{aligned} P_d : o_r \uparrow \rightarrow G_s^+ \\ P_i : o_r \uparrow \rightarrow f_0 \uparrow \rightarrow G_s^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output request must inhibit the flushing-state update before stop-code detection can qualify it.

C.5.4 Bubble Reshuffling

QBuffer

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned} \neg l_a &\rightarrow G_{i_r}^+ \\ \neg p_a &\rightarrow G_{r_r}^+ \\ \neg r_a &\rightarrow G_{q_r}^+ \\ \neg o_a &\rightarrow G_{l_a}^+ \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve $l_a, i_r, r_a, q_r, p_a, r_r, o_a$, together with the input and output rails, in their original sense. For the remaining internal state variables, we implement f_0, f_1, f_2 in complemented form. This directly provides the complemented state required by several subsequent guards without changing any fork-critical polarity. The resulting PRS is:

$$\begin{aligned}
& l_a \rightarrow \neg f_0 \downarrow \\
& p_a \vee r \rightarrow \neg f_0 \uparrow \\
& p_a \rightarrow \neg f_1 \downarrow \\
& r_a \vee r \rightarrow \neg f_1 \uparrow \\
& r_a \rightarrow \neg f_2 \downarrow \\
& o_a \vee r \rightarrow \neg f_2 \uparrow \\
& \neg f_0 \wedge \neg l_a \rightarrow i_r \uparrow \\
& (\neg f_0 \wedge \neg f_1) \vee r \rightarrow i_r \downarrow \\
& l_r \wedge \neg f_2 \wedge \neg o_a \wedge \neg r \rightarrow l_a \uparrow \\
& (\neg l_r \wedge \neg f_0) \vee r \rightarrow l_a \downarrow \\
& \neg f_2 \wedge \neg r_a \rightarrow q_r \uparrow \\
& \neg f_2 \rightarrow q_r \downarrow \\
& \neg f_1 \wedge \neg p_a \rightarrow r_r \uparrow \\
& (\neg f_2 \wedge \neg f_1) \vee r \rightarrow r_r \downarrow \\
& i_{fk} \Rightarrow p_{fk} \quad k=0,\dots,3 \\
& i_{tk} \Rightarrow p_{tk} \quad k=0,\dots,3 \\
& q_{fk} \Rightarrow o_{fk} \quad k=0,\dots,3 \\
& q_{tk} \Rightarrow o_{tk} \quad k=0,\dots,3
\end{aligned}$$

At this point, the fork-critical polarities are fixed. We therefore propagate the remaining bubbles through the guards, introducing explicit inverters wherever the opposite polarity of a polarity-fixed signal is required. The input and output rails already retain their original external polarity, so no restoration inverter is required for them.

$$\begin{aligned}
& l_a \rightarrow \neg f_0 \downarrow \\
& \neg p_a \vee \neg r \rightarrow \neg f_0 \uparrow \\
& p_a \rightarrow \neg f_1 \downarrow \\
& \neg r_a \vee \neg r \rightarrow \neg f_1 \uparrow \\
& r_a \rightarrow \neg f_2 \downarrow \\
& \neg o_a \vee \neg r \rightarrow \neg f_2 \uparrow \\
& \neg f_0 \wedge \neg l_a \rightarrow i_r \uparrow \\
& (\neg f_0 \wedge f_1) \vee r \rightarrow i_r \downarrow \\
& \neg l_r \wedge \neg f_2 \wedge \neg o_a \wedge \neg r \rightarrow l_a \uparrow \\
& (l_r \wedge f_0) \vee r \rightarrow l_a \downarrow \\
& \neg f_2 \wedge \neg r_a \rightarrow q_r \uparrow \\
& \neg f_2 \rightarrow q_r \downarrow \\
& \neg f_1 \wedge \neg p_a \rightarrow r_r \uparrow \\
& (f_2 \wedge \neg f_1) \vee r \rightarrow r_r \downarrow \\
& i_{fk} \Rightarrow p_{fk} \quad k=0,\dots,3 \\
& i_{tk} \Rightarrow p_{tk} \quad k=0,\dots,3 \\
& q_{fk} \Rightarrow o_{fk} \quad k=0,\dots,3 \\
& q_{tk} \Rightarrow o_{tk} \quad k=0,\dots,3 \\
& \neg f_0 \Rightarrow f_0 \\
& \neg f_1 \Rightarrow f_1 \\
& \neg f_2 \Rightarrow f_2 \\
& \neg l_r \Rightarrow l_r \\
& \neg p_a \Rightarrow \neg p_a \\
& \neg r \Rightarrow r \\
& \neg r_a \Rightarrow r_a \\
& \neg o_a \Rightarrow o_a
\end{aligned}$$

InputAnalyzer

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned}
 f_1 &\rightarrow G_{o_{fk}}^+ \\
 i_a &\rightarrow G_{f_1}^- \\
 \neg i_a &\rightarrow G_s^- \\
 \neg l_a &\rightarrow G_{r_r}^+ \\
 \neg i_{fk} &\rightarrow G_{o_{fk}}^- \\
 l_r &\rightarrow G_{f_2}^+ \\
 \neg l_r &\rightarrow G_{r_r}^+ \\
 \neg o_r &\rightarrow G_v^+ \\
 \neg o_r &\rightarrow G_{i_a}^+ \\
 \neg o_r &\rightarrow G_s^+
 \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve o_r , i_a , f_1 , f_0 , l_r , l_a , r_r , together with the input rails, in their original sense, while implementing s , f_2 and the output rails in complemented form. For the remaining internal signals, we preserve v and r_a in their original sense. The resulting PRS is:

$$\begin{aligned}
 o_r \wedge (i_{f3} \wedge i_{f2} \wedge i_{f1} \wedge i_{f0}) &\rightarrow f_0 \uparrow \\
 (\neg i_{f3} \wedge \neg i_{f2} \wedge \neg i_{f1} \wedge \neg i_{f0}) \vee r &\rightarrow f_0 \downarrow \\
 o_r \wedge (i_{t3} \vee i_{t2} \vee i_{t1} \vee i_{t0}) &\rightarrow f_1 \uparrow \\
 i_a \wedge (\neg o_{t3} \wedge \neg o_{f3} \wedge \dots \wedge \neg o_{t0} \wedge \neg o_{f0}) \vee r &\rightarrow f_1 \downarrow \\
 l_r \wedge \neg v \wedge s &\rightarrow f_2 \downarrow \\
 r_r \vee r &\rightarrow f_2 \uparrow \\
 f_0 \wedge v \wedge \neg o_r &\rightarrow s \downarrow \\
 (l_r \wedge \neg i_a \wedge \neg f_0) \vee r &\rightarrow s \uparrow \\
 (\neg s \wedge l_r \wedge f_0) \vee (v \wedge f_1 \wedge \neg o_r) \vee (\neg v \wedge f_0) &\rightarrow i_a \uparrow \\
 \neg f_0 \wedge \neg f_1 &\rightarrow i_a \downarrow \\
 \neg l_r \wedge \neg o_r \wedge f_1 &\rightarrow v \uparrow \\
 l_r \vee r &\rightarrow v \downarrow \\
 \neg f_2 \wedge l_r &\rightarrow l_a \uparrow \\
 \neg l_r \vee r &\rightarrow l_a \downarrow \\
 (\neg f_2 \wedge \neg l_a \wedge \neg l_r) \vee r &\rightarrow r_r \uparrow \\
 r_a \wedge \neg f_2 &\rightarrow r_r \downarrow \\
 (f_1 \wedge i_{fk}) \vee (o_r \wedge v \wedge f_0) &\rightarrow o_{fk} \downarrow & k=0,\dots,3 \\
 \neg i_{fk} \vee (\neg s \wedge \neg o_r) &\rightarrow o_{fk} \uparrow & k=0,\dots,3 \\
 f_1 \wedge i_{tk} &\rightarrow o_{tk} \downarrow & k=0,\dots,3 \\
 \neg i_{tk} &\rightarrow o_{tk} \uparrow & k=0,\dots,3
 \end{aligned}$$

At this point, the fork-critical polarities are fixed. We therefore propagate the remaining bubbles through the guards, introducing explicit inverters wherever the opposite polarity of a polarity-fixed signal is required. The output rails are restored to their original polarity before leaving the module.

$$\begin{aligned}
& \neg o_r \wedge (\neg i_{f3} \wedge \neg i_{f2} \wedge \neg i_{f1} \wedge \neg i_{f0}) \rightarrow f_0 \uparrow \\
& (i_{f3} \wedge i_{f2} \wedge i_{f1} \wedge i_{f0}) \vee r \rightarrow f_0 \downarrow \\
& \neg o_r \wedge (\neg i_{t3} \vee \neg i_{t2} \vee \neg i_{t1} \vee \neg i_{t0}) \rightarrow f_1 \uparrow \\
& i_a \wedge (o_{t3} \wedge o_{f3} \wedge \dots \wedge o_{t0} \wedge o_{f0}) \vee r \rightarrow f_1 \downarrow \\
& l_r \wedge v \wedge s \rightarrow f_2 \downarrow \\
& \neg r_r \vee \neg r \rightarrow f_2 \uparrow \\
& f_0 \wedge v \wedge o_r \rightarrow s \downarrow \\
& (\neg l_r \wedge \neg i_a \wedge \neg f_0) \vee \neg r \rightarrow s \uparrow \\
& (\neg s \wedge \neg l_r \wedge \neg f_0) \vee (\neg v \wedge \neg f_1 \wedge \neg o_r) \vee (\neg v \wedge \neg f_0) \rightarrow i_a \uparrow \\
& f_0 \wedge f_1 \rightarrow i_a \downarrow \\
& \neg l_r \wedge \neg o_r \wedge \neg f_1 \rightarrow v \uparrow \\
& l_r \vee r \rightarrow v \downarrow \\
& \neg f_2 \wedge \neg l_r \rightarrow l_a \uparrow \\
& l_r \vee r \rightarrow l_a \downarrow \\
& (\neg f_2 \wedge \neg l_a \wedge \neg l_r) \vee \neg r \rightarrow r_r \uparrow \\
& r_a \wedge f_2 \rightarrow r_r \downarrow \\
& (f_1 \wedge i_{fk}) \vee (o_r \wedge v \wedge f_0) \rightarrow o_{fk} \downarrow \quad k=0,\dots,3 \\
& \neg i_{fk} \vee (\neg s \wedge \neg o_r) \rightarrow o_{fk} \uparrow \quad k=0,\dots,3 \\
& f_1 \wedge i_{tk} \rightarrow o_{tk} \downarrow \quad k=0,\dots,3 \\
& \neg i_{tk} \rightarrow o_{tk} \uparrow \quad k=0,\dots,3 \\
& \neg f_0 \Rightarrow f_0 \\
& \neg f_1 \Rightarrow f_1 \\
& \neg v \Rightarrow v \\
& \neg r_r \Rightarrow r_r \\
& \neg i_{fk} \Rightarrow i_{fk} \quad k=0,\dots,3 \\
& \neg i_{tk} \Rightarrow i_{tk} \quad k=0,\dots,3 \\
& \neg l_r \Rightarrow l_r \\
& \neg r \Rightarrow r \\
& \neg o_{fk} \Rightarrow o_{fk} \quad k=0,\dots,3 \\
& \neg o_r \Rightarrow o_r \\
& \neg o_{tk} \Rightarrow o_{tk} \quad k=0,\dots,3
\end{aligned}$$

C.6 PayloadDispatch

The *InputRouter* takes all the flits entering the *MulticastUnit* and routes them either to the parsing datapath or to the common-payload datapath. A multicast transmission consists of a sequence of multicast route flits followed by the common payload. The common payload is directed to the *PayloadDispatch*, which copies each input flit to all active children. Before this process starts, however, the *QuadRouter* sends the very first nibble of the multicast route to the *PayloadDispatch*. This nibble determines which children will receive the data once the multicast route flits have been processed and the *InputRouter* starts directing the common payload to the *PayloadDispatch*.

C.6.1 CHP

$$\begin{aligned}
 \text{PayloadDispatch}(I?, C_h?, O_0!, O_1!, O_2!, O_3!) \equiv & \\
 * [(C_h?c, b := 0); & \\
 * [\neg b.\text{tail} \rightarrow I?b; & \\
 ([c.\text{msb} \rightarrow O_0!b \parallel \neg c.\text{msb} \rightarrow \text{skip}], [c.(\text{msb} - 1) \rightarrow O_1!b \parallel \neg c.(\text{msb} - 1) \rightarrow \text{skip}], & \\
 [c.(\text{msb} - 2) \rightarrow O_2!b \parallel \neg c.(\text{msb} - 2) \rightarrow \text{skip}], & \\
 [c.(\text{msb} - 3) \rightarrow O_3!b \parallel \neg c.(\text{msb} - 3) \rightarrow \text{skip}] & \\
)] & \\
] &
 \end{aligned}$$

To simplify the HSE implementation, we restructure the process as follows.

$$\begin{aligned}
 \text{PayloadDispatch}(I?, C_h?, O_0!, O_1!, O_2!, O_3!) \equiv & \\
 (c_0 := \text{false}, c_1 := \text{false}, c_2 := \text{false}, c_3 := \text{false}); & \\
 (* [[\overline{C_h} \rightarrow c_0 := \overline{C_h}?.\text{msb} \parallel \bar{I} \wedge c_0 \rightarrow w_0 := \neg \bar{I}?.\text{tail}; O_0!\bar{I}?; c_0 := w_0]] \parallel & \\
 * [[\overline{C_h} \rightarrow c_1 := \overline{C_h}?.(\text{msb} - 1) \parallel \bar{I} \wedge c_1 \rightarrow w_1 := \neg \bar{I}?.\text{tail}; O_1!\bar{I}?; c_1 := w_1]] \parallel & \\
 * [[\overline{C_h} \rightarrow c_2 := \overline{C_h}?.(\text{msb} - 2) \parallel \bar{I} \wedge c_2 \rightarrow w_2 := \neg \bar{I}?.\text{tail}; O_2!\bar{I}?; c_2 := w_2]] \parallel & \\
 * [[\overline{C_h} \rightarrow c_3 := \overline{C_h}?.(\text{msb} - 3) \parallel \bar{I} \wedge c_3 \rightarrow w_3 := \neg \bar{I}?.\text{tail}; O_3!\bar{I}?; c_3 := w_3]] & \\
) &
 \end{aligned}$$

We now only need to implement a single block, which we call *ChildDispatch*. Notice that the C_h channel now carries only one bit of data.

$$\begin{aligned}
 \text{ChildDispatch}(I?, C_h?, O) \equiv & \\
 c := \text{false}; & \\
 * [[\overline{C_h} \rightarrow C_h?c \parallel \bar{I} \wedge c \rightarrow w := \neg \bar{I}?.\text{tail}; O!\bar{I}?; c := w]] &
 \end{aligned}$$

However, we are still left with a problem: when combining the acknowledge of the input signals, not all children necessarily receive data. Some children may remain inactive throughout the entire multicast transmission, and therefore their output acknowledge will never be asserted. To address this issue, we add another branch to the CHP that simply acknowledges the input transaction whenever the child is inactive.

$$\begin{aligned}
 \text{ChildDispatch}(I?, C_h?, O) \equiv & \\
 c := \text{false}; & \\
 * [[\overline{C_h} \rightarrow C_h?c \parallel \bar{I} \wedge c \rightarrow w := \neg \bar{I}?.\text{tail}; O!\bar{I}?; c := w \parallel \bar{I} \wedge \neg c \rightarrow I]] &
 \end{aligned}$$

C.6.2 HSE

Within each *ChildDispatch*, c is the stored child-enable bit and w_0, w_1 encode its next value. The auxiliary variables introduced below are f_0 for the inactive-child branch and f_1 for completion of input return to spacer.

ChildDispatch $\equiv$

$$\begin{aligned}
 & * [[c_{ht} \vee c_{hf} \rightarrow [c_{ht} \rightarrow c \uparrow \parallel c_{hf} \rightarrow c \downarrow]; c_{ha} \uparrow; [\neg c_{ht} \wedge \neg c_{hf}]; c_{ha} \downarrow \\
 & \quad \parallel (i_{t31} \vee i_{f31}) \wedge c \rightarrow \\
 & \quad [i_{f0} \rightarrow w_1 \uparrow \parallel i_{t0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{tk} \uparrow \parallel i_{fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; i_a \uparrow; \\
 & \quad \langle , k : 31..0 : [\neg i_{tk} \rightarrow o_{tk} \downarrow], [\neg i_{fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg o_a]; i_a \downarrow; [w_0 \rightarrow c \downarrow; w_0 \downarrow \parallel w_1 \rightarrow c \uparrow; w_1 \downarrow] \\
 & \quad \parallel (i_{t31} \vee i_{f31}) \wedge \neg c \rightarrow i_a \uparrow; [\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; i_a \downarrow \\
 &]]
 \end{aligned}$$

We can reshuffle the program by moving the update of c to a point characterized by a unique program state, immediately before $i_a \downarrow$. In the third branch, i_a must wait until all input rails have returned to spacer. We therefore introduce the state variable f_1 to explicitly identify the completion of this phase.

ChildDispatch $\equiv$

$$\begin{aligned}
 & * [[c_{ht} \vee c_{hf} \rightarrow [c_{ht} \rightarrow c \uparrow \parallel c_{hf} \rightarrow c \downarrow]; c_{ha} \uparrow; [\neg c_{ht} \wedge \neg c_{hf}]; c_{ha} \downarrow \\
 & \quad \parallel (i_{t31} \vee i_{f31}) \wedge c \rightarrow \\
 & \quad [i_{f0} \rightarrow w_1 \uparrow \parallel i_{t0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{tk} \uparrow \parallel i_{fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; i_a \uparrow; \\
 & \quad \langle , k : 31..0 : [\neg i_{tk} \rightarrow o_{tk} \downarrow], [\neg i_{fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg o_a]; [w_0 \rightarrow c \downarrow; w_0 \downarrow \parallel w_1 \rightarrow c \uparrow; w_1 \downarrow]; i_a \downarrow \\
 & \quad \parallel (i_{t31} \vee i_{f31}) \wedge \neg c \rightarrow i_a \uparrow; [\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; f_1 \uparrow; i_a \downarrow; f_1 \downarrow \\
 &]]
 \end{aligned}$$

However, the state in which f_1 is asserted in the third branch may overlap with the state reached near the end of the second branch after c has been lowered. We therefore introduce a new state variable, f_0 , which is asserted at the beginning of the third branch and uniquely identifies its execution.

ChildDispatch $\equiv$

$$\begin{aligned}
 & * [[c_{ht} \vee c_{hf} \rightarrow [c_{ht} \rightarrow c \uparrow \parallel c_{hf} \rightarrow c \downarrow]; c_{ha} \uparrow; [\neg c_{ht} \wedge \neg c_{hf}]; c_{ha} \downarrow \\
 & \quad \parallel (i_{t31} \vee i_{f31}) \wedge c \rightarrow \\
 & \quad [i_{f0} \rightarrow w_1 \uparrow \parallel i_{t0} \rightarrow w_0 \uparrow], \langle , k : 31..0 : [i_{tk} \rightarrow o_{tk} \uparrow \parallel i_{fk} \rightarrow o_{fk} \uparrow] \rangle; [o_a]; i_a \uparrow; \\
 & \quad \langle , k : 31..0 : [\neg i_{tk} \rightarrow o_{tk} \downarrow], [\neg i_{fk} \rightarrow o_{fk} \downarrow] \rangle; [\neg o_a]; [w_0 \rightarrow c \downarrow; w_0 \downarrow \parallel w_1 \rightarrow c \uparrow; w_1 \downarrow]; i_a \downarrow \\
 & \quad \parallel (i_{t31} \vee i_{f31}) \wedge \neg c \rightarrow f_0 \uparrow; i_a \uparrow; [\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}]; f_1 \uparrow; f_0 \downarrow; i_a \downarrow; f_1 \downarrow \\
 &]]
 \end{aligned}$$

C.6.3 PRS

f_0 is asserted when there is a pending input and c is low, and deasserted when f_1 becomes high. However, if two inputs arrive consecutively while c is low, it is possible for the new input to arrive before f_1 has returned low. In that case, the positive and negative guards could interfere. We therefore strengthen the positive guard with $\neg f_1$.

$$\begin{aligned}
 & \neg f_1 \wedge \neg c \wedge (i_{t31} \vee i_{f31}) \rightarrow f_0 \uparrow \\
 & f_1 \vee r \rightarrow f_0 \downarrow
 \end{aligned}$$

f_1 is asserted when all input rails have returned to spacer while both f_0 and i_a are high. It is deasserted once i_a returns low.

$$\begin{aligned} f_0 \wedge i_a \wedge (\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}) &\rightarrow f_1 \uparrow \\ \neg i_a \vee r &\rightarrow f_1 \downarrow \end{aligned}$$

i_a is asserted in the second branch when o_a is high, and in the third branch when f_0 is high. Conversely, it is lowered in the second branch when o_a , w_0 , and w_1 are low, and in the third branch once f_0 has returned low while f_1 is high. The first condition for lowering i_a is also true when the third branch begins, so we strengthen it by requiring $\neg f_0$.

$$\begin{aligned} f_0 \vee o_a &\rightarrow i_a \uparrow \\ (\neg f_0 \wedge (f_1 \vee ((\neg w_0 \wedge \neg w_1) \wedge \neg o_a))) \vee r &\rightarrow i_a \downarrow \end{aligned}$$

c is asserted only by c_{ht} in the first branch, since in the second branch it is already high when it is updated. Conversely, it is deasserted only in the second branch, when w_0 is high, i_a is high, and o_a is low. In the first branch, c is low by default whenever c_{hf} is received, since this branch is entered from the idle state.

$$\begin{aligned} c_{ht} &\rightarrow c \uparrow \\ (w_0 \wedge i_a \wedge \neg o_a) \vee r &\rightarrow c \downarrow \end{aligned}$$

Both w_0 and w_1 are asserted by the input data whenever c is high, and lowered after c has captured their value, i_a is high, and o_a is low.

$$\begin{aligned} i_{t0} \wedge c &\rightarrow w_0 \uparrow \\ (\neg c \wedge i_a \wedge \neg o_a) \vee r &\rightarrow w_0 \downarrow \\ i_{f0} \wedge c &\rightarrow w_1 \uparrow \\ (c \wedge i_a \wedge \neg o_a) \vee r &\rightarrow w_1 \downarrow \end{aligned}$$

c_{ha} is asserted only once the value of c has been captured correctly, and deasserted once the data on C_h becomes invalid.

$$\begin{aligned} (c_{ht} \wedge c) \vee (c_{hf} \wedge \neg c) &\rightarrow c_{ha} \uparrow \\ \neg c_{ht} \wedge \neg c_{hf} &\rightarrow c_{ha} \downarrow \end{aligned}$$

The output rails follow the corresponding input rails only while c is high.

$$\begin{aligned} c \wedge i_{fk} &\rightarrow o_{fk} \uparrow & k=0,\dots,31 \\ \neg i_{fk} &\rightarrow o_{fk} \downarrow & k=0,\dots,31 \\ c \wedge i_{tk} &\rightarrow o_{tk} \uparrow & k=0,\dots,31 \\ \neg i_{tk} &\rightarrow o_{tk} \downarrow & k=0,\dots,31 \end{aligned}$$

The following constraints apply within one *ChildDispatch* instance. Its acknowledge signals i_a and o_a are therefore scalar. The index $\ell \in \{0,1\}$ identifies the two next-enable rails, not the four child instances.

¶ An isochronic fork on i_a constrains arrival at G_c^- . The two paths are

$$\begin{aligned} P_d : i_a \downarrow &\rightarrow G_c^- \\ P_i : i_a \downarrow \rightarrow i_{t0} \uparrow \rightarrow w_0 \uparrow &\rightarrow G_c^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The child-enable clearing guard must stop using the old input acknowledge before a new tail is captured.

The same fork also constrains arrival at $G_{w_\ell}^-$. The two paths are

$$\begin{aligned} P_d &: i_a \downarrow \rightarrow G_{w_\ell}^- \\ P_i &: i_a \downarrow \rightarrow i_{\rho 0} \uparrow \rightarrow G_{w_\ell}^+ \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The next-state clearing guard must be disabled before the corresponding new input rail can assert the state. Here $(\rho, \ell) \in \{(t, 0), (f, 1)\}$.

¶ An isochronic fork on o_a constrains arrival at $G_{w_\ell}^-$. The two paths are

$$\begin{aligned} P_d &: o_a \uparrow \rightarrow G_{w_\ell}^- \\ P_i &: o_a \uparrow \rightarrow i_a \uparrow \rightarrow G_{w_\ell}^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit next-state clearing before the input acknowledge rises.

The same fork also constrains arrival at G_c^- . The two paths are

$$\begin{aligned} P_d &: o_a \uparrow \rightarrow G_c^- \\ P_i &: o_a \uparrow \rightarrow i_a \uparrow \rightarrow G_c^- \end{aligned}$$

We require $\tau(P_d) < \tau(P_i)$. The output acknowledge must inhibit child deactivation before the input acknowledge rises.

C.6.4 Bubble Reshuffling

We first identify the non-acknowledged branches of all the isochronic forks.

$$\begin{aligned} i_a &\rightarrow G_c^- \\ i_a &\rightarrow G_{w_\ell}^- \\ \neg o_a &\rightarrow G_{w_\ell}^- \\ \neg o_a &\rightarrow G_c^- \end{aligned}$$

To keep every non-acknowledged branch inverter-free, we preserve w_0, w_1, c, o_a, i_a , as well as the input rails, in their original sense, while implementing the output rails in complemented form. For the remaining signals, we preserve f_1 in its original sense, while implementing f_0 and c_{ha} in complemented form. Complementing f_0 avoids requiring the complemented input rails in its positive guard, while complementing c_{ha} directly matches the polarities required by its two guards. Since c_{ha} is externally visible, its original polarity is restored before it leaves the module. With these assignments applied, the affected PRS becomes

$$\begin{aligned}
& \neg f_1 \wedge \neg c \wedge (i_{t31} \vee i_{f31}) \rightarrow \neg f_0 \downarrow \\
& f_1 \vee r \rightarrow \neg f_0 \uparrow \\
& \neg f_0 \wedge i_a \wedge (\neg i_{t31} \wedge \neg i_{f31} \wedge \dots \wedge \neg i_{t0} \wedge \neg i_{f0}) \rightarrow f_1 \uparrow \\
& \neg i_a \vee r \rightarrow f_1 \downarrow \\
& \neg f_0 \vee o_a \rightarrow i_a \uparrow \\
& (\neg f_0 \wedge (f_1 \vee (\neg w_0 \wedge \neg w_1 \wedge \neg o_a))) \vee r \rightarrow i_a \downarrow \\
& c_{ht} \rightarrow c \uparrow \\
& (w_0 \wedge i_a \wedge \neg o_a) \vee r \rightarrow c \downarrow \\
& i_{t0} \wedge c \rightarrow w_0 \uparrow \\
& (\neg c \wedge i_a \wedge \neg o_a) \vee r \rightarrow w_0 \downarrow \\
& i_{f0} \wedge c \rightarrow w_1 \uparrow \\
& (c \wedge i_a \wedge \neg o_a) \vee r \rightarrow w_1 \downarrow \\
& (c_{ht} \wedge c) \vee (c_{hf} \wedge \neg c) \rightarrow \neg c_{ha} \downarrow \\
& \neg c_{ht} \wedge \neg c_{hf} \rightarrow \neg c_{ha} \uparrow \\
& c \wedge i_{fk} \rightarrow \neg o_{fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow \neg o_{fk} \uparrow \quad k=0,\dots,31 \\
& c \wedge i_{tk} \rightarrow \neg o_{tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow \neg o_{tk} \uparrow \quad k=0,\dots,31
\end{aligned}$$

At this point, the polarities of the signals on the right-hand side are fixed. We therefore only need to adjust the guards, introducing explicit inverters whenever the opposite polarity of a polarity-fixed signal is required. Inverters are also added to c_{ha} and to each output rail to restore their original external polarities. Finally, we decompose the input-completion condition used by f_1 with a completion tree to reduce the capacitive load. The root of this tree is denoted by z .

$$\begin{aligned}
& ((i_{t(4k+3)} \vee i_{f(4k+3)}) \wedge \dots \wedge (i_{t4k} \vee i_{f4k})) \rightarrow \neg m_k \downarrow & k=0,\dots,7 \\
& (\neg i_{t(4k+3)} \wedge \neg i_{f(4k+3)} \wedge \dots \wedge \neg i_{t4k} \wedge \neg i_{f4k}) \rightarrow \neg m_k \uparrow & k=0,\dots,7 \\
& \neg m_{2k+1} \wedge \neg m_{2k} \rightarrow d_k \uparrow & k=0,\dots,3 \\
& m_{2k+1} \wedge m_{2k} \rightarrow d_k \downarrow & k=0,\dots,3 \\
& d_{2k+1} \wedge d_{2k} \rightarrow e_k \downarrow & k=0,1 \\
& \neg d_{2k+1} \wedge \neg d_{2k} \rightarrow e_k \uparrow & k=0,1 \\
& \neg e_1 \wedge \neg e_0 \rightarrow z \uparrow \\
& e_1 \wedge e_0 \rightarrow z \downarrow
\end{aligned}$$

$$\begin{aligned}
& \neg f_1 \wedge \neg c \wedge (i_{t31} \vee i_{f31}) \rightarrow \neg f_0 \downarrow \\
& \neg \neg f_1 \vee \neg \neg r \rightarrow \neg f_0 \uparrow \\
& \neg \neg f_0 \wedge \neg \neg i_a \wedge \neg \neg z \rightarrow f_1 \uparrow \\
& i_a \vee r \rightarrow f_1 \downarrow \\
& \neg \neg f_0 \vee \neg \neg o_a \rightarrow i_a \uparrow \\
& (\neg f_0 \wedge (f_1 \vee (\neg w_0 \wedge \neg w_1 \wedge \neg o_a))) \vee r \rightarrow i_a \downarrow \\
& \neg c_{ht} \rightarrow c \uparrow \\
& (w_0 \wedge i_a \wedge \neg o_a) \vee r \rightarrow c \downarrow \\
& \neg i_{t0} \wedge \neg \neg c \rightarrow w_0 \uparrow \\
& (\neg c \wedge i_a \wedge \neg o_a) \vee r \rightarrow w_0 \downarrow \\
& \neg i_{f0} \wedge \neg \neg c \rightarrow w_1 \uparrow \\
& (c \wedge i_a \wedge \neg o_a) \vee r \rightarrow w_1 \downarrow \\
& (c_{ht} \wedge c) \vee (c_{hf} \wedge \neg c) \rightarrow \neg c_{ha} \downarrow \\
& \neg c_{ht} \wedge \neg c_{hf} \rightarrow \neg c_{ha} \uparrow \\
& c \wedge i_{fk} \rightarrow \neg o_{fk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{fk} \rightarrow \neg o_{fk} \uparrow \quad k=0,\dots,31 \\
& c \wedge i_{tk} \rightarrow \neg o_{tk} \downarrow \quad k=0,\dots,31 \\
& \neg i_{tk} \rightarrow \neg o_{tk} \uparrow \quad k=0,\dots,31 \\
& \neg f_1 \Rightarrow \neg f_1 \\
& \neg i_a \Rightarrow \neg i_a \\
& \neg c \Rightarrow \neg c \\
& \neg w_0 \Rightarrow \neg w_0 \\
& \neg w_1 \Rightarrow \neg w_1 \\
& \neg \neg c_{ha} \Rightarrow c_{ha} \\
& \neg c_{ht} \Rightarrow \neg c_{ht} \\
& \neg i_{f0} \Rightarrow \neg i_{f0} \\
& \neg i_{t0} \Rightarrow \neg i_{t0} \\
& \neg r \Rightarrow \neg r \\
& \neg o_a \Rightarrow \neg o_a \\
& \neg \neg o_{fk} \Rightarrow o_{fk} \quad k=0,\dots,31 \\
& \neg \neg o_{tk} \Rightarrow o_{tk} \quad k=0,\dots,31
\end{aligned}$$

Appendix D

Router Characterization Lookup Tables

This appendix reports the complete transistor-level characterization data used to parameterize the discrete-event network simulator. Unless otherwise stated, L_f denotes the forward latency, II the initiation interval, and E the dynamic energy normalized per input bit. All timing values are reported in nanoseconds and all energy values in fJ/bit.

The initiation interval represents the minimum interval between successive operations of the same type and is therefore used as the local cycle-time constraint in the network model.

D.1 Unicast Forwarding

The unicast characterization is divided into forward latency, initiation interval, and dynamic energy to keep the complete path-dependent lookup tables readable. Header, intermediate data, and tail flits are reported separately. A direct $P^i \rightarrow P^o$ path is not supported by the router architecture.

D.1.1 Forward Latency

Table D.1: Forward latency L_f for all supported unicast paths. All values are in ns.

Path	Header	Data	Tail
$C_0^i \rightarrow C_0^o$	2.360	0.824	0.835
$C_0^i \rightarrow C_1^o$	2.296	0.823	0.837
$C_0^i \rightarrow C_2^o$	2.324	0.824	0.837
$C_0^i \rightarrow C_3^o$	2.291	0.823	0.836
$C_0^i \rightarrow P^o$	1.737	0.533	0.539
$C_1^i \rightarrow C_0^o$	2.611	0.825	0.835
$C_1^i \rightarrow C_1^o$	2.554	0.823	0.837
$C_1^i \rightarrow C_2^o$	2.583	0.823	0.837
$C_1^i \rightarrow C_3^o$	2.549	0.824	0.838
$C_1^i \rightarrow P^o$	1.842	0.531	0.537
$C_2^i \rightarrow C_0^o$	2.611	0.823	0.836
$C_2^i \rightarrow C_1^o$	2.550	0.822	0.838
$C_2^i \rightarrow C_2^o$	2.584	0.824	0.836
$C_2^i \rightarrow C_3^o$	2.549	0.823	0.838
$C_2^i \rightarrow P^o$	1.845	0.530	0.537
$C_3^i \rightarrow C_0^o$	2.586	0.822	0.834
$C_3^i \rightarrow C_1^o$	2.526	0.823	0.836
$C_3^i \rightarrow C_2^o$	2.562	0.824	0.835
$C_3^i \rightarrow C_3^o$	2.526	0.822	0.836
$C_3^i \rightarrow P^o$	1.818	0.530	0.537
$P^i \rightarrow C_0^o$	2.324	0.791	0.795
$P^i \rightarrow C_1^o$	2.265	0.789	0.795
$P^i \rightarrow C_2^o$	2.317	0.790	0.799
$P^i \rightarrow C_3^o$	2.278	0.790	0.798

D.1.2 Initiation Interval

Table D.2: Initiation interval II for all supported unicast paths. All values are in ns.

Path	Header	Data	Tail
$C_0^i \rightarrow C_0^o$	3.510	3.246	3.164
$C_0^i \rightarrow C_1^o$	3.508	3.243	3.163
$C_0^i \rightarrow C_2^o$	3.512	3.247	3.164
$C_0^i \rightarrow C_3^o$	3.504	3.245	3.160
$C_0^i \rightarrow P^o$	3.500	3.113	3.159
$C_1^i \rightarrow C_0^o$	3.507	3.235	3.157
$C_1^i \rightarrow C_1^o$	3.507	3.234	3.157
$C_1^i \rightarrow C_2^o$	3.505	3.237	3.164
$C_1^i \rightarrow C_3^o$	3.507	3.231	3.160
$C_1^i \rightarrow P^o$	3.500	3.112	3.157
$C_2^i \rightarrow C_0^o$	3.506	3.235	3.164
$C_2^i \rightarrow C_1^o$	3.503	3.230	3.157
$C_2^i \rightarrow C_2^o$	3.510	3.236	3.157
$C_2^i \rightarrow C_3^o$	3.506	3.232	3.154
$C_2^i \rightarrow P^o$	3.492	3.109	3.156
$C_3^i \rightarrow C_0^o$	3.501	3.222	3.156
$C_3^i \rightarrow C_1^o$	3.504	3.222	3.157
$C_3^i \rightarrow C_2^o$	3.497	3.227	3.151
$C_3^i \rightarrow C_3^o$	3.508	3.221	3.153
$C_3^i \rightarrow P^o$	3.491	3.104	3.149
$P^i \rightarrow C_0^o$	2.691	2.495	2.364
$P^i \rightarrow C_1^o$	2.691	2.492	2.366
$P^i \rightarrow C_2^o$	2.701	2.498	2.372
$P^i \rightarrow C_3^o$	2.701	2.496	2.374

D.1.3 Dynamic Energy

Table D.3: Dynamic energy E for all supported unicast paths. All values are in fJ/bit of input data.

Path	Header	Data	Tail
$C_0^i \rightarrow C_0^o$	158.802	131.959	138.686
$C_0^i \rightarrow C_1^o$	149.291	131.409	138.215
$C_0^i \rightarrow C_2^o$	152.070	131.547	138.299
$C_0^i \rightarrow C_3^o$	149.491	131.435	138.211
$C_0^i \rightarrow P^o$	93.308	80.393	83.601
$C_1^i \rightarrow C_0^o$	159.276	132.181	138.966
$C_1^i \rightarrow C_1^o$	152.913	131.812	138.644
$C_1^i \rightarrow C_2^o$	156.011	131.999	138.745
$C_1^i \rightarrow C_3^o$	153.295	131.867	138.579
$C_1^i \rightarrow P^o$	92.148	80.078	83.299
$C_2^i \rightarrow C_0^o$	159.311	132.185	138.887
$C_2^i \rightarrow C_1^o$	153.118	131.911	138.670
$C_2^i \rightarrow C_2^o$	155.886	132.013	138.623
$C_2^i \rightarrow C_3^o$	153.269	131.903	138.512
$C_2^i \rightarrow P^o$	92.277	79.967	83.375
$C_3^i \rightarrow C_0^o$	158.973	132.171	138.898
$C_3^i \rightarrow C_1^o$	152.787	131.805	138.525
$C_3^i \rightarrow C_2^o$	155.691	131.985	138.628
$C_3^i \rightarrow C_3^o$	153.022	131.873	138.494
$C_3^i \rightarrow P^o$	92.088	80.052	83.295
$P^i \rightarrow C_0^o$	141.977	120.838	126.807
$P^i \rightarrow C_1^o$	137.071	120.586	126.609
$P^i \rightarrow C_2^o$	139.770	120.640	126.610
$P^i \rightarrow C_3^o$	137.219	120.600	126.373

D.2 Multicast Payload Distribution

The multicast payload characterization depends on the local fan-out F , defined as the number of child outputs activated by the current router. Fan-outs $F = 1$ and $F = 4$ were directly characterized, whereas $F = 2$ and $F = 3$ were obtained by interpolation. Forward latency, initiation interval, and dynamic energy are reported separately below.

D.2.1 Forward Latency

Table D.4: Multicast common-payload forward latency L_f . All values are in ns.

Input	F	First data	Intermediate data	Tail
C_0^i	1	1.480	0.879	0.878
C_0^i	2	1.479	0.879	0.878
C_0^i	3	1.478	0.880	0.878
C_0^i	4	1.478	0.880	0.879
C_1^i	1	1.382	0.879	0.877
C_1^i	2	1.382	0.879	0.878
C_1^i	3	1.382	0.879	0.879
C_1^i	4	1.382	0.879	0.880
C_2^i	1	1.382	0.878	0.877
C_2^i	2	1.383	0.878	0.877
C_2^i	3	1.384	0.879	0.877
C_2^i	4	1.385	0.880	0.878
C_3^i	1	1.378	0.875	0.877
C_3^i	2	1.379	0.876	0.877
C_3^i	3	1.381	0.877	0.878
C_3^i	4	1.382	0.878	0.878
P^i	1	1.391	0.886	0.887
P^i	2	1.390	0.887	0.888
P^i	3	1.390	0.888	0.889
P^i	4	1.390	0.889	0.890

D.2.2 Initiation Interval

Table D.5: Multicast common-payload initiation interval II . All values are in ns.

Input	F	First data	Intermediate data	Tail
C_0^i	1	4.950	4.191	4.211
C_0^i	2	4.950	4.191	4.211
C_0^i	3	4.950	4.191	4.210
C_0^i	4	4.950	4.191	4.210
C_1^i	1	4.916	4.189	4.209
C_1^i	2	4.920	4.189	4.210
C_1^i	3	4.925	4.189	4.210
C_1^i	4	4.930	4.189	4.210
C_2^i	1	4.923	4.190	4.210
C_2^i	2	4.923	4.190	4.209
C_2^i	3	4.923	4.189	4.208
C_2^i	4	4.923	4.189	4.208
C_3^i	1	4.891	4.188	4.210
C_3^i	2	4.893	4.187	4.210
C_3^i	3	4.896	4.186	4.211
C_3^i	4	4.898	4.186	4.212
P^i	1	1.604	3.211	3.233
P^i	2	1.604	3.211	3.235
P^i	3	1.605	3.212	3.237
P^i	4	1.606	3.213	3.240

D.2.3 Dynamic Energy

Table D.6: Multicast common-payload dynamic energy E . All values are in fJ/bit of input data.

Input	F	First data	Intermediate data	Tail
C_0^i	1	179.688	174.177	176.806
C_0^i	2	241.196	232.077	235.158
C_0^i	3	302.704	289.977	293.509
C_0^i	4	364.211	347.876	351.860
C_1^i	1	178.267	174.106	176.747
C_1^i	2	238.846	231.948	235.184
C_1^i	3	299.425	289.789	293.622
C_1^i	4	360.005	347.630	352.060
C_2^i	1	178.417	174.028	176.741
C_2^i	2	238.948	231.888	235.058
C_2^i	3	299.479	289.747	293.375
C_2^i	4	360.009	347.606	351.692
C_3^i	1	178.237	173.998	176.624
C_3^i	2	238.802	231.857	235.002
C_3^i	3	299.367	289.716	293.380
C_3^i	4	359.932	347.575	351.758
P^i	1	183.831	179.269	181.768
P^i	2	244.354	237.224	240.077
P^i	3	304.878	295.179	298.387
P^i	4	365.401	353.135	356.697

D.3 Multicast Setup

The multicast setup operation corresponds to the header that transfers the packet from conventional forwarding to multicast processing. Multicast-route parsing is accounted for separately in Section D.4. For the setup operation, the measured processing time and initiation interval coincide.

Table D.7: Multicast setup characterization.

Input	II [ns]	Dynamic energy [fJ/bit]
C_0^i	1.469	7.978
C_1^i	1.467	7.942
C_2^i	1.468	7.993
C_3^i	1.468	7.977
P^i	1.949	34.400

D.4 Multicast Route Parsing

The cost of parsing a multicast-route word depends on the parser phase and on two data-dependent quantities. Parameter z denotes the number of zero nibbles skipped after the first zero nibble triggers the flush operation, while s denotes the number of changes in the child being served. Not all combinations of z and s are structurally possible. Each route flit contains seven route nibbles, and observing at least one child transition requires at least two non-zero nibbles. More generally, s child transitions require at least $s + 1$ non-zero nibbles, so a combination is valid only if $s \leq 6 - z$. For example, if $s \geq 1$, at least two nibbles must be non-zero and therefore $z \leq 5$. Impossible combinations are omitted from the characterization tables below.

Three parser phases are distinguished: *initial*, for the first route word; *continuation*, for subsequent route words; and *terminator*, for the final termination word. The original energy measurements are associated with a complete 32-bit route flit. The values reported below have therefore been divided by 32 and are expressed in fJ per input bit.

D.4.1 Initial Route Word

For the initial route word, the characterization explicitly captures the dependence on s when $z = 0$. When $z > 0$, s was not necessarily fixed to zero during characterization, since variations due to s were found to be comparatively small relative to the dominant effect introduced by changing z . The corresponding entries are therefore reported over the structurally valid range of s for each value of z .

Table D.8: Characterization of the first multicast-route word.

z	s	II [ns]	E [fJ/bit]
0	0	61.940	830.468
0	1	57.796	808.554
0	2	53.652	786.640
0	3	49.508	764.727
0	4	45.364	742.813
0	5–6	41.220	720.899
1	0–5	74.853	984.838
2	0–4	63.502	776.399
3	0–3	51.699	722.541
4	0–2	39.879	591.216
5	0–1	32.713	497.008
6	0	32.713	497.008

D.4.2 Continuation Route Words

Table D.9: Initiation interval II , in ns, for continuation multicast-route words. Slashes indicate structurally impossible combinations of z and s .

$z \backslash s$	0	1	2	3	4	5	6
0	31.000	34.379	37.758	41.137	44.516	47.894	51.273
1	29.423	32.156	34.860	37.563	40.266	42.969	/
2	27.846	29.934	31.961	33.989	36.016	/	/
3	26.269	27.712	29.063	30.415	/	/	/
4	24.692	25.489	26.165	/	/	/	/
5	23.115	23.267	/	/	/	/	/
6	23.115	/	/	/	/	/	/

The 31 ns value at $z = 0, s = 0$ is an optimistic modeling assumption for full continuation words that remain on the same child, both for the measured nibble patterns used to characterize this condition.

Table D.10: Dynamic energy E , in fJ/bit of input data, for continuation multicast-route words. Slashes indicate structurally impossible combinations of z and s .

$z \backslash s$	0	1	2	3	4	5	6
0	581.230	638.242	695.253	752.264	809.275	866.286	923.297
1	534.228	579.256	624.865	670.474	716.083	761.692	/
2	487.226	520.271	554.477	588.684	622.891	/	/
3	440.224	461.285	484.090	506.894	/	/	/
4	393.222	402.300	413.702	/	/	/	/
5	346.219	343.314	/	/	/	/	/
6	346.219	/	/	/	/	/	/



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

AN ASYNCHRONOUS TREE NETWORK-ON-CHIP FOR EXACT HIERARCHICAL MULTICAST

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):

Marco

First name(s):

Ferroni

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date

Stanford, 30th of September 2026

Signature(s)

Marco Ferroni

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.